

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC ĐẠI NAM**



Tài liệu học tập

LẬP TRÌNH CƠ BẢN

Ngành: Công nghệ thông tin
Trình độ đào tạo: Đại học
Người biên soạn: TS. Đậu Hải Phong

Hà Nội , 2024

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC ĐẠI NAM**



Tài liệu học tập

LẬP TRÌNH CƠ BẢN

(Lưu hành nội bộ)

Ngành: Công nghệ thông tin

Trình độ đào tạo: Đại học

Người biên soạn: TS. Đậu Hải Phong

Thư ký

Chủ tịch

Hà Nội , 2024

MỞ ĐẦU

Kỹ năng lập trình là một trong những kỹ năng thiết yếu trong thời đại công nghệ thông tin hiện nay. Lập trình không chỉ giúp chúng ta giải quyết các vấn đề phức tạp một cách có hệ thống mà còn rèn luyện tư duy logic và khả năng phân tích, nhờ vậy có thể ứng dụng vào nhiều lĩnh vực khác nhau trong cuộc sống.

Tài liệu này nhằm mục đích trang bị cho người học những kiến thức nền tảng về lập trình thông qua ngôn ngữ lập trình C++. Nội dung được thiết kế theo lộ trình từ đơn giản đến phức tạp, từ những khái niệm cơ bản về máy tính và lập trình đến các kỹ thuật lập trình nâng cao hơn.

Người học sẽ được giới thiệu về lĩnh vực khoa học máy tính, cơ sở của việc lập trình, các thành phần cấu thành một chương trình cũng như các công cụ hỗ trợ trong quá trình thiết kế và phát triển phần mềm. Tiếp đó, các kiểu dữ liệu, cách khai báo và thao tác với biến, hằng số trong C++ sẽ được trình bày.

Với các chương trình và bài tập thực hành đa dạng, phong phú, tài liệu này sẽ giúp người học nắm vững lý thuyết đồng thời rèn luyện kỹ năng lập trình thực tế. Hi vọng rằng tài liệu sẽ trở thành người bạn đồng hành đáng tin cậy trong hành trình chinh phục ngôn ngữ lập trình C++ của các bạn.

Quyển tài liệu này bao gồm 9 chương, cung cấp cho người học những kiến thức nền tảng về lập trình bằng ngôn ngữ C++, từ những khái niệm cơ bản nhất cho đến các kỹ thuật nâng cao hơn.

Chương 1: Giới thiệu về Máy tính và Lập trình

Chương 1 giới thiệu về lĩnh vực khoa học máy tính và các khái niệm cơ bản về lập trình, giải quyết vấn đề và thiết kế phần mềm. Các thành phần của chương trình như từ khóa, biến, toán tử và dấu câu được đề cập. Các công cụ lập trình như lược đồ, sơ đồ khối và sơ đồ phân cấp cũng được giới thiệu.

Chương 2: Giới thiệu về ngôn ngữ lập trình C++

Chương 2 giúp sinh viên bắt đầu với C++ bằng cách giới thiệu các kiểu dữ liệu, định danh, khai báo biến, hằng số, chú thích, đầu ra chương trình, phép toán số học đơn giản và chuỗi ký tự. Các quy ước về phong cách lập trình cũng được giới thiệu và mô hình hóa trong suốt bài.

Chương 3: Biểu thức và thao tác cơ bản

Trong chương 3, sinh viên học cách viết chương trình nhập và xử lý dữ liệu số, ký tự và chuỗi. Việc sử dụng toán tử số học và tạo biểu thức toán học được đề cập chi tiết hơn, nhấn mạnh tới thứ tự ưu tiên của toán tử. Gỡ lỗi cũng được giới thiệu, với một phần về theo dõi chương trình bằng tay. Các phần về định dạng đầu ra đơn giản, chuyển đổi kiểu dữ liệu và sử dụng các hàm thư viện làm việc với số cũng được bao gồm.

Chương 4: Cấu trúc rẽ nhánh

Trong chương 4, sinh viên học về toán tử quan hệ, biểu thức quan hệ và cách điều khiển luồng chương trình với các câu lệnh if, if/else và if/else if. Toán tử điều kiện và câu lệnh switch cũng được đề cập. Các ứng dụng quan trọng của các cấu trúc này như chương trình lựa chọn từ menu và xác nhận đầu vào cũng được bao gồm.

Chương 5: Vòng lặp và Tập tin

Chương 5 đề cập đến các cấu trúc điều khiển lặp lại. Vòng lặp while, do-while và for được dạy, cùng với các ví dụ ứng dụng thông thường. Bộ đếm, tích lũy, tổng chạy, giá trị đánh dấu cuối, và các chủ đề liên quan đến ứng dụng khác cũng được thảo luận. Đầu ra/Vào tập tin tuần tự cũng được giới thiệu. Sinh viên học cách đọc và ghi tập tin văn bản, và sử dụng vòng lặp để xử lý dữ liệu trong tập tin.

Chương 6: Hàm

Trong chương 6, sinh viên học cách và lý do để chia nhỏ chương trình thành nhiều đơn vị sử dụng cả hàm void và hàm trả về giá trị. Việc truyền đối số cũng được đề cập, nhấn mạnh khi nào thì đối số nên được truyền theo giá trị và khi nào cần được truyền theo tham chiếu. Phạm vi của các biến cũng được bao gồm, với các phần về biến cục bộ so với biến toàn cục và biến cục bộ tĩnh. Hàm nạp chồng cũng được giới thiệu và minh họa.

Chương 7: Mảng và Thuật toán tìm kiếm, sắp xếp

Trong chương 7, sinh viên học cách tạo và làm việc với mảng một chiều và nhiều chiều. Nhiều ví dụ về xử lý mảng được cung cấp bao gồm tìm tổng, trung bình, giá trị lớn nhất và nhỏ nhất trong mảng, cũng như tính tổng theo hàng, cột và toàn bộ phần tử của mảng hai chiều. Các kỹ thuật lập trình sử dụng mảng song song cũng được minh họa, và sinh viên được hướng dẫn cách sử dụng tập tin dữ liệu làm nguồn đầu vào để khởi tạo mảng.

Trong chương này, sinh viên giới thiệu các kiến thức cơ bản về sắp xếp mảng và tìm kiếm. Bao gồm các thuật toán Sắp xếp Nổi bọt, Sắp xếp Lựa chọn, Tìm kiếm Tuyến tính và Tìm kiếm Nhị phân.

Chương 8: Con trỏ

Chương 8 giải thích cách sử dụng con trỏ. So sánh và đối chiếu con trỏ với biến tham chiếu. Các chủ đề khác bao gồm toán tử con trỏ, khởi tạo con trỏ, so sánh quan hệ của con trỏ, con trỏ và mảng, con trỏ và hàm, cấp phát bộ nhớ động, và nhiều hơn nữa.

Chương 9: Dữ liệu Cấu trúc

Sinh viên được giới thiệu về kiểu dữ liệu trừu tượng và được dạy cách tạo ra chúng bằng cách sử dụng cấu trúc, liên hợp và các kiểu dữ liệu liệt kê. Các thảo luận và ví dụ bao gồm sử dụng con trỏ đến cấu trúc, truyền cấu trúc cho hàm và trả về cấu trúc từ hàm.

Xin chân thành cảm ơn!

Đậu Hải Phong

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU VỀ MÁY TÍNH VÀ NGÔN NGỮ LẬP TRÌNH.....1

1.1. Tại sao phải viết chương trình.....	1
1.2. Hệ thống máy tính: phần cứng và phần mềm.....	2
1.3. Chương trình và Ngôn ngữ lập trình	9
1.4. Chương trình gồm những thành phần gì?.....	15
1.5. Nhập, Xử lý và Xuất.....	19
1.6. Quá trình viết chương trình	20
1.7. Lập trình hướng cấu trúc và hướng đối tượng.....	25
1.8. Câu hỏi và Bài tập	26
1.8.1. Câu hỏi.....	26
1.8.2. Thuật toán	27

CHƯƠNG 2. GIỚI THIỆU VỀ NGÔN NGỮ LẬP TRÌNH C++.....29

2.1. Các thành phần của chương trình C++.....	29
2.2. Đối tượng cout.....	33
2.3. Chỉ dẫn #include.....	34
2.4. Biến, giá trị, phép gán và khởi tạo giá trị cho biến	35
2.4.1. Biến	35
2.4.2. Giá trị (Literals).....	36
2.4.3. Phép gán	36
2.4.4. Khởi tạo giá trị cho biến	37
2.5. Mã định danh.....	38
2.6. Kiểu dữ liệu: nguyên, kí tự, thực, bool, chuỗi.....	39
2.6.1. Các kiểu dữ liệu số nguyên	39
2.6.2. Kiểu dữ liệu số thực	41
2.6.3. Kiểu dữ liệu kí tự.....	44
2.6.4. Kiểu bool	45
2.6.5. Kiểu chuỗi (string)	46
2.7. Phạm vi của biến	47
2.8. Toán tử số học	47

2.9.	Ghi chú	49
2.10.	Hằng số	50
2.11.	Câu hỏi và Bài tập.....	51
2.11.1.	Câu hỏi	51
2.11.2.	Bài tập	52
CHƯƠNG 3. BIỂU THỨC VÀ THAO TÁC CƠ BẢN TRÊN DỮ LIỆU		56
3.1.	Đối tượng cin.....	56
3.2.	Biểu thức toán học.....	58
3.3.	Chuyển đổi dữ liệu	60
3.4.	Tràn trên (Overflow) và tràn dưới (Underflow).....	62
3.5.	Ép kiểu.....	63
3.6.	Gán nhiều và gán kết hợp.....	64
3.7.	Định dạng đầu ra	66
3.8.	Làm việc với đối tượng chuỗi và kí tự	68
3.9.	Hàm trong thư viện toán học	71
3.10.	Câu hỏi và Bài tập.....	74
3.10.1.	Câu hỏi	74
3.10.2.	Bài tập	76
CHƯƠNG 4. CẤU TRÚC Rẽ NHÁNH.....		84
4.1.	Toán tử quan hệ	84
4.2.	Câu lệnh if khuyết	85
4.3.	Câu lệnh if/else	88
4.4.	Câu lệnh if lồng nhau	90
4.5.	Câu lệnh if/ else if	94
4.6.	Biến cờ (Flags)	96
4.7.	Toán tử lôgic.....	97
4.8.	Xác thực đầu vào	100
4.9.	So sánh chuỗi và các kí tự	102
4.10.	Toán tử điều kiện ?.....	106
4.11.	Câu lệnh switch.....	107

4.12.	Thực đơn.....	110
4.13.	Câu hỏi và Bài tập.....	112
4.13.1.	Câu hỏi	112
4.13.2.	Bài tập	113
CHƯƠNG 5. VÒNG LẶP VÀ TỆP		120
5.1.	Các Toán Tử Tăng và Giảm.....	120
5.2.	Giới thiệu về Vòng Lặp: Vòng Lặp while.....	123
5.3.	Sử Dụng Vòng Lặp while cho Kiểm Tra Dữ Liệu Đầu Vào	125
5.4.	Bộ Đếm.....	126
5.5.	Vòng lặp do-while	127
5.6.	Vòng lặp for.....	131
5.7.	Vòng lặp lồng nhau	135
5.8.	Sử dụng tệp để lưu trữ dữ liệu.....	138
5.9.	Câu lệnh Break/Continue	154
5.10.	Câu hỏi và Bài tập.....	156
5.10.1.	Câu hỏi	156
5.10.2.	Bài tập	157
CHƯƠNG 6. HÀM		164
6.1.	Mô đun chương trình.....	164
6.2.	Định nghĩa và lời gọi hàm	164
6.3.	Hàm nguyên mẫu.....	168
6.4.	Truyền tham số cho hàm	170
6.5.	Minh họa: thực đơn với mỗi chức năng là 1 hàm	174
6.6.	Giá trị trả về từ hàm.....	176
6.7.	Biến cục bộ và toàn cục.....	181
6.8.	Đối số mặc định.....	184
6.9.	Sử dụng biến tham chiếu là tham số.....	185
6.10.	Một số vấn đề khác	187
6.11.	Câu hỏi và Bài tập.....	191
6.11.1.	Câu hỏi	191

6.11.2. Bài tập	191
CHƯƠNG 7. MẢNG VÀ THUẬT TOÁN SẮP XẾP.....	203
7.1. Khái niệm mảng	203
7.2. Truy cập các phần tử mảng.....	204
7.3. Phạm vi mảng dựa trên vòng lặp.....	208
7.4. Minh họa: sử dụng mảng song song.....	209
7.5. Mảng là tham số của hàm.....	210
7.6. Mảng 2 chiều	213
7.7. Mảng nhiều chiều	216
7.8. Một số thuật toán tìm kiếm và sắp xếp.....	218
7.8.1. Thuật toán tìm kiếm	218
7.8.2. Thuật toán sắp xếp	221
7.9. Câu hỏi và Bài tập	225
7.9.1. Câu hỏi.....	225
7.9.2. Bài tập	226
CHƯƠNG 8. CON TRỎ.....	237
8.1. Địa chỉ của biến.....	237
8.2. Biến con trỏ	238
8.3. Mối quan hệ giữa Mảng và Con trỏ	243
8.4. Khởi tạo con trỏ.....	244
8.5. So sánh các con trỏ	245
8.6. Con trỏ là tham số cho hàm.....	247
8.7. Cấp phát động.....	250
8.8. Hàm trả về con trỏ	254
8.9. Câu hỏi và Bài tập	257
8.9.1. Câu hỏi.....	257
8.9.2. Bài tập	258
CHƯƠNG 9. DỮ LIỆU CÓ CẤU TRÚC	261
9.1. Kiểu dữ liệu trừu tượng	261
9.2. Cấu trúc	262

9.3.	Truy cập thành viên cấu trúc	264
9.4.	Khởi tạo cấu trúc	266
9.5.	Mảng cấu trúc	267
9.6.	Cấu trúc là tham số hàm	269
9.7.	Con trỏ trỏ đến cấu trúc	272
9.8.	Câu hỏi và Bài tập	274
9.8.1.	<i>Câu hỏi</i>	274
9.8.2.	<i>Bài tập</i>	275
TÀI LIỆU THAM KHẢO.....		280

DANH MỤC HÌNH ẢNH, BẢNG BIỂU

Bảng 1.1. Các ngôn ngữ lập trình.....	11
Bảng 1.2. Các thành phần phổ biến trong chương trình.....	16
Bảng 2.1. Bảng từ khóa trong C++.....	38
Bảng 4.1. Giá trị ASCII của các ký tự thường được sử dụng.....	102
Bảng 5.1. Bảng luồng tệp tin	142
Bảng 6.1. Bảng các thành phần của hàm.....	165
Bảng 9.1. Các kiểu dữ liệu cơ bản.....	261
Hình 1.1. Chương trình xử lý văn bản và chương trình trình chiếu	2
Hình 1.2. Các thiết bị điển hình trong hệ thống máy tính	3
Hình 1.3. Máy tính ENIAC	4
Hình 1.4. Một bộ vi xử lý	5
Hình 1.5. Tổ chức của CPU.....	5
Hình 1.6. Ngôn ngữ cấp thấp và ngôn ngữ cấp cao	11
Hình 1.7. Dịch tệp nguồn C++ sang tệp thực thi.....	14
Hình 1.8. Môi trường phát triển tích hợp (IDE).....	15
Hình 1.9. Tên biến đại diện cho vị trí bộ nhớ.....	18
Hình 1.10. Các bước viết chương trình	21
Hình 1.11. Biểu đồ phân cấp	22
Hình 1.12. Các thủ tục trong một chương trình.....	25
Hình 1.13. Các đối tượng trong một chương trình	26
Hình 2.1. Cửa sổ giao diện điều khiển	33
Hình 4.1. Cấu trúc quyết định	86
Hình 4.2. Logic câu lệnh if/else.....	89
Hình 4.3. Sơ đồ thuật toán điều kiện vay lãi suất đặc biệt	92
Hình 5.1. Logic của vòng lặp	123
Hình 5.2. Sơ đồ thuật toán cho vòng lặp while	124
Hình 5.3. Sơ đồ thuật toán xác thực giá trị đầu vào	126
Hình 5.4. Logic vòng lặp do-while.....	128
Hình 5.5. Ghi dữ liệu vào tệp	139

Hình 5.6. Đọc dữ liệu từ tệp	139
Hình 5.7. Một số loại tệp tin.....	141
Hình 5.8. Kết quả được ghi trong tệp demofile.txt trên từng dòng	145
Hình 5.9. Kết quả được ghi trong tệp demofile.txt trên 1 dòng	146
Hình 5.10. Kết quả được ghi vào tệp Numbers.txt.....	147
Hình 5.11. Nội dung tệp Friends	148
Hình 5.12. Nội dung tệp NumericData.txt	150
Hình 5.13. Nội dung tệp Sales.txt.....	152
Hình 5.14. Nội dung tệp ListOfNumbers.txt.....	153
Hình 6.1. Các thành phần của hàm.....	165
Hình 6.2. Truyền giá trị cho tham số	171
Hình 6.3. Truyền nhiều giá trị cho các tham số.....	173
Hình 6.4. Hàm trả về giá trị	177
Hình 6.5. Định nghĩa hàm trả về giá trị	177
Hình 6.6. Truyền các đối số cho hàm và hàm trả về giá trị	179
Hình 6.7. Hàm là đối số của hàm khác.....	181
Hình 7.1. Nội dung tệp Data1.txt	207
Hình 8.1. Biến và địa chỉ của biến.....	237
Hình 8.2. Truyền một mảng cho hàm	239
Hình 8.3. Truyền đối số bằng tham chiếu.....	240
Hình 8.4. Mối quan hệ giữa con trỏ và biến.....	241
Hình 8.5. Các phần tử của mảng	245
Hình 8.6. Biến con trỏ iptr trỏ đến một đoạn bộ nhớ được cấp phát động.....	251
Hình 9.1. Các biến có cùng kiểu cấu trúc.....	264

CHƯƠNG 1. GIỚI THIỆU VỀ MÁY TÍNH VÀ NGÔN NGỮ LẬP TRÌNH

Chủ đề	
1.1. Tại sao phải có chương trình?	1.5. Đầu vào, Xử lý và Đầu ra
1.2. Hệ thống máy tính	1.6. Quá trình viết một chương trình
1.3. Chương trình và Ngôn ngữ lập trình	1.7. Lập trình hướng Thủ tục và Lập trình hướng đối tượng
1.4. Chương trình được tạo ra như thế nào?	

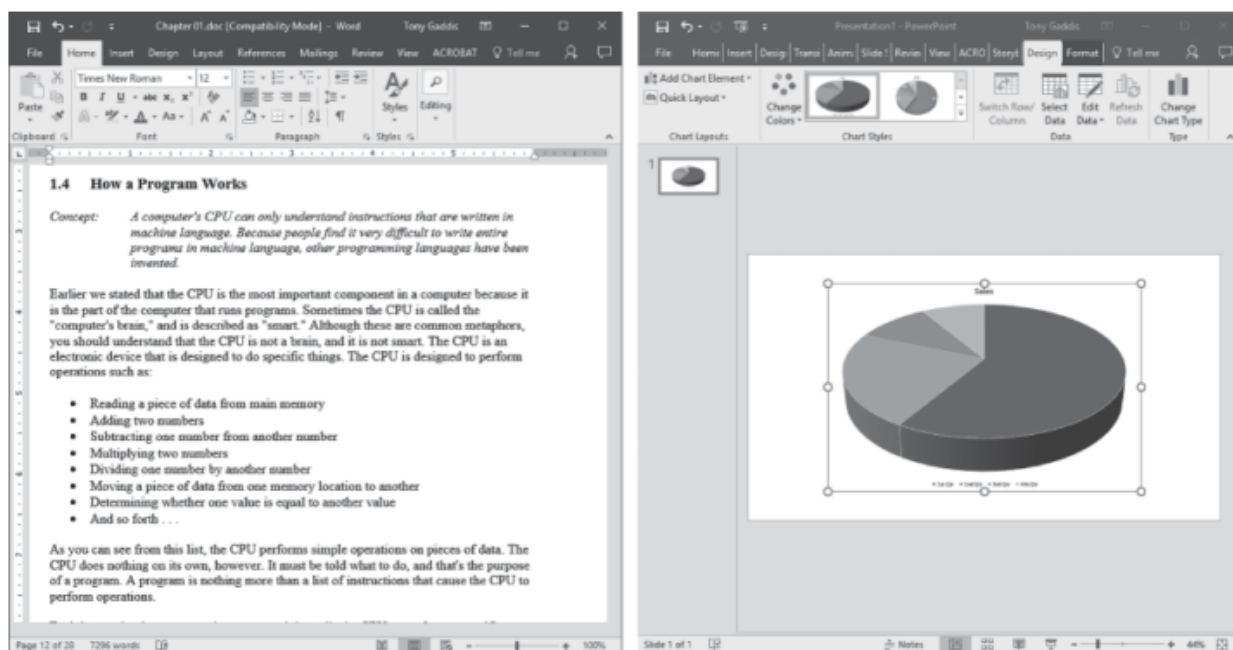
1.1. Tại sao phải viết chương trình

KHÁI NIỆM: Máy tính có thể thực hiện nhiều công việc khác nhau vì chúng có thể lập trình được.

Mọi người sử dụng máy tính với nhiều công việc khác nhau. Ở trường, học sinh sử dụng máy tính cho các công việc như viết bài, tìm kiếm bài báo, gửi e-mail và tham gia các lớp học trực tuyến. Tại nơi làm việc, mọi người sử dụng máy tính để thực hiện các giao dịch kinh doanh, giao tiếp với khách hàng và đồng nghiệp, phân tích dữ liệu, thuyết trình, điều khiển máy móc trong cơ sở sản xuất và nhiều nhiệm vụ khác. Ở nhà, mọi người sử dụng máy tính cho các công việc như thanh toán hóa đơn, mua sắm trực tuyến, mạng xã hội và chơi trò chơi trên máy tính. Và đừng quên rằng điện thoại thông minh, máy nghe nhạc MP3, hệ thống định vị ô tô và nhiều thiết bị khác cũng là máy tính. Việc sử dụng máy tính gần như vô hạn trong cuộc sống hàng ngày của.

Máy tính có thể làm được rất nhiều việc vì chúng có thể được lập trình. Điều này có nghĩa là máy tính không được thiết kế để chỉ thực hiện một công việc mà là bất kỳ công việc nào mà chương trình yêu cầu chúng thực hiện. Chương trình là một tập hợp các hướng dẫn mà máy tính tuân theo để thực hiện một tác vụ. Ví dụ, Hình 1-1 hiển thị màn hình sử dụng Microsoft Word và PowerPoint, đây là hai chương trình được sử dụng phổ biến.

Các chương trình thường được gọi là phần mềm. Phần mềm là điều cần thiết cho máy tính vì không có phần mềm, máy tính không thể làm được gì. Tất cả phần mềm sử dụng để làm cho máy tính của mình trở nên hữu ích đều được làm bởi các cá nhân được gọi là lập trình viên hoặc nhà phát triển phần mềm. Lập trình viên hoặc nhà phát triển phần mềm là người được đào tạo và có kỹ năng cần thiết để thiết kế, tạo và thử nghiệm các chương trình máy tính. Lập trình máy tính là một nghề nghiệp thú vị và bổ ích. Ngày nay, bạn sẽ tìm thấy các lập trình viên làm việc trong lĩnh vực kinh doanh, y tế, chính phủ, thực thi pháp luật, nông nghiệp, học viện, giải trí và hầu hết mọi lĩnh vực khác.



Hình 1.1. Chương trình xử lý văn bản và chương trình trình chiếu

Lập trình máy tính vừa là một nghệ thuật vừa là một khoa học. Đó là một nghệ thuật vì mọi khía cạnh của chương trình đều phải được thiết kế cẩn thận. Dưới đây là một số điều phải được thiết kế cho bất kỳ chương trình máy tính trong thực tế:

- Luồng logic của các hướng dẫn
- Các tính toán toán học
- Sự hiển thị trên màn hình
- Cách trình bày thông tin cho người dùng
- “Tính thân thiện với người dùng” của chương trình
- Tài liệu, tệp trợ giúp, hướng dẫn, v.v.

Ngoài ra, còn có khía cạnh khoa học và kỹ thuật của lập trình. Bởi vì các chương trình hiếm khi hoạt động tốt ngay lần đầu tiên được viết nên cần phải kiểm tra, sửa chữa và thiết kế lại rất nhiều. Điều này đòi hỏi sự kiên nhẫn và bền bỉ của người lập trình. Viết phần mềm cũng đòi hỏi sự kỷ luật. Lập trình viên phải học các ngôn ngữ đặc biệt như C vì máy tính không hiểu tiếng Anh hoặc các ngôn ngữ khác của con người. Các ngôn ngữ như C có những quy tắc nghiêm ngặt phải được tuân thủ cẩn thận.

Cả bản chất khoa học và nghệ thuật của việc lập trình đều khiến việc viết phần mềm máy tính giống như thiết kế một chiếc ô tô: Cả ô tô và chương trình đều phải hoạt động hiệu quả, mạnh mẽ, dễ sử dụng và dễ nhìn.

1.2. Hệ thống máy tính: phần cứng và phần mềm

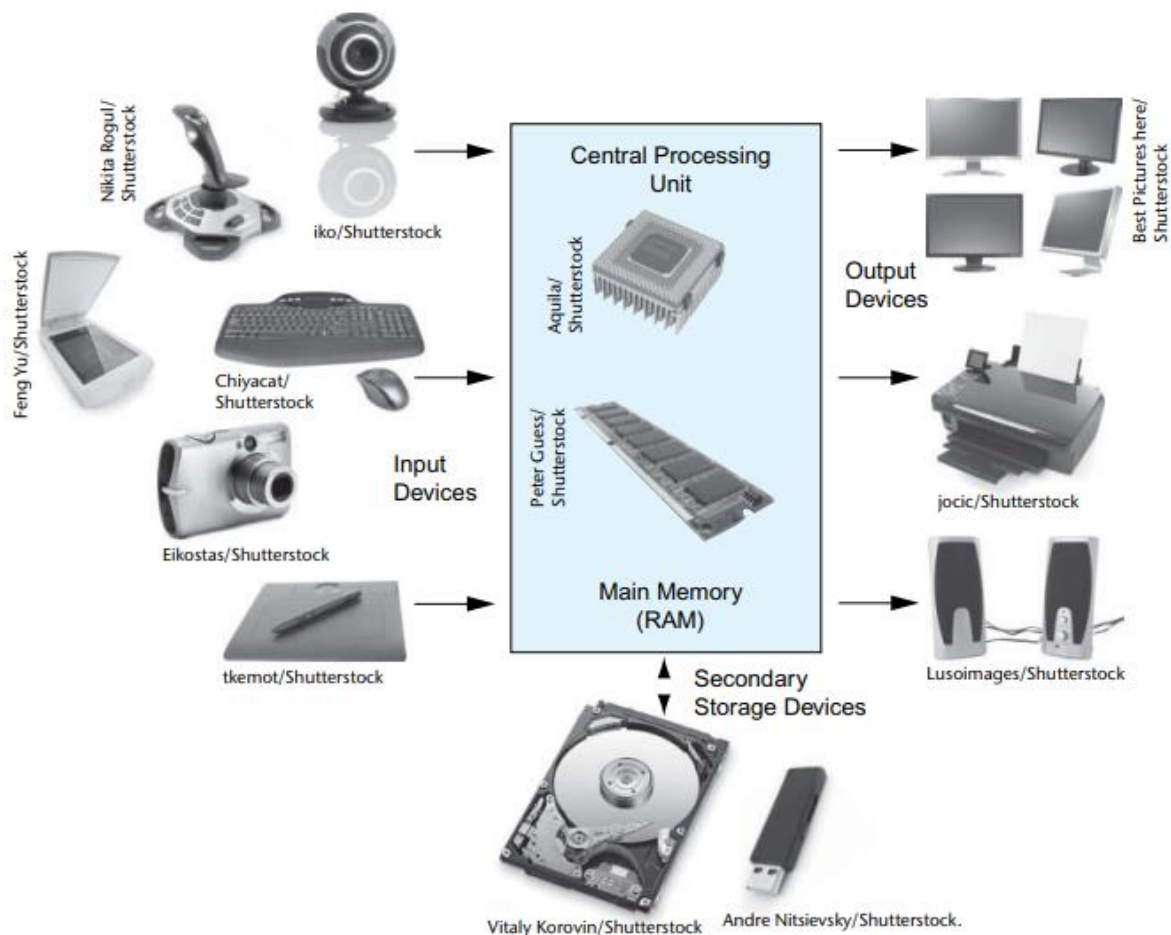
Khái niệm: Tất cả các hệ thống máy tính đều bao gồm các thiết bị phần cứng và thành phần phần mềm tương thích nhau. Phần này cung cấp cái nhìn tổng quan về tổ chức phần cứng và phần mềm máy tính tiêu chuẩn.

PHẦN CỨNG

Phần cứng đề cập đến các thành phần vật lý mà máy tính được tạo ra. Máy tính, như thường nghĩ, không phải là một thiết bị riêng lẻ mà là một hệ thống các thiết bị. Giống như các nhạc cụ trong dàn nhạc giao hưởng, mỗi thiết bị đều đóng một vai trò riêng. Một hệ thống máy tính điển hình bao gồm các thành phần chính sau:

- Bộ xử lý trung tâm (CPU):
- Bộ nhớ chính
- Thiết bị lưu trữ thứ cấp
- Thiết bị đầu vào
- Thiết bị đầu ra

Tổ chức của một hệ thống máy tính được mô tả trong Hình 1-2.

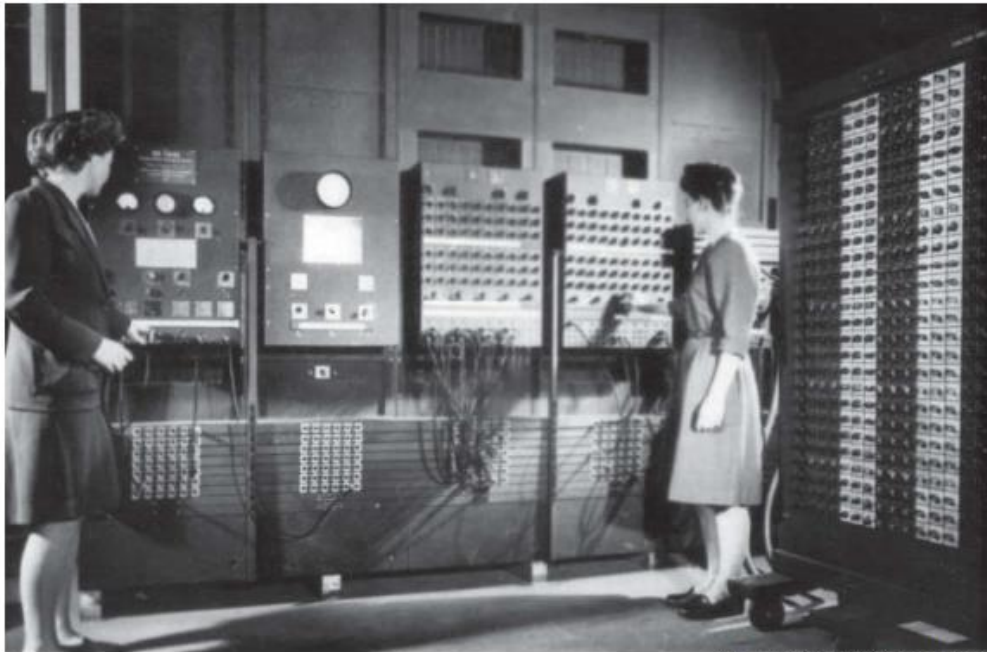


Hình 1.2. Các thiết bị điển hình trong hệ thống máy tính

CPU

Khi một máy tính đang thực hiện các nhiệm vụ mà một chương trình yêu cầu nó thực hiện, nói rằng máy tính đang chạy hoặc đang thực thi chương trình đó. Bộ xử lý trung tâm hay CPU là một bộ phận của máy tính thực sự chạy các chương trình. CPU là thành phần quan trọng nhất trong máy tính vì nếu không có nó, máy tính không thể chạy phần mềm.

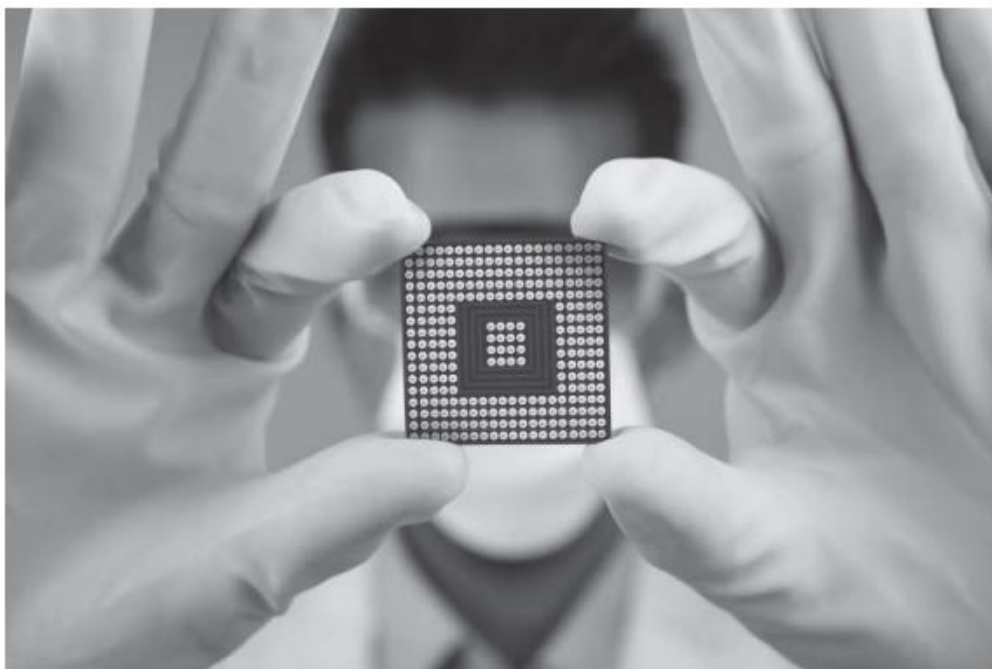
Trong những chiếc máy tính đầu tiên, CPU là những thiết bị khổng lồ được làm từ các bộ phận điện và cơ khí như ống chân không và công tắc. Hình 1-3 cho thấy một thiết bị như vậy. Hai người phụ nữ trong ảnh đang làm việc với chiếc máy tính ENIAC lịch sử. ENIAC, được nhiều người coi là máy tính điện tử có thể lập trình đầu tiên trên thế giới, được chế tạo vào năm 1945 để tính toán bảng đường đạn pháo cho Quân đội Hoa Kỳ. Chiếc máy này, chủ yếu là một CPU lớn, cao 8 feet, dài 100 feet và nặng 30 tấn.



U.S. Army Center of Military History

Hình 1.3. Máy tính ENIAC

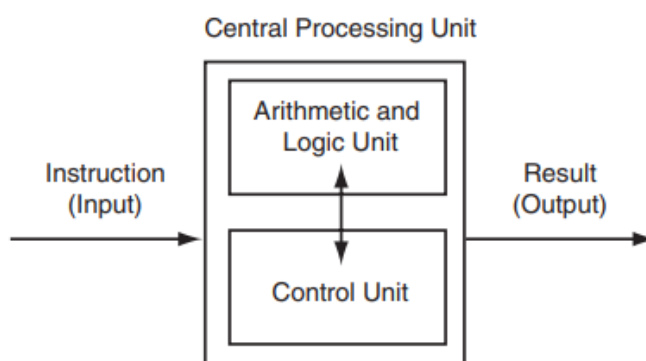
Ngày nay, CPU là những con chip nhỏ được gọi là bộ vi xử lý. Hình 1-4 thể hiện bức ảnh một kỹ thuật viên phòng thí nghiệm đang cầm một bộ vi xử lý hiện đại. Ngoài việc nhỏ hơn nhiều so với CPU cơ điện cũ trong các máy tính đời đầu, bộ vi xử lý còn mạnh hơn nhiều.



Creativa/Shutterstock

Hình 1.4. Một bộ vi xử lý

Công việc của CPU là tìm nạp hướng dẫn, làm theo hướng dẫn và đưa ra một số kết quả. Bên trong, bộ xử lý trung tâm bao gồm hai phần: bộ điều khiển và bộ số học và logic (ALU). Bộ điều khiển điều phối tất cả các hoạt động của máy tính. Nó có trách nhiệm xác định nơi nhận lệnh tiếp theo và điều chỉnh các thành phần chính khác của máy tính bằng tín hiệu điều khiển. Đơn vị số học và logic, như tên gọi của nó, được thiết kế để thực hiện các phép toán. Tổ chức của CPU được thể hiện trong Hình 1-5.



Hình 1.5. Tổ chức của CPU

Chương trình là một chuỗi các hướng dẫn được lưu trữ trong bộ nhớ của máy tính. Khi máy tính đang chạy một chương trình, CPU sẽ tham gia vào một quá trình được gọi chính thức là tìm nạp.

Tìm nạp FZetch	– Bộ điều khiển của CPU tìm nạp lệnh tiếp theo trong chuỗi các lệnh chương trình từ bộ nhớ chính.
-----------------------	---

Giải mã Decode	-	Lệnh được mã hóa dưới dạng một số. Bộ điều khiển giải mã lệnh và tạo tín hiệu điện tử.
Thực thi Execute	-	Tín hiệu được định tuyến đến thành phần thích hợp của máy tính (chẳng hạn như ALU, ổ đĩa hoặc một số thiết bị khác). Tín hiệu làm cho thành phần thực hiện một thao tác.

Các bước này được lặp lại miễn là có hướng dẫn thực hiện.

Bộ nhớ chính

Bạn có thể coi bộ nhớ chính là vùng làm việc của máy tính. Đây là nơi máy tính lưu trữ chương trình trong khi chương trình đang chạy cũng như dữ liệu mà chương trình đang hoạt động. Ví dụ: giả sử bạn đang sử dụng một chương trình xử lý văn bản để viết một bài luận cho một trong các lớp học của mình. Trong khi bạn thực hiện việc này, cả chương trình xử lý văn bản và bài luận đều được lưu trữ trong bộ nhớ chính.

Bộ nhớ chính thường được gọi là bộ nhớ truy cập ngẫu nhiên hoặc RAM. Nó được gọi như vậy vì CPU có thể truy cập nhanh chóng dữ liệu được lưu trữ ở bất kỳ vị trí ngẫu nhiên nào trong RAM. Bộ điều khiển của CPU tìm nạp lệnh tiếp theo trong chuỗi lệnh chương trình từ bộ nhớ chính. RAM thường là loại bộ nhớ dễ thay đổi, chỉ được sử dụng để lưu trữ tạm thời trong khi chương trình đang chạy. Khi tắt máy tính, nội dung trong RAM sẽ bị xóa. Bên trong máy tính của bạn, RAM được lưu trữ trong các chip nhỏ.

Bộ nhớ của máy tính được chia thành các vị trí lưu trữ nhỏ gọi là byte. Một byte là đủ bộ nhớ để chỉ lưu trữ một chữ cái trong bảng chữ cái hoặc một số nhỏ. Để làm bất cứ điều gì có ý nghĩa, một máy tính phải có rất nhiều byte. Hầu hết các máy tính ngày nay đều có hàng triệu, thậm chí hàng tỷ byte bộ nhớ.

Mỗi byte được chia thành tám vị trí lưu trữ nhỏ hơn được gọi là bit. Thuật ngữ bit là viết tắt của chữ số nhị phân. Các nhà khoa học máy tính thường coi bit như những công tắc nhỏ có thể bật hoặc tắt. Tuy nhiên, bit không phải là “công tắc” thực sự, ít nhất là không theo nghĩa thông thường. Trong hầu hết các hệ thống máy tính, bit là những thành phần điện cực nhỏ có thể chứa điện tích dương hoặc điện tích âm. Các nhà khoa học máy tính nghĩ về điện tích dương như một công tắc bật vị trí và điện tích âm đóng vai trò là công tắc ở vị trí tắt.

Bộ nhớ thứ cấp là loại bộ nhớ có thể lưu trữ dữ liệu trong thời gian dài, ngay cả khi máy tính không có nguồn điện. Các chương trình thường được lưu trữ trong bộ nhớ phụ và được tải vào bộ nhớ chính khi cần thiết. Dữ liệu quan trọng như tài liệu xử lý văn bản, dữ liệu bảng lương và hồ sơ kiểm kê cũng được lưu vào bộ lưu trữ thứ cấp. vị trí và điện tích âm đóng vai trò là công tắc ở vị trí tắt.

Bộ nhớ thứ cấp

Bộ nhớ thứ cấp là loại bộ nhớ có thể lưu trữ dữ liệu trong thời gian dài, ngay cả khi máy tính không có nguồn điện. Các chương trình thường được lưu trữ trong bộ nhớ phụ và được tải vào bộ nhớ chính khi cần thiết. Dữ liệu quan trọng như tài liệu xử lý văn bản, dữ liệu bảng lương và hồ sơ kiểm kê cũng được lưu vào bộ lưu trữ thứ cấp.

Loại thiết bị lưu trữ thứ cấp phổ biến nhất là ổ đĩa. Ổ đĩa truyền thống lưu trữ dữ liệu bằng cách mã hóa từ tính vào một đĩa tròn. Ổ đĩa thể rắn, lưu trữ dữ liệu trong bộ nhớ thể rắn, ngày càng trở nên phổ biến. Ổ đĩa thể rắn không có bộ phận chuyển động và hoạt động nhanh hơn ổ đĩa truyền thống. Hầu hết các máy tính đều có một số loại thiết bị lưu trữ thứ cấp, ổ đĩa truyền thống hoặc ổ cứng thể rắn, được gắn bên trong vỏ máy. Các thiết bị lưu trữ bên ngoài có thể được sử dụng để tạo bản sao lưu dữ liệu quan trọng hoặc di chuyển dữ liệu sang máy tính khác. Ví dụ: ổ USB (Universal Serial Bus) và thẻ nhớ SD (Secure Digital) là những thiết bị nhỏ xuất hiện trong hệ thống dưới dạng ổ đĩa. Chúng không tốn kém, đáng tin cậy và đủ nhỏ để mang theo trong túi của bạn.

Thiết bị đầu vào

Đầu vào là bất kỳ dữ liệu nào máy tính thu thập từ thế giới bên ngoài. Thiết bị thu thập thông tin và gửi đến máy tính được gọi là thiết bị đầu vào. Các thiết bị đầu vào phổ biến là bàn phím, chuột, màn hình cảm ứng, máy quét, máy ảnh kỹ thuật số và micro. Ổ đĩa, ổ CD/DVD và ổ USB cũng có thể được coi là thiết bị đầu vào vì các chương trình và thông tin được lấy ra từ chúng và tải vào bộ nhớ của máy tính.

Các thiết bị đầu ra

Đầu ra là bất kỳ thông tin nào máy tính gửi ra thế giới bên ngoài. Nó có thể là một báo cáo bán hàng, một danh sách tên hoặc một hình ảnh đồ họa. Thông tin được gửi đến một thiết bị đầu ra, thiết bị này định dạng và trình bày nó. Các thiết bị đầu ra phổ biến là màn hình, máy in và loa. Thiết bị lưu trữ cũng có thể được coi là thiết bị đầu ra vì CPU gửi cho chúng dữ liệu cần lưu.

PHẦN MỀM

Nếu một máy tính hoạt động thì phần mềm không phải là tùy chọn. Mọi việc máy tính thực hiện, từ khi bạn bật công tắc nguồn cho đến khi tắt hệ thống, đều nằm dưới sự kiểm soát của phần mềm. Có hai loại phần mềm chung: phần mềm hệ thống và phần mềm ứng dụng. Hầu hết các chương trình máy tính rõ ràng phù hợp với một trong hai loại này. hãy xem xét kỹ hơn về từng cái.

Phần mềm hệ thống

Các chương trình điều khiển và quản lý các hoạt động cơ bản của máy tính thường được gọi là phần mềm hệ thống. Phần mềm hệ thống thường bao gồm các loại chương trình sau:

Các hệ điều hành

Hệ điều hành là tập hợp các chương trình cơ bản nhất trên máy tính. Hệ điều hành kiểm soát các hoạt động bên trong của phần cứng máy tính, quản lý tất cả các thiết bị được kết nối với máy tính, cho phép lưu và lấy dữ liệu từ các thiết bị lưu trữ cũng như cho phép các chương trình khác chạy trên máy tính.

Chương trình tiện ích

Một chương trình tiện ích thực hiện một nhiệm vụ chuyên biệt nhằm nâng cao hoạt động của máy tính hoặc bảo vệ dữ liệu. Ví dụ về các chương trình tiện ích là chương trình quét vi-rút, chương trình nén tệp và chương trình sao lưu dữ liệu.

Công cụ phát triển phần mềm

Các công cụ phần mềm mà lập trình viên sử dụng để tạo, sửa đổi và kiểm tra phần mềm được gọi là công cụ phát triển phần mềm. Trình biên dịch và môi trường phát triển tích hợp mà sẽ thảo luận sau trong chương này là những ví dụ về các chương trình thuộc loại này.

Phần mềm ứng dụng

Các chương trình giúp máy tính trở nên hữu ích trong công việc hàng ngày được gọi là phần mềm ứng dụng. Đây là những chương trình mà mọi người thường dành phần lớn thời gian để chạy trên máy tính của mình. Hình 1-1, ở đầu chương này, hiển thị màn hình của hai ứng dụng thường được sử dụng-Microsoft Word, một chương trình xử lý văn bản và Microsoft PowerPoint, một chương trình thuyết trình. Một số ví dụ khác về phần mềm ứng dụng là chương trình bảng tính, chương trình e-mail, trình duyệt web và chương trình trò chơi.

Kiểm tra

1. Tại sao máy tính lại được nhiều người, nhiều ngành nghề khác nhau sử dụng?
2. Liệt kê năm thành phần phần cứng chính của hệ thống máy tính
3. Liệt kê năm thành phần phần cứng chính của hệ thống máy tính
4. Mô tả các bước trong chu trình tìm nạp/giải mã/thực thi.
5. Địa chỉ bộ nhớ là gì? Mục đích của nó là gì?
6. Địa chỉ bộ nhớ là gì? Mục đích của nó là gì?
7. Hai loại phần mềm chung là gì?
8. Bộ chương trình cơ bản nào kiểm soát hoạt động bên trong của phần cứng máy tính?
9. Bạn gọi một chương trình thực hiện một tác vụ chuyên biệt, chẳng hạn như trình quét vi-rút, chương trình nén tệp hoặc chương trình sao lưu dữ liệu là gì?
10. Các chương trình xử lý văn bản, chương trình bảng tính, chương trình e-mail, trình duyệt web và chương trình trò chơi thuộc loại phần mềm nào?

1.3. Chương trình và Ngôn ngữ lập trình

KHÁI NIỆM: Chương trình là một tập hợp các hướng dẫn mà máy tính tuân theo để thực hiện một tác vụ. Ngôn ngữ lập trình là ngôn ngữ đặc biệt dùng để viết chương trình máy tính.

Một chương trình là gì?

Máy tính được thiết kế để làm theo hướng dẫn. chương trình máy tính là một tập hợp các hướng dẫn cho máy tính biết cách giải quyết vấn đề hoặc thực hiện một tác vụ. ví dụ: giả sử muốn máy tính tính tổng lương của ai đó. Dưới đây là danh sách những việc máy tính nên làm:

1. Hiện thị thông báo trên màn hình hỏi “bạn đã làm việc bao nhiêu giờ?”
2. Đợi người dùng nhập số giờ đã làm việc. khi người dùng nhập một số, hãy lưu nó vào bộ nhớ.
3. Hiện thị thông báo trên màn hình hỏi “bạn được trả bao nhiêu một giờ?”
4. Đợi người dùng nhập mức lương theo giờ. khi người dùng nhập một số, hãy lưu nó trong trí nhớ.
5. Nhân số giờ với số tiền được trả mỗi giờ và lưu kết quả vào bộ nhớ.
6. Hiện thị thông báo trên màn hình cho biết số tiền kiếm được. thông báo phải bao gồm kết quả tính toán được thực hiện ở bước 5.

Nói chung, các hướng dẫn này được gọi là một thuật toán. *Thuật toán là một tập hợp các bước được xác định rõ ràng để thực hiện một nhiệm vụ hoặc giải quyết một vấn đề.* Lưu ý các bước này được sắp xếp tuần tự. Bước 1 phải được thực hiện trước bước 2, v.v. điều quan trọng là những hướng dẫn này phải được thực hiện theo trình tự thích hợp.

Mặc dù bạn và tôi có thể dễ dàng hiểu được các hướng dẫn trong thuật toán tính lương nhưng nó vẫn chưa sẵn sàng để thực thi trên máy tính. CPU của máy tính chỉ có thể xử lý các hướng dẫn được viết bằng ngôn ngữ máy. Nếu bạn xem một chương trình ngôn ngữ máy, bạn sẽ thấy một dòng số nhị phân (các số chỉ bao gồm 1 và 0). Các số nhị phân tạo thành các hướng dẫn ngôn ngữ máy mà CPU hiểu là các lệnh. đây là một ví dụ về hướng dẫn ngôn ngữ máy có thể trông như thế nào:

1011010000000101

Như bạn có thể tưởng tượng, quá trình mã hóa một thuật toán thành ngôn ngữ máy rất tẻ nhạt và khó khăn. Ngoài ra, mỗi loại CPU khác nhau đều có ngôn ngữ máy riêng. nếu bạn viết chương trình ngôn ngữ máy cho máy tính A rồi muốn chạy chương trình đó trên máy tính B có loại CPU khác, bạn sẽ phải viết lại chương trình bằng ngôn ngữ máy của máy tính B.

Ngôn ngữ lập trình, sử dụng từ thay vì số, được phát minh để giảm bớt công việc lập trình. Một chương trình có thể được viết bằng ngôn ngữ lập trình, chẳng hạn như C++, để

hiều hơn nhiều so với ngôn ngữ máy. các lập trình viên lưu chương trình của họ vào các tệp văn bản, sau đó sử dụng phần mềm đặc biệt để chuyển đổi chương trình của họ sang ngôn ngữ máy.

Chương trình 1-1 cho thấy thuật toán tính lương có thể được viết bằng C++ như thế nào.

“Đầu ra chương trình với ví dụ Đầu vào” hiển thị những gì chương trình sẽ hiển thị trên màn hình khi nó đang chạy. Trong ví dụ này, người dùng nhập 10 cho số giờ làm việc và 15 cho mức lương theo giờ. Chương trình hiển thị thu nhập là 150 USD.

LƯU Ý: số dòng được hiển thị trong chương trình 1-1 không phải là một phần của chương trình. cuốn sách này hiển thị số dòng trong tất cả danh sách chương trình để giúp chỉ ra các phần cụ thể của chương trình.

Chương trình 1.1. Chương trình này tính toán mức lương của người dùng

```
// Chương trình này tính toán mức lương của người dùng
#include <iostream>
using namespace std;

int main()
{
    double hours, rate, pay;

    // Lấy số giờ làm việc.
    cout << " Bạn đã làm việc bao nhiêu giờ?";
    cin >> hours;

    // Nhận mức lương theo giờ.
    cout << " Bạn được trả bao nhiêu mỗi giờ?";
    cin >> rate;

    // Tính lương.
    pay = hours * rate;

    // Hiển thị tiền lương.
    cout << " Bạn đã kiếm được số tiền là " << pay << endl;
    return 0;
}
```

Kết quả chương trình:

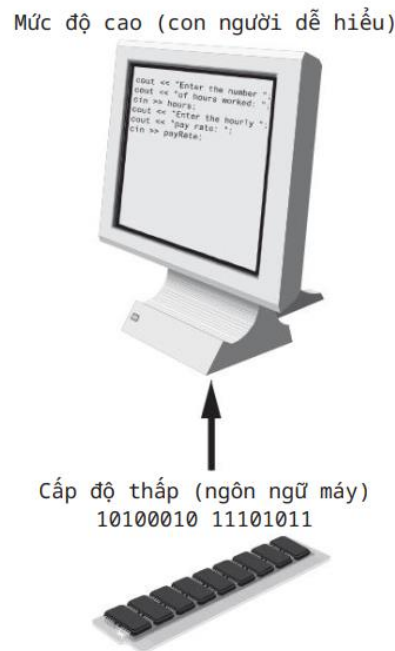
Bạn đã làm bao nhiêu giờ? 10
Bạn được trả bao nhiêu một giờ? 15
Bạn kiếm được số tiền là 150

Ngôn ngữ lập trình

Theo nghĩa rộng, có hai loại ngôn ngữ lập trình: cấp thấp và cấp cao. Ngôn ngữ cấp thấp gần với trình độ của máy tính, nghĩa là nó giống với ngôn ngữ máy số của máy tính hơn là ngôn ngữ tự nhiên của con người.

Ngôn ngữ dễ học nhất là ngôn ngữ cấp cao. Chúng được gọi là “cấp cao” vì chúng gần với mức độ con người có thể đọc được hơn là khả năng đọc của máy tính.

Hình 1-7 minh họa khái niệm về cấp độ ngôn ngữ.



Hình 1.6. Ngôn ngữ cấp thấp và ngôn ngữ cấp cao

Nhiều ngôn ngữ cấp cao đã được tạo ra. Bảng 1-1 liệt kê một số ngôn ngữ nổi tiếng.

Ngoài các tính năng cấp cao cần thiết để viết các ứng dụng như hệ thống trả lương và chương trình kiểm kê, C++ còn có nhiều tính năng cấp thấp. C++ dựa trên ngôn ngữ C, được phát minh cho các mục đích như viết hệ điều hành và trình biên dịch.

Vì C++ phát triển từ C nên nó mang theo tất cả các khả năng cấp thấp của C.

Bảng 1.1. Các ngôn ngữ lập trình

Ngôn ngữ	Mô tả
BASIC	Tất cả các mục đích Mã Instruction Symbolic mới bắt đầu. Một ngôn ngữ lập trình chung ban đầu được thiết kế đủ đơn giản cho người mới bắt đầu học.
FORTRAN	Trình dịch công thức. Một ngôn ngữ được thiết kế để lập trình các thuật toán toán học phức tạp.
COBOL	Ngôn ngữ định hướng nghiệp vụ. Một ngôn ngữ được thiết kế cho các ứng dụng nghiệp vụ.
Pascal	Một ngôn ngữ có cấu trúc, có mục đích chung được thiết kế chủ yếu để dạy lập trình.
C	Một ngôn ngữ có cấu trúc, có mục đích chung được phát triển tại Phòng thí nghiệm Bell. C cung cấp cả tính năng cấp cao và cấp thấp. Dựa trên

Ngôn ngữ	Mô tả
	ngôn ngữ C, C++ cung cấp các tính năng hướng đối tượng không có trong C. Cũng được phát minh tại Phòng thí nghiệm Bell.
C++	Dựa trên ngôn ngữ C, C++ cung cấp các tính năng hướng đối tượng không có trong C. Cũng được phát minh tại Phòng thí nghiệm Bell.
C#	Phát âm là “C shape” Một ngôn ngữ do Microsoft phát minh để phát triển các ứng dụng dựa trên nền tảng Microsoft .NET
Java	Một ngôn ngữ hướng đối tượng có thể được sử dụng để phát triển các chương trình chạy trên nhiều loại thiết bị khác nhau.
JavaScript	JavaScript có thể được sử dụng để viết các chương trình nhỏ chạy trên các trang web. Mặc dù có tên như vậy nhưng JavaScript không liên quan đến Java.
Python	Python là ngôn ngữ có mục đích chung được tạo ra vào đầu những năm 1990. Nó đã trở nên phổ biến trong cả ứng dụng kinh doanh và học thuật.
Ruby	Ruby là ngôn ngữ có mục đích chung được tạo ra vào những năm 1990. Nó ngày càng trở thành ngôn ngữ phổ biến cho các chương trình chạy trên máy chủ web.
Visual Basic	Môi trường phát triển phần mềm và ngôn ngữ lập trình của Microsoft cho phép các lập trình viên nhanh chóng tạo ra các ứng dụng dựa trên Windows.

C++ phổ biến không chỉ vì sự kết hợp các tính năng cấp thấp và cấp cao mà còn vì tính di động của nó. Điều này có nghĩa là chương trình C++ có thể được viết trên một loại máy tính, sau đó chạy trên nhiều loại hệ thống khác. Điều này thường yêu cầu chương trình phải được biên dịch lại trên từng loại hệ thống, nhưng bản thân chương trình có thể cần ít hoặc không cần thay đổi.

LƯU Ý: Các chương trình được viết cho môi trường đồ họa cụ thể thường yêu cầu những thay đổi đáng kể khi chuyển sang một loại hệ thống khác. Ví dụ về các môi trường đồ họa như vậy là Windows, hệ thống X-Window và hệ điều hành macOS.

Mã nguồn, Mã đối tượng và Mã thực thi

Khi một chương trình C++ được viết, nó phải được gõ vào máy tính và lưu vào một tập tin. Một trình soạn thảo văn bản, tương tự như một chương trình xử lý văn bản, được sử dụng cho tác vụ này. Các câu lệnh do lập trình viên viết được gọi là mã nguồn và tệp chứa chúng được gọi là tệp nguồn.

Sau khi mã nguồn được lưu vào một tập tin, quá trình dịch nó sang ngôn ngữ máy có thể bắt đầu. Trong giai đoạn đầu tiên của quá trình này, một chương trình được gọi là bộ tiền xử lý sẽ đọc mã nguồn. Bộ tiền xử lý tìm kiếm các dòng đặc biệt bắt đầu bằng ký hiệu

#. Những dòng này chứa các lệnh khiến bộ tiền xử lý sửa đổi mã nguồn theo một cách nào đó.

Trong giai đoạn tiếp theo, trình biên dịch sẽ thực hiện các bước thông qua mã nguồn được xử lý trước, dịch từng lệnh mã nguồn sang lệnh ngôn ngữ máy thích hợp. Quá trình này sẽ phát hiện ra bất kỳ lỗi cú pháp nào có thể có trong chương trình. Lỗi cú pháp là việc sử dụng không đúng các từ khóa, toán tử, dấu câu và các thành phần ngôn ngữ khác. Nếu chương trình không có lỗi cú pháp, trình biên dịch sẽ lưu các hướng dẫn ngôn ngữ máy đã dịch, được gọi là mã đối tượng, trong một tệp đối tượng.

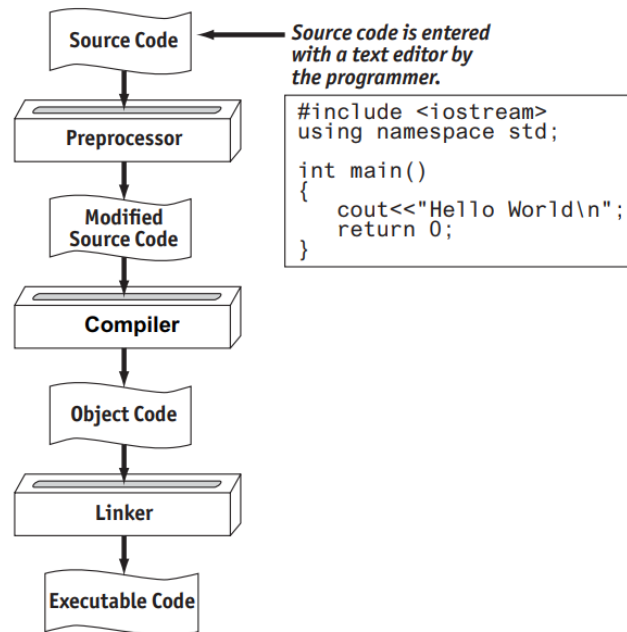
Mặc dù tệp đối tượng chứa các hướng dẫn ngôn ngữ máy nhưng nó không phải là một chương trình hoàn chỉnh. Đây là lý do: C++ được trang bị thuận tiện với một thư viện mã viết sẵn để thực hiện các thao tác thông thường hoặc các tác vụ đôi khi khó khăn. Ví dụ: thư viện chứa mã dành riêng cho phần cứng để hiển thị thông báo trên màn hình và đọc dữ liệu nhập từ bàn phím. Nó cũng cung cấp các quy trình cho các hàm toán học, chẳng hạn như tính căn bậc hai của một số. Bộ sưu tập mã này, được gọi là thư viện thời gian chạy (runtime) rất phong phú. Các chương trình hầu như luôn sử dụng một phần thư viện này. Tuy nhiên, khi trình biên dịch tạo ra một tệp đối tượng, nó không bao gồm mã máy cho bất kỳ quy trình thư viện thời gian chạy nào mà lập trình viên có thể đã sử dụng. Trong giai đoạn cuối của quá trình dịch thuật, một chương trình khác gọi là trình liên kết (Linker) sẽ kết hợp tệp đối tượng với các quy trình thư viện cần thiết. Sau khi trình liên kết hoàn tất bước này, một tệp thực thi sẽ được tạo. Tệp thực thi chứa các hướng dẫn ngôn ngữ máy hoặc mã thực thi và sẵn sàng chạy trên máy tính.

Hình 1.6 Dịch tệp nguồn C++ sang tệp thực thi Các chương trình hầu như luôn sử dụng một phần của nó. Tuy nhiên, khi trình biên dịch tạo ra một tệp đối tượng, nó không bao gồm mã máy cho bất kỳ quy trình thư viện thời gian chạy nào mà lập trình viên có thể đã sử dụng. Trong giai đoạn cuối của quá trình dịch thuật, một chương trình khác gọi là trình liên kết sẽ kết hợp tệp đối tượng với các quy trình thư viện cần thiết. Sau khi trình liên kết hoàn tất bước này, một tệp thực thi sẽ được tạo. Tệp thực thi chứa các hướng dẫn ngôn ngữ máy hoặc mã thực thi và sẵn sàng chạy trên máy tính.

Hình 1.6 minh họa quá trình dịch một tệp nguồn C++ thành một tệp thực thi.

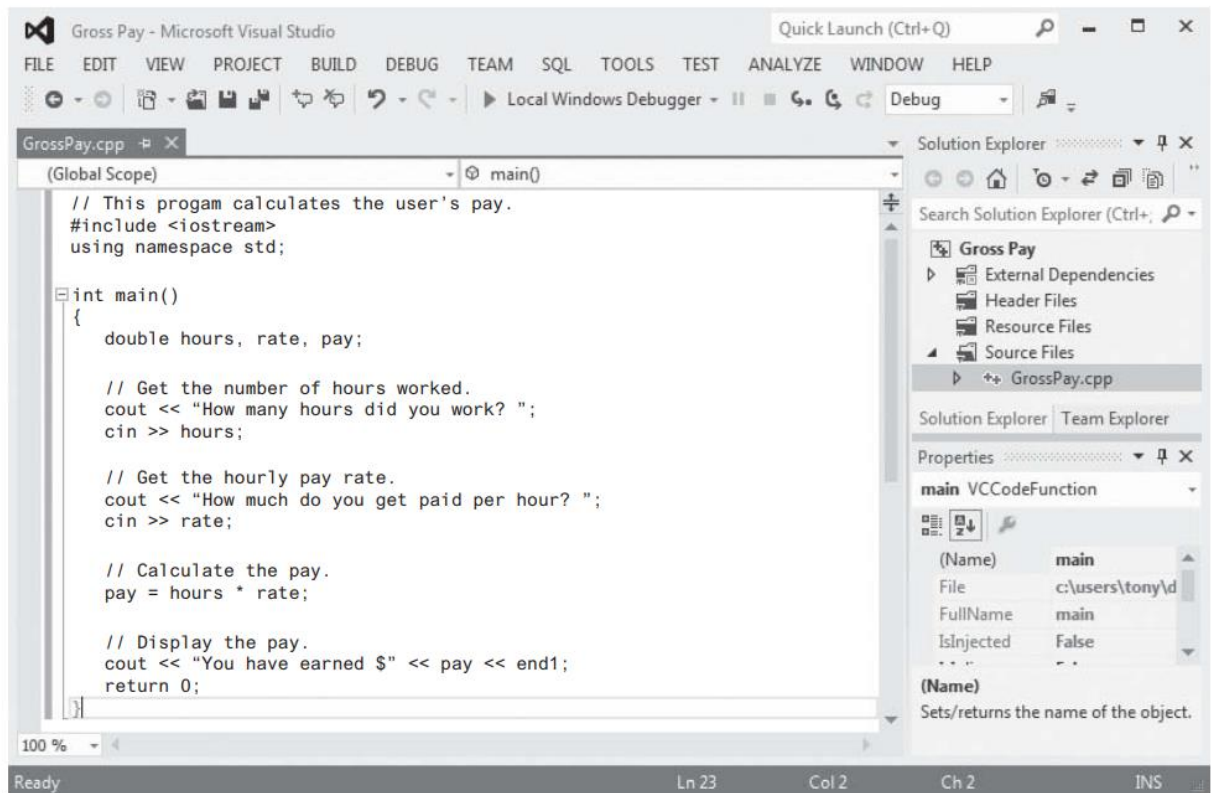
Toàn bộ quá trình gọi bộ tiền xử lý, trình biên dịch và trình liên kết có thể được bắt đầu bằng một hành động duy nhất. Ví dụ: trên hệ thống Linux, lệnh sau sẽ khiến chương trình C++ có tên hello.cpp được tiền xử lý, biên dịch và liên kết. Mã thực thi được lưu trữ trong một tệp có tên hello.

```
g++ -o hello hello.cpp
```



Hình 1.7. Dịch tệp nguồn C++ sang tệp thực thi

Nhiều hệ thống phát triển, đặc biệt là các hệ thống trên máy tính cá nhân, có môi trường phát triển tích hợp (*IDE- integrated development environments*). Các môi trường này bao gồm trình soạn thảo văn bản, trình biên dịch, trình gỡ lỗi và các tiện ích khác được tích hợp vào một gói có một bộ menu duy nhất. Quá trình tiền xử lý, biên dịch, liên kết và thậm chí thực thi một chương trình được thực hiện chỉ bằng một cú nhấp chuột hoặc bằng cách chọn một mục từ menu. Hình 1.7 hiển thị một màn hình từ Microsoft Visual Studio IDE.



Hình 1.8. Môi trường phát triển tích hợp (IDE)

Kiểm tra

1. Thuật toán là gì?
2. Tại sao ngôn ngữ lập trình máy tính được phát minh?
3. Sự khác biệt giữa ngôn ngữ cấp cao và ngôn ngữ cấp thấp là gì?
4. Tính linh động có nghĩa là gì?
5. Giải thích các hoạt động được thực hiện bởi bộ tiền xử lý, trình biên dịch và trình liên kết.
6. Giải thích những gì được lưu trữ trong tệp nguồn, tệp đối tượng và tệp thực thi.
7. Môi trường phát triển tích hợp là gì?

1.4. Chương trình gồm những thành phần gì?

KHÁI NIỆM: Có một số thành phần chung cho tất cả các ngôn ngữ lập trình.

Thành phần ngôn ngữ

Tất cả các ngôn ngữ lập trình đều có một số điểm chung. Bảng 1-2 liệt kê các thành phần phổ biến mà bạn sẽ tìm thấy ở hầu hết mọi ngôn ngữ.

Bảng 1.2. Các thành phần phổ biến trong chương trình

Thành phần	Mô tả
Từ khóa	Những từ có ý nghĩa đặc biệt. Từ khóa chỉ có thể được sử dụng cho mục đích dự định của họ. Từ khóa còn được gọi là từ dành riêng.
Định danh do lập trình viên định nghĩa	Các từ hoặc tên được xác định bởi người lập trình. Chúng là những tên tượng trưng đề cập đến các biến hoặc các chương trình lập trình.
Toán tử	Toán tử thực hiện các thao tác trên một hoặc nhiều toán hạng. Toán hạng thường là một phần dữ liệu, giống như một số.
Chấm câu (Punctuation)	Các ký tự dấu chấm câu đánh dấu sự bắt đầu hoặc kết thúc của câu lệnh hoặc phân tách các mục trong danh sách.
Cú pháp	Các quy tắc phải tuân theo khi xây dựng một chương trình. Cú pháp quy định cách sử dụng các từ khóa và toán tử cũng như vị trí các ký hiệu dấu chấm câu phải xuất hiện.

Hãy xem xét một số phần cụ thể của Chương trình 1.1 (chương trình tính lương) để xem ví dụ về từng yếu tố được liệt kê trong bảng trên. Để thuận tiện cho bạn, Chương trình 1-1 được liệt kê lại.

Chương trình 1.2. Chương trình này tính toán mức lương của người dùng

```
// Chương trình này tính toán mức lương của người dùng
#include <iostream>
using namespace std;

int main()
{
    double hours, rate, pay;

    // Lấy số giờ làm việc.
    cout << " Bạn đã làm việc bao nhiêu giờ?";
    cin >> hours;

    // Nhận mức lương theo giờ.
    cout << " Bạn được trả bao nhiêu mỗi giờ?";
    cin >> rate;

    // Tính lương.
    pay = hours * rate;

    // Hiển thị tiền lương.
    cout << " Bạn đã kiếm được số tiền là " << pay << endl;
    return 0;
}
```

Kết quả chương trình:

Bạn đã làm bao nhiêu giờ? 10
Bạn được trả bao nhiêu một giờ? 15
Bạn kiếm được số tiền là 150

Từ khóa (Từ dành riêng)

Ba từ khóa của C++ xuất hiện ở dòng 3 và 5: `use`, `namespace` và `int`. Từ `double` xuất hiện ở dòng 7 cũng là một từ khóa của C++. Những từ này luôn được viết bằng chữ thường, mỗi từ có một ý nghĩa đặc biệt trong C++ và chỉ có thể được sử dụng cho mục đích riêng của nó. Như bạn sẽ thấy, lập trình viên được phép đặt tên riêng của mình cho một số thứ nhất định trong chương trình. Tuy nhiên, các từ khóa được bảo lưu và *không thể được sử dụng cho bất kỳ mục đích nào khác ngoài mục đích được chỉ định của chúng*. Một phần của việc học ngôn ngữ lập trình là tìm hiểu các từ khóa là gì, ý nghĩa của chúng và cách sử dụng chúng.

LƯU Ý:

- Câu lệnh `#include <iostream>` ở dòng 2 là một lệnh tiền xử lý.
- Trong C++, các từ khóa được viết bằng chữ thường.

Định danh do lập trình viên định nghĩa

Các từ `giờ` (`hours`), `mức lương` (`rate`) và `lương` (`pay`), xuất hiện trong chương trình ở các dòng là các mã định danh do người lập trình xác định. Chúng không phải là một phần của ngôn ngữ C++ mà là những cái tên do người lập trình đặt ra. Trong chương trình cụ thể này, đây là tên của các biến. Như bạn sẽ tìm hiểu ở phần sau của chương này, các biến là tên của các vị trí bộ nhớ có thể chứa dữ liệu.

Trên đoạn trên có đoạn mã sau xuất hiện:

```
pay = hours * rate;
```

Ký hiệu `=` và `*` đều là toán tử. Chúng thực hiện các thao tác trên các phần dữ liệu được gọi là toán hạng. Toán tử `*` nhân hai toán hạng của nó, trong ví dụ này là các biến số `giờ` (`hour`) và `tiền công mỗi giờ` (`rate`). Ký hiệu `=` được gọi là toán tử gán. Nó lấy giá trị của biểu thức ở bên phải và lưu nó vào biến có tên xuất hiện ở bên trái. Trong ví dụ này, toán tử `=` lưu trữ trong biến `lương` (`pay`) là kết quả của biến số `giờ` nhân với biến `tiền công mỗi giờ` (`rate`). Nói cách khác là: "Biến `tiền lương` bằng số `giờ` nhân với `tiền công một giờ`, hoặc "Biến `lương` (`pay`) được gán bằng giá trị của số `giờ` nhân với `tiền công mỗi giờ`."

Chấm câu (Punctuation)

Lưu ý các dòng lệnh đều kết thúc bằng dấu chấm phẩy. Dấu chấm phẩy trong C++ tương tự như dấu chấm trong tiếng Anh: Nó đánh dấu sự kết thúc của một câu hoàn chỉnh (hoặc câu lệnh, như cách gọi trong thuật ngữ lập trình). Tuy nhiên, dấu chấm phẩy không xuất

hiện ở cuối mỗi dòng trong chương trình C++. Có những quy tắc chi phối nơi bắt buộc phải có dấu chấm phẩy và chỗ nào không có. Một phần của việc học C++ là học cách đặt dấu chấm phẩy và các ký hiệu chấm câu khác.

Dòng và câu lệnh

Thông thường, nội dung của một chương trình được hình dung dưới dạng các dòng và các câu lệnh. “Dòng” chỉ là một dòng duy nhất xuất hiện trong phần nội dung của chương trình. Hầu hết các dòng đều chứa đựng điều gì đó có ý nghĩa; tuy nhiên, một số dòng trống. Các dòng trống chỉ ở đó để làm cho chương trình dễ đọc hơn.

Câu lệnh là một lệnh hoàn chỉnh khiến máy tính thực hiện một số hành động. Đây là lệnh xuất hiện trong Chương trình 1-1:

```
cout << "Bạn đã làm việc bao nhiêu giờ?";
```

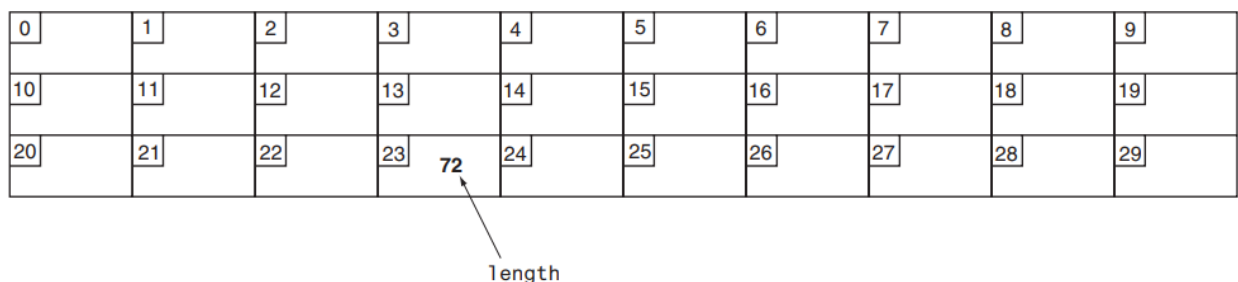
Câu lệnh này khiến máy tính hiển thị thông báo “Bạn đã làm việc bao nhiêu giờ?” trên màn hình. Các câu lệnh có thể là sự kết hợp của các từ khóa, toán tử và ký hiệu do người lập trình xác định. Các câu lệnh thường chỉ chiếm một dòng trong chương trình, nhưng đôi khi chúng nằm rải rác trên nhiều dòng.

Biến

Biến là một vị trí lưu trữ được đặt tên trong bộ nhớ của máy tính để chứa một phần thông tin. Thông tin được lưu trữ trong các biến có thể thay đổi trong khi chương trình đang chạy (do đó có tên là “biến”). Lưu ý trong Chương trình 1.1 các từ *hour*, *rate* và *pay* xuất hiện ở nhiều nơi. Cả ba đều là tên của các biến. Biến *hour* dùng để lưu trữ số giờ người dùng đã làm việc. Biến *rate* lưu trữ mức lương theo giờ của người dùng. Biến *pay* chứa kết quả của *hour* nhân với *rate*, là tổng lương của người dùng.

LƯU Ý: Lưu ý các biến trong Chương trình 1-1 có tên phản ánh mục đích của chúng. Trên thực tế, có thể dễ dàng đoán được các biến được dùng để làm gì chỉ bằng cách đọc tên của chúng. Điều này sẽ được thảo luận thêm trong phần tiếp theo.

Biến là tên tượng trưng đại diện cho các vị trí trong bộ nhớ truy cập ngẫu nhiên (RAM) của máy tính. Khi thông tin được lưu trữ trong một biến, nó thực sự được lưu trữ trong RAM. Giả sử một chương trình có một biến có tên là chiều dài (*length*). Hình 1-10 minh họa cách tên biến thể hiện vị trí bộ nhớ.



Hình 1.9. Tên biến đại diện cho vị trí bộ nhớ

Trong Hình 1.8, biến `length` đang giữ giá trị 72. Số 72 thực sự được lưu trữ trong RAM tại địa chỉ 23, nhưng chiều dài tên tượng trưng cho vị trí lưu trữ này. Có thể coi biến như một hộp chứa thông tin. Trong Hình 1.8, số 72 được lưu trong hộp có tên là `length`. Chỉ một phần tử có thể được lưu trữ trong hộp tại bất kỳ thời điểm nào. Nếu chương trình lưu một giá trị khác vào hộp, nó sẽ thay thế số 72.

Định nghĩa biến

Trong lập trình, có hai loại dữ liệu chung: số và ký tự. Các số được sử dụng để thực hiện các phép toán và ký tự được sử dụng để in dữ liệu trên màn hình hoặc trên giấy.

Dữ liệu số có thể được phân loại hơn nữa. Ví dụ:

Sau đây là tất cả các số nguyên:

5
7
129
32154

Sau đây là số thực hoặc số dấu phẩy động:

3.14159
6.7
1.0002

Khi tạo một biến trong chương trình C++, bạn phải biết loại dữ liệu nào chương trình sẽ lưu trữ trong đó. Nhìn vào dòng sau của Chương trình 1.1:

```
double hours, rate, pay;
```

Từ `double` trong câu lệnh này chỉ ra rằng các biến `hours`, `rate` và `pay` sẽ được sử dụng để chứa các số dấu phẩy động. Câu lệnh này được gọi là định nghĩa biến. Nó được sử dụng để xác định một hoặc nhiều biến sẽ được sử dụng trong chương trình và để chỉ ra loại dữ liệu mà chúng sẽ chứa. Việc định nghĩa biến làm cho các biến được tạo trong bộ nhớ, vì vậy *tất cả các biến phải được định nghĩa trước khi chúng có thể được sử dụng*. Nếu bạn xem lại Chương trình 1-1, bạn sẽ thấy rằng các định nghĩa biến xuất hiện trước bất kỳ câu lệnh nào khác sử dụng các biến đó.

LƯU Ý: Các lập trình viên thường sử dụng thuật ngữ “khai báo biến” có nghĩa tương tự như “định nghĩa biến”. Nói đúng ra, có một sự khác biệt giữa hai việc này. Câu lệnh định nghĩa luôn tạo ra một biến trong bộ nhớ. Tuy nhiên, một số loại câu lệnh khai báo không tạo ra biến trong bộ nhớ.

1.5. Nhập, Xử lý và Xuất

KHÁI NIỆM: Ba hoạt động chính của một chương trình là đầu vào, xử lý và đầu ra.

Các chương trình máy tính thường thực hiện quy trình ba bước để thu thập đầu vào, thực hiện một số quy trình trên thông tin được thu thập, sau đó tạo ra đầu ra. Đầu vào là thông tin một chương trình thu thập từ thế giới bên ngoài. Nó có thể được gửi đến chương trình từ người dùng đang nhập dữ liệu bằng bàn phím hoặc sử dụng chuột. Nó cũng có thể được đọc từ các tập tin hoặc các thiết bị phần cứng được kết nối với máy tính. Chương trình 1-1 cho phép người dùng nhập hai thông tin: số giờ làm việc (hour) và mức lương theo giờ (rate). Dòng 11 và 15 sử dụng đối tượng cin (phát âm là “see in”) để thực hiện các thao tác nhập sau:

```
cin >> hour;  
cin >> rate;
```

Khi thông tin được thu thập từ thế giới bên ngoài, chương trình thường xử lý thông tin đó theo cách nào đó. Trong Chương trình 1-1, số giờ làm việc (hour) và mức lương theo giờ (rate) được nhân với nhau ở dòng 18 và kết quả được gán cho biến lương (pay):

```
pay = hours * rate;
```

Đầu ra là thông tin mà chương trình gửi ra thế giới bên ngoài. Nó có thể là các từ hoặc đồ họa được hiển thị trên màn hình, một báo cáo được gửi tới máy in, dữ liệu được lưu trữ trong một tập tin hoặc thông tin được gửi đến bất kỳ thiết bị nào được kết nối với máy tính. Các dòng 10, 14 và 21 trong Chương trình 1-1 đều thực hiện đầu ra:

```
cout << "Bạn đã làm bao nhiêu giờ? ";  
cout << "Bạn được trả bao nhiêu một giờ? ";  
cout << "Bạn kiếm được số tiền là " << pay << endl;
```

Những dòng này sử dụng đối tượng cout (phát âm là “see out”) để hiển thị thông báo trên màn hình máy tính. Bạn sẽ tìm hiểu thêm chi tiết về các đối tượng cin và cout trong chương tiếp theo.

Kiểm tra

- 1.1.8 Mô tả sự khác biệt giữa từ khóa và mã định danh do người lập trình định nghĩa.
- 1.19 Mô tả sự khác biệt giữa toán tử và ký hiệu dấu chấm câu.
- 1.20 Mô tả sự khác biệt giữa dòng chương trình và câu lệnh.
- 1.21 Tại sao gọi biến là “biến”?
- 1.22 Điều gì xảy ra với nội dung hiện tại của một biến khi một giá trị mới được lưu trữ ở đó?
- 1.23 Điều gì phải làm trong chương trình trước khi sử dụng một biến?
- 1.24 Ba hoạt động chính của một chương trình là gì?

1.6. Quá trình viết chương trình

KHÁI NIỆM: Quá trình lập trình bao gồm một số bước, bao gồm thiết kế, các hoạt động tạo, thử nghiệm và gỡ lỗi.

Thiết kế và tạo một chương trình

Bây giờ bạn đã được giới thiệu chương trình là gì, đã đến lúc xem xét quá trình tạo chương trình. Thông thường, khi những sinh viên chưa có kinh nghiệm được giao bài tập lập trình thì sẽ gặp khó khăn lúc bắt đầu vì không biết phải làm gì trước. Nếu bạn rơi vào tình thế tiến thoái lưỡng nan này, các bước được liệt kê trong Hình 1-11 có thể là gợi ý tốt. Đây là các bước được khuyến nghị cho quá trình viết chương trình.

1. Xác định rõ chương trình sẽ làm gì?
2. Hình dung chương trình đang chạy trên máy tính?
3. Sử dụng các công cụ thiết kế như biểu đồ phân cấp, biểu đồ luồng hoặc giả mã để tạo mô hình của chương trình.
4. Kiểm tra mô hình xem có lỗi logic không?
5. Nhập mã, lưu và biên dịch mã.
6. Sửa các lỗi phát hiện được trong quá trình biên dịch. Lặp lại Bước 5 và 6 nhiều lần nếu cần.
7. Chạy chương trình với dữ liệu thử nghiệm làm đầu vào.
8. Sửa các lỗi phát hiện được khi chạy chương trình. Lặp lại các bước từ 5 đến 8 nhiều lần nếu cần.
9. Xác thực kết quả của chương trình.

Hình 1.10. Các bước viết chương trình

Các bước được liệt kê trong Hình 1.9 nhấn mạnh tầm quan trọng của việc lập kế hoạch. Cũng giống như có những cách tốt và những cách xấu để sơn một ngôi nhà, cũng có những cách tốt và những cách xấu để tạo ra một chương trình. Một chương trình tốt luôn bắt đầu bằng việc lập kế hoạch.

Với chương trình tính lương ở trên làm ví dụ, hãy xem xét từng bước chi tiết hơn.

1. Xác định rõ chương trình sẽ làm gì?

Bước này yêu cầu bạn xác định mục đích của chương trình, thông tin cần nhập, quá trình xử lý sẽ diễn ra và đầu ra mong muốn. Hãy xem xét từng yêu cầu này cho ví dụ:

<i>Mục đích</i>	Để tính tổng lương của người dùng
<i>Đầu vào</i>	Số giờ làm việc, Tiền công trả cho 1 giờ
<i>Xử lý</i>	Nhân số giờ làm việc với tiền công trả cho 1 giờ. Kết quả là tổng tiền lương của người dùng
<i>Đầu ra</i>	Hiển thị thông báo cho biết tổng tiền lương của người dùng

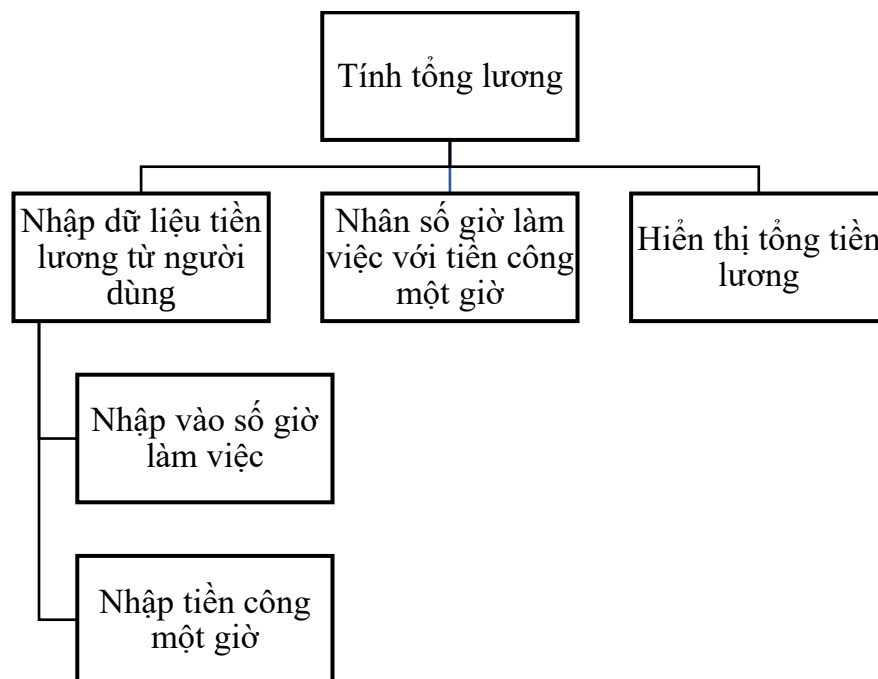
2. Hình dung chương trình đang chạy trên máy tính.

Trước khi tạo một chương trình trên máy tính, trước tiên bạn nên hình dung nó trong đầu. Bước 2 là trực quan hóa chương trình. Hãy thử tưởng tượng màn hình máy tính trông như thế nào khi chương trình đang chạy. Nếu thấy hợp lý, hãy vẽ hình ảnh của màn hình với đầu vào và đầu ra mẫu tại các giá trị khác nhau của chương trình. Ví dụ: đây là màn hình do chương trình tính lương tạo ra:

Bạn đã làm việc bao nhiêu giờ? 10 Bạn được trả bao nhiêu mỗi giờ? 15 Bạn đã kiếm được \$150

Ở bước này, bạn phải đặt mình vào vị trí của người dùng. Chương trình sẽ hiển thị những thông báo gì? Nó nên hỏi những câu hỏi gì? Bằng cách giải quyết những mối lo ngại này, bạn sẽ xác định được hầu hết kết quả đầu ra của chương trình.

3. *Sử dụng các công cụ thiết kế như biểu đồ phân cấp, biểu đồ luồng hoặc giả mã để tạo mô hình của chương trình.*



Hình 1.11. Biểu đồ phân cấp

Khi lập kế hoạch cho một chương trình, người lập trình sử dụng một hoặc nhiều công cụ thiết kế để tạo ra mô hình của chương trình. Ba công cụ thiết kế phổ biến là biểu đồ phân cấp, lưu đồ và mã giả. Biểu đồ phân cấp là một sơ đồ mô tả bằng đồ họa cấu trúc của một chương trình. Nó có các hộp đại diện cho từng bước trong chương trình. Các hộp được kết nối theo cách minh họa mối quan hệ của chúng với nhau. Hình 1.10 thể hiện biểu đồ phân cấp của chương trình tính lương.

Biểu đồ phân cấp bắt đầu bằng nhiệm vụ tổng thể, sau đó tinh chỉnh nó thành các nhiệm vụ phụ nhỏ hơn. Mỗi nhiệm vụ phụ sẽ được tinh chỉnh thành các nhóm nhiệm vụ nhỏ hơn nữa, cho đến khi mỗi nhiệm vụ đủ nhỏ để có thể dễ dàng thực hiện. Ví dụ: trong Hình 1.10,

nhiệm vụ tổng thể “Tính tổng lương” được liệt kê trong hộp cấp cao nhất. Nhiệm vụ đó được chia thành ba nhiệm vụ con. Nhiệm vụ con đầu tiên, “Nhập dữ liệu tiền lương từ người dùng” được chia thành hai nhiệm vụ con. Quá trình “phân chia và giải quyết” này được gọi là thiết kế từ trên xuống.

Giả mã là sự giao thoa giữa ngôn ngữ của con người và ngôn ngữ lập trình. Mặc dù máy tính không thể hiểu giả mã nhưng các lập trình viên thường thấy hữu ích khi viết thuật toán bằng ngôn ngữ “gần giống” ngôn ngữ lập trình nhưng vẫn rất giống với ngôn ngữ tự nhiên. Ví dụ: đây là mã giả mô tả cách tính lương

Nhập dữ liệu tiền lương từ người dùng
Tính tổng lương
Hiển thị tổng lương

Mặc dù giả mã ở trên cung cấp một cái nhìn bao quát về chương trình nhưng nó không tiết lộ tất cả các chi tiết của chương trình. Một phiên bản chi tiết hơn của mã giả sau:

Hiển thị "Bạn đã làm bao nhiêu giờ? "
Nhập số giờ (hour)
Hiển thị ""Bạn được trả bao nhiêu một giờ? "
Nhập số tiền trả cho 1 giờ (rate)
Lưu giá trị số giờ làm việc (hour) nhân với số tiền trả cho một giờ (hour) vào biến tổng lương (pay)
Hiển thị giá trị biến tổng lương (pay)

4. Kiểm tra mô hình xem có lỗi logic không.

Lỗi logic là những lỗi khiến chương trình tạo ra kết quả sai. Khi một biểu đồ phân cấp, lưu đồ hoặc mô hình giả mã của chương trình được lập thì cần kiểm tra các lỗi này. Lập trình viên nên theo dõi các biểu đồ hoặc giả mã, kiểm tra tính logic của từng bước. Nếu tìm thấy lỗi, mô hình có thể được sửa trước khi thử bước tiếp theo.

5. Nhập mã, lưu và biên dịch mã.

Khi mô hình của chương trình (biểu đồ phân cấp, lưu đồ hoặc giả mã) đã được tạo, kiểm tra và sửa thì lập trình viên có thể viết mã nguồn trên máy tính.

Lập trình viên lưu mã nguồn vào một tệp và bắt đầu quá trình dịch nó sang ngôn ngữ máy. Trong bước này, trình biên dịch sẽ tìm bất kỳ lỗi cú pháp nào có thể tồn tại trong chương trình.

6. Sửa các lỗi phát hiện được trong quá trình biên dịch. Lặp lại Bước 5 và 6 nhiều lần đến khi hết lỗi

Nếu trình biên dịch báo cáo bất kỳ lỗi nào thì các lỗi này phải được sửa. Bước 5 và 6 được lặp lại cho đến khi chương trình không còn lỗi biên dịch.

7. Chạy chương trình với dữ liệu thử nghiệm làm đầu vào.

Khi tệp thực thi được tạo, chương trình đã sẵn sàng để kiểm tra lỗi thời gian chạy chương trình (runtime errors). *Lỗi thời gian chạy là lỗi xảy ra khi chương trình đang chạy.* Đây thường là những lỗi logic, chẳng hạn như lỗi toán học.

Việc kiểm tra lỗi thời gian chạy yêu cầu chương trình phải được thực thi với dữ liệu hoặc đầu vào mẫu. Dữ liệu mẫu phải sao cho có thể dự đoán được kết quả đầu ra chính xác. Nếu chương trình không tạo ra đầu ra chính xác thì có lỗi logic trong chương trình.

8. Sửa các lỗi phát hiện được khi chạy chương trình. Lặp lại các bước từ 5 đến 8 nhiều lần nếu cần.

Khi có lỗi thời gian chạy trong chương trình thì lỗi đó phải được sửa. Phải xác định nguyên nhân và bước xảy ra lỗi. Kiểm tra bàn (desk-checking) là một quá trình có thể giúp xác định lỗi thời gian chạy. *Thuật ngữ kiểm tra bàn (desk-checking) là quá trình lập trình viên bắt đầu đọc chương trình hoặc một phần của chương trình và thực hiện từng bước qua từng câu lệnh.* Trong quá trình này thường được ghi lại từng bước ra giấy về nội dung hiện tại của tất cả các biến và hiển thị ra màn hình khi có sự thay đổi. Bằng cách xem qua từng câu lệnh, nhiều lỗi có thể được phát hiện và sửa chữa. Nếu là lỗi logic (chẳng hạn như công thức toán được trình bày không đúng) thì bạn phải sửa lại câu lệnh hoặc các câu liên quan đến logic. Nếu lỗi xảy ra do hiểu chưa đầy đủ về các yêu cầu của chương trình thì bạn phải trình bày lại mục đích của chương trình và sửa đổi lại biểu đồ phân cấp, lưu đồ, giả mã và mã nguồn. Chương trình sau đó phải được lưu, biên dịch lại và kiểm tra lại. Điều này có nghĩa là các Bước 5 đến 8 phải được lặp lại cho đến khi chương trình tạo ra kết quả khả quan một cách đáng tin cậy.

9. Xác thực kết quả của chương trình.

Khi bạn tin rằng mình đã sửa được tất cả các lỗi trong thời gian chạy, hãy nhập dữ liệu kiểm tra và xác định xem chương trình có giải quyết được vấn đề ban đầu hay không.

Kỹ thuật phân mềm là gì?

Lĩnh vực công nghệ phần mềm bao gồm toàn bộ quá trình tạo ra phần mềm máy tính. Nó bao gồm thiết kế, lập trình, thử nghiệm, gỡ lỗi, đặc tả tài liệu, sửa đổi và duy trì dự án phát triển phần mềm. Giống như các kỹ sư truyền thống, kỹ sư phần mềm sử dụng một số công cụ trong công việc của họ.

Hầu hết các ứng dụng phần mềm thương mại đều rất lớn. Trong nhiều trường hợp, các sản phẩm do một hoặc nhiều nhóm lập trình viên phát triển chứ không phải một cá nhân. Điều quan trọng là các yêu cầu của chương trình phải được phân tích kỹ lưỡng và chia thành các nhiệm vụ phụ do từng nhóm hoặc các cá nhân trong nhóm xử lý.

Ở Bước 3 của quá trình lập trình, bạn đã được giới thiệu về biểu đồ phân cấp như một công cụ để thiết kế từ trên xuống. Các nhiệm vụ con được xác định trong thiết kế từ trên xuống có thể dễ dàng trở thành các mô-đun hoặc các thành phần riêng biệt của chương

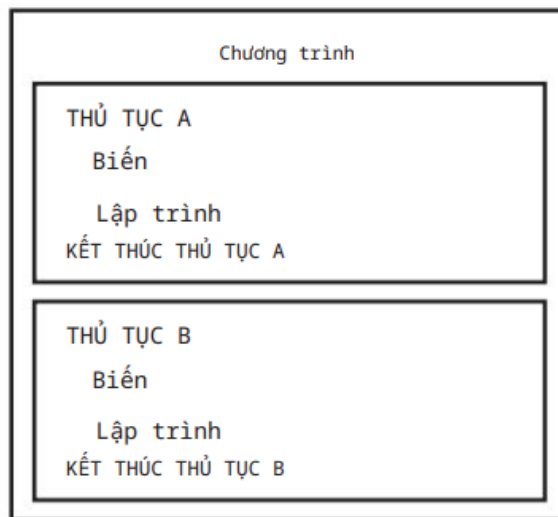
trình. Nếu chương trình rất lớn hoặc phức tạp, một nhóm kỹ sư phần mềm có thể được phân công để làm việc trên từng mô-đun riêng. Khi dự án phát triển, các mô-đun được phối hợp để cuối cùng trở thành một ứng dụng phần mềm duy nhất.

1.7. Lập trình hướng cấu trúc và hướng đối tượng

KHÁI NIỆM: Lập trình thủ tục và lập trình hướng đối tượng là hai cách phương pháp về phát triển phần mềm và thiết kế chương trình.

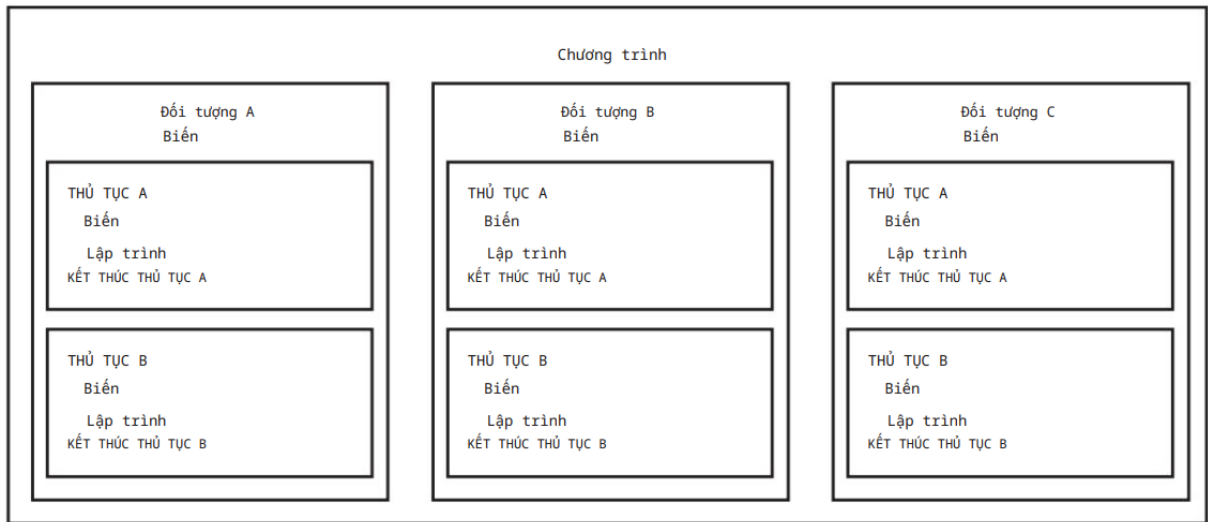
C++ là ngôn ngữ có thể được sử dụng cho hai phương pháp viết chương trình máy tính: lập trình thủ tục và lập trình hướng đối tượng. Cuốn sách này được thiết kế để dạy cho bạn một số điều trong cả hai điều đó.

Trong lập trình thủ tục, người lập trình xây dựng các thủ tục (hoặc hàm, như chúng được gọi trong C++). Các thủ tục là tập hợp các câu lệnh lập trình thực hiện một nhiệm vụ cụ thể. Mỗi thủ tục chứa các biến riêng của chúng và thường chia sẻ các biến với các thủ tục khác. Điều này được minh họa trong Hình 1-13.



Hình 1.12. Các thủ tục trong một chương trình

Lập trình thủ tục tập trung vào thủ tục hoặc hàm. Mặt khác, lập trình hướng đối tượng (OOP) lấy đối tượng làm trung tâm. Đối tượng là một phần tử lập trình chứa dữ liệu và các thủ tục thao tác trên dữ liệu đó. Nó là một đơn vị khép kín. Điều này được minh họa trong Hình 1.12.



Hình 1.13. Các đối tượng trong một chương trình

Các đối tượng chứa đựng bên trong chúng cả thông tin và khả năng thao tác với thông tin. Các hoạt động được thực hiện trên thông tin trong một đối tượng bằng cách gửi cho đối tượng một tin nhắn. Khi một đối tượng nhận được một thông báo hướng dẫn nó thực hiện một số thao tác thì nó sẽ thực hiện hướng dẫn đó.

Kiểm tra

1. Bạn nên xác định bốn mục nào khi xác định nội dung một chương trình?
2. Sơ đồ phân cấp là gì?
3. Mô tả quy trình kiểm tra bàn làm việc.
4. Mô tả trình biên dịch làm gì với mã nguồn của chương trình.
5. Lỗi thời gian chạy là gì?
6. Có phải lỗi cú pháp (chẳng hạn như viết sai chính tả một từ khóa) được trình biên dịch tìm thấy hoặc khi chương trình đang chạy?
7. Mục đích của việc thử nghiệm một chương trình với dữ liệu hoặc đầu vào mẫu là gì?
8. Mô tả ngắn gọn sự khác biệt giữa lập trình hướng thủ tục và lập trình hướng đối tượng?

1.8. Câu hỏi và Bài tập

1.8.1. Câu hỏi

1. Cả bộ nhớ chính và bộ nhớ thứ cấp đều là loại bộ nhớ. Mô tả sự khác biệt giữa 2 loại bộ nhớ này?
2. Sự khác biệt giữa phần mềm hệ thống và phần mềm ứng dụng là gì?
3. Loại phần mềm nào điều khiển hoạt động bên trong của phần cứng máy tính?
4. Tại sao các chương trình viết bằng ngôn ngữ cấp cao phải được dịch sang mạng máy trước khi nó chạy?

5. Tại sao viết chương trình bằng ngôn ngữ cấp cao lại dễ hơn viết chương trình bằng ngôn ngữ máy?

6. Giải thích sự khác biệt giữa tệp đối tượng và tệp thực thi.

7. Sự khác biệt giữa lỗi cú pháp và lỗi logic là gì?

1.8.2. Thuật toán

Vẽ biểu đồ phân cấp hoặc lưu đồ mô tả các chương trình được mô tả phía dưới.

8. Tài khoản

Cần thực hiện các bước sau trong chương trình tính số tiền còn lại trong tài khoản của khách hàng:

1. Hiện thị thông báo “Nhập tổng tiền trong tài khoản của khách hàng”.
2. Đợi người dùng nhập tổng tiền trong tài khoản của khách hàng.
3. Hiện thị thông báo “Nhập số tiền khách hàng thanh toán”.
4. Đợi người dùng nhập số tiền khách hàng thanh toán.
5. Trừ tổng tiền trong tài khoản cho số tiền khách hàng thanh toán được số tiền còn lại trong tài khoản của khách hàng.
6. Hiện thị thông báo số tiền còn lại trong tài khoản của khách hàng.

9. Thuế bán hàng

Thiết kế biểu đồ phân cấp hoặc lưu đồ cho một chương trình tính tổng doanh số bán lẻ.

Chương trình sẽ yêu cầu người dùng nhập:

- Giá bán lẻ của mặt hàng mua
- Thuế bán hàng
- Sau khi nhập xong các mục này, chương trình sẽ tính toán và hiển thị:
- Thuế bán hàng khi mua hàng
- Tổng tiền phải trả

10. Số dư tài khoản

Thiết kế biểu đồ phân cấp hoặc lưu đồ cho một chương trình tính toán số dư hiện tại trong tài khoản tiết kiệm. Chương trình phải yêu cầu người dùng nhập:

- Số dư ban đầu
- Tổng số tiền gửi
- Tổng tiền rút
- Lãi suất ngân hàng

Sau khi chương trình tính toán và hiển thị trên màn hình số dư hiện tại.

Dự đoán kết quả

Các câu sau là các chương trình được diễn đạt dưới dạng câu phát biểu bằng lời. Mỗi chương trình sẽ hiển thị gì trên màn hình nếu chúng là các chương trình thực tế?

11. Biến x bắt đầu bằng giá trị 0.
Biến y bắt đầu bằng giá trị 5.
Thêm 1 vào x.
Thêm 1 vào y.
Thêm x vào y và lưu kết quả vào y.
Hiển thị giá trị y lên màn hình.
12. Biến x bắt đầu bằng giá trị 20.
Biến y bắt đầu bằng giá trị 10.
Lưu giá trị của x vào biến t.
Lưu giá trị của y vào x.
Lưu trữ giá trị của t trong y.
Hiển thị giá trị của x lên màn hình.
Hiển thị giá trị của y lên màn hình.
13. Biến a bắt đầu bằng giá trị 1.
Biến b bắt đầu bằng giá trị 10.
Biến c bắt đầu bằng giá trị 100.
Biến x bắt đầu bằng giá trị 0.
Lưu trữ giá trị của c nhân với 3 vào x.
Cộng giá trị của b nhân với 6 vào giá trị đã có trong x.
Thêm giá trị của a nhân 5 vào giá trị đã có trong x.
Hiển thị giá trị x lên màn hình.

Tìm lỗi

14. Thuật toán giả mã sau đây có lỗi. Chương trình có nhiệm vụ hỏi người dùng về chiều dài và chiều rộng của một căn phòng hình chữ nhật, sau đó hiển thị diện tích của căn phòng. Chương trình phải nhân chiều rộng với chiều dài để xác định diện tích.

Hãy tìm lỗi của đoạn giả mã sau:

diện tích = chiều rộng x chiều dài.
Hiển thị "Chiều rộng của căn phòng là bao nhiêu?".
Nhập vào chiều rộng
Hiển thị "Chiều dài của căn phòng là bao nhiêu?".
Nhập vào chiều dài
Hiển thị diện tích căn phòng.

CHƯƠNG 2. GIỚI THIỆU VỀ NGÔN NGỮ LẬP TRÌNH C++

Chủ đề	
2.1	Các phần của chương trình C++
2.2	Đối tượng cout
2.3	Chỉ thị #include
2.4	Biến, giá trị và phép gán
2.5	Định danh
2.6	Kiểu dữ liệu số nguyên
2.7	Kiểu dữ liệu char
2.8	Lớp chuỗi C++
2.9	Kiểu dữ liệu dấu phẩy động
2.10	Kiểu dữ liệu bool
2.11	Xác định kích thước của kiểu dữ liệu
2.12	Thông tin thêm về Gán và khởi tạo biến
2.13	Phạm vi
2.14	Toán tử số học
2.15	Bình luận
2.16	Các hằng số được đặt tên
2.17	Phong cách lập trình

2.1. Các thành phần của chương trình C++

KHÁI NIỆM: Trong chương trình C++ có các phần và thành phần phục vụ các mục đích khác nhau.

Hãy kiểm tra từng dòng Chương trình 2-1. Đây là dòng đầu tiên:

// Một chương trình C++ đơn giản

// đánh dấu sự bắt đầu của một lời chú thích. Trình biên dịch bỏ qua mọi thứ từ dấu gạch chéo kép đến cuối dòng. Điều đó có nghĩa là bạn có thể gõ bất cứ thứ gì bạn muốn trên dòng đó và trình biên dịch sẽ không quan tâm. Mặc dù không bắt buộc phải có chú thích nhưng chúng rất quan trọng đối với các lập trình viên. Hầu hết các chương trình đều phức tạp hơn nhiều so với ví dụ trong Chương trình 2-1 và các chú thích sẽ giúp giải thích những gì đang diễn ra.

Chương trình 2.1. Một chương trình đơn giản

```
// Một chương trình C++ đơn giản
#include <iostream>
using namespace std;
int main()
{
    cout << " Lập trình rất thú vị! ";
    return 0;
}
```

Kết quả chương trình:

Lập trình rất thú vị!

Dòng :

```
#include <iostream>
```

Vì dòng này bắt đầu bằng # nên nó được gọi là chỉ thị tiền xử lý. Bộ tiền xử lý đọc chương trình của bạn trước khi nó được biên dịch và chỉ thực thi những dòng bắt đầu bằng ký hiệu #. Hãy coi bộ tiền xử lý như một chương trình “thiết lập” mã nguồn của bạn cho trình biên dịch.

Khi gặp lệnh #include thì bộ tiền xử lý đưa nội dung của một tệp khác, được gọi là tệp tiêu đề (header file) vào chương trình. Nó được gọi là tệp tiêu đề vì nó phải được đặt ở đầu hoặc trên cùng của chương trình. Từ được đặt trong ngoặc là iostream, là tên của tệp tiêu đề sẽ được đưa vào. (Tên của tệp <iostream> cho biết đó là tệp tiêu đề C++ tiêu chuẩn.) Tệp tiêu đề <iostream> chứa mã cho phép chương trình C++ hiển thị đầu ra trên màn hình và đọc đầu vào từ bàn phím. Vì chương trình này sử dụng cout để hiển thị kết quả màn hình nên tập tin tiêu đề <iostream> phải có trong chương trình. Nội dung của tệp <iostream> được đưa vào chương trình tại thời điểm câu lệnh #include xuất hiện.

Dòng :

```
using namespace std;
```

Các chương trình thường chứa các phân tử có tên duy nhất. Trong chương này, bạn sẽ học cách định nghĩa biến. Trong chương trình thì biến, hàm và đối tượng phải có tên. C++ sử dụng các không gian tên (namespace) để sắp xếp các tên các thực thể trong chương trình. Câu lệnh `using namespace std;` để cho biết rằng chương trình sẽ truy cập các thực thể có tên là một phần của không gian tên được gọi là std. (ngay cả các không gian tên cũng có tên.) Lý do chương trình cần quyền truy cập vào không gian tên std là vì mọi tên được tạo bởi tệp iostream đều là một phần của không gian tên đó. Để một chương trình sử dụng các thực thể trong iostream, nó phải có quyền truy cập vào không gian tên std.

Dòng :

```
int main()
```

Điều này đánh dấu sự bắt đầu của một hàm. *Một hàm có thể được coi là một nhóm gồm một hoặc nhiều câu lệnh lập trình có chung một tên.* Tên của hàm này là main và hai dấu ngoặc đơn theo sau tên cho biết đó là một hàm. Từ int là viết tắt của “số nguyên”. Nó chỉ ra rằng hàm sẽ trả về một giá trị nguyên khi nó thực thi xong.

Mặc dù hầu hết các chương trình C++ có nhiều hơn một hàm, *nhưng mọi chương trình C++ đều phải có một hàm gọi là main.* Đó là điểm khởi đầu của chương trình. Nếu bạn đọc chương trình C++ và muốn tìm xem nó bắt đầu từ đâu thì chỉ cần tìm hàm có tên main.

LƯU Ý:

- *C++ là ngôn ngữ lập trình phân biệt chữ hoa chữ thường.* Điều đó có nghĩa là nó coi các chữ cái viết hoa là các ký tự hoàn toàn khác so với các chữ cái viết thường.
- Trong C++, tên hàm main phải được viết bằng tất cả các chữ cái viết thường. C++ không coi “Main” giống với “main” hoặc “INT” giống với “int”. Điều này đúng với tất cả các từ khóa C++.

Dòng chứa một ký tự ngoặc nhọn:

```
{
```

Đây được gọi là dấu ngoặc nhọn trái hoặc dấu ngoặc nhọn mở và nó được liên kết với phần đầu của hàm main. Tất cả các câu lệnh tạo nên một hàm đều được đặt trong một bộ dấu ngoặc nhọn.

Mọi thứ nằm giữa hai dấu ngoặc nhọn đều là nội dung của hàm main.

CẢNH BÁO! Đảm bảo rằng bạn có dấu ngoặc đóng cho mỗi dấu ngoặc mở trong chương trình của mình!

Sau dấu ngoặc nhọn mở, bạn sẽ thấy câu lệnh sau ở dòng 7:

```
cout << "Lập trình rất thú vị!";
```

Nói một cách đơn giản, dòng này hiển thị một thông báo trên màn hình. Bạn sẽ đọc thêm về cout và toán tử << ở phần sau của chương này. Thông báo “Lập trình rất thú vị!” được in không có dấu ngoặc kép. Trong thuật ngữ lập trình, nhóm ký tự bên trong dấu ngoặc kép được gọi là chuỗi ký tự hoặc hằng chuỗi.

LƯU Ý: Đây là dòng duy nhất trong chương trình khiến mọi thứ được in trên màn hình. Các dòng khác, như `#include <iostream>` và `int main()`, là cần phải có của một chương trình, nhưng chúng không thực hiện đưa kết quả ra màn hình. Hãy nhớ rằng, chương trình là một tập hợp các hướng dẫn cho máy tính. Nếu muốn hiển thị thứ gì đó trên màn hình, bạn phải sử dụng câu lệnh lập trình cho việc đó.

Cuối dòng là dấu chấm phẩy. Giống như dấu chấm đánh dấu sự kết thúc của một câu, *dấu chấm phẩy đánh dấu sự kết thúc của một câu lệnh hoàn chỉnh trong C++*. Các chú thích sẽ được trình biên dịch bỏ qua, vì vậy dấu chấm phẩy không bắt buộc ở cuối chú thích. Các chỉ thị tiền xử lý, như câu lệnh `#include`, chỉ cần kết thúc ở cuối dòng và không bao giờ yêu cầu dấu chấm phẩy. Phần đầu của hàm, như `int main()`, không phải là một câu lệnh hoàn chỉnh, do đó bạn cũng không đặt dấu chấm phẩy ở đó.

Có vẻ như các quy tắc về vị trí đặt dấu chấm phẩy không rõ ràng chút nào. Thay vì lo lắng về điều đó bây giờ, hãy tập trung vào việc học các phần của chương trình. Bạn sẽ sớm cảm nhận được nơi nào nên và không nên sử dụng dấu chấm phẩy.

Dòng:

```
return 0;
```

Điều này sẽ gửi giá trị số nguyên 0 trở lại hệ điều hành sau khi hoàn thành chương trình. Giá trị 0 thường chỉ ra rằng chương trình được thực hiện thành công.

Dòng chứa dấu ngoặc nhọn đóng:

```
}
```

Dấu } này đánh dấu sự kết thúc của hàm main. Vì hàm main là hàm duy nhất trong chương trình này nên nó cũng đánh dấu sự kết thúc của chương trình.

Trong chương trình mẫu, bạn gặp phải một số bộ ký tự đặc biệt. Bảng 2-1 cung cấp một bản tóm tắt ngắn gọn về cách chúng được sử dụng.

Bảng 2.1. Ký tự đặc biệt

Kí tự	Tên	Mô tả
//	Dấu gạch chéo kép	Đánh dấu sự bắt đầu của một giải thích
#	Dấu thăng	Đánh dấu sự bắt đầu của một lệnh tiền xử lý.
< >	Dấu ngoặc mở và đóng	Kèm theo tên tệp khi được sử dụng với lệnh #include.
()	Dấu ngoặc đơn mở và đóng	Được sử dụng để đặt tên hàm, như trong int main().
{ }	Dấu ngoặc nhọn mở và đóng	Bao gồm một nhóm câu lệnh, chẳng hạn như nội dung của một hàm.
" "	Dấu ngoặc kép mở và đóng	Kèm theo một chuỗi ký tự, chẳng hạn như thông báo sẽ được in trên màn hình.
;	Dấu chấm phẩy	Đánh dấu sự kết thúc của một câu lệnh.

Kiểm tra

2.1 Chương trình C++ sau đây sẽ không biên dịch được vì các dòng đã bị trộn lẫn.

```
int main()
{
    // Một chương trình hỗn hợp điên rồ
    retur #include <iostream>
    cout << " Năm 1492 Columbus giương buồm vượt đại dương xanh.";
{
    using namespace std;
```

Khi các dòng được sắp xếp hợp lý, chương trình Sẽ hiển thị thông tin Sau trên màn hình:

Năm 1492 Columbus giương buồm vượt đại dương xanh.

Sắp xếp lại các dòng theo đúng thứ tự. Kiểm tra chương trình bằng cách nhập vào máy tính, biên dịch và chạy thử.

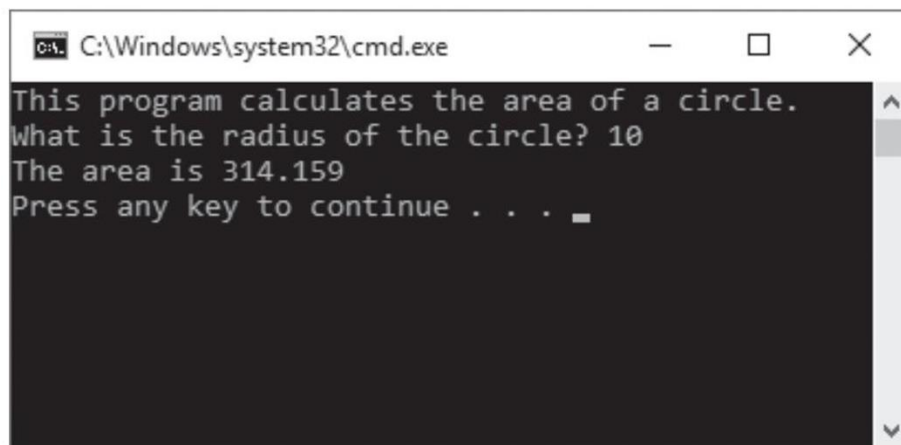
2.2. Đối tượng cout

KHÁI NIỆM: Sử dụng đối tượng cout để hiển thị thông tin trên màn hình máy tính.

Trong phần này, bạn Sẽ học cách viết chương trình xuất ra màn hình. Loại đầu ra màn hình đơn giản nhất mà chương trình có thể hiển thị là đầu ra bảng điều khiển, chỉ là văn bản thuần túy. Bảng điều khiển từ là một thuật ngữ máy tính cũ. Nó xuất hiện từ thời mà người vận hành máy tính tương tác với hệ thống bằng cách gõ trên thiết bị đầu cuối. Thiết bị đầu cuối bao gồm một màn hình và bàn phím đơn giản, được gọi là bảng điều khiển.

Trên các máy tính hiện đại, chạy hệ điều hành đồ họa như Windows hoặc Mac OS X, đầu ra của bàn điều khiển thường được hiển thị trong một cửa Sổ như cửa Sổ được hiển thị trong Hình 2.1. Trong C++, bạn Sử dụng đối tượng cout để tạo đầu ra của bàn điều khiển. (Bạn có thể nghĩ đến từ cout có nghĩa là đầu ra của bàn điều khiển.)

cout được phân loại là đối tượng luồng, nghĩa là nó hoạt động với các luồng dữ liệu. Để in thông báo lên màn hình, bạn gửi một dòng ký tự tới cout.



Hình 2.1. Cửa sổ giao diện điều khiển

Lưu ý toán tử << được Sử dụng để gửi chuỗi “Lập trình rất thú vị!” để cout.

Khi ký hiệu << được Sử dụng theo cách này, nó được gọi là toán tử chèn luồng. Mục ngay bên phải người vận hành được gửi tới cout Sau đó hiển thị trên màn hình.

Toán tử chèn luồng luôn được viết dưới dạng hai dấu nhỏ hơn không có khoảng cách giữa chúng. Bởi vì bạn đang Sử dụng nó để gửi một luồng dữ liệu đến đối tượng cout, nên bạn có thể coi toán tử chèn luồng như một mũi tên phải trở về phía cout.

Chương trình 2.2 là một cách khác để viết chương trình tương tự.

Chương trình 2.2. Cách khác để viết chương trình 2.1

```
// Một chương trình C++ đơn giản
#include <iostream>
using namespace std;

int main()
{
    cout << "Lập trình " << "rất thú vị! ";
    return 0;
}
```

Kết quả chương trình:

Lập trình rất thú vị!

Như bạn có thể thấy, toán tử chèn luồng có thể được Sử dụng để gửi nhiều mục tới cout. Đầu ra của chương trình này giống hệt với Chương trình 2-1. Chương trình 2-3 chỉ ra một cách khác để đạt được điều tương tự.

Chương trình 2-3 chỉ ra một cách khác để đạt được điều tương tự.

Chương trình 2.3. Cách viết khác về chương trình 2.1

```
// Một chương trình C++ đơn giản
#include <iostream>
using namespace std;

int main()
{
    cout << "Lập trình ";
    cout<<"rất thú vị! ";
    return 0;
}
```

Kết quả chương trình:

Lập trình rất thú vị!

2.3. Chỉ dẫn #include

Trong ngôn ngữ lập trình C++, #include là một chỉ thị tiền xử lý (preprocessor directive) được sử dụng để bao gồm các tệp header hoặc thư viện vào chương trình của bạn. Chỉ thị

này cho phép bạn sử dụng các định nghĩa, hàm, và cấu trúc dữ liệu được khai báo trong các tệp header đó.

Khi bạn sử dụng một thư viện hoặc tệp header, bạn cần sử dụng chỉ thị `#include` để chương trình hiểu rằng có bao gồm chúng vào chương trình. Điều này giúp trình biên dịch hiểu được các định nghĩa và hàm được khai báo trong các tệp header đó.

```
#include <iostream> // Bao gồm thư viện iostream, cho phép sử dụng nhập/xuất dữ liệu
#include "my_header.h" // Bao gồm tệp header có tên my_header.h, nơi chứa các định nghĩa tùy chỉnh
```

2.4. Biến, giá trị, phép gán và khởi tạo giá trị cho biến

2.4.1. Biến

KHÁI NIỆM: Biến là một vùng nhớ được sử dụng để lưu trữ dữ liệu. Mỗi biến có một tên duy nhất, một kiểu dữ liệu và một địa chỉ ô nhớ trong bộ nhớ máy tính. sử dụng biến để lưu trữ và thao tác các giá trị trong chương trình.

Ví dụ, khai báo một biến `age` kiểu `int` (integer) và gán cho nó giá trị 25:

```
int tuoi = 25;
```

Ở đây, `int` là kiểu dữ liệu của biến, `tuoi` là tên của biến, và 25 là giá trị được gán cho biến.

Sau khi khai báo và gán giá trị cho biến, có thể sử dụng biến này trong các phép tính toán hoặc truy xuất giá trị của nó:

```
cout << "Tuoi cua ban la: " << tuoi << endl; // Output: Tuoi cua ban la: 25
tuoi = tuoi + 5; // Tăng giá trị biến age lên 5 năm
cout << "Tuoi moi cua ban la: " << tuoi << endl; // Output: Tuoi moi cua ban la: 30
```

Chú ý:

- *Quy tắc đặt tên biến:* Tên biến phải bắt đầu bằng một chữ cái hoặc dấu gạch dưới (`'_'`), và chỉ có thể chứa các chữ cái, chữ số và dấu gạch dưới. Tên biến phân biệt chữ hoa/chữ thường.
- *Phạm vi sử dụng biến:* Biến được khai báo trong một khối lệnh (ví dụ: trong hàm) chỉ có thể sử dụng trong phạm vi của khối lệnh đó.
- *Khởi tạo giá trị ban đầu:* Khi khai báo một biến, bạn nên khởi tạo giá trị ban đầu cho biến. Nếu không, biến sẽ có một giá trị ngẫu nhiên trong bộ nhớ.
- *Kiểu dữ liệu của biến:* Mỗi biến có một kiểu dữ liệu xác định, như `int`, `float`, `char`, `bool`,... Kiểu dữ liệu này quyết định kích thước, định dạng và phạm vi giá trị của biến.

2.4.2. Giá trị (Literals)

KHÁI NIỆM: Trong lập trình C++, giá trị (Literals) là những giá trị cụ thể được sử dụng trong chương trình, như số nguyên, số thực, ký tự, chuỗi ký tự, v.v. Chúng là những giá trị cố định được sử dụng trực tiếp trong mã nguồn.

Ví dụ:

```
int x = 42; // Số nguyên
float pi = 3.14159; // Số thực
double d = 2.5E6; // Số thực dạng số mũ (2.5 x 106)
char c = 'A'; // Ký tự được đặt trong cặp dấu nháy đơn (')
string s = "Hello, C++!"; // Chuỗi ký tự được đặt trong cặp dấu nháy kép
```

Chú ý:

- Số nguyên: Các số nguyên có thể được biểu diễn ở các hệ cơ số khác nhau, như hệ thập phân (10), hệ nhị phân (2), hệ bát phân (8) hoặc hệ thập lục phân (16).
- Số thực: Số thực có thể được biểu diễn ở dạng số mũ, như 2.5E6 (=2.5 x 10⁶).
- Ký tự: Ngoài các ký tự in thường, cũng có thể sử dụng các ký tự đặc biệt như '\n' (xuống dòng), '\t' (tab), '\"' (dấu nháy kép), v.v.
- Chuỗi ký tự: Các chuỗi ký tự có thể được nối với nhau bằng cách sử dụng dấu '+’.

2.4.3. Phép gán

Khái niệm: phép gán (assignment) là thao tác gán một giá trị cho một biến. Nó được thể hiện bằng toán tử gán '='. Sau khi thực hiện phép gán, giá trị cũ của biến sẽ bị thay thế bằng giá trị mới.

Ví dụ, có thể gán giá trị 10 cho biến x:

```
int x;
x = 10; // Gán giá trị 10 cho biến x
Hoặc gán giá trị của biến y cho biến z:
int y = 20;
int z;
z = y; // Gán giá trị của y (20) cho z
```

Cũng có thể kết hợp phép gán với các phép toán khác:

```
int a = 5;
a += 3; // Tương đương với a = a + 3 (a = 8)
a -= 2; // Tương đương với a = a - 2 (a = 6)
a *= 4; // Tương đương với a = a * 4 (a = 24)
a /= 6; // Tương đương với a = a / 6 (a = 4)
```

Chú ý:

- Kiểu dữ liệu: Khi gán giá trị cho một biến, giá trị được gán phải tương thích với kiểu dữ liệu của biến. Nếu không, sẽ xảy ra lỗi.
- Phép gán chuỗi: Với biến kiểu string, có thể gán trực tiếp chuỗi ký tự cho biến:

```
string name = "John Doe";
```

- Phép gán liên tiếp: có thể thực hiện nhiều phép gán liên tiếp:

```
int x = y = z = 10; // Gán giá trị 10 cho x, y và z
```

- Toán tử gán kép: Các toán tử gán kép như ‘+=’, ‘-=’, ‘*=’, ‘/=’ và ‘%=’ rất hữu ích khi thực hiện các phép toán và gán giá trị cùng lúc.

2.4.4. Khởi tạo giá trị cho biến

Khái niệm: Khi khai báo một biến trong C++, có thể khởi tạo giá trị ban đầu cho biến đó. Việc khởi tạo giá trị cho biến rất quan trọng vì nó giúp tránh các lỗi do sử dụng biến trước khi gán giá trị cho chúng.

Ví dụ, có thể khai báo và khởi tạo giá trị cho biến ‘x’ như sau:

```
int x = 10; // Khởi tạo biến x với giá trị 10
```

Hoặc khởi tạo nhiều biến cùng lúc:

```
int a = 5, b = 7, c = 9; // Khởi tạo 3 biến a, b, c với các giá trị tương ứng
```

Cũng có thể khởi tạo biến bằng cách gán cho nó giá trị của một biến khác:

```
int y = x; // Khởi tạo biến y bằng giá trị của biến x
```

Ngoài ra, còn có các cách khởi tạo biến khác như:

```
int z{20}; // Khởi tạo biến z với giá trị 20 (C++11)  
int w(15); // Khởi tạo biến w với giá trị 15 (C++11)
```

Chú ý:

- Kiểu dữ liệu: Khi khởi tạo giá trị cho biến, giá trị phải tương thích với kiểu dữ liệu của biến.
- Gán giá trị mặc định: Nếu không khởi tạo giá trị cho biến, biến sẽ được gán giá trị mặc định tùy thuộc vào kiểu dữ liệu (0 cho số nguyên, 0.0 cho số thực, false cho bool, v.v.).
- Khởi tạo biến kiểu ‘string’: Với biến kiểu ‘string’, có thể khởi tạo bằng cách gán trực tiếp một chuỗi ký tự.
- Khởi tạo biến kiểu ‘const’: Với biến ‘const’ (hằng số), phải khởi tạo giá trị ngay khi khai báo, vì giá trị của chúng không thể thay đổi sau này.

Kiểm tra

1. Giải thích khái niệm "biến" trong lập trình C++. Biến có những thành phần chính nào?
2. Hãy viết ví dụ khai báo và gán giá trị cho một biến số nguyên và một biến số thực trong C++.
3. Những quy tắc đặt tên biến trong C++ là gì? Tại sao việc đặt tên biến quan trọng?
4. Giải thích sự khác nhau giữa phép gán ‘=’ và phép gán kép như ‘+=’, ‘-=’, ‘*=’, ‘/=’. Hãy cho ví dụ minh họa.

5. Khi khai báo một biến, tại sao nên khởi tạo giá trị ban đầu cho nó? Hãy cho ví dụ về cách khởi tạo giá trị cho biến.
6. Giải thích sự khác nhau giữa cách khởi tạo biến bằng dấu '=' và dấu '{}' trong C++.
7. Khi khai báo một biến 'const', phải làm gì để đảm bảo giá trị của nó không thể thay đổi?
8. Giả sử có 3 biến 'x', 'y' và 'z' với các kiểu dữ liệu tương ứng là 'int', 'float' và 'string'. Hãy viết đoạn code để khai báo và khởi tạo giá trị cho chúng.
9. Giải thích ý nghĩa của đoạn code sau:

```
int a = 10, b = 20, c;  
c = a++;
```

10. Trong C++, khi gán giá trị của một biến cho một biến khác, điều gì sẽ xảy ra nếu kiểu dữ liệu của hai biến không tương thích?

2.5. Mã định danh

KHÁI NIỆM: Mã định danh (Identifiers) trong C++ là những tên được dùng để đặt cho các thực thể trong chương trình, như biến, hàm, lớp, v.v. Mã định danh giúp dễ dàng nhận biết và sử dụng các thành phần này trong quá trình lập trình.

Trong C++, có một số từ khóa (keywords) đã được định nghĩa sẵn và không thể dùng làm mã định danh. Một số ví dụ về từ khóa trong C++ bao gồm:

Bảng 2.1. Bảng từ khóa trong C++

<i>alignas</i>	<i>const</i>	<i>for</i>	<i>private</i>	<i>throw</i>
<i>alignof</i>	<i>constexpr</i>	<i>friend</i>	<i>protected</i>	<i>true</i>
<i>and</i>	<i>const_cast</i>	<i>goto</i>	<i>public</i>	<i>try</i>
<i>and_eq</i>	<i>continue</i>	<i>if</i>	<i>register</i>	<i>typedef</i>
<i>asm</i>	<i>decltype</i>	<i>inline</i>	<i>reinterpret_cast</i>	<i>typeid</i>
<i>auto</i>	<i>default</i>	<i>int</i>	<i>return</i>	<i>typename</i>
<i>bitand</i>	<i>delete</i>	<i>long</i>	<i>short</i>	<i>union</i>
<i>bitor</i>	<i>do</i>	<i>mutable</i>	<i>signed</i>	<i>unsigned</i>
<i>bool</i>	<i>double</i>	<i>namespace</i>	<i>sizeof</i>	<i>using</i>
<i>break</i>	<i>dynamic_cast</i>	<i>new</i>	<i>static</i>	<i>virtual</i>
<i>case</i>	<i>else</i>	<i>noexcept</i>	<i>static_assert</i>	<i>void</i>
<i>catch</i>	<i>enum</i>	<i>not</i>	<i>static_cast</i>	<i>volatile</i>
<i>char</i>	<i>explicit</i>	<i>not_eq</i>	<i>struct</i>	<i>wchar_t</i>
<i>char16_t</i>	<i>export</i>	<i>nullptr</i>	<i>switch</i>	<i>while</i>
<i>char32_t</i>	<i>extern</i>	<i>operator</i>	<i>template</i>	<i>xor</i>
<i>class</i>	<i>false</i>	<i>or</i>	<i>this</i>	<i>xor_eq</i>
<i>compl</i>	<i>float</i>	<i>or_eq</i>	<i>thread_local</i>	

Không thể dùng những từ khóa này để đặt tên cho biến, hàm, lớp, v.v.

Chú ý:

Khi đặt tên cho mã định danh, cần tuân theo một số quy tắc sau:

- Chỉ dùng chữ cái (hoa hoặc thường), chữ số và dấu gạch dưới (_):
 - Không được dùng các ký tự đặc biệt như '@', '#', '\$', v.v.
 - Không được bắt đầu bằng chữ số.
- Không được trùng với các từ khóa của C++:
 - Như đã nêu ở trên, không thể dùng các từ khóa của C++ làm mã định danh.
- Phân biệt chữ hoa và chữ thường:
 - C++ phân biệt chữ hoa và chữ thường, nghĩa là 'myVariable' và 'myvariable' là hai mã định danh khác nhau.
- Độ dài tối đa của mã định danh:
 - Theo tiêu chuẩn, độ dài tối đa của mã định danh là 255 ký tự.
- Đặt tên mã định danh có ý nghĩa:
 - Nên đặt tên mã định danh theo cách có ý nghĩa, giúp dễ hiểu và duy trì code.
 - Ví dụ: 'totalScore' thay vì 'x', 'calculateAverage()' thay vì 'f()'.
- Sử dụng các quy ước đặt tên:
 - Có nhiều quy ước đặt tên được sử dụng phổ biến, như:
 - Camel case: 'myVariableName'
 - Snake case: 'my_variable_name'
 - Pascal case: 'MyVariableName'

Tuân thủ các quy tắc trên sẽ giúp bạn đặt tên mã định danh một cách hiệu quả, tạo code dễ đọc, dễ hiểu và dễ bảo trì.

2.6. Kiểu dữ liệu: nguyên, kí tự, thực, bool, chuỗi

2.6.1. Các kiểu dữ liệu số nguyên

KHÁI NIỆM: Trong C++, các kiểu dữ liệu kiểu nguyên (Integer Types) được sử dụng để biểu diễn các số nguyên. Các kiểu dữ liệu này có kích thước và phạm vi giá trị khác nhau, tùy thuộc vào việc chúng được khai báo như thế nào.

Các kiểu dữ liệu kiểu nguyên bao gồm:

Kiểu dữ liệu	Kích thước	Giá trị
short int	2 bytes	232,768 to 132,767
unsigned short int	2 bytes	0 to +65,535
int	4 bytes	22,147,483,648 to 12,147,483,647
unsigned int	4 bytes	0 to 4,294,967,295
long int	4 bytes	22,147,483,648 to 12,147,483,647
unsigned long int	4 bytes	0 to 4,294,967,295

long long int	8 bytes	29,223,372,036,854,775,808 to 9,223,372,036,854,775,807
unsigned long long int	8 bytes	0 to 18,446,744,073,709,551,615

Như đã đề cập trước đó, các biến được xác định bằng cách nêu từ khóa kiểu dữ liệu theo sau là tên của biến. Trong Chương trình 2.4, một số nguyên, một số nguyên không dấu và một số nguyên dài đã được xác định.

Chương trình 2.4. Chương trình này có các biến thuộc một số kiểu số nguyên.

```
#include <iostream>
using namespace std;
int main() {
    int a = 42;
    short b = 12345;
    long c = 123456789;
    long long d = 9223372036854775807LL;

    cout << "a (int): " << a << endl;
    cout << "b (short): " << b << endl;
    cout << "c (long): " << c << endl;
    cout << "d (long long): " << d << endl;

    unsigned int e = 4294967295;
    unsigned short f = 65535;
    unsigned long g = 18446744073709551615UL;
    unsigned long long h = 18446744073709551615ULL;
    cout << "e (unsigned int): " << e << endl;
    cout << "f (unsigned short): " << f << endl;
    cout << "g (unsigned long): " << g << endl;
    cout << "h (unsigned long long): " << h << endl;
    return 0;
}
```

Kết quả chương trình:

```
a (int): 42
b (short): 12345
c (long): 123456789
d (long long): 9223372036854775807
e (unsigned int): 4294967295
```

```
f (unsigned short): 65535
g (unsigned long): 18446744073709551615
h (unsigned long long): 18446744073709551615
```

Trong ví dụ sau, thực hiện các phép toán cộng, nhân, trừ và chia trên các kiểu dữ liệu kiểu nguyên khác nhau ('int', 'short', 'long', 'unsigned int'). Kết quả của các phép toán được in ra màn hình.

Chương trình 2.5. Chương trình tính tổng, hiệu, tích, thương

```
#include <iostream>
using namespace std;
int main() {
    int x = 10;
    short y = 5;
    long z = 100;
    int sum = x + y;
    long product = x * z;
    short difference = y - x;
    unsigned int quotient = z / x;
    cout << "Tổng: " << sum << endl;
    cout << "Tích: " << product << endl;
    cout << "Hiệu: " << difference << endl;
    cout << "Thương: " << quotient << endl;
    return 0;
}
```

Kết quả chương trình:

```
Tổng: 15
Tích: 1000
Hiệu: -5
Thương: 10
```

2.6.2. Kiểu dữ liệu số thực

Khái niệm

Trong C++, kiểu số thực (floating-point) được sử dụng để biểu diễn các số có thể có các chữ số thập phân. Chúng được sử dụng để lưu trữ và thao tác với các giá trị số có độ chính

xác cao hơn so với các kiểu số nguyên. Kiểu số thực được chia thành hai loại chính: ‘float’ và ‘double’.

Các kiểu dữ liệu số thực

float:

- Kiểu dữ liệu ‘float’ sử dụng 4 byte (32 bit) để lưu trữ một số thực.
- Phạm vi giá trị của ‘float’ là từ khoảng $3.4E-38$ đến $3.4E+38$.
- Độ chính xác của ‘float’ là 7 chữ số thập phân.

double:

- Kiểu dữ liệu ‘double’ sử dụng 8 byte (64 bit) để lưu trữ một số thực.
- Phạm vi giá trị của ‘double’ là từ khoảng $1.7E-308$ đến $1.7E+308$.
- Độ chính xác của ‘double’ là 15 chữ số thập phân.

Ví dụ 1: Tính diện tích hình tròn

Chương trình 2.6. Tính diện tích hình tròn

```
#include <iostream>
using namespace std;
int main() {
    float radius, area;
    const float PI = 3.14159;
    cout << "Nhap ban kinh duong tron: ";
    cin >> radius;
    area = PI * radius * radius;
    cout << "Dien tich cua duong tron la: " << area << " don vi vuong" <<
endl;
    return 0;
}
```

Kết quả chương trình:

Nhap ban kinh duong tron: 5
Dien tich cua duong tron la: 78.5397 don vi vuong

Trong ví dụ 1, sử dụng kiểu float để lưu trữ bán kính và diện tích của hình tròn. Sau khi người dùng nhập vào bán kính, tính toán diện tích bằng công thức ‘ $area = PI * radius * radius$ ’ và in kết quả ra màn hình.

Ví dụ 2: Tính khoảng cách giữa hai điểm

Chương trình 2.7. Tính khoảng cách giữa hai điểm

```
#include <iostream>
#include <cmath>
using namespace std;
int main() {
    double x1, y1, x2, y2, distance;
    cout << "Nhap toa do diem dau tien (x1, y1): ";
```

```

cin >> x1 >> y1;
cout << "Nhap toa do diem thu hai (x2, y2): ";
cin >> x2 >> y2;
distance = sqrt(pow(x2 - x1, 2) + pow(y2 - y1, 2));
cout << "Khoang cach giua 2 diem la: " << distance << " don vi" <<
endl;
return 0;
}

```

Kết quả chương trình:

```

Nhap toa do diem dau tien (x1, y1): 1 2
Nhap toa do diem thu hai (x2, y2): 3 4
Khoang cach giua 2 diem la: 2.82843 don vi

```

Trong ví dụ 2, sử dụng kiểu ‘double’ để lưu trữ tọa độ của hai điểm và khoảng cách giữa chúng. Sau khi người dùng nhập vào tọa độ của hai điểm, tính toán khoảng cách bằng công thức $distance = \sqrt{(x2 - x1)^2 + (y2 - y1)^2}$ và in kết quả ra màn hình.

Trong các ứng dụng thực tế, việc lựa chọn kiểu dữ liệu thực phù hợp sẽ phụ thuộc vào yêu cầu về độ chính xác và phạm vi giá trị của các phép tính.

Chú ý:

- Độ chính xác: Khi sử dụng các kiểu số thực, cần lưu ý rằng chúng có giới hạn về độ chính xác. Các phép tính với số thực có thể gây ra lỗi làm tròn (rounding error), đặc biệt là khi thực hiện nhiều phép tính lặp đi lặp lại. Vì vậy, cần cân nhắc độ chính xác cần thiết khi lựa chọn kiểu dữ liệu.
- So sánh: Khi so sánh hai số thực, cần phải kiểm tra sự khác biệt tuyệt đối (absolute difference) giữa chúng thay vì so sánh trực tiếp. Điều này là do có thể xảy ra lỗi làm tròn.
- Khởi tạo: Nên khởi tạo các biến kiểu số thực với giá trị ban đầu để tránh các giá trị rác (garbage value).

Định dạng in ra màn hình

Trong các ví dụ trước, đã sử dụng `cout << "Dien tich cua duong tron la: " << area << " don vi vuong" << endl;` để in kết quả ra màn hình. Tuy nhiên, có thể sử dụng các định dạng in khác để kiểm soát chính xác hơn về cách hiển thị kết quả.

Trong ví dụ tính diện tích hình tròn, có thể in kết quả với 2 chữ số thập phân như sau:

```

cout << fixed << setprecision(2) << "Dien tich hinh tron la: " << area << "
don vi vuong" << endl;

```

Trong đó:

- `fixed`: Hiển thị số thực dưới dạng số thập phân cố định.
- `setprecision(2)`: Chỉ định hiển thị 2 chữ số thập phân.

Tương tự, trong ví dụ tính khoảng cách giữa hai điểm, có thể in kết quả với 4 chữ số thập phân:

```
cout << fixed << setprecision(4) << "Khoang cach giua 2 diem la: " << distance
<< " don vi" << endl;
```

Việc định dạng kết quả in ra màn hình giúp đảm bảo rằng thông tin được hiển thị một cách rõ ràng và dễ đọc.

2.6.3. Kiểu dữ liệu kí tự

KHÁI NIỆM: Trong C++, kiểu kí tự (character) được sử dụng để biểu diễn các kí tự đơn lẻ, như chữ cái, số, và các kí hiệu đặc biệt. Kiểu kí tự được ký hiệu bằng ‘char’ và sử dụng 1 byte (8 bit) để lưu trữ một kí tự.

Các kiểu dữ liệu kí tự

char

- Kiểu dữ liệu ‘char’ sử dụng 1 byte (8 bit) để lưu trữ một kí tự.
- Phạm vi giá trị của ‘char’ thường là từ -128 đến 127, hoặc 0 đến 255, tùy thuộc vào cách biểu diễn (signed hoặc unsigned).
- Các kí tự được biểu diễn bằng cách sử dụng bảng mã, chẳng hạn như ASCII hoặc Unicode.

wchar_t

- Kiểu dữ liệu ‘wchar_t’ được sử dụng để lưu trữ các kí tự rộng (wide character), như các kí tự trong các ngôn ngữ phương Đông.
- Kích thước của ‘wchar_t’ có thể là 2 byte (16 bit) hoặc 4 byte (32 bit), tùy thuộc vào cài đặt của hệ thống.

Chú ý

- Lưu trữ kí tự: các kí tự được lưu trữ dưới dạng mã số trong bảng mã, chẳng hạn như ascii hoặc unicode. Khi sử dụng các kí tự, cần lưu ý đến bảng mã đang được sử dụng.
- So sánh kí tự: khi so sánh hai kí tự, việc so sánh thực chất là so sánh mã số tương ứng của chúng trong bảng mã.
- Thao tác với kí tự: có thể thực hiện các phép toán cơ bản như cộng, trừ, nhân, chia với các kí tự, tuy nhiên kết quả sẽ là mã số tương ứng của kí tự, không phải kí tự.
- Nhập/Xuất kí tự: Sử dụng cin và cout để nhập và xuất kí tự.

Ví dụ 1: Chuyển đổi chữ thường sang chữ hoa

Chương trình 2.8. Chuyển chữ thường thành chữ hoa

```
#include <iostream>
```

```
using namespace std;
int main() {
    char c;
    cout << "Nhap ki tu viet thuong: ";
    cin >> c;
    cout << "Ki tu viet hoa la: " << static_cast<char>(c - 'a' + 'A') <<
endl;
    return 0;
}
```

Kết quả chương trình:

Nhap ki tu viet thuong: a
Ki tu viet hoa la: A

Trong ví dụ 1, sử dụng kiểu char để lưu trữ một kí tự do người dùng nhập vào. Sau đó, chuyển đổi kí tự thường sang kí tự hoa bằng cách trừ mã số của 'a' và cộng mã số của 'A'. Hàm `static_cast<char>` để ép kiểu số nguyên thành kiểu kí tự. Cuối cùng, in kết quả ra màn hình.

2.6.4. Kiểu bool

KHÁI NIỆM: Kiểu bool được sử dụng để biểu diễn các giá trị logic, nghĩa là chỉ có hai giá trị có thể: 'true' (đúng) và 'false' (sai).

Chú ý

- Giá trị mặc định: khi khai báo một biến kiểu 'bool' mà không gán giá trị khởi tạo, giá trị mặc định của nó sẽ là 'false'.
- So sánh giá trị: các biến kiểu 'bool' có thể được so sánh bằng các toán tử so sánh như ==, !=, >, <, >=, <=. Kết quả của phép so sánh sẽ là 'true' hoặc 'false'.
- Các phép toán logic: các biến kiểu bool' có thể được sử dụng trong các phép toán logic như &&' (và), ||' (hoặc), !' (phủ định).
- Chuyển đổi kiểu: các kiểu dữ liệu khác có thể được chuyển đổi sang kiểu bool'. Số 0 sẽ được coi là 'false', còn các giá trị khác sẽ được coi là 'true'.

Ví dụ 2: Giá trị kiểu bool

Chương trình 2.9. In giá trị kiểu bool

```
#include <iostream>
using namespace std;
int main()
{
    bool boolValue;

    boolValue = true;
    cout << boolValue << endl;
    boolValue = false;
    cout << boolValue << endl;
    return 0;
}
```

}
Kết quả chương trình:
1
0

Trong ví dụ 2, sử dụng kiểu bool' để lưu trữ kết quả kiểm tra xem số đã nhập có phải là số chẵn hay không. Sau đó, sử dụng câu lệnh if-else' để in ra thông báo tương ứng.

2.6.5. Kiểu chuỗi (string)

KHÁI NIỆM: Kiểu string (chuỗi kí tự) được sử dụng để biểu diễn và thao tác với các chuỗi kí tự. Kiểu string' là một kiểu dữ liệu được định nghĩa trong thư viện <string>. Nó cung cấp nhiều phương thức và toán tử để dễ dàng thao tác với chuỗi.

Chú ý

- Khởi tạo chuỗi: có nhiều cách để khởi tạo một chuỗi string
 - Ví dụ:

```
string s1;
string s2 = "hello";
string s3(5, 'a');
```

- Truy cập kí tự: các kí tự trong chuỗi có thể được truy cập bằng chỉ số, ví dụ: s[0] để lấy kí tự đầu tiên.
- Thao tác chuỗi: có nhiều phương thức và toán tử để thao tác với chuỗi, ví dụ: '+' để nối chuỗi, length()' để lấy độ dài, find()' để tìm kiếm chuỗi con.
- Nhập/xuất chuỗi: sử dụng cin' và cout' để nhập và xuất chuỗi.
- Để nhập 1 chuỗi gồm cả dấu cách (space) thì cần dùng hàm getline
 - Ví dụ:

```
getline(cin,s1);
```

Ví dụ 3, Chuyển chuỗi chữ thường thành chuỗi chữ hoa

Chương trình 2.10. Chuyển chuỗi chữ thường thành chuỗi chữ hoa

<pre>#include <iostream> #include <string> using namespace std; int main() { string input; cout << "Nhập một chuỗi: "; getline(cin, input); cout << "Chuỗi vừa nhập là: " << input << endl; return 0; }</pre>
Kết quả chương trình:
Nhập một chuỗi: Dau Hai Phong

2.7. Phạm vi của biến

KHÁI NIỆM: Phạm vi (scope) của một biến trong C++ xác định vùng mà biến đó có thể được truy cập và sử dụng. C++ có các loại phạm vi sau:

- Phạm vi toàn cục (Global Scope): Biến toàn cục được khai báo bên ngoài tất cả các hàm và có thể được truy cập từ bất kỳ nơi nào trong chương trình.
- Phạm vi cục bộ (Local Scope): Biến cục bộ được khai báo bên trong một hàm hoặc khối lệnh, chỉ có thể được truy cập trong vùng đó.
- Phạm vi tham số (Parameter Scope): Các biến tham số được khai báo trong danh sách tham số của một hàm, chỉ có thể được sử dụng bên trong hàm đó.

Cụ thể vấn đề này sẽ được đề cập trong chương về định nghĩa hàm.

Chương trình 2.11. Lỗi vì chưa khai báo biến

```
#include <iostream>
using namespace std;
int main()
{
    cout << value; // Lỗi! biến chưa được định nghĩa!
    int value = 100;
    return 0;
}
```

Kết quả chương trình:

error: 'value' was not declared in this scope

2.8. Toán tử số học

KHÁI NIỆM: Trong C++, các toán tử số học được sử dụng để thực hiện các phép tính số học trên các giá trị số. Các toán tử này hoạt động trên các kiểu dữ liệu số như int, float, double, v.v.

Các toán tử cơ bản

C++ cung cấp các toán tử số học cơ bản sau:

- Cộng (+): Thực hiện phép cộng giữa hai toán hạng.
- Trừ (-): Thực hiện phép trừ giữa hai toán hạng.
- Nhân (*): Thực hiện phép nhân giữa hai toán hạng.
- Chia (/): Thực hiện phép chia giữa hai toán hạng.
- Chia lấy phần dư (%): Thực hiện phép chia lấy phần dư giữa hai toán hạng.

Chia nguyên

Khi thực hiện phép chia '/' giữa hai số nguyên, kết quả sẽ là một số nguyên (phép chia nguyên). Phần thập phân sẽ bị cắt đi. Ví dụ: $10 / 3 = 3$.

Chú ý

- Thứ tự ưu tiên các toán tử: các toán tử có mức độ ưu tiên khác nhau, ví dụ: '*' và '/' có mức độ ưu tiên cao hơn '+' và '-'.
- Toán tử chia lấy phần dư: toán tử '%' chỉ hoạt động được với các số nguyên, không thể sử dụng với các số thực.
- Phép chia cho 0: chia cho 0 sẽ gây ra lỗi chia cho 0 (division by zero error).

Ví dụ 1: Tính diện tích và chu vi hình tròn

Chương trình 2.12. Tính diện tích, chu vi hình tròn

```
#include <iostream>
using namespace std;
int main()
{
    float radius, area, circumference;
    const float PI = 3.14159;
    cout << "Nhap ban kinh hinh tron: ";
    cin >> radius;
    area = PI * radius * radius;
    circumference = 2 * PI * radius;

    cout << "Dien tich hinh tron: " << area << " don vi vuong" << endl;
    cout << "Chu vi hinh tron: " << circumference << " don vi" << endl;
    return 0;
}
```

Kết quả chương trình:

```
Nhap ban kinh hinh tron: 5
Dien tich hinh tron: 78.5397 don vi vuong
Chu vi hinh tron: 31.4159 don vi
```

Trong ví dụ 1, sử dụng các toán tử số học để tính diện tích và chu vi của hình tròn dựa trên bán kính đã nhập.

Ví dụ 2: Tính toán chia lấy phần dư

Chương trình 2.13. Phép chia lấy dư

```
#include <iostream>
using namespace std;
int main() {
    int dividend, divisor, quotient, remainder;
    cout << "Nhap so bi chia: ";
    cin >> dividend;
    cout << "Nhap so chia: ";
    cin >> divisor;
    quotient = dividend / divisor;
    remainder = dividend % divisor;
    cout << "Ket qua phép chia: " << quotient << endl;
    cout << "Phan du cua phép chia: " << remainder << endl;
}
```

```
    return 0;  
}
```

Kết quả chương trình:

```
Nhap so bi chia: 7  
Nhap so chia: 2  
Ket qua phep chia: 3  
Phan du cua phep chia: 1
```

Trong ví dụ 2, sử dụng toán tử chia nguyên / và toán tử chia lấy phần dư % để tính toán kết quả và phần dư của phép chia giữa hai số nguyên.

2.9. Ghi chú

KHÁI NIỆM: Ghi chú (comment) trong C++ là những dòng văn bản được thêm vào chương trình, nhưng không được thông dịch và thực thi bởi trình biên dịch. Ghi chú được sử dụng để:

- Giải thích và tài liệu hóa mã nguồn.
- Tạm thời loại bỏ một đoạn mã khỏi chương trình.
- Ngăn ngừa trình biên dịch thực thi một đoạn mã.

Các kiểu ghi chú:

C++ cung cấp hai cách để thêm ghi chú vào chương trình:

- Ghi chú trên một dòng: Bắt đầu bằng // và kéo dài đến cuối dòng.
- Ghi chú nhiều dòng: Bắt đầu bằng /* và kết thúc bằng */.

Chú ý:

- Thông tin trong ghi chú: ghi chú nên cung cấp thông tin hữu ích như mục đích, chức năng, tác giả, ngày tháng, v.v.
- Duy trì ghi chú: cần duy trì và cập nhật ghi chú khi có sự thay đổi trong mã nguồn để đảm bảo tài liệu hóa luôn chính xác.
- Cách sử dụng ghi chú hợp lý: không nên ghi chú quá nhiều, ghi chú không cần thiết hoặc không đúng cách có thể làm mã nguồn trở nên khó đọc.

Ví dụ 1: Ghi chú trên một dòng

Chương trình 2.14. Cách ghi chú trên 1 dòng và nhiều dòng

```
/*Đây là chương trình hướng dẫn ghi chú  
trên 1 dòng và nhiều dòng*/  
  
#include <iostream>  
using namespace std;  
int main() {  
    // Khai báo hai số nguyên  
    int a = 10, b = 20;  
    // Tính tổng hai số
```

```
int sum = a + b;
// In ra kết quả
cout << "Tong cua " << a << " va " << b << " la: " << sum << endl;
return 0;
}
```

Kết quả chương trình:

Tong cua 10 va 20 la: 30

Trong ví dụ này, sử dụng ghi chú trên một dòng để giải thích các bước trong chương trình.

Việc sử dụng ghi chú một cách hợp lý giúp mã nguồn trở nên dễ đọc, dễ hiểu và dễ bảo trì hơn. Ghi chú là một phần quan trọng trong quá trình phát triển phần mềm.

2.10. Hằng số

KHÁI NIỆM: Hằng số (constant) trong C++ là một giá trị cố định, không thể thay đổi trong suốt thời gian chạy của chương trình. Hằng số được sử dụng để lưu trữ các giá trị không thay đổi, như các hằng số toán học, hằng số vật lý, hoặc các cấu hình của chương trình.

Chú ý:

- Khai báo hằng số: hằng số được khai báo bằng từ khóa `const` hoặc `#define`. Ví dụ: `const float PI = 3.14159;` hoặc `#define PI 3.14159`.
- Phạm vi của hằng số: hằng số có thể có phạm vi toàn cục hoặc cục bộ, tương tự như biến.
- Đặt tên hằng số: tên của hằng số thường viết hoa để dễ phân biệt với biến.
- Sử dụng hằng số: hằng số có thể được sử dụng trong các biểu thức, gán giá trị, hoặc truyền vào hàm.
- Tối ưu hóa hằng số: trình biên dịch có thể tối ưu hóa các hằng số bằng cách thay thế chúng bằng giá trị thực tế trong quá trình biên dịch.

Ví dụ 1: Tính diện tích hình tròn

Chương trình 2.15. Tính diện tích hình tròn

```
#include <iostream>
using namespace std;
int main() {
    const float PI = 3.14159; // Hằng số PI
    float radius, area;
    cout << "Nhap ban kinh hinh tron: ";
    cin >> radius;
    area = PI * radius * radius;
    cout << "Dien tich hinh tron la: " << area << " don vi vuong" << endl;
    return 0;
}
```

Kết quả chương trình:

Nhap ban kinh hinh tron: 5
Dien tich hinh tron la: 78.5397 don vi vuong

Trong ví dụ này, sử dụng hằng số PI' để tính diện tích hình tròn. Giá trị của PI' không thể thay đổi trong suốt thời gian chạy chương trình.

Ví dụ 2: Tính tiền lương

Chương trình 2.16. Tính tiền lương nhân viên

```
#include <iostream>
using namespace std;
#define HOURLY_RATE 20.0 // Hang so muc luong theo gio
#define OVERTIME_RATE 1.5 // Hang so muc luong lam them gio
int main() {
    int normalHours, overtimeHours;
    float totalPay;
    cout << "Nhap so gio lam viec thuong: ";
    cin >> normalHours;
    cout << "Nhap so gio lam them: ";
    cin >> overtimeHours;
    totalPay = (normalHours * HOURLY_RATE) + (overtimeHours * HOURLY_RATE *
OVERTIME_RATE);
    cout << "Tong tien luong: $" << totalPay << endl;
    return 0;
}
```

Kết quả chương trình:

Nhap so gio lam viec thuong: 10
Nhap so gio lam them: 2
Tong tien luong: \$260

Trong ví dụ này, sử dụng hai hằng số HOURLY_RATE' và OVERTIME_RATE' để tính tổng tiền lương dựa trên số giờ làm việc thường và số giờ làm thêm. Các giá trị của các hằng số này không thể thay đổi trong suốt chương trình.

Việc sử dụng hằng số giúp mã nguồn trở nên rõ ràng, dễ bảo trì và ít xảy ra lỗi do nhập sai giá trị. Ngoài ra, trình biên dịch cũng có thể tối ưu hóa hiệu suất bằng cách thay thế các hằng số bằng giá trị thực tế.

2.11. Câu hỏi và Bài tập

2.11.1. Câu hỏi

1. Đối tượng cout' trong C++ dùng để làm gì?

2. Câu lệnh `#include <iostream>` có tác dụng gì?
3. Biến trong C++ được khai báo như thế nào?
4. Phạm vi của biến cục bộ và biến toàn cục khác nhau như thế nào?
5. Ghi chú trong C++ có thể được viết bằng cách nào?
6. Tại sao việc sử dụng ghi chú là quan trọng trong lập trình?
7. Hằng số trong C++ được khai báo bằng từ khóa nào?
8. Khi nào nên sử dụng hằng số thay vì biến?
9. Giải thích sự khác biệt giữa khai báo hằng số bằng `const` và `#define`.
10. Trong ví dụ tính diện tích hình tròn, hằng số `PI` được sử dụng như thế nào?

2.11.2. Bài tập

1. Tổng của hai số (*Bắt buộc*)

Viết một chương trình lưu các số nguyên 50 và 100 trong các biến, và lưu tổng của hai số này trong một biến có tên là `total`.

2. Dự đoán sale

Phòng sale The East Coast của một công ty tạo ra 58 phần trăm của tổng doanh thu. Dựa trên tỉ lệ phần trăm đó, viết một chương trình đoán xem Phòng sale The East Coast này sẽ tạo ra con số là bao nhiêu nếu công ty có doanh thu là 8.6 triệu đô trong năm nay.

3. Thuế bán hàng (*Bắt buộc*)

Viết một chương trình tính tổng thuế bán hàng trên một giỏ hàng 95 đô. Giả sử thuế bán hàng của một tiểu bang là 4 phần trăm, và thuế bán hàng của quận là 2 phần trăm.

4. Hóa đơn nhà hàng

Viết một chương trình tính thuế và tip trên một hóa đơn nhà hàng cho vị khách quen với số tiền phải trả cho bữa ăn là 88.67 đô. Thuế là 6.75 phần trăm chi phí bữa ăn. Tiền tip là 20 phần trăm tổng tiền sau khi cộng thêm thuế. Hiển thị chi phí bữa ăn, số tiền thuế, số tiền tip, và tổng hóa đơn ra màn hình.

5. Giá trị trung bình

Để có được giá trị trung bình của một loạt các giá trị, bạn cộng các giá trị lại sau đó chia cho tổng số các giá trị đếm được. Viết một chương trình để lưu trữ các giá trị sau trong năm biến khác nhau: 28, 32, 37, 24, và 33. Chương trình trước hết phải tính được tổng của năm biến và lưu kết quả vào trong một biến sum riêng biệt. Sau đó, chương trình lấy biến sum chia cho 5 để lấy được giá trị trung bình. Hiển thị nó giá trị trung bình ra màn hình.

Gợi ý: Nên sử dụng kiểu dữ liệu `double` cho các biến trong chương trình.

6. Thanh toán hàng năm

Giả sử một nhân viên được trả lương hai tuần một lần và kiếm được 2.200 đô là mỗi kỳ lương. Trong một năm, nhân viên được trả 26 lần. Viết chương trình định nghĩa các biến sau đây:

Chương trình tính toán tổng thanh toán hàng năm cho nhân viên bằng cách nhân số tiền thanh toán mỗi lần với số kỳ lương được nhận trong một năm và lưu kết quả vào trong biến `annualPay`. Hiển thị tổng `annualPay` ra màn hình.

7. Mức nước biển (Bắt buộc)

Giả sử mức nước biển hiện tại dâng lên khoảng 1.5 mm mỗi năm, viết một chương trình hiển thị:

- Số mm sẽ tăng lên so với mức nước biển hiện tại sau 5 năm.
- Số mm sẽ tăng lên so với mức nước biển hiện tại sau 7 năm.
- Số mm sẽ tăng lên so với mức nước biển hiện tại sau 10 năm.

8. Tổng tiền mua hàng

Một khách hàng trong một cửa hàng mua 5 món đồ. Giá của các món đồ như sau:

- Giá của món đồ thứ nhất = 15.59 đô la
- Giá của món đồ thứ hai = 24.59 đô la
- Giá của món đồ thứ ba = 6.59 đô la
- Giá của món đồ thứ tư = 12.59 đô la
- Giá của món đồ thứ năm = 3.59 đô la

Viết một chương trình lưu giữ giá của năm món đồ này trong năm biến. Hiển thị giá của mỗi món đồ, tổng tạm tính của giỏ hàng, số tiền thuế, và tổng phải trả. Giả sử thuế bán hàng là 7 phần trăm.

9. Kích thước kiểu dữ liệu

Bạn được giao công việc của một lập trình viên trong Cyborg supercomputer. Một trong các bước để hoàn thành các công việc tính toán là bạn cần phải biết được các kiểu dữ liệu sẽ sử dụng sau đây có kích thước bao nhiêu byte: kiểu `char`, kiểu `int`, kiểu `float`, và kiểu `double`. Tuy nhiên bạn lại không có bất kỳ tài liệu kỹ thuật nào để tra cứu thông tin này cả. Hãy viết chương trình C++ để xác định xem không gian bộ nhớ cần cho việc chứa các kiểu dữ liệu này là bao nhiêu và hiển thị ra màn hình.

10. Số dặm trên mỗi gallon

Một chiếc xe ô tô có thể chứa được 15 gallon xăng và có thể di chuyển được 375 dặm trước khi cần nạp lại. Viết chương trình tính toán số dặm trên mỗi gallon mà xe có thể đi được. Hiển thị nó ra màn hình.

11. Quãng đường với mỗi bình xăng (*Bắt buộc*)

Một chiếc xe với một bình xăng 20 gallon trung bình 23.5 dặm cho mỗi gallon khi lái xe trong thành phố, và 28.9 dặm cho mỗi gallon khi lái xe trên đường cao tốc. Viết chương trình tính toán và hiển thị quãng đường ô tô đi được với một bình xăng khi di chuyển trên thị trấn và khi di chuyển trên đường cao tốc.

12. Tính toán mẫu đất

Một mẫu đất tương đương với 43.560 feet vuông. Viết chương trình tính toán số mẫu đất trên một khu đất có diện tích 391.876 feet vuông.

13. Giá một bo mạch

Một công ty điện tử bán một bo mạch với lợi nhuận là 35 phần trăm. Viết chương trình tính toán giá bán của một bo mạch có chi phí là 14.95 đô la. Hiện kết quả lên màn hình.

14. Thông tin cá nhân

Viết một chương trình hiển thị các thông tin sau, mỗi thông tin trên một dòng.

- Tên của bạn
- Địa chỉ của bạn, với quận, thành phố, và ZIP code
- Số điện thoại của bạn
- Chuyên ngành của bạn

Sử dụng duy nhất một câu lệnh cout để hiển thị tất cả những thông tin trên.

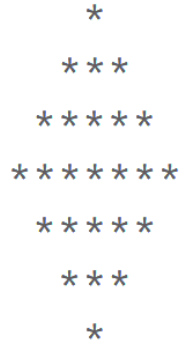
15. Mẫu hình tam giác (*Bắt buộc*)

Viết chương trình hiển thị hình tam giác theo mẫu sau trên màn hình:

```
      *
     ***
    *****
   ********
```

16. Mẫu hình kim cương

Viết chương trình hiển thị hình mẫu sau:



17. Hoa hồng chứng khoán

Kathryn đã mua 750 cổ phiếu với giá 35.00 USD / cổ phiếu. Cô ấy phải trả cho người môi giới chứng khoán của mình khoản hoa hồng 2% cho giao dịch. Viết một chương trình tính toán và hiển thị như sau:

- Số tiền thanh toán cho cổ phiếu (không có hoa hồng).
- Số tiền hoa hồng.
- Tổng số tiền đã thanh toán (tổng cho cổ phiếu cộng với tiền hoa hồng).

18. Tiêu thụ nước tăng lực

Một công ty nước ngọt gần đây đã khảo sát 16,500 khách hàng của mình và thấy rằng khoảng 15% trong số những người được khảo sát mua một hoặc nhiều nước tăng lực mỗi tuần. Trong số những khách mua nước tăng lực, có khoảng 58% trong số họ thích nước tăng lực có vị cam. Viết chương trình hiển thị như sau:

Số lượng khách hàng gần đúng trong cuộc khảo sát mua một hoặc nhiều nước tăng lực trong mỗi tuần.

Số lượng khách hàng gần đúng trong cuộc khảo sát thích nước tăng lực có vị cam.

19. Nhiệt độ cao hàng năm

Nhiệt độ cao trung bình tháng 7 là 85 độ F ở New York City, 88 độ F ở Denver, và 106 độ F ở Phoenix. Viết chương trình tính toán và báo cáo nhiệt độ trung bình mới trong tháng 7 của các thành phố nếu nhiệt độ ở mỗi thành phố tăng lên 2%.

20. Bao nhiêu sơn

Một nhãn hiệu sơn quảng cáo rằng sơn của họ có thể phủ được 340 feet vuông trên mỗi gallon. Viết chương trình xác định và báo cáo xấp xỉ số lượng gallon sơn cần thiết để sơn cho hai lớp áo trên hàng rào bằng gỗ cao 6 feet và dài 100 feet.

CHƯƠNG 3. BIỂU THỨC VÀ THAO TÁC CƠ BẢN TRÊN DỮ LIỆU

Chủ đề	
3.1	Đối tượng cin
3.2	Biểu thức toán học
3.3	Chuyển đổi kiểu dữ liệu
3.4	Tràn trên và tràn dưới
3.5	Ép kiểu
3.6	Gán nhiều và gán kết hợp
3.7	Định dạng đầu ra
3.8	Làm việc với chuỗi và kí tự
3.9	Một số hàm trong thư viện toán học

3.1. Đối tượng cin

Khái niệm: cin là một đối tượng được sử dụng để nhập dữ liệu vào chương trình từ bàn phím. Nó là một phần của thư viện iostream và cho phép người dùng nhập các kiểu dữ liệu khác nhau, như số nguyên, số thực, ký tự, chuỗi, v.v.

Chú ý

- Đọc dữ liệu với cin: cin đọc dữ liệu từ bàn phím và lưu trữ vào biến được chỉ định. Nó sẽ đọc dữ liệu cho đến khi gặp khoảng trắng (space, tab, newline).
- Xử lý lỗi đầu vào: Nếu người dùng nhập một kiểu dữ liệu không phù hợp với biến, cin sẽ đặt trạng thái lỗi và bạn cần xử lý lỗi này trước khi tiếp tục.
- Nhập chuỗi với cin: Khi nhập chuỗi với cin, nó chỉ đọc dữ liệu cho đến khi gặp khoảng trắng đầu tiên. Nếu bạn muốn nhập cả dòng, hãy sử dụng getline().

Ví dụ 1: Nhập và hiển thị số nguyên

Chương trình 3.1. Nhập và hiển thị một số nguyên

```
#include <iostream>
using namespace std;
int main()
{
    int num1;
    float num2;
    cout << "Nhập số nguyên: ";
    cin >> num1;
    cout << "Nhập số thực: ";
    cin >> num2;

    cout << "Số nguyên vừa nhập là: " << num1 << endl;
    cout << "Số thực vừa nhập là: " << num2 << endl;
    return 0;
}
```

Kết quả chương trình:

Nhập số nguyên: 5

Nhap so thuc: 8.99
So nguyen vua nhap la: 5
So thuc vua nhap la: 8.99

Ví dụ 2: Nhập và hiển thị chuỗi

Chương trình 3.2. Nhập và hiển thị một chuỗi

```
#include <iostream>
#include <string>
using namespace std;
int main() {
    string name;
    cout << "Nhap ten cua ban: ";
    cin >> name;
    cout << "Xin chao, " << name << "!" << endl;
    return 0;
}
```

Kết quả chương trình:

Nhap ten cua ban: Phong
Xin chao, Phong!

Để nhập nhập 1 chuỗi có dấu cách thì cần dùng hàm getline. Chú ý khi dùng hàm getline thì trước đó cần dùng hàm cin.ignore() để bỏ kí tự enter trong bộ nhớ đệm.

Ví dụ 3, nhập và in chuỗi có dấu cách

Chương trình 3.3. Nhập chuỗi có dấu cách

```
#include <iostream>
#include <string>
using namespace std;
int main() {
    string name;
    cout << "Nhap ten cua ban: ";
    cin >> name;
    cout << "Xin chao, " << name << "!" << endl;
    return 0;
}
```

Kết quả chương trình:

Nhap ho va ten cua ban: Dau Hai Phong
Xin chao, Dau Hai Phong!

Kiểm tra

Dưới đây là một số câu kiểm tra về đối tượng cin trong ngôn ngữ lập trình C++:

1. Câu hỏi: Trong C++, đối tượng nào được sử dụng để nhận dữ liệu từ bàn phím?
2. Đáp án: Đối tượng cin được sử dụng để nhận dữ liệu từ bàn phím trong C++.
3. Câu hỏi: Khi nhập chuỗi với cin, dữ liệu sẽ được đọc cho đến khi gặp kí tự nào?

4. Đáp án: Khi nhập chuỗi với cin, dữ liệu sẽ được đọc cho đến khi gặp khoảng trắng (space, tab, newline) đầu tiên.
5. Câu hỏi: Nếu người dùng nhập một kiểu dữ liệu không phù hợp với biến, cin sẽ làm gì?
6. Đáp án: Nếu người dùng nhập một kiểu dữ liệu không phù hợp với biến, cin sẽ đặt trạng thái lỗi và bạn cần xử lý lỗi này trước khi tiếp tục.
7. Câu hỏi: Để nhập cả dòng văn bản với cin, bạn nên sử dụng phương thức nào?
8. Đáp án: Để nhập cả dòng văn bản với cin, bạn nên sử dụng phương thức getline().
9. Câu hỏi: Trong ví dụ sau, hãy chỉ ra lỗi và giải thích tại sao lại xuất hiện lỗi đó:

```
#include <iostream>
using namespace std;
int main() {
    int num;
    cout << "Nhap so nguyen: ";
    num = cin;
    cout << "So nguyen vua nhap la: " << num << endl;
    return 0;
}
```

3.2. Biểu thức toán học

KHÁI NIỆM: biểu thức toán học là một chuỗi các toán hạng (operands) và toán tử (operators) được kết hợp lại với nhau để tạo thành một biểu thức có giá trị. Biểu thức toán học được sử dụng để thực hiện các phép tính toán trong chương trình C++.

Thứ tự ưu tiên các toán tử

Các toán tử có mức độ ưu tiên khác nhau. Khi đánh giá một biểu thức, các toán tử có mức độ ưu tiên cao sẽ được thực hiện trước. Bảng toán tử ưu tiên trong C++ như sau:

1. Toán tử đơn ngôi (unary operators): ++, --, +, -, !, ~
2. Toán tử nhân, chia, chia lấy dư: *, /, %
3. Toán tử cộng, trừ: +, -
4. Toán tử so sánh: <, >, <=, >=, ==, !=
5. Toán tử logic: &&, ||
6. Toán tử gán: =, +=, -=, *=, /=, %=, &=, |=, ^=, <<=, >>=

Sử dụng dấu ngoặc đơn

Dấu ngoặc đơn () được sử dụng để nhóm các toán hạng và toán tử lại với nhau. Khi có nhiều toán tử cùng mức độ ưu tiên, các toán tử trong cùng một cặp ngoặc sẽ được thực hiện trước.

Chú ý

- Sử dụng dấu ngoặc đơn để rõ ràng hơn trong việc thực hiện các phép tính.

- Chú ý đến mức độ ưu tiên của các toán tử khi viết biểu thức.
- Kiểm tra kỹ biểu thức trước khi sử dụng để tránh lỗi.

Ví dụ 1: Tính biểu thức toán học

Chương trình 3.4. Tính toán biểu thức toán học

```
#include <iostream>
using namespace std;

int main() {
    int a = 10, b = 5, c;
    c = (a + b) * 2 - 3;
    cout << "Ket qua của bieu thuc: " << c << endl;
    return 0;
}
```

Kết quả chương trình:

Ket qua cua bieu thuc: 27

Ví dụ 2: Sử dụng toán tử ưu tiên

Chương trình 3.5. Sử dụng toán tử ưu tiên

```
#include <iostream>
using namespace std;

int main() {
    int a = 10, b = 5, c = 3;
    int result = a * b + c;
    cout << "Ket qua cua bieu thuc: " << result << endl;
    return 0;
}
```

Kết quả chương trình:

Ket qua cua bieu thuc: 53

Ví dụ 3: Chuyển đổi biểu thức đại số sau thành biểu thức lập trình

$$\frac{(x * y + z)}{(y - x)} \text{ với } x = 5, y = 10, z = 3$$

Chương trình 3.6. Chuyển đổi biểu thức đại số sau thành biểu thức lập trình

```
#include <iostream>
using namespace std;

int main() {
    int x = 5, y = 10, z = 3;
    int result = (x * y + z) / (y - x);
    cout << "Ket qua cua bieu thuc: " << result << endl;
    return 0;
}
```

Kết quả chương trình:

Ket qua cua bieu thuc: 10

Kiểm tra

1. Câu hỏi: Trong C++, biểu thức toán học là gì?
2. Câu hỏi: Trong C++, toán tử nào có mức độ ưu tiên cao nhất?
3. Câu hỏi: Khi có nhiều toán tử cùng mức độ ưu tiên, làm thế nào để thực hiện chúng?
4. Câu hỏi: Trong ví dụ sau, kết quả của biểu thức $(a + b) * 2 - 3$ sẽ là bao nhiêu, biết rằng $a = 10$ và $b = 5$?
5. Câu hỏi: Trong ví dụ sau, kết quả của biểu thức $a * b + c$ sẽ là bao nhiêu, biết rằng $a = 10$, $b = 5$, và $c = 3$?

3.3. Chuyển đổi dữ liệu

KHÁI NIỆM: Khi toán hạng của một toán tử có các kiểu dữ liệu khác nhau, C++ sẽ tự động chuyển đổi chúng sang cùng một kiểu dữ liệu. Điều này có thể ảnh hưởng đến kết quả của các biểu thức toán học.

Nếu nhân một int với một float thì kết quả sẽ là kiểu dữ liệu gì? Điều gì sẽ xảy ra nếu một double được chia cho một unsigned int? Có cách nào để dự đoán điều gì sẽ xảy ra trong những trường hợp này không? Câu trả lời là có. C++ tuân theo một bộ quy tắc khi thực hiện các phép toán trên các biến có kiểu dữ liệu khác nhau. Sẽ rất hữu ích khi hiểu các quy tắc này để ngăn chặn các lỗi tinh vi xâm nhập vào chương trình của bạn.

Cũng giống như các sĩ quan trong quân đội, các loại dữ liệu được xếp hạng. Một loại dữ liệu sẽ xếp hạng cao hơn loại khác nếu nó có thể chứa số lượng lớn hơn. Ví dụ: float xếp hạng cao hơn int. Phần dưới đây sẽ liệt kê các loại dữ liệu theo thứ tự xếp hạng của chúng, từ cao nhất đến thấp nhất.

```
long double
double
float
unsigned long long int
long long int
unsigned long int
long int
unsigned int
int
```

Khi C++ làm việc với một toán tử, nó sẽ cố gắng chuyển đổi các toán hạng về cùng loại.

Việc chuyển đổi tự động này được gọi là ép kiểu. Khi một giá trị được chuyển đổi sang loại dữ liệu cao hơn, nó được cho là được thăng cấp. Giảm hạng một giá trị có nghĩa là

chuyển đổi nó thành loại dữ liệu thấp hơn. hãy xem xét các quy tắc cụ thể chi phối việc đánh giá các biểu thức toán học.

Quy tắc 1: char, short và unsigned short được tự động thăng cấp thành int.

Quy tắc 2: Khi toán tử làm việc với hai giá trị thuộc các loại dữ liệu khác nhau, giá trị xếp hạng thấp hơn sẽ được thăng cấp thành loại giá trị xếp hạng cao hơn.

Trong biểu thức sau đây, giả sử Nam là int và TyLeLaiSuat là float:

```
Nam * TyLeLaiSuat
```

Trước khi phép nhân diễn ra, Nam sẽ được thăng cấp thành số float.

Quy tắc 3: Khi giá trị cuối cùng của biểu thức được gán cho một biến, nó sẽ được chuyển đổi thành kiểu dữ liệu của biến đó.

Trong câu lệnh sau, giả sử dientich là kiểu long int, trong khi chieudai và chieurong đều là int:

```
dientich = chieudai* chieurong;
```

Vì chieudai và chieurong đều là int nên chúng sẽ không được chuyển đổi sang bất kỳ loại dữ liệu nào khác. Tuy nhiên, kết quả của phép nhân sẽ được chuyển đổi thành long int để có thể lưu trữ trong biến dientich.

Hãy chú ý đến các tình huống trong đó một biểu thức dẫn đến một giá trị phân số được gán cho một biến số nguyên. Đây là một ví dụ:

```
int x, y = 4;
float z = 2.7;
x = y * z;
```

Trong biểu thức $y * z$, y sẽ được thăng cấp thành float và 10.8 sẽ là kết quả của phép nhân. Tuy nhiên, vì x là số nguyên nên 10.8 sẽ bị cắt bớt và 10 sẽ được lưu trong x.

Phép chia số nguyên

Khi bạn chia một số nguyên cho một số nguyên khác trong C++, kết quả luôn là số nguyên. Nếu còn dư sẽ bị loại bỏ. Ví dụ: trong đoạn mã sau, các phần được gán giá trị 2.0:

```
double parts;
parts = 15 / 6;
```

Mặc dù 15 chia cho 6 thực sự là 2.5, phần .5 của kết quả bị loại bỏ vì đang chia một số nguyên cho một số nguyên. Việc các phần được khai báo là double không thành vấn đề bởi vì phần phân số của kết quả sẽ bị loại bỏ trước khi phép gán diễn ra. Để phép chia trả về giá trị dấu phẩy động, ít nhất một trong các toán hạng phải thuộc kiểu dữ liệu dấu phẩy động. Ví dụ: mã trước đó có thể được viết là:

```
double parts;
parts = 15.0 / 6;
```

Trong mã này, giá trị bằng chữ 15.0 được hiểu là số dấu phẩy động, do đó phép chia sẽ trả về số dấu phẩy động. Giá trị 2.5 sẽ được gán cho parts.

3.4. Tràn trên (Overflow) và tràn dưới (Underflow)

KHÁI NIỆM:

- Tràn trên (Overflow): Tràn trên xảy ra khi một phép toán số học tạo ra một kết quả vượt quá giới hạn lớn nhất của kiểu dữ liệu mà biến đang lưu trữ. Ví dụ, khi bạn cố gắng lưu trữ một giá trị lớn hơn INT_MAX (giới hạn lớn nhất của kiểu int) vào một biến kiểu int.

- Tràn dưới (Underflow): Tràn dưới xảy ra khi một phép toán số học tạo ra một kết quả nhỏ hơn giới hạn nhỏ nhất của kiểu dữ liệu mà biến đang lưu trữ. Ví dụ, khi bạn cố gắng lưu trữ một giá trị nhỏ hơn FLT_MIN (giới hạn nhỏ nhất của kiểu float) vào một biến kiểu float.

Ví dụ 1: tràn trên

Chương trình 3.7. Tràn trên dẫn đến kết quả sai

```
#include <iostream>
#include <limits>
using namespace std;
int main() {
    int x = numeric_limits<int>::max(); // x = 2147483647
    x = x + 1; // Tràn trên, x = -2147483648
    cout << "x = " << x << endl;
    return 0;
}
```

Kết quả chương trình:

x = -2147483648

Ví dụ 2: tràn dưới

Chương trình 3.8. Tràn dưới dẫn đến kết quả sai

```
#include <iostream>
#include <limits>
using namespace std;
int main() {
    float y = numeric_limits<float>::min(); // y = 1.175494e-38
    y = y / 10.0; // Tràn dưới, y = 0
    cout << "y = " << y << endl;
    return 0;
}
```

Kết quả chương trình:

y = 1.17549e-39

Giải thích chi tiết

1. Tràn trên (Overflow):

- Tràn trên xảy ra khi kết quả của một phép toán số học vượt quá giới hạn lớn nhất của kiểu dữ liệu được sử dụng.
- Ví dụ, với kiểu `int` 32-bit, giới hạn lớn nhất là `INT_MAX = 2147483647`. Nếu bạn cố gắng lưu trữ giá trị lớn hơn `INT_MAX` vào một biến kiểu `int`, sẽ xảy ra tràn trên.
- Khi tràn trên xảy ra, giá trị của biến sẽ "quay lại" từ giới hạn lớn nhất về giới hạn nhỏ nhất của kiểu dữ liệu đó. Ví dụ, $INT_MAX + 1 = -2147483648$.
- Tràn trên có thể dẫn đến kết quả không mong muốn và lỗi trong chương trình.

2. Tràn dưới (Underflow):

- Tràn dưới xảy ra khi kết quả của một phép toán số học nhỏ hơn giới hạn nhỏ nhất của kiểu dữ liệu được sử dụng.
- Ví dụ, với kiểu `float` 32-bit, giới hạn nhỏ nhất là `FLT_MIN = 1.175494e-38`. Nếu bạn cố gắng lưu trữ giá trị nhỏ hơn `FLT_MIN` vào một biến kiểu `float`, sẽ xảy ra tràn dưới.
- Khi tràn dưới xảy ra, giá trị của biến sẽ được làm tròn về 0.
- Tràn dưới thường không gây ra lỗi nghiêm trọng như tràn trên, nhưng có thể dẫn đến kết quả không chính xác.

Để tránh các vấn đề do tràn trên và tràn dưới gây ra, bạn nên:

- Kiểm tra giới hạn của kiểu dữ liệu trước khi thực hiện các phép tính.
- Sử dụng các kiểu dữ liệu có phạm vi giá trị lớn hơn nếu cần thiết.
- Xử lý các trường hợp tràn trên và tràn dưới phù hợp với yêu cầu của ứng dụng.

3.5. Ép kiểu

KHÁI NIỆM: Ép kiểu (type casting) là một kỹ thuật cho phép chuyển đổi một kiểu dữ liệu này thành một kiểu dữ liệu khác. Điều này thường cần thiết khi thực hiện các phép toán giữa các kiểu dữ liệu khác nhau hoặc khi muốn lưu trữ một giá trị vào một biến có kiểu dữ liệu khác.

C++ cung cấp các cách ép kiểu sau:

1. Static cast: Là cách ép kiểu phổ biến nhất, sử dụng từ khóa `static_cast`.
2. Dynamic cast: Được sử dụng để ép kiểu giữa các lớp có quan hệ kế thừa.
3. Const cast: Được sử dụng để thay đổi tính chất `const` của một biến.
4. Reinterpret cast: Được sử dụng để ép kiểu giữa các kiểu dữ liệu hoàn toàn khác nhau.

Trong chương này, chỉ giới thiệu cách ép kiểu sử dụng từ khóa `static_cast`. Hai cách còn lại giới thiệu sau.

Một số chú ý khi ép kiểu

- Kiểm tra giới hạn: Khi ép kiểu, bạn cần phải đảm bảo rằng giá trị mới nằm trong phạm vi của kiểu dữ liệu mục tiêu, nếu không có thể xảy ra tràn trên (overflow) hoặc tràn dưới (underflow).
- Giữ nguyên ý nghĩa: Khi ép kiểu, bạn cần phải đảm bảo rằng ý nghĩa của dữ liệu vẫn được giữ nguyên.

Ví dụ 1: sử dụng từ khóa `static_cast`

Chương trình 3.9. Ép kiểu sử dụng từ khóa `static_cast`

```
#include <iostream>
using namespace std;
int main()
{
    int x = 10;
    double y = static_cast<double>(x); // Ép kiểu từ int sang double
    cout << "x = " << x << ", y = " << y << endl; // Output: x = 10, y = 10.0
    return 0;
}
```

Kết quả chương trình:

x = 10, y = 10.0

3.6. Gán nhiều và gán kết hợp

KHÁI NIỆM

1. Gán nhiều (Multiple Assignment): Gán nhiều là tính năng cho phép gán giá trị cho nhiều biến cùng một lúc. Ví dụ: `a = b = c = 10`; sẽ gán giá trị 10 cho cả ba biến a, b và c.
2. Gán kết hợp (Combined Assignment Operators): Gán kết hợp là các toán tử gán kết hợp với các phép toán số học. Ví dụ: `a += 5`; sẽ tăng giá trị của a lên 5.

Một số chú ý:

- Thứ tự gán: Khi sử dụng gán nhiều, thứ tự gán sẽ bắt đầu từ bên phải sang bên trái. Ví dụ: `a = b = c = 10`; sẽ gán 10 cho c trước, sau đó gán c cho b, cuối cùng gán b cho a.
- Kiểu dữ liệu: Khi sử dụng gán kết hợp, kiểu dữ liệu của biến bên trái phải tương thích với kiểu dữ liệu của biểu thức bên phải.
- Tính toán trước khi gán: Khi sử dụng gán kết hợp, phép toán sẽ được thực hiện trước khi gán giá trị mới cho biến.

Ví dụ 1: gán 1 giá trị cho nhiều biến

Chương trình 3.10. gán giá trị 10 cho 03 biến a, b, c

```
#include <iostream>
```

```
using namespace std;
int main()
{
    int a, b, c;
    a = b = c = 10; // Gán giá trị 10 cho cả a, b và c
    cout << "a = " << a << ", b = " << b << ", c = " << c << endl;
    // Output: a = 10, b = 10, c = 10
    return 0;
}
```

Kết quả chương trình:

a = 10, b = 10, c = 10

Trong ví dụ trên, sử dụng gán nhiều để gán giá trị 10 cho cả ba biến a, b và c. Lưu ý rằng, thứ tự gán sẽ bắt đầu từ bên phải sang bên trái, tức là c = 10 trước, sau đó b = c và cuối cùng a = b.

Ví dụ 2: phép gán kết hợp đơn giản

Chương trình 3.11. Phép gán kết hợp đơn giản

```
#include <iostream>
using namespace std;
int main() {
    int x = 10;
    x += 5; // Tương đương với x = x + 5
    cout << "x = " << x << endl; // Output: x = 15

    x -= 3; // Tương đương với x = x - 3
    cout << "x = " << x << endl; // Output: x = 12
    x *= 2; // Tương đương với x = x * 2
    cout << "x = " << x << endl; // Output: x = 24
    x /= 4; // Tương đương với x = x / 4
    cout << "x = " << x << endl; // Output: x = 6
    x %= 3; // Tương đương với x = x % 3
    cout << "x = " << x << endl; // Output: x = 0
    return 0;
}
```

Kết quả chương trình:

x = 15
x = 12
x = 24
x = 6
x = 0

Trong ví dụ này, sử dụng các toán tử gán kết hợp như +=, -=, *=, /= và %= để thực hiện các phép toán số học trên biến x. Lưu ý rằng, các phép toán sẽ được thực hiện trước khi gán giá trị mới cho biến.

Ví dụ 3: sử dụng gán kết hợp với biểu thức phức tạp

Chương trình 3.12. Sử dụng gán kết hợp với biểu thức phức tạp

```
#include <iostream>
using namespace std;
int main()
{
    int a = 10, b = 5, c = 3;
    a += b * c; // Tương đương với a = a + (b * c)
    cout << "a = " << a << endl; // Output: a = 25
    b -= a / c; // Tương đương với b = b - (a / c)
    cout << "b = " << b << endl; // Output: b = 20
    c *= a + b; // Tương đương với c = c * (a + b)
    cout << "c = " << c << endl; // Output: c = 900
    return 0;
}
```

Kết quả chương trình:

```
a = 25
b = -3
c = 66
```

Trong ví dụ này, sử dụng gán kết hợp với các biểu thức phức tạp hơn. Ví dụ, `a += b * c` sẽ tính `b * c` trước, sau đó cộng với `a` và gán lại cho `a`.

3.7. Định dạng đầu ra

KHÁI NIỆM: Định dạng đầu ra là quá trình sắp xếp và kiểu dữ liệu của dữ liệu được xuất ra trên màn hình hoặc các thiết bị đầu ra khác. C++ cung cấp nhiều cách để định dạng đầu ra, chủ yếu thông qua việc sử dụng các thao tác trên luồng đầu ra `cout`.

Một số chú ý

- Sử dụng các thao tác trên luồng đầu ra: Các thao tác định dạng đầu ra chủ yếu được thực hiện thông qua các thao tác trên luồng đầu ra `cout`.
- Sử dụng các thao tác định dạng: C++ cung cấp nhiều thao tác định dạng như `setw()`, `setprecision()`, `setfill()`, `setiosflags()`, v.v.
- Lưu ý về hiệu suất: Việc định dạng đầu ra quá nhiều có thể ảnh hưởng đến hiệu suất của chương trình, do đó cần cân nhắc khi sử dụng.

Ví dụ 1: Sử dụng `setw()` để định dạng cột

Chương trình 3.13. Sử dụng `setw()` để định dạng cột

```
#include <iostream>
#include <iomanip>
using namespace std;
int main() {
    int x = 123, y = 45, z = 6789;
    cout << setw(10) << "So" << setw(10) << "So" << setw(10) << "So" << endl;
```

```
cout << setw(10) << x << setw(10) << y << setw(10) << z << endl;
return 0;
}
```

Kết quả chương trình:

So	So	So
123	45	6789

Trong ví dụ 1, sử dụng setw() để thiết lập độ rộng của cột.

Ví dụ 2: Sử dụng setprecision() để định dạng số thực

Chương trình 3.14. Sử dụng setprecision() để định dạng số thực

```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    double x = 3.14159265358979;
    cout << "So thuc: " << x << endl;
    cout << "So thuc voi 4 chu so sau dau phay: " << setprecision(4) << x
    << endl;
    cout << "So thuc voi 8 chu so sau dau phay: " << setprecision(8) << x
    << endl;
    return 0;
}
```

Kết quả chương trình:

```
So thuc: 3.14159
So thuc voi 4 chu so sau dau phay: 3.142
So thuc voi 8 chu so sau dau phay: 3.1415927
```

Trong ví dụ 2, sử dụng setprecision() để thiết lập số lượng chữ số sau dấu phẩy khi xuất ra số thực.

Ví dụ 3: Sử dụng setprecision() và fixed / scientific để định dạng số thực

Chương trình 3.15. Sử dụng setprecision() và fixed / scientific để định dạng số thực

```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    double x = 1234.56789;
    cout << "So thuc (mac dinh): " << x << endl;
    cout << "So thuc (fixed): " << fixed << setprecision(2) << x << endl;
    cout << "So thuc (scientific): " << scientific << setprecision(4) << x
    << endl;
    return 0;
}
```

Kết quả chương trình:

So thuc (mac dinh): 1234.57
So thuc (fixed): 1234.57
So thuc (scientific): 1.2346e+03

Trong ví dụ 3, sử dụng `fixed` để hiển thị số thực ở định dạng số thập phân, và `scientific` để hiển thị số thực ở định dạng số khoa học. cũng sử dụng `setprecision()` để thiết lập số chữ số sau dấu phẩy.

3.8. Làm việc với đối tượng chuỗi và ký tự

KHÁI NIỆM: Có các hàm đặc biệt để làm việc với các ký tự và đối tượng chuỗi.

Mặc dù có thể sử dụng `cin` với toán tử `>>` để nhập chuỗi nhưng nó có thể gây ra các vấn đề mà bạn cần lưu ý. Khi `cin` đọc đầu vào, nó sẽ chuyển qua và bỏ qua mọi ký tự khoảng trắng ở đầu (dấu cách, tab hoặc ngắt dòng). Khi đến ký tự không trống đầu tiên và bắt đầu đọc, nó sẽ dừng đọc khi đến ký tự khoảng trắng tiếp theo. Chương trình 3-18 minh họa vấn đề này.

Chương trình 3.16. Vấn đề nhập chuỗi có khoảng cách với cin

```
// Chương trình này minh họa một vấn đề có thể xảy ra nếu
// cin được sử dụng để đọc dữ liệu ký tự vào đối tượng chuỗi.
#include <iostream>
#include <string>
using namespace std;
int main()
{
    string name;
    string city;
    cout << "Xin vui lòng nhập tên của bạn: ";
    cin >> name;
    cout << "Nhập thành phố bạn đang sống: ";
    cin >> city;
    cout << "Xin chào, " << name << endl;
    cout << "Bạn sống ở " << city << endl;
    return 0;
}
```

Kết quả chương trình:

Xin vui lòng nhập tên của bạn: Hai Phong
Nhập thành phố bạn đang sống: Xin chào, Hai
Bạn sống ở Phong

Lưu ý rằng người dùng không bao giờ có cơ hội nhập vào thành phố. Trong câu lệnh đầu vào đầu tiên, khi `cin` đến khoảng trống giữa Hai và Phong, nó ngừng đọc, chỉ lưu trữ Hai như giá trị của biến tên (`name`). Trong câu lệnh đầu vào thứ hai, `cin` sử dụng các ký tự còn lại nó được tìm thấy trong bộ đệm bàn phím và lưu trữ Phong dưới dạng giá trị của

thành phố (city). Để khắc phục sự cố này, bạn có thể sử dụng hàm C++ có tên getline. Hàm getline như sau, trong đó cin là luồng đầu vào chúng tôi đang đọc và inputLine là tên của đối tượng string để nhận đầu vào:

```
getline(cin, inputLine);
```

Chương trình 3.17. Chương trình sử dụng hàm getline

```
// Chương trình này minh họa một vấn đề có thể xảy ra nếu
// cin được sử dụng để đọc dữ liệu ký tự vào đối tượng chuỗi.
#include <iostream>
#include <string>
using namespace std;
int main()
{
    string name;
    string city;
    cout << " Xin vui lòng nhập tên của bạn: ";
    getline(cin, name);
    cout << " Nhập thành phố bạn đang sống: ";
    getline(cin, city);
    cout << " Xin chào, " << name << endl;
    cout << " Bạn sống ở " << city << endl;
    return 0;
}
```

Kết quả chương trình:

```
Xin vui lòng nhập tên của bạn: Hai Phong
Nhập thành phố bạn đang sống: Ha Noi
Xin chào, Hai Phong
Bạn sống ở Ha Noi
```

Sử dụng cin.ignore()

Việc sử dụng cin.ignore() là rất quan trọng trong trường hợp này, vì nó đảm bảo rằng ký tự newline không bị coi là kết thúc chuỗi khi nhập tên. Nếu không có cin.ignore(), thì khi nhập tên, chương trình sẽ dừng lại ngay sau ký tự newline đầu tiên, và phần còn lại của tên sẽ không được nhập vào.

Chương trình 3.18. Vấn đề xảy ra khi sử dụng getline() sau khi nhập số

```
#include <iostream>
#include <string>

using namespace std;

int main() {
    int number;
    string name;

    cout << "Nhập một số: ";
    cin >> number;
```

```

    cout << "Nhap ten cua ban: ";
    getline(cin, name);

    cout << "So: " << number << endl;
    cout << "Ten: " << name << endl;

    return 0;
}

```

Kết quả chương trình:

```

Nhap mot so: 9
Nhap ten cua ban: So: 9
Ten:

```

Khi chạy chương trình này, nó sẽ hoạt động như sau:

- Chương trình sẽ yêu cầu người dùng nhập một số nguyên.
- Sau khi người dùng nhập số và nhấn Enter, *cin >> number* sẽ lưu trữ số đó vào biến *number*.
- Tiếp theo, chương trình sẽ yêu cầu người dùng nhập một chuỗi.
- Tại đây sẽ xảy ra vấn đề: *getline(cin, name)* sẽ không nhận được chuỗi mà người dùng nhập, thay vào đó, nó sẽ lấy khoảng trắng (space) hoặc ký tự xuống dòng (newline) từ bộ đệm của *cin* và gán cho biến *name*.

Vì vậy, kết quả sẽ sai, như sau:

```

Nhap mot so: 9
Nhap ten cua ban: So: 9
Ten:

```

Như bạn thấy, chương trình không thể nhập được chuỗi như mong muốn.

Để khắc phục vấn đề này, cần sử dụng *cin.ignore()* để loại bỏ ký tự xuống dòng (newline) từ bộ đệm trước khi đọc chuỗi. Ví dụ:

*Chương trình 3.19. Sử dụng *cin.ignore()* để xử lý vấn đề của chương trình trên*

```

#include <iostream>
#include <string>

using namespace std;

int main() {
    int number;
    string name;

    cout << "Nhap mot so: ";
    cin >> number;

    // Loai bo ky tu newline con sot lai trong bo dem nhap

```

```

cin.ignore();

cout << "Nhap ten cua ban: ";
getline(cin, name);

cout << "So: " << number << endl;
cout << "Ten: " << name << endl;

return 0;
}

```

Kết quả chương trình:

```

Nhap mot so: 9
Nhap ten cua ban: Dau Hai Phong
So: 9
Ten: Dau Hai Phong

```

Toán tử và hàm thành viên của đối tượng chuỗi

- Toán tử gán =: Gán một chuỗi khác cho một chuỗi.
- Toán tử +: Nối hai chuỗi.
- Toán tử so sánh <, >, <=, >=, ==, !=: So sánh hai chuỗi.
- Hàm length(): Trả về độ dài của chuỗi.
- Hàm substr(start, length): Trả về một chuỗi con của chuỗi gốc.
- Hàm find(str, pos): Tìm vị trí của chuỗi str trong chuỗi gốc, bắt đầu từ vị trí pos.

3.9. Hàm trong thư viện toán học

KHÁI NIỆM: Thư viện cmath cung cấp một số hàm để thực hiện các phép toán phức tạp.

Ở phần trước của chương này, bạn đã học cách sử dụng hàm pow để nâng lũy thừa của một số. Thư viện C++ có nhiều hàm khác thực hiện các phép toán chuyên biệt. Các chức năng này rất hữu ích trong các chương trình khoa học và có mục đích đặc biệt. Bảng dưới đây cho thấy một số trong số này, mỗi cái đều yêu cầu `#include <cmath>`.

Hàm	Ví dụ	Mô tả
abs	y = abs(x);	Trả về giá trị tuyệt đối của đối số. Đối số và giá trị trả về là số nguyên.
cos	y = cos(x);	Trả về cosine của đối số. Đối số nên là một góc được biểu diễn trong radian. Kiểu trả về và đối số là kiểu double.

Hàm	Ví dụ	Mô tả
exp	$y = \exp(x);$	Tính giá trị mũ của đối số, là x. Kiểu trả về và đối số là kiểu double.
fmod	$y = \text{fmod}(x, z);$	Trả về, dưới dạng một số thực (double), phần dư của đối số thứ nhất chia cho đối số thứ hai. Hoạt động tương tự như toán tử modulus, nhưng các đối số là kiểu double. (Toán tử modulus chỉ hoạt động với số nguyên.) Hãy chú ý không truyền số không làm đối số thứ hai. Làm như vậy sẽ gây ra phép chia cho không.
log	$y = \log(x);$	Trả về logarithm tự nhiên của đối số. Kiểu trả về và đối số là kiểu double. $\log_{10} y = \log_{10}(x);$ Trả về logarithm cơ số 10 của đối số. Kiểu trả về và đối số là kiểu double.
round	$y = \text{round}(x)$	Đối số, x, có thể là một double, một float, hoặc một long double. Trả về giá trị của x được làm tròn đến số nguyên gần nhất. Ví dụ, nếu x là 2.8, hàm trả về 3.0, hoặc nếu x là 2.1, hàm trả về 2.0. Kiểu trả về là giống với kiểu của đối số.
sin	$y = \sin(x);$	Trả về sine của đối số. Đối số nên là một góc được biểu diễn trong radian. Kiểu trả về và đối số là kiểu double.
sqrt	$y = \text{sqrt}(x);$	Trả về căn bậc hai của đối số. Kiểu trả về và đối số là kiểu double. $\tan y = \tan(x);$ Trả về tangent của đối số. Đối số nên là một góc được biểu diễn trong radian. Kiểu trả về và đối số là kiểu double.

Mỗi hàm này đều dễ sử dụng như hàm pow. Đoạn chương trình sau đây minh họa hàm sqrt, trả về căn bậc hai của một số:

```
cout << "Nhập vào 1 số: ";
cin >> num;
s = sqrt(num);
cout << "Căn bậc 2 của " << num << " là " << s << endl;
```

Kết quả

Nhập vào 1 số: 25

Căn bậc 2 của 25 là 5

Ví dụ 2:

```
/*Chương trình này hỏi độ dài hai cạnh của một hình tam giác vuông. Khi đó
độ dài cạnh huyền là tính toán và hiển thị.*/
```

```
#include <iostream>
#include <iomanip> // Ham thiết lập do chính xác
#include <cmath> // Ham sqrt và pow
using namespace std;
int main()
{
    double a, b, c;

    cout << "Nhập do dài cạnh a: ";
    cin >> a;
    cout << "Nhập do dài cạnh b: ";
    cin >> b;
    c = sqrt(pow(a, 2.0) + pow(b, 2.0));
    cout << " Do dài cạnh huyền là ";
    cout << setprecision(2) << c << endl;
    return 0;
}
```

Kết quả chương trình:

```
Nhập do dài cạnh a: 5
Nhập do dài cạnh b: 6
Do dài cạnh huyền là 7.8
```

Số ngẫu nhiên

Số ngẫu nhiên rất hữu ích cho nhiều nhiệm vụ lập trình khác nhau. Sau đây chỉ là một vài ví dụ:

- Số ngẫu nhiên thường được sử dụng trong trò chơi. Ví dụ: trò chơi máy tính cho phép người chơi tung xúc xắc sử dụng các số ngẫu nhiên để biểu thị giá trị của xúc xắc.
- Các chương trình hiển thị các lá bài được rút từ một bộ bài được xáo trộn sử dụng các số ngẫu nhiên để thể hiện mệnh giá của các lá bài.
- Số ngẫu nhiên rất hữu ích trong các chương trình mô phỏng. Trong một số mô phỏng, máy tính phải quyết định ngẫu nhiên cách hành xử của con người, động vật, côn trùng hoặc sinh vật khác. Các công thức có thể được xây dựng trong đó một số ngẫu nhiên được sử dụng để xác định các hành động và sự kiện khác nhau diễn ra trong chương trình.
- Số ngẫu nhiên rất hữu ích trong các chương trình thống kê phải chọn ngẫu nhiên dữ liệu để phân tích.
- Số ngẫu nhiên thường được sử dụng trong bảo mật máy tính để mã hóa dữ liệu nhạy cảm.

Thư viện C++ có một hàm rand() mà bạn có thể sử dụng để tạo số ngẫu nhiên. Hàm rand() yêu cầu #include<cstdlib>. Số được hàm trả về là số nguyên. Đây là một ví dụ về cách sử dụng của nó:

```
y = rand();
```

Sau khi câu lệnh này được thực thi, biến `y` sẽ chứa một số ngẫu nhiên. Trên thực tế, các số được tạo bởi `rand()` là giả ngẫu nhiên. Hàm sử dụng thuật toán tạo ra cùng một chuỗi số mỗi lần chương trình được lặp lại trên cùng một hệ thống. Ví dụ: giả sử các câu lệnh sau được thực thi:

```
cout << rand() << endl;
cout << rand() << endl;
cout << rand() << endl;
```

Ba số được hiển thị sẽ có vẻ ngẫu nhiên, nhưng mỗi khi chương trình chạy, ba giá trị giống nhau sẽ được tạo ra. Để ngẫu nhiên hóa kết quả của `rand()`, phải sử dụng hàm `srand()`. `srand()` chấp nhận một đối số `unsigned int`, hoạt động như một giá trị gốc cho thuật toán. Bằng cách chỉ định các giá trị hạt giống khác nhau, `rand()` sẽ tạo ra các chuỗi số ngẫu nhiên khác nhau.

Một cách phổ biến để nhận được các giá trị hạt giống duy nhất là gọi hàm thời gian, là một phần của thư viện chuẩn. Hàm `time` trả về số giây đã trôi qua kể từ nửa đêm ngày 1 tháng 1 năm 1970. Hàm `time` yêu cầu `#include<ctime>` và bạn chuyển 0 làm đối số cho hàm. Chương trình sau chứng minh điều này. Chương trình sẽ tạo ra ba số ngẫu nhiên khác nhau mỗi lần nó được thực thi.

```
// Chương trình này minh họa các số ngẫu nhiên.
#include <iostream>
#include <cstdlib> // Đối với rand và srand
#include <ctime> // Đối với hàm thời gian
using namespace std;

int main()
{
    // Lấy thời gian của hệ thống.
    unsigned seed = time(0);
    // Khởi tạo trình tạo số ngẫu nhiên.
    srand(seed);
    // Hiển thị ba số ngẫu nhiên.
    cout << rand() << endl;
    cout << rand() << endl;
    cout << rand() << endl;
    return 0;
}
```

Kết quả chương trình:

```
22461
14641
22314
```

3.10. Câu hỏi và Bài tập

3.10.1. Câu hỏi

1. Giả sử một chương trình có các khai báo biến sau:

```
int units;  
float mass;  
double weight;  
và câu lệnh sau:  
weight = mass * units;
```

Chuyển đổi kiểu dữ liệu tự động nào sẽ xảy ra?

- A) mass bị hạ cấp xuống int, units vẫn là int, và kết quả của mass * units là int.
- B) units được nâng cấp lên float, mass vẫn là float, và kết quả của mass * units là float.
- C) units được nâng cấp lên float, mass vẫn là float, và kết quả của mass * units là double.

2. Giả sử một chương trình có các khai báo biến sau:

```
int a, b = 2;  
float c = 4.2;
```

và câu lệnh sau:

```
a = b * c;
```

Giá trị nào sẽ được lưu trong a?

- A) 8.4
- B) 8
- C) 0
- D) Không có đáp án nào trên

3. Giả sử qty và salesReps đều là các số nguyên (int). Sử dụng biểu thức ép kiểu để viết lại câu lệnh sau để nó không còn thực hiện phép chia nguyên nữa.

```
unitsEach = qty / salesReps;
```

4. Viết lại khai báo biến sau để biến trở thành một hằng có tên.

```
int rate;
```

5. Viết một câu lệnh gán đa trị có thể được sử dụng thay thế cho nhóm các câu lệnh gán sau:

```
east = 1;  
west = 1;  
north = 1;  
south = 1;
```

6. Viết một câu lệnh cout để hiển thị biến divSales trong một trường rộng 8 khoảng trắng, ở dạng số cố định, với độ chính xác 2 chữ số thập phân. Dấu chấm thập phân nên luôn được hiển thị.

7. Viết một câu lệnh cout để hiển thị biến totalAge trong một trường rộng 12 khoảng trắng, ở dạng số cố định, với độ chính xác 4 chữ số thập phân.

8. Một biến velocity có giá trị 180.67245612456. Viết một câu lệnh cout để hiển thị nó trong một trường rộng 6 khoảng trắng, căn trái. Viết một câu lệnh cout khác để hiển thị nó trong 18 khoảng trắng với độ chính xác 12 chữ số thập phân, ở dạng số cố định, với các số 0 thừa.

3.10.2. Bài tập

1. Số dặm trên mỗi Gallon (Bắt buộc)

Viết chương trình tính toán mức tiêu thụ xăng của ô tô. Chương trình này sẽ yêu cầu người dùng nhập số lít xăng của bình xăng có thể chứa, và nhập số km mà xe có thể đi được khi được đổ đầy bình. Sau đó nó sẽ hiển thị số km có thể được lái đi trên mỗi lít xăng.

2. Vị trí ngồi trên sân vận động

Có 3 loại ghế ngồi trên một sân vận động. Với một môn bóng, các ghế hạng A với giá 15 đô la, các ghế hạng B có giá 12 đô la, và các ghế hạng C có giá là 9 đô la. Viết chương trình yêu cầu người dùng nhập số lượng vé cho mỗi hạng ghế và hiển thị tổng số tiền thu được từ các vé được bán ra. Hãy định dạng số tiền ở dạng kí hiệu fixed-point, với độ chính xác 2 số thập phân đằng sau dấu phẩy, và luôn hiển thị dấu phẩy thập phân trong mọi trường hợp.

3. Điểm trung bình của bài kiểm tra (Bắt buộc)

Viết một chương yêu cầu nhập điểm của 5 bài kiểm tra. Chương trình thực hiện tính toán giá trị trung bình của 5 bài kiểm tra này sau đó hiển thị lên màn hình. Số được hiển thị phải được định dạng theo kí hiệu fixed-point, với độ chính xác là 1 đằng sau dấu phẩy.

4. Lượng mưa trung bình

Viết một chương trình tính toán lượng mưa trung bình trong 3 tháng. Chương trình yêu cầu người dùng nhập vào tên của mỗi tháng, như là June, July; và lượng mưa (đơn vị đo là inches) của mỗi tháng. Sau đó chương trình hiển thị ra thông báo tương tự như sau:

“Lượng mưa trung bình của tháng June, July, và August là 6.72 inches.”.

5. Tỷ lệ nam và nữ

Viết một chương trình yêu cầu người dùng nhập vào số lượng người nam và số lượng người nữ của một lớp. Chương trình sẽ hiển thị tỉ lệ phần trăm của nam và nữ ra màn hình.

Gợi ý: Giả sử có 8 nam và 12 nữ trong một lớp. Do đó tổng số của lớp là 20. Tỷ lệ phần trăm của nam có thể được tính bằng công thức $8 : 20 = 0.4$ hoặc 40 phần trăm. Tỷ lệ phần trăm của nữ có thể được tính bằng công thức $12 : 20 = 0.6$ hoặc 60 phần trăm.

6. Điều chỉnh các thành phần (Bắt buộc)

Công thức làm bánh với các thành phần như sau:

- 1.5 cốc đường
- 1 cốc bơ
- 2.75 cốc bột

Với các phần như trên cho ra 48 chiếc bánh. Bạn hãy viết một chương trình yêu cầu người dùng nhập số lượng chiếc bánh mà họ muốn làm, sau đó hiển thị ra tỷ lệ của các thành phần theo công thức trên để đủ làm được số chiếc bánh mà họ muốn.

7. Rạp chiếu phim

Một rạp chiếu phim được giữ lại 20% tổng doanh thu thu được từ việc bán vé. Phần còn lại chuyển cho nhà phân phối phim. Viết chương trình tính số tiền cho phòng vé và nhà phân phối. Chương trình cần yêu cầu nhập tên phim, số vé cho người lớn và số vé cho trẻ em đã được bán.

Biết:

- Giá vé cho người lớn là 10 đô la;
- Giá vé cho trẻ em là 6 đô la.

8. Có bao nhiêu vật dụng?

Công ty Yukon Widget sản xuất các vật dụng nặng 12,5 pound mỗi vật. Viết chương trình tính toán có bao nhiêu vật dụng được xếp trên một tấm bảng pallet, dựa trên tổng trọng lượng của pallet. Chương trình sẽ hỏi người dùng xem pallet nặng bao nhiêu và trọng lượng của pallet với các vật dụng được xếp chồng lên nhau. Sau đó, nó sẽ tính toán và hiển thị số lượng vật dụng được xếp chồng lên nhau trên pallet.

9. Bao nhiêu Calories?

Một túi bánh có 30 chiếc bánh. Thông tin về calorie trên túi ghi rằng 10 khẩu phần trong túi và một khẩu phần tương đương với 300 calories. Viết chương trình yêu cầu người dùng nhập số lượng bánh mà họ đã ăn, sau đó in ra thông báo tổng số calories đã tiêu thụ.

10. Bảo hiểm bao nhiêu?

Nhiều chuyên gia tài chính khuyên rằng chủ sở hữu bất động sản nên bảo hiểm ngôi nhà hoặc tòa nhà của họ ít nhất 80% số tiền mà nó sẽ chi phí để thay thế cấu trúc. Viết một chương trình yêu cầu người dùng nhập chi phí thay thế của một tòa nhà, sau đó hiển thị số tiền bảo hiểm tối thiểu mà họ nên mua cho tài sản đó.

11. Chi phí ô tô (Bắt buộc)

Viết một chương trình yêu cầu người dùng nhập chi phí hàng tháng cho các chi phí sau đây phát sinh từ việc vận hành ô tô của họ: thanh toán khoản vay, bảo hiểm, xăng, dầu, lốp và bảo dưỡng. Sau đó, chương trình sẽ hiển thị tổng chi phí hàng tháng của các chi phí này và tổng chi phí hàng năm của các chi phí này.

12. Celsius to Fahrenheit (Bắt buộc)

Viết một chương trình chuyển đổi giá trị giữa hai đơn vị đo nhiệt độ là Celsius và Fahrenheit. Với công thức chuyển đổi là

$$F = \frac{9}{5}C + 32$$

Trong đó F là nhiệt độ trong đơn vị Fahrenheit, và C là nhiệt độ trong đơn vị Celsius.

13. Currency

Viết chương trình chuyển đổi giá trị của U.S. dollar sang Japanese yen và euros, lưu trữ hệ số chuyển đổi trong các hằng số YEN_PER_DOLLAR và EUROS_PER_DOLLAR. Để có được tỉ lệ chuyển đổi được cập nhật, hãy tìm trên Internet với từ khóa “currency exchange rate”. Nếu bạn không thể tìm được tỉ lệ chuyển đổi trong thời gian gần đây nhất thì có thể sử dụng tỉ lệ sau:

- 1 Dollar = 98.93 Yen
- 1 Dollar = 0.74 Euros

Hãy định dạng kết quả ở dạng kí hiệu fixed-point, với độ chính xác 2 số thập phân sau dấu phẩy, và luôn để dấu phẩy thập phân được hiển thị.

14. Thuế bán hàng theo tháng

Một công ty bán lẻ phải nộp một báo cáo thuế bán hàng hàng tháng liệt kê doanh thu trong tháng và số thuế bán hàng đã thu được. Viết một chương trình yêu cầu nhập tháng, năm và tổng số tiền thu được tại máy tính tiền (tức là doanh thu cộng với thuế bán hàng). Giả sử thuế bán hàng của tiểu bang là 4 phần trăm, và thuế bán hàng của quận là 2 phần trăm.

Nếu biết tổng số tiền thu được và tổng số thuế bán hàng là 6 phần trăm, thì số tiền bán sản phẩm có thể được tính như sau:

$$S = \frac{T}{1.06}$$

Trong đó, S là doanh thu bán sản phẩm, và T là tổng thu nhập (doanh thu bán sản phẩm cộng với thuế bán hàng).

Một báo cáo của chương trình có thể tương tự như dưới đây:

Month: October

Total Collected: \$ 26572.89
Sales: \$ 25068.76
County Sales Tax: \$ 501.38
State Sales Tax: \$ 1002.75
Total Sales Tax: \$ 1504.13

15. Thuế bất động sản

Một quận thu thuế bất động sản trên giá trị đánh giá của tài sản, bằng 60 phần trăm giá trị thực của tài sản. Nếu một mẫu đất được định giá là 10.000 đô la, thì giá trị đánh giá của nó là 6.000 đô la. Thuế tài sản sau đó là 75 ¢ cho mỗi \$ 100 của giá trị đánh giá. Thuế cho diện tích được đánh giá là \$ 6.000 sẽ là \$ 45. Viết chương trình yêu cầu nhập giá trị thực của một phần tài sản, sau đó hiển thị giá trị đánh giá và thuế tài sản.

(1 đô la = 100 cent)

16. Thuế tài sản công dân cao tuổi

Quận Madison cung cấp một khoản miễn thuế chủ nhà \$ 5,000 cho những người cao tuổi của nó. Ví dụ: nếu ngôi nhà của một người cao tuổi được định giá là 158.000 đô la, giá trị được đánh giá của nó sẽ là 94.800 đô la, như đã giải thích ở trên. Tuy nhiên, anh ta sẽ chỉ trả thuế trên 89.800 đô la. Với mức thuế năm ngoái là 2,64 đô la cho mỗi 100 đô la giá trị được đánh giá, thuế tài sản sẽ là 2.370,72 đô la. Ngoài việc giảm thuế, công dân cao tuổi được phép trả thuế tài sản của họ thành bốn lần như nhau. Khoản thanh toán hàng quý đến hạn cho tài sản này sẽ là \$ 592,68. Viết một chương trình yêu cầu người dùng nhập giá trị thực của một phần tài sản và thuế suất hiện hành cho mỗi \$ 100 giá trị được đánh giá. Sau đó, chương trình sẽ tính toán và báo cáo mức thuế bất động sản hàng năm mà một chủ nhà cao cấp sẽ phải trả cho bất động sản này và hóa đơn thuế hàng quý sẽ là bao nhiêu.

17. Gia sư toán

Viết một chương trình có thể sử dụng như một gia sư về toán cho các bạn nhỏ. Chương trình sẽ hiển thị hai số ngẫu nhiên để thực hiện phép cộng, ví dụ:

$$\begin{array}{r} 247 \\ +129 \\ \hline \end{array}$$

Sau đó chương trình sẽ dừng lại trong khi học sinh thực hiện việc tính toán. Khi học sinh đã sẵn sàng để kiểm tra đáp án, họ có thể nhấn một phím bất kỳ để chương trình hiển thị kết quả chính xác.

$$\begin{array}{r} 247 \\ +129 \\ \hline 376 \end{array}$$

18. Lãi suất kiếm được

Giả sử không có khoản tiền gửi nào ngoài khoản đầu tư ban đầu, số dư trong tài khoản tiết kiệm sau một năm có thể được tính theo công thức

$$\text{Amount} = \text{Principal} * (1 + \frac{\text{Rate}}{T})^T$$

Trong đó, Principal là số dư trong tài khoản tiết kiệm, Rate là lãi suất, và T là số lần tính lãi gộp trong một năm (T là 4 nếu lãi gộp hàng quý).

Viết một chương trình yêu cầu tiền gốc, lãi suất và số lần lãi gộp. Nó sẽ hiển thị một báo cáo tương tự như:

Interest Rate:	4.25%
Times Compounded:	12
Principal:	\$ 1000.00
Interest:	\$ 43.34
Amount in Savings:	\$ 1043.34

19. Thanh toán hàng tháng

Khoản thanh toán hàng tháng cho một khoản vay có thể được tính theo công thức sau:

$$\text{Payment} = \frac{\text{Rate} * (1 + \text{Rate})^N}{((1 + \text{Rate})^N - 1)} * L$$

Trong đó, Rate là lãi suất hàng tháng, được tính bằng lãi suất hàng năm chia cho 12. (12% lãi suất hàng năm sẽ là 1% lãi suất hàng tháng.) N là số lần thanh toán và L là số tiền của khoản vay.

Hãy viết một chương trình yêu cầu nhập vào các giá trị này và hiển thị báo cáo dưới dạng tương tự như sau:

Loan Amount:	\$ 10000.00
Monthly Interest Rate:	1%
Number of Payments:	36
Monthly Payment:	\$ 332.14
Amount Paid Back:	\$ 11957.15
Interest Paid:	\$ 1957.15

20. Pizza Pi

Tiệm Pizza của Joe đang cần một chương trình tính toán số lượng miếng cắt của một chiếc pizza có kích cỡ bất kỳ. Chương trình thực hiện các bước như sau:

- A. Yêu cầu người dùng nhập vào đường kính của chiếc pizza (đơn vị là inches).
- B. Tính toán số lượng miếng cắt có thể chia ra từ chiếc pizza với kích cỡ đó.
- C. Hiển thị một thông báo rằng số lượng miếng cắt nên được thực hiện.

Để tính toán được số lượng miếng cắt hợp lý, bạn cần biết một số thông tin sau:

- Mỗi miếng cắt cần phải có diện tích 14.125 inches.
- Để tính toán số lượng miếng cắt, đơn giản ta chia diện tích của chiếc pizza cho 14.125.
- Diện tích của chiếc pizza được tính theo công thức: $Area = \pi r^2$

Hãy đảm bảo rằng output của chương trình hiển thị chính xác số lượng các miếng cắt, và nó ở định dạng kí hiệu fixed-point, làm tròn đến số đầu tiên đằng sau dấu phẩy. Nên sử dụng pi như một hằng số.

21. Bao nhiêu chiếc Pizza?

Chỉnh sửa chương trình bạn đã viết trong Programming Challenge 20 (Pizza Pi) để báo cáo số lượng pizza bạn cần mua cho một bữa tiệc nếu mỗi người tham dự có khả năng sẽ ăn khoảng 4 miếng cắt. Chương trình cần yêu cầu người dùng nhập vào số lượng người tham dự bữa tiệc, và đường kính loại pizza sẽ được gọi mua. Sau đó tính toán và hiển thị số lượng pizza cần mua lên màn hình.

22. Tính góc

Viết một chương trình yêu cầu người dùng nhập vào một góc, đơn vị đo radians. Chương trình sẽ hiển thị các giá trị sine, cosine, và tangent của góc đó. (Sử dụng hàm sin, cos, và tan trong thư viện có sẵn để xác định các giá trị này.) Output sẽ hiển thị các số dạng kí hiệu fixed-point, làm tròn đến 4 chữ số đằng sau dấu phẩy.

23. Chương trình giao dịch chứng khoán

Chương trình giao dịch cổ phiếu

Tháng trước Joe đã mua một số cổ phiếu tại Acme Software, Inc. Dưới đây là chi tiết của việc mua:

- Số lượng cổ phiếu mà Joe mua là 1.000.
- Khi Joe mua cổ phiếu, anh ta trả 45,50 đô la cho mỗi cổ phiếu.
- Joe đã trả cho người môi giới chứng khoán của mình một khoản hoa hồng lên tới 2% số tiền anh ta trả cho cổ phiếu.

Hai tuần sau, Joe bán cổ phiếu. Dưới đây là chi tiết của đợt giảm giá:

- Số cổ phiếu mà Joe đã bán là 1.000.

- Anh đã bán cổ phiếu với giá 56,90 USD / cổ phiếu.
- Anh ta đã trả cho người môi giới chứng khoán của mình một khoản hoa hồng khác lên tới 2% số tiền anh ta nhận được cho cổ phiếu.

Viết chương trình hiển thị thông tin sau:

- Số tiền Joe đã trả để mua cổ phiếu.
- Số tiền hoa hồng Joe trả cho người môi giới của mình khi anh ta mua cổ phiếu.
- Số tiền mà Joe đã có được khi bán cổ phiếu
- Số tiền hoa hồng Joe đã trả cho người môi giới của mình khi anh ta bán cổ phiếu.

Hiển thị số lợi nhuận mà Joe kiếm được sau khi bán cổ phiếu của mình và trả hai khoản hoa hồng cho người môi giới của anh ấy. (Nếu số lợi nhuận mà chương trình của bạn hiển thị là một số âm, thì Joe đã mất tiền trong giao dịch.)

24. Trồng nho

Một chủ vườn nho đang trồng một số hàng nho mới và cần biết có bao nhiêu cây nho để trồng trong mỗi hàng. Cô ấy đã xác định rằng sau khi đo chiều dài của một hàng trong tương lai, cô ấy có thể sử dụng công thức sau để tính số lượng dây leo sẽ phù hợp với hàng đó, cùng với các cụm cuối của giàn sẽ cần được xây dựng ở mỗi đầu của hàng:

$$V = \frac{R - 2E}{S}$$

Các thuật ngữ trong công thức là:

- V là số nho sẽ phù hợp trong hàng.
- R là chiều dài của hàng, tính bằng feet.
- E là khoảng trống, tính bằng feet, được sử dụng bởi một cụm cuối trụ.
- S là khoảng cách giữa các dây leo, tính bằng feet.

Viết chương trình thực hiện phép tính cho chủ vườn nho. Chương trình sẽ yêu cầu người dùng nhập các thông tin sau:

- Chiều dài của hàng, tính bằng feet
- Khoảng không gian được sử dụng bởi một cụm cuối bài, tính bằng feet
- Khoảng cách giữa các dây leo, tính bằng feet
- Sau khi dữ liệu đầu vào đã được nhập, chương trình sẽ tính toán và hiển thị số lượng nho sẽ phù hợp trong hàng.

25. Word Game

Viết một chương trình chơi chữ với người dùng. Chương trình sẽ thực hiện hỏi những người dùng những thông tin sau đây:

- Tên của người chơi
- Tuổi của người chơi
- Tên thành phố
- Tên trường đại học
- Lĩnh vực chuyên môn
- Loại động vật
- Tên thú cưng

Sau khi người dùng nhập vào những thông tin này, chương trình sẽ hiển thị một câu truyện như sau, với việc chèn các thông tin của người dùng vào các vị trí tương ứng thích hợp:

There once was a person named **NAME** who lived in **CITY**. At the age of **AGE**, **NAME** went to college at **COLLEGE**. **NAME** graduated and went to work as a **PROFESSION**. Then, **NAME** adopted a(n) **ANIMAL** named **PETNAME**. They both lived happily ever after!

CHƯƠNG 4. CẤU TRÚC RỄ NHÁNH

Chủ đề	
4.1	Toán tử quan hệ
4.2	Câu lệnh if khuyết
4.3	Câu lệnh if/else
4.4	Câu lệnh if lồng nhau
4.5	Câu lệnh if/else if
4.6	Biến cờ (Flags)
4.7	Toán tử logic
4.8	Xác thực đầu vào
4.9.	So sánh chuỗi và các kí tự
4.10	Toán tử điều kiện ?
4.11	Câu lệnh switch
4.12	Thực đơn

4.1. Toán tử quan hệ

KHÁI NIỆM: Toán tử quan hệ là các toán tử được sử dụng để so sánh hai giá trị hoặc biểu thức. Kết quả của phép so sánh sẽ là một giá trị logic (true hoặc false).

Liệt kê các toán tử quan hệ và giải thích

Kí hiệu	Đọc	Giải thích
<	nhỏ hơn	Trả về true nếu giá trị bên trái nhỏ hơn giá trị bên phải.
>	lớn hơn	Trả về true nếu giá trị bên trái lớn hơn giá trị bên phải.
<=	nhỏ hơn hoặc bằng	Trả về true nếu giá trị bên trái nhỏ hơn hoặc bằng giá trị bên phải.
>=	lớn hơn hoặc bằng	Trả về true nếu giá trị bên trái lớn hơn hoặc bằng giá trị bên phải.
==	bằng	Trả về true nếu giá trị bên trái bằng giá trị bên phải.
!=	khác	Trả về true nếu giá trị bên trái không bằng giá trị bên phải.

Kết quả của phép so sánh sử dụng các toán tử quan hệ sẽ là một giá trị logic:

- true (đúng): Biểu diễn bằng số 1.
- false (sai): Biểu diễn bằng số 0.

Chú ý:

- Kiểu dữ liệu: Các toán tử quan hệ có thể được sử dụng để so sánh các kiểu dữ liệu số (int, float, double) và các kiểu dữ liệu khác như char.

- So sánh chuỗi: Khi so sánh chuỗi, toán tử quan hệ sẽ so sánh từng ký tự trong chuỗi theo thứ tự từ điển.
- Ưu tiên toán tử: Các toán tử quan hệ có độ ưu tiên thấp hơn các toán tử số học, do đó chúng thường được sử dụng trong các biểu thức phức tạp hơn.

Ví dụ 1: Chương trình này hiển thị các giá trị của trạng thái đúng và sai.

Chương trình 4.1. Chương trình này hiển thị các giá trị của trạng thái đúng và sai.

```
// Chương trình này hiển thị các giá trị của trạng thái đúng và sai.
#include <iostream>
using namespace std;
int main()
{
    bool trueValue, falseValue;
    int x = 5, y = 10;
    trueValue = x < y;
    falseValue = y == x;
    cout << "True is " << trueValue << endl;
    cout << "False is " << falseValue << endl;
    return 0;
}
```

Kết quả chương trình:

True is 1
False is 0

Ví dụ 2: (Giả sử x là 10, y là 7 và z, a và b là số nguyên hoặc bool.)

Câu lệnh	Đầu ra
<code>z = x < y</code>	z được gán 0 vì x không nhỏ hơn y.
<code>cout << (x > y);</code>	Hiển thị 1 vì x lớn hơn y.
<code>a = x >= y;</code>	a được gán 1 vì x lớn hơn hoặc bằng y.
<code>cout << (x <= y);</code>	Hiển thị 0 vì x không nhỏ hơn hoặc bằng y.
<code>b = y != x;</code>	b được gán 1 vì y không bằng x.

4.2. Câu lệnh if khuyết

KHÁI NIỆM: Câu lệnh if không có else (if khuyết) là một dạng câu lệnh điều kiện trong lập trình C++. Câu lệnh này cho phép kiểm tra một điều kiện logic và thực hiện một hành động tương ứng nếu điều kiện đó đúng. Điểm đặc biệt của câu lệnh if khuyết là chỉ thực hiện hành động khi điều kiện đúng, không có hành động nào được thực hiện khi điều kiện sai.

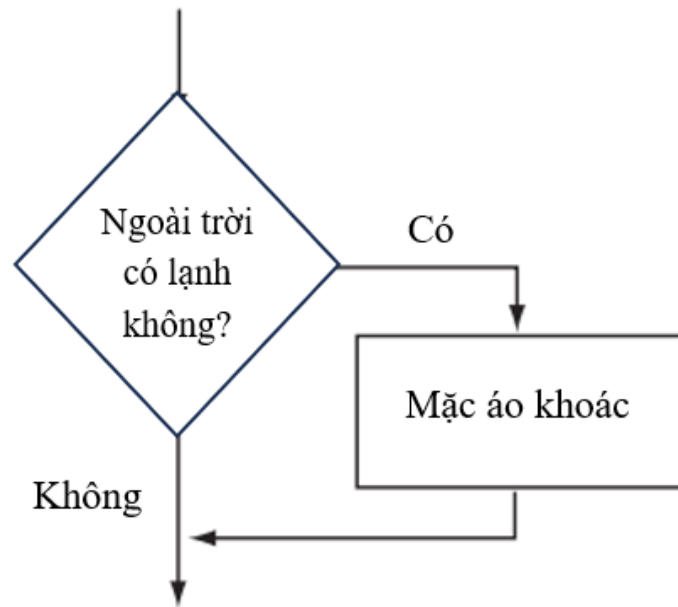
Cú pháp

```
if (điều_kiện) {  
    // Khối lệnh được thực hiện nếu điều_kiện đúng  
}
```

Trong đó:

- điều_kiện: là một biểu thức logic (có giá trị true hoặc false)
- Khối lệnh được đặt trong cặp dấu {} và chỉ được thực hiện khi điều_kiện đúng.

Trong sơ đồ luồng sau, hành động "Mặc áo khoác" chỉ được thực hiện khi trời lạnh ở bên ngoài. Nếu không lạnh ở ngoài, hành động sẽ bị bỏ qua. Hành động được thực thi có điều kiện vì nó chỉ được thực hiện khi có một điều kiện nhất định (trời lạnh ở ngoài). Hình 4-1 cho thấy một sơ đồ luồng chi tiết hơn, nơi ba hành động chỉ được thực hiện khi trời lạnh ở ngoài.



Hình 4.1. Cấu trúc quyết định

Tương tự, dưới đây là một số ví dụ khác:

Nếu xe đang thiếu xăng, hãy dừng lại tại một trạm dịch vụ và đổ xăng.

Nếu trời đang mưa ở ngoài, hãy vào bên trong.

Nếu bạn đang đói, hãy lấy thứ gì đó để ăn.

Chú ý

- Nếu khối lệnh chỉ chứa một câu lệnh, thì có thể bỏ dấu {}.
- Điều kiện trong câu lệnh if có thể là bất kỳ biểu thức logic nào, bao gồm các toán tử so sánh, logic,...

- Câu lệnh if có thể được lồng nhau (if bên trong if) để kiểm tra nhiều điều kiện.

Ví dụ 1: Kiểm tra số âm, dương

Chương trình 4.2. Kiểm tra một số là số âm hay số dương

```
#include <iostream>
using namespace std;
int main()
{
    int so = -5;
    if (so > 0) {
        cout << "So " << so << " la so duong" << endl;
    }
    if (so < 0) {
        cout << "So " << so << " la so am" << endl;
    }
}
```

Kết quả chương trình:

So -5 la so am

Giải thích:

- Chương trình khai báo biến số có giá trị -5.
- Câu lệnh if đầu tiên kiểm tra xem số có lớn hơn 0 không. Do số = -5 nên điều kiện số > 0 sai, nên khối lệnh bên trong không được thực hiện.
- Câu lệnh if thứ hai kiểm tra xem số có nhỏ hơn 0 không. Do số = -5 nên điều kiện số < 0 đúng, nên khối lệnh bên trong được thực hiện, in ra thông báo "Số -5 là số âm".

Ví dụ 2: Kiểm tra số chẵn, lẻ

Chương trình 4.3. Kiểm tra một số là số chẵn hay số lẻ

```
#include <iostream>
using namespace std;
int main()
{
    int so = 8;
    if (so % 2 == 0) {
        cout << "So " << so << " la so chan" << endl;
    }
    if (so % 2 != 0) {
        cout << "So " << so << " la so le" << endl;
    }
}
```

Kết quả chương trình:

So 8 la so chan

Giải thích:

- Chương trình khai báo biến so có giá trị 8.
- Câu lệnh if đầu tiên kiểm tra xem so chia hết cho 2 hay không (giá trị dư là 0). Do $so = 8$ nên điều kiện $so \% 2 == 0$ đúng, nên khối lệnh bên trong được thực hiện, in ra thông báo "Số 8 là số chẵn".
- Câu lệnh if thứ hai kiểm tra xem so chia cho 2 có dư hay không (giá trị dư khác 0). Do $so = 8$ nên điều kiện $so \% 2 != 0$ sai, nên khối lệnh bên trong không được thực hiện.

Kiểm tra

1. Viết chương trình nhập vào một số nguyên, in ra "Số âm" nếu số đó là số âm, "Số dương" nếu số đó là số dương.
2. Viết chương trình nhập vào một số nguyên, in ra "Số chẵn" nếu số đó là số chẵn, "Số lẻ" nếu số đó là số lẻ.
3. Viết chương trình nhập vào một số nguyên, in ra "Số chia hết cho 3" nếu số đó chia hết cho 3, "Số không chia hết cho 3" nếu số đó không chia hết cho 3.

4.3. Câu lệnh if/else

KHÁI NIỆM: Câu lệnh if/else là một dạng câu lệnh điều kiện trong lập trình C++. Câu lệnh này cho phép kiểm tra một điều kiện logic và thực hiện một hành động tương ứng nếu điều kiện đó đúng, hoặc thực hiện một hành động khác nếu điều kiện sai.

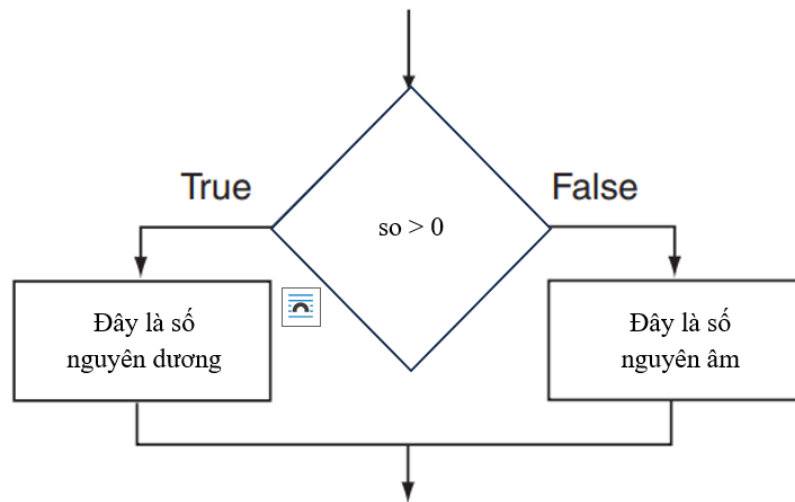
Cú pháp

```
if (điều_kiện) {
    // Khối lệnh được thực hiện nếu điều_kiện đúng
} else {
    // Khối lệnh được thực hiện nếu điều_kiện sai
}
```

Trong đó:

- điều_kiện: là một biểu thức logic (có giá trị true hoặc false)
- Khối lệnh được đặt trong cặp dấu {} và chỉ được thực hiện khi điều_kiện đúng hoặc sai.

Giống như câu lệnh if, một biểu thức được đánh giá. Nếu biểu thức đúng, một câu lệnh hoặc khối các câu lệnh sẽ được thực thi. Tuy nhiên, nếu biểu thức sai, một nhóm các câu lệnh khác sẽ được thực thi. Hình dưới minh họa logic câu lệnh if/else cùng với toán tử so sánh lớn hơn. Nếu $so > 0$ là đúng thì thông báo “Đây là số nguyên dương”. Ngược lại, thông báo “Đây là số nguyên âm”.



Hình 4.2. Logic câu lệnh if/else

Chú ý

- Nếu khối lệnh chỉ chứa một câu lệnh, thì có thể bỏ dấu {}.
- Điều kiện trong câu lệnh if có thể là bất kỳ biểu thức logic nào, bao gồm các toán tử so sánh, logic, ...
- Câu lệnh if/else có thể được lồng nhau (if/else bên trong if/else) để kiểm tra nhiều điều kiện.

Ví dụ 1: Kiểm tra 1 số là số nguyên âm hay nguyên dương

Chương trình 4.4. Kiểm tra 1 số là số nguyên âm hay nguyên dương

```

#include <iostream>
using namespace std;

int main() {
    int so;
    cout << "Nhập vào một số nguyên: ";
    cin >> so;

    if (so > 0) {
        cout << so << " là số nguyên dương." << endl;
    }
    else {
        cout << so << " là số nguyên âm." << endl;
    }

    return 0;
}
  
```

Kết quả chương trình:

Nhập vào một số nguyên: 4
4 là số nguyên dương.

Ví dụ 2: Viết chương trình nhập vào 1 năm và cho biết năm đó có phải năm nhuận không

Chương trình 4.5. Kiểm tra năm nhuận

```
#include <iostream>
using namespace std;

int main() {
    int nam;
    cout << "Nhap vao mot nam: ";
    cin >> nam;

    if ((nam % 4 == 0 && nam % 100 != 0) || nam % 400 == 0) {
        cout << nam << " la nam nhuan." << endl;
    }
    else {
        cout << nam << " khong phai la nam nhuan." << endl;
    }

    return 0;
}
```

Kết quả chương trình:

Nhap vao mot nam: 2024
2024 la nam nhuan.

Kiểm tra

1. Viết chương trình nhập vào một số nguyên, in ra "Số dương" nếu số đó là số dương, "Số âm" nếu số đó là số âm, "Số 0" nếu số đó bằng 0.
2. Viết chương trình nhập vào tuổi của một người, in ra "Trẻ em" nếu tuổi nhỏ hơn 18, "Người lớn" nếu tuổi lớn hơn hoặc bằng 18.
3. Viết chương trình nhập vào điểm số của một học sinh, in ra "Đậu" nếu điểm số lớn hơn hoặc bằng 5, "Rớt" nếu điểm số nhỏ hơn 5.
4. Viết chương trình nhập vào một số nguyên, in ra "Số chia hết cho 3" nếu số đó chia hết cho 3, "Số không chia hết cho 3" nếu số đó không chia hết cho 3.

4.4. Câu lệnh if lồng nhau

KHÁI NIỆM Câu lệnh if/else lồng nhau là một dạng câu lệnh điều kiện nâng cao trong lập trình C++. Trong câu lệnh này, một hoặc nhiều câu lệnh if/else được đặt bên trong một câu lệnh if/else khác, cho phép kiểm tra và thực hiện các hành động tương ứng với nhiều điều kiện phức tạp.

Cú pháp

```
if (điều_kiện_1) {
    // Khối lệnh 1 được thực hiện nếu điều_kiện_1 đúng
    if (điều_kiện_2) {
        // Khối lệnh 2 được thực hiện nếu điều_kiện_2 đúng
    } else {
```

```

        // Khối lệnh 3 được thực hiện nếu điều_kiện_2 sai
    }
} else {
    // Khối lệnh 4 được thực hiện nếu điều_kiện_1 sai
}

```

Trong đó:

- điều_kiện_1 và điều_kiện_2: là các biểu thức logic (có giá trị true hoặc false)
- Các khối lệnh được đặt trong cặp dấu {} và chỉ được thực hiện khi các điều kiện tương ứng đúng hoặc sai.

Chú ý

- Câu lệnh if/else lồng nhau có thể có nhiều cấp, tức là có thể có if/else bên trong if/else.
- Điều kiện trong câu lệnh if có thể là bất kỳ biểu thức logic nào, bao gồm các toán tử so sánh, logic, ...
- Khối lệnh bên trong các câu lệnh if/else có thể chứa bất kỳ câu lệnh nào trong C++, bao gồm cả các câu lệnh if/else khác.

Ví dụ 1: Viết chương trình nhập vào 1 điểm số. Sau đó in kết quả xếp loại học lực

Chương trình 4.6. Xếp loại học lực

```

#include <iostream>
using namespace std;

int main() {
    int diem;

    cout<<"Nhập vào diem so (0-100):";
    cin>>diem;

    if (diem >= 90) {
        cout << "Loai Xuat sac" << endl;
    } else {
        if (diem >= 80) {
            cout << "Loai Gioi" << endl;
        } else {
            if (diem >= 70) {
                cout << "Loai Kha" << endl;
            } else {
                if (diem >= 50) {
                    cout << "Loai Trung binh" << endl;
                } else {
                    cout << "Loai Yeu" << endl;
                }
            }
        }
    }

    return 0;
}

```

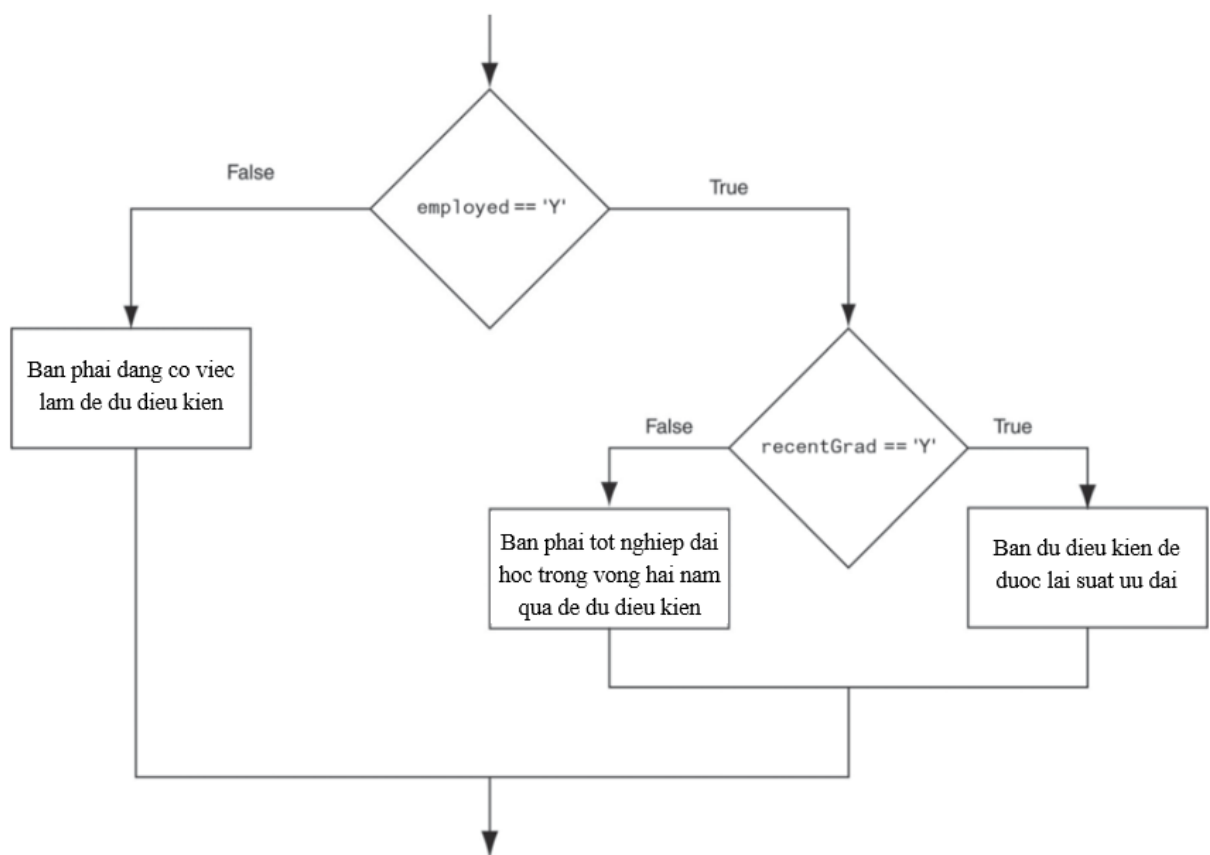
Kết quả chương trình:

Nhap vao diem so (0-100):85

Loai Gioi

Ví dụ 2: Chương trình minh họa câu lệnh if lồng nhau

Đôi khi một câu lệnh if phải được lồng vào bên trong một câu lệnh if khác. Ví dụ, hãy xem xét một chương trình ngân hàng xác định xem khách hàng ngân hàng có đủ điều kiện để được hưởng lãi suất thấp đặc biệt trên một khoản vay hay không. Để đủ điều kiện, hai điều kiện phải tồn tại: (1) khách hàng phải đang có việc làm, và (2) khách hàng phải mới tốt nghiệp đại học gần đây (trong vòng 2 năm qua). Hình 4.3 cho thấy một sơ đồ cho một thuật toán có thể được sử dụng trong chương trình.



Hình 4.3. Sơ đồ thuật toán điều kiện vay lãi suất đặc biệt

Nếu theo dõi luồng thực thi trong sơ đồ thì thấy biểu thức `employed == 'Y'` được kiểm tra. Nếu biểu thức này sai, không cần thiết phải thực hiện các kiểm tra tiếp theo; chúng ta biết rằng khách hàng không đủ điều kiện để được hưởng lãi suất đặc biệt. Tuy nhiên, nếu biểu thức đúng, chúng ta cần kiểm tra điều kiện thứ hai. Điều này được thực hiện với một cấu trúc quyết định lồng nhau kiểm tra biểu thức `recentGrad == 'Y'`. Nếu biểu thức này đúng, thì khách hàng đủ điều kiện để được hưởng lãi suất đặc biệt. Nếu biểu thức này sai,

thì khách hàng không đủ điều kiện. Chương trình 4.8 cho thấy mã cho chương trình hoàn chỉnh.

Chương trình 4.7. Điều kiện vay lãi suất đặc biệt

```
// Chương trình này minh họa câu lệnh if lồng nhau.
#include <iostream>
using namespace std;

int main()
{
    char employed, // Hiện đang có việc làm hay không, Nhập Y hoặc N
        recentGrad; // Mọi tốt nghiệp gần đây, Nhập Y hoặc N

    // Người dùng có đang làm việc và mọi tốt nghiệp gần đây không?
    cout << "Tra lời các câu hỏi sau đây\n";
    cout << "bằng Y cho Có hoặc ";
    cout << "N cho Không.\n";
    cout << "Bạn đang có việc làm? ";
    cin >> employed;
    cout << "Bạn đã tốt nghiệp đại học ";
    cout << "trong vòng hai năm qua? ";
    cin >> recentGrad;

    // Xác định điều kiện vay của người dùng.
    if (employed == 'Y')
    {
        if (recentGrad == 'Y') // Câu lệnh if lồng
        {
            cout << "Bạn đủ điều kiện để được lãi suất ưu đãi.\n";
        }
        else // Đang làm việc nhưng không phải tốt nghiệp gần đây
        {
            cout << "Bạn phải tốt nghiệp đại học ";
            cout << "trong vòng hai năm qua để đủ điều kiện.\n";
        }
    }
    else // Không có việc làm
    {
        cout << "Bạn phải đang có việc làm để đủ điều kiện.\n";
    }

    return 0;
}
```

Kết quả chương trình:

Tra lời các câu hỏi sau đây
bằng Y cho Có hoặc N cho Không.
Bạn đang có việc làm? y
Bạn đã tốt nghiệp đại học trong vòng hai năm qua? y
Bạn phải đang có việc làm để đủ điều kiện.

Giải thích:

Chương trình yêu cầu người dùng nhập vào hai ký tự 'Y' hoặc 'N' để trả lời hai câu hỏi: "Bạn đang có việc làm?" và "Bạn đã tốt nghiệp đại học trong vòng hai năm qua?".

Nếu người dùng đang có việc làm (`employed == 'Y'`), chương trình sẽ kiểm tra xem họ có phải là tân sinh viên tốt nghiệp gần đây hay không (`recentGrad == 'Y'`).

Nếu người dùng là tân sinh viên tốt nghiệp gần đây, chương trình sẽ in ra thông báo "Bạn đủ điều kiện để được lãi suất ưu đãi."

Nếu người dùng không phải là tân sinh viên tốt nghiệp gần đây, chương trình sẽ in ra thông báo "Bạn phải tốt nghiệp đại học trong vòng hai năm qua để đủ điều kiện."

Nếu người dùng không có việc làm (`employed != 'Y'`), chương trình sẽ in ra thông báo "Bạn phải đang có việc làm để đủ điều kiện."

4.5. Câu lệnh `if/ else if`

KHÁI NIỆM: Câu lệnh `if/else if` là một dạng câu lệnh điều kiện nâng cao trong lập trình C++. Câu lệnh này cho phép kiểm tra một loạt các điều kiện logic và thực hiện các hành động tương ứng khi các điều kiện đó đúng.

Cú pháp

```
if (điều_kiện_1) {  
    // Khối lệnh 1 được thực hiện nếu điều_kiện_1 đúng  
} else if (điều_kiện_2) {  
    // Khối lệnh 2 được thực hiện nếu điều_kiện_1 sai và điều_kiện_2 đúng  
} else if (điều_kiện_3) {  
    // Khối lệnh 3 được thực hiện nếu điều_kiện_1 và điều_kiện_2 sai,  
    // nhưng điều_kiện_3 đúng  
}  
...  
else {  
    // Khối lệnh n+1 được thực hiện nếu tất cả các điều kiện trên sai  
}
```

Trong đó:

- `điều_kiện_1`, `điều_kiện_2`, `điều_kiện_3`, ...: là các biểu thức logic (có giá trị `true` hoặc `false`)
- Các khối lệnh được đặt trong cặp dấu `{ }` và chỉ được thực hiện khi các điều kiện tương ứng đúng.

Chú ý

- Câu lệnh `if/else if` có thể có nhiều `else if` trong một câu lệnh.
- Chỉ một trong các khối lệnh tương ứng với các điều kiện đúng sẽ được thực hiện.
- Nếu tất cả các điều kiện đều sai, khối lệnh `else` cuối cùng sẽ được thực hiện.
- Điều kiện trong câu lệnh `if/else if` có thể là bất kỳ biểu thức logic nào, bao gồm các toán tử so sánh, logic, ...

Ví dụ 1: Viết chương trình nhập vào 1 điểm. Sau đó hiển thị xếp loại sinh viên

Chương trình 4.8. Xếp loại sinh viên

```
#include <iostream>
using namespace std;

int main() {
    double diemTrungBinh;
    cout << "Nhap vao diem trung binh: ";
    cin >> diemTrungBinh;

    if (diemTrungBinh >= 9.0) {
        cout << "Xep loai xuat sac." << endl;
    }
    else if (diemTrungBinh >= 8.0) {
        cout << "Xep loai gioi." << endl;
    }
    else if (diemTrungBinh >= 6.5) {
        cout << "Xep loai kha." << endl;
    }
    else if (diemTrungBinh >= 5.0) {
        cout << "Xep loai trung binh." << endl;
    }
    else {
        cout << "Xep loai yeu." << endl;
    }

    return 0;
}
```

Kết quả chương trình:

Nhap vao diem trung binh: 6.5
Xep loai kha.

Ví dụ 2: Viết chương trình nhập vào thu nhập, sau đó hiển thị số thuế phải nộp

Chương trình 4.9. Tính thuế thu nhập cá nhân

```
#include <iostream>
using namespace std;

int main() {
    double thuNhapNam, thueTNCN;
    cout << "Nhap vao thu nhap nam (trieu dong): ";
    cin >> thuNhapNam;

    if (thuNhapNam <= 108) {
        thueTNCN = thuNhapNam * 0.05;
    }
    else if (thuNhapNam <= 216) {
        thueTNCN = 5.4 + (thuNhapNam - 108) * 0.1;
    }
    else if (thuNhapNam <= 384) {
```

```

    thueTNCN = 16.2 + (thuNhapNam - 216) * 0.15;
}
else if (thuNhapNam <= 624) {
    thueTNCN = 37.8 + (thuNhapNam - 384) * 0.2;
}
else if (thuNhapNam <= 960) {
    thueTNCN = 86.2 + (thuNhapNam - 624) * 0.25;
}
else {
    thueTNCN = 181.2 + (thuNhapNam - 960) * 0.3;
}

cout << "Thue thu nhap ca nhan phai nop la: " << thueTNCN << " trieu
dong." << endl;

return 0;
}

```

Kết quả chương trình:

Nhap vao thu nhap nam (trieu dong): 50
 Thue thu nhap ca nhan phai nop la: 2.5 trieu dong.

4.6. Biến cờ (Flags)

KHÁI NIỆM Biến cờ (flags) là một kỹ thuật lập trình dùng để lưu trạng thái của một sự kiện hoặc điều kiện nào đó. Nó thường được sử dụng trong các câu lệnh điều khiển như if/else, while, for, ... để kiểm tra và thực hiện các hành động tương ứng.

Một biến cờ (flag) thường là một biến kiểu bool (đúng/sai) để báo hiệu khi một điều kiện nào đó tồn tại trong chương trình. Khi biến cờ được gán giá trị false, nó chỉ ra rằng điều kiện đó không tồn tại. Khi biến cờ được gán giá trị true, nó có nghĩa là điều kiện đó tồn tại. Ví dụ, giả sử một chương trình tính hoa hồng bán hàng có một biến bool, được định nghĩa và khởi tạo như sau:

```
bool salesQuotaMet = false;
```

Trong chương trình, biến salesQuotaMet được sử dụng như một biến cờ để chỉ ra liệu một nhân viên bán hàng đã đạt được chỉ tiêu bán hàng hay chưa. Khi chúng ta định nghĩa biến, chúng ta khởi tạo nó với giá trị false bởi vì lúc đó chúng ta chưa biết liệu nhân viên bán hàng đã đạt chỉ tiêu hay chưa. Giả sử một biến tên là sales chứa số lượng bán hàng, đoạn mã tương tự như sau có thể được sử dụng để gán giá trị cho biến salesQuotaMet:

```

if (sales >= QUOTA_AMOUNT)
    salesQuotaMet = true;
else
    salesQuotaMet = false;

```

Kết quả của đoạn mã này, biến salesQuotaMet có thể được sử dụng như một biến cờ để chỉ ra liệu chỉ tiêu bán hàng đã được đạt hay chưa. Sau đó trong chương trình, chúng ta có thể kiểm tra biến cờ theo cách sau:

```
if (salesQuotaMet)
    cout << "Bạn đã đạt được chỉ tiêu bán hàng!\n";
```

Đoạn mã này sẽ hiển thị "Bạn đã đạt được chỉ tiêu bán hàng!" nếu biến bool salesQuotaMet có giá trị true. Lưu ý rằng chúng ta không cần phải sử dụng toán tử == để so sánh rõ ràng biến salesQuotaMet với giá trị true. Đoạn mã này tương đương với đoạn mã sau:

```
if (salesQuotaMet == true)
    cout << "Bạn đã đạt được chỉ tiêu bán hàng!\n";
```

Biến cờ kiểu số nguyên

Các biến kiểu số nguyên cũng có thể được sử dụng như là biến cờ. Điều này là do trong C++, giá trị 0 được coi là false, và bất kỳ giá trị khác 0 nào đều được coi là true. Trong chương trình tính hoa hồng bán hàng được mô tả trước đó, chúng ta có thể định nghĩa biến salesQuotaMet với câu lệnh sau:

```
int salesQuotaMet = 0; // 0 có nghĩa là false.
```

Giống như trước đây, chúng ta khởi tạo biến với giá trị 0 bởi vì lúc đó chúng ta chưa biết liệu chỉ tiêu bán hàng đã được đạt hay chưa. Sau khi tính toán doanh số bán hàng, chúng ta có thể sử dụng đoạn mã tương tự như sau để gán giá trị cho biến salesQuotaMet:

```
if (sales >= QUOTA_AMOUNT)
    salesQuotaMet = 1;
else
    salesQuotaMet = 0;
```

Sau đó trong chương trình, chúng ta có thể kiểm tra biến cờ theo cách sau:

```
if (salesQuotaMet)
    cout << "Bạn đã đạt được chỉ tiêu bán hàng!\n";
```

4.7. Toán tử logic

KHÁI NIỆM Toán tử logic là các toán tử được sử dụng để thực hiện các phép toán logic trên các biểu thức logic (biểu thức có kết quả là true hoặc false). Các toán tử logic này thường được sử dụng trong các câu lệnh điều kiện (if/else, while, ...) để kiểm tra và thực hiện các hành động tương ứng.

Các toán tử logic

Toán tử	Mô tả	Diễn giải
&&	AND	Trả về true nếu cả hai toán hạng đều true, ngược lại trả về false.
	OR	Trả về true nếu ít nhất một trong hai toán hạng là true, ngược lại trả về false.

Toán tử	Mô tả	Diễn giải
!	NOT	Trả về true nếu toán hạng là false, và trả về false nếu toán hạng là true.

Dưới đây là bảng giá trị chân lý cho toán tử && (AND) trong C++:

X	Y	X && Y
true	true	true
true	false	false
false	true	false
false	false	false

Bảng giá trị chân lý cho toán tử || (OR) trong C++:

X	Y	X Y
true	true	true
true	false	true
false	true	true
false	false	false

Ví dụ 1: Chương trình sau đây thực hiện hoạt động giúp khách hàng của ngân hàng đủ điều kiện nhận mức lãi suất đặc biệt. Chương trình này sử dụng toán tử logic.

Chương trình 4.10. Điều kiện nhận lãi suất đặc biệt

```
// Chương trình này minh họa toán tử logic &&.
#include <iostream>
using namespace std;

int main()
{
    char employed, // Hiện đang có việc làm hay không, Nhập Y hoặc N
        recentGrad; // Mới tốt nghiệp gần đây, Nhập Y hoặc N

    // Người dùng có đang làm việc và mới tốt nghiệp gần đây không?
    cout << "Tra lời các câu hỏi sau đây\n";
    cout << "Nhập Y cho Có hoặc N cho Không.\n";
    cout << "Bạn đang có việc làm? ";
    cin >> employed;
    cout << "Bạn đã tốt nghiệp đại học "
        << "trong vòng hai năm qua? ";
    cin >> recentGrad;

    // Xác định điều kiện vay của người dùng.
    if (employed == 'Y' && recentGrad == 'Y')
    {
        cout << "Bạn đủ điều kiện để được lãi suất "
            << "ưu đãi.\n";
    }
    else
```

```

{
    cout << "Ban phai dang co viec lam va\n"
          << "tot nghiep dai hoc trong vong\n"
          << "hai nam qua de du dieu kien.\n";
}

return 0;
}

```

Kết quả chương trình:

Tra loi cac cau hoi sau day
 bang Y cho Co hoac N cho Khong.
 Ban dang co viec lam? y
 Ban da tot nghiep dai hoc trong vong hai nam qua? n
 Ban phai dang co viec lam va
 tot nghiep dai hoc trong vong
 hai nam qua de du dieu kien.

Ví dụ 2: Chương trình sau thực hiện các bài kiểm tra khác nhau để xác định một người có đủ điều kiện vay vốn hay không. Điều này xác định xem khách hàng có kiếm được ít nhất 35.000 USD mỗi năm hay đã làm việc được hơn 5 năm.

Chương trình 4.11. Điều kiện vay vốn

```

// Chuong trinh nay minh hoa toan tu logic ||.
#include <iostream>
using namespace std;

int main()
{
    // Hang so cho thu nhap toi thieu va so nam lam viec
    const double THU_NHAP_TOI_THIEU = 35000.0;
    const int SO_NAM_TOI_THIEU = 5;
    double thuNhap; // Thu nhap hang nam
    int soNam; // So nam lam viec tai cong viec hien tai

    // Nhap thu nhap hang nam
    cout << "Thu nhap hang nam cua ban la bao nhieu? ";
    cin >> thuNhap;

    // Nhap so nam lam viec tai cong viec hien tai
    cout << "Ban da lam viec tai cong viec hien tai "
          << "trong bao nhieu nam? ";
    cin >> soNam;

    // Xac dinh dieu kien vay cua nguoi dung
    if (thuNhap >= THU_NHAP_TOI_THIEU || soNam > SO_NAM_TOI_THIEU)
        cout << "Ban du dieu kien.\n";
    else
    {

```

```

        cout << "Ban phai co thu nhap toi thieu $"
            << THU_NHAP_TOI_THIEU << " hoac da "
            << "lam viec hon " << SO_NAM_TOI_THIEU
            << " nam.\n";
    }

    return 0;
}

```

Kết quả chương trình:

```

Thu nhap hang nam cua ban la bao nhieu? 120000
Ban da lam viec tai cong viec hien tai trong bao nhieu nam? 10
Ban du dieu kien.

```

4.8. Xác thực đầu vào

KHÁI NIỆM Xác thực đầu vào (Input Validation) là quá trình kiểm tra và đảm bảo rằng dữ liệu được người dùng nhập vào là hợp lệ, đúng định dạng và đáp ứng các yêu cầu của chương trình. Việc xác thực đầu vào là một trong những kỹ năng quan trọng nhất trong lập trình C++, giúp tăng cường tính bảo mật và độ tin cậy của ứng dụng.

Có lẽ câu nói nổi tiếng nhất của thế giới máy tính là "Garbage in, garbage out" (Đầu vào rác, đầu ra cũng rác). Tính toàn vẹn của đầu ra chương trình chỉ tốt bằng đầu vào của nó, vì vậy bạn nên cố gắng đảm bảo rằng không có "rác" đi vào chương trình của mình. Xác thực đầu vào là quá trình kiểm tra dữ liệu được người dùng cung cấp cho chương trình và xác định liệu nó có hợp lệ hay không. Một chương trình tốt nên cung cấp hướng dẫn rõ ràng về loại đầu vào có thể chấp nhận được, và không giả định rằng người dùng đã tuân theo những hướng dẫn đó. Đây chỉ là một vài ví dụ về xác thực đầu vào được thực hiện bởi các chương trình:

- Số được kiểm tra để đảm bảo chúng nằm trong phạm vi giá trị có thể. Ví dụ: có 168 giờ trong một tuần. Không thể có người nào làm việc lâu hơn 168 giờ trong một tuần.
- Giá trị được kiểm tra về tính "hợp lý". Mặc dù có thể có người làm việc 168 giờ/tuần, nhưng điều đó không phải là có thể xảy ra.
- Các mục được chọn từ menu hoặc các tập hợp lựa chọn khác được kiểm tra để đảm bảo chúng là các tùy chọn có sẵn.
- Các biến được kiểm tra về giá trị có thể gây ra sự cố, chẳng hạn như chia cho 0.

Ví dụ 1: Chương trình sau là một chương trình tính điểm bài kiểm tra sẽ từ chối bất kỳ điểm số nào nhỏ hơn 0 hoặc lớn hơn 100.

Chương trình 4.12. Xác thực điểm bài kiểm tra

```

// Chương trình tính điểm bài kiểm tra nay không chấp nhận
// điểm số nhỏ hơn 0 hoặc lớn hơn 100.
#include <iostream>

```

```

using namespace std;

int main()
{
    // Hang so cho nguong diem
    const int DIEM_A = 90,
            DIEM_B = 80,
            DIEM_C = 70,
            DIEM_D = 60,
            DIEM_TOI_THIEU = 0, // Diem so hop le toi thieu
            DIEM_TOI_DA = 100; // Diem so hop le toi da
    int diemSo; // De luu diem so

    // Nhap diem so
    cout << "Nhap diem so cua ban, toi se \n"
          << "cho ban biet diem chu tuong ung: ";
    cin >> diemSo;

    // Xac thuc dau vao va xac dinh diem chu
    if (diemSo >= DIEM_TOI_THIEU && diemSo <= DIEM_TOI_DA)
    {
        // Xac dinh diem chu
        if (diemSo >= DIEM_A)
            cout << "Diem chu cua ban la A.\n";
        else if (diemSo >= DIEM_B)
            cout << "Diem chu cua ban la B.\n";
        else if (diemSo >= DIEM_C)
            cout << "Diem chu cua ban la C.\n";
        else if (diemSo >= DIEM_D)
            cout << "Diem chu cua ban la D.\n";
        else
            cout << "Diem chu cua ban la F.\n";
    }
    else
    {
        // Diem so nhap vao khong hop le
        cout << "Diem so do khong hop le. Chay lai chuong trinh\n"
              << "va nhap gia tri trong khoang tu \n"
              << DIEM_TOI_THIEU << " den " << DIEM_TOI_DA << ".\n";
    }

    return 0;
}

```

Kết quả chương trình:

Nhap diem so cua ban, toi se
 cho ban biet diem chu tuong ung: 80
 Diem chu cua ban la B.

Giải thích:

- Chương trình định nghĩa các hằng số cho ngưỡng điểm để xác định điểm chữ tương ứng (A, B, C, D, F).

- Chương trình cũng định nghĩa các hằng số DIEM_TOI_THIEU và DIEM_TOI_DA để lưu trữ giá trị điểm số hợp lệ tối thiểu và tối đa.
- Chương trình yêu cầu người dùng nhập vào điểm số.
- Câu lệnh if (diemSo >= DIEM_TOI_THIEU && diemSo <= DIEM_TOI_DA) kiểm tra xem điểm số nhập vào có hợp lệ hay không.
 - Nếu điểm số hợp lệ, chương trình sẽ sử dụng cấu trúc if...else if để xác định và in ra điểm chữ tương ứng.
 - Nếu điểm số không hợp lệ, chương trình sẽ in ra thông báo yêu cầu người dùng nhập lại điểm số trong khoảng hợp lệ.

Chú ý

- Việc xác thực đầu vào là rất quan trọng để đảm bảo chương trình chạy đúng và không bị lỗi do dữ liệu không hợp lệ.
- Chương trình này sử dụng các hằng số để lưu trữ ngưỡng điểm và khoảng điểm hợp lệ, giúp dễ dàng thay đổi và bảo trì sau này.
- Cấu trúc if...else if được sử dụng để xác định điểm chữ dựa trên điểm số hợp lệ.

4.9. So sánh chuỗi và các ký tự

KHÁI NIỆM Các toán tử quan hệ cũng có thể được sử dụng để so sánh các ký tự và đối tượng chuỗi.

So sánh ký tự

Như chúng ta đã biết, các ký tự thực sự được lưu trữ trong bộ nhớ dưới dạng số nguyên. Trên hầu hết các hệ thống, số nguyên này là giá trị ASCII của ký tự. Ví dụ: chữ 'A' được biểu thị bằng số 65, chữ 'B' được biểu thị bằng số 66, v.v. Bảng dưới cho thấy các số ASCII tương ứng với một số ký tự thường được sử dụng.

Bảng 4.1. Giá trị ASCII của các ký tự thường được sử dụng

Character	ASCII Value
'0'–'9'	48–57
'A'–'Z'	65–90
'a'–'z'	97–122
Blank	32
Period	46

Lưu ý mọi ký tự, kể cả ký tự trống, đều có mã ASCII. Cũng lưu ý rằng mã ASCII của ký tự đại diện cho một chữ số, chẳng hạn như '1' hoặc '2', không giống với giá trị của chính chữ số đó.

Khi hai ký tự được so sánh, thực tế giá trị ASCII của ký tự đó đang được so sánh. 'A' < 'B' vì giá trị ASCII của 'A' (65) nhỏ hơn giá trị ASCII của 'B' (66). Tương tự như vậy '1' < '2' vì giá trị ASCII của '1' (49) nhỏ hơn giá trị ASCII của '2' (50). Tuy nhiên, như thể hiện

trong bảng giá trị mã ASCII trên, chữ thường có mã ASCII cao hơn chữ in hoa, vì vậy 'a' > 'Z'. Chương trình dưới đây cho thấy cách so sánh các ký tự với các toán tử quan hệ.

Chương trình 4.13. So sánh ký tự

```
// Chương trình này minh họa cách các ký tự có thể được
// so sánh với các toán tử quan hệ.
#include <iostream>
using namespace std;

int main()
{
    char ch;
    // Get a character from the user.
    cout << "Nhập một chữ số hoặc một chữ cái: ";
    ch = cin.get();

    // Xác định nội dung người dùng đã nhập
    if (ch >= '0' && ch <= '9')
        cout << "Bạn đã nhập một chữ số.\n";
    else if (ch >= 'A' && ch <= 'Z')
        cout << "Bạn đã nhập một chữ in hoa.\n";
    else if (ch >= 'a' && ch <= 'z')
        cout << "Bạn đã nhập một chữ cái viết thường.\n";
    else
        cout << "Đó không phải là chữ số hay chữ cái.\n";

    return 0;
}
```

Kết quả chương trình:

```
Nhập một chữ số hoặc một chữ cái: a
Bạn đã nhập một chữ cái viết thường.
```

So sánh chuỗi

Chuỗi cũng có thể được so sánh bằng các toán tử quan hệ. Giống như các ký tự riêng lẻ, khi so sánh hai đối tượng chuỗi, thực tế đó là giá trị ASCII của các ký tự tạo nên chuỗi đang được so sánh.

Ví dụ 1:

```
string str1 = "ABC";
string str2 = "XYZ";
```

Đối tượng chuỗi str1 được coi là nhỏ hơn đối tượng chuỗi str2 vì các ký tự "ABC" đứng trước theo thứ tự bảng chữ cái (có giá trị ASCII thấp hơn) các ký tự "XYZ". Vì vậy, câu lệnh if sau sẽ gây ra thông báo "str1 nhỏ hơn str2." để được hiển thị trên màn hình.

```
if (str1 < str2)
```

```
cout << " str1 nhỏ hơn str2.";
```

Lần lượt từng ký tự trong toán hạng thứ nhất được so sánh với ký tự ở vị trí tương ứng trong toán hạng thứ hai. Nếu tất cả các ký tự trong cả hai đối tượng chuỗi giống nhau thì hai chuỗi đó bằng nhau.

Ví dụ 2:

```
string name1 = "Mary";  
string name2 = "Mark";
```

Giá trị trong name1 là "Mary" lớn hơn giá trị trong name2 là "Mark". Điều này là do ba ký tự đầu tiên trong name1 có cùng giá trị ASCII như ba ký tự đầu tiên trong name2, nhưng 'y' ở vị trí thứ tư của "Mary" có giá trị ASCII lớn hơn 'k' ở vị trí tương ứng của "Mark".

Bất kỳ toán tử quan hệ nào cũng có thể được sử dụng để so sánh hai đối tượng chuỗi. Dưới đây là một số so sánh hợp lệ của name1 và name2:

```
name1 > name2 // true  
name1 <= name2 // false
```

Chú ý

- Khi so sánh ký tự, C++ sử dụng bảng mã ASCII để so sánh giá trị của các ký tự.
- Khi so sánh chuỗi, C++ so sánh từng ký tự một trong chuỗi, theo thứ tự từ trái sang phải.
- Các hàm so sánh chuỗi như strcmp(), strcasecmp() cũng có thể được sử dụng.
 - Hàm strcmp(): Hàm so sánh 2 chuỗi có phân biệt chữ hoa chữ thường.
 - Hàm strcasecmp(): Hàm so sánh 2 chuỗi không phân biệt chữ hoa chữ thường.

Ví dụ 1, Chương trình sau trình bày thêm cách sử dụng các toán tử quan hệ với các đối tượng chuỗi.

Chương trình 4.14. Minh họa so sánh các chuỗi kí tự

```
// Chương trình này sử dụng các toán tử quan hệ để so sánh một chuỗi  
// được nhập bởi người dùng với các mã số phân hợp lệ.  
#include <iostream>  
#include <iomanip>  
#include <string>  
using namespace std;  
  
int main()  
{  
    const double GIA_A = 249.0, // Giá cho phần A  
                GIA_B = 199.0; // Giá cho phần B  
    string maSoPhan; // Lưu trữ mã số phần  
  
    // Hiện thị các phần cơ sở và lựa chọn của người dùng  
    cout << "Các mã số phần tại nghe là:\n"  
          << "Tại nghe chong on: mã số phần S-29A\n"
```

```

        << "Tai nghe khong day: ma so phan S-29B\n"
        << "Nhap ma so phan cua tai nghe ban\n"
        << "muon mua: ";
    cin >> maSoPhan;

    // Dat dinh dang dau ra so
    cout << fixed << showpoint << setprecision(2);

    // Xac dinh va hien thi gia chinh xac
    if (maSoPhan == "S-29A")
        cout << "Gia la $" << GIA_A << endl;
    else if (maSoPhan == "S-29B")
        cout << "Gia la $" << GIA_B << endl;
    else
        cout << maSoPhan << " khong phai la ma so phan hop le.\n";

    return 0;
}

```

Kết quả chương trình:

Cac ma so phan tai nghe la:
 Tai nghe chong on: ma so phan S-29A
 Tai nghe khong day: ma so phan S-29B
 Nhap ma so phan cua tai nghe ban
 muon mua: S-29A
 Gia la \$249.00

Giải thích:

- Chương trình định nghĩa hai hằng số GIA_A và GIA_B lưu trữ giá tiền cho hai loại tai nghe khác nhau.
- Chương trình hiển thị danh sách các mã số phần tai nghe có sẵn và yêu cầu người dùng nhập mã số phần của tai nghe họ muốn mua.
- Chương trình đặt định dạng đầu ra số với hai chữ số thập phân.
- Câu lệnh if (maSoPhan == "S-29A") sử dụng toán tử so sánh == để kiểm tra xem mã số phần nhập vào có khớp với mã số phần "S-29A" hay không.
 - Nếu khớp, chương trình sẽ in ra giá tiền cho phần "S-29A".
- Câu lệnh else if (maSoPhan == "S-29B") kiểm tra xem mã số phần có khớp với "S-29B" hay không.
 - Nếu khớp, chương trình sẽ in ra giá tiền cho phần "S-29B".
- Nếu mã số phần nhập vào không khớp với cả hai mã số hợp lệ, chương trình sẽ in ra thông báo "không phải là mã số phần hợp lệ".

Chú ý:

- Chương trình sử dụng toán tử so sánh == để so sánh chuỗi, không phải giá trị ASCII của các ký tự trong chuỗi.

- Việc sử dụng string thay vì char cho phép chương trình xử lý các chuỗi có nhiều ký tự.
- Định dạng đầu ra được đặt để hiển thị giá tiền với hai chữ số thập phân sau dấu chấm.

4.10. Toán tử điều kiện ?

KHÁI NIỆM Toán tử điều kiện (Conditional Operator), còn được gọi là toán tử ba ngôi (Ternary Operator), là một toán tử đặc biệt trong C++ cho phép thực hiện một câu lệnh if/else rút gọn. Nó có cú pháp ngắn gọn và thường được sử dụng khi cần thực hiện một phép gán có điều kiện.

Cú pháp

điều_kiện ? giá_trị_nếu_đúng : giá_trị_nếu_sai

Trong đó:

- điều_kiện: là một biểu thức logic (có giá trị true hoặc false)
- giá_trị_nếu_đúng: là giá trị được gán nếu điều_kiện đúng
- giá_trị_nếu_sai: là giá trị được gán nếu điều_kiện sai

Chú ý

- Toán tử điều kiện có thể được sử dụng trong các biểu thức, cả trong việc gán giá trị và trong các câu lệnh if/else.
- Toán tử điều kiện thường được sử dụng khi cần một câu lệnh if/else đơn giản và gọn gàng.
- Không nên lạm dụng toán tử điều kiện khi câu lệnh if/else trở nên quá phức tạp.

Ví dụ 1: Tìm số lớn nhất giữa hai số

Chương trình 4.15. Tìm số lớn nhất trong 2 số

```
#include <iostream>
using namespace std;

int main() {
    int a = 10, b = 20;
    int max = (a > b) ? a : b;
    cout << "Số lớn nhất là: " << max << endl;
    return 0;
}
```

Kết quả chương trình:

Số lớn nhất là: 20

Ví dụ 2: Kiểm tra số dương, số âm

Chương trình 4.16. Số âm hay số dương

```
#include <iostream>
using namespace std;
int main() {
    int so = -5;
    string ket_qua = (so > 0) ? "So duong" : "So am";
    cout << ket_qua << endl;
    return 0;
}
```

Kết quả chương trình:

So am

Ví dụ 3: Tìm số lớn nhất giữa ba số

Chương trình 4.17. Tìm số lớn nhất trong 3 số

```
#include <iostream>
using namespace std;
int main() {
    int a = 10, b = 20, c = 30;
    int max = (a > b) ? ((a > c) ? a : c) : ((b > c) ? b : c);
    cout << "So lon nhat la: " << max << endl;
    return 0;
}
```

Kết quả chương trình:

So lon nhat la: 30

4.11. Câu lệnh switch

KHÁI NIỆM Câu lệnh switch-case là một câu lệnh điều kiện trong lập trình C++. Nó cho phép kiểm tra giá trị của một biến hoặc biểu thức và thực hiện các hành động tương ứng với từng giá trị được kiểm tra.

Câu lệnh switch-case thường được sử dụng khi cần kiểm tra nhiều trường hợp khác nhau, thay vì sử dụng nhiều câu lệnh if-else lồng nhau.

Cú pháp

```
switch (biến_hoặc_biểu_thức) {
    case giá_trị_1:
        // Khối lệnh được thực hiện nếu biến_hoặc_biểu_thức = giá_trị_1
        break;
    case giá_trị_2:
        // Khối lệnh được thực hiện nếu biến_hoặc_biểu_thức = giá_trị_2
        break;
    ...
    default:
        // Khối lệnh được thực hiện nếu biến_hoặc_biểu_thức không khớp
        với bất kỳ giá trị nào trong các case
}
```

```
        break;
    }
```

Trong đó:

- biến_hoặc_biểu_thức: là giá trị được kiểm tra trong câu lệnh switch.
- giá_trị_1, giá_trị_2, ...: là các giá trị cụ thể mà biến_hoặc_biểu_thức được so sánh.
- Khối lệnh được đặt trong cặp dấu {} và chỉ được thực hiện khi biến_hoặc_biểu_thức khớp với giá trị tương ứng.
- break; là câu lệnh để thoát khỏi câu lệnh switch sau khi thực hiện xong khối lệnh tương ứng.
- default: là trường hợp được thực hiện nếu biến_hoặc_biểu_thức không khớp với bất kỳ giá trị nào trong các case.

Chú ý

- Các giá trị trong các case phải là các hằng số (kiểu nguyên, kiểu kí tự hoặc hằng số) hoặc hằng số biểu thức.
- Các biểu thức ở case không được trùng lặp.
- Nếu một case không có câu lệnh break;, thì chương trình sẽ tiếp tục thực hiện các case tiếp theo cho đến khi gặp break;.
- Câu lệnh switch-case có thể lồng nhau.

Ví dụ 1: Chương trình sau cho thấy cách một câu lệnh switch đơn giản hoạt động.

Chương trình 4.18. Thực hiện phép tính

```
#include <iostream>
using namespace std;
int main() {
    char phep_tinh;
    int a, b, ket_qua;

    cout << "Nhap hai so nguyen: ";
    cin >> a >> b;
    cout << "Nhap phep tinh (+, -, *, /): ";
    cin >> phep_tinh;

    switch (phep_tinh) {
        case '+':
            ket_qua = a + b;
            cout << a << " + " << b << " = " << ket_qua << endl;
            break;
        case '-':
            ket_qua = a - b;
            cout << a << " - " << b << " = " << ket_qua << endl;
            break;
        case '*':
            ket_qua = a * b;
```

```

        cout << a << " * " << b << " = " << ket_qua << endl;
        break;
    case '/':
        if (b == 0) {
            cout << "Loi: Khong the chia cho 0" << endl;
        } else {
            ket_qua = a / b;
            cout << a << " / " << b << " = " << ket_qua << endl;
        }
        break;
    default:
        cout << "Loi: Phep tinh khong hop le" << endl;
        break;
}

return 0;
}

```

Kết quả chương trình:

Nhập hai số nguyên: 2 3
 Nhập phép tính (+, -, *, /): +
 2 + 3 = 5

Ví dụ 2: Chương trình cho phép nhập vào 1 điểm số, sau đó in ra xếp loại học lực

Chương trình 4.19. Xếp loại học lực

```

#include <iostream>
using namespace std;
int main() {
    int diem;
    cout << "Nhap diem so: ";
    cin >> diem;

    switch (diem) {
        case 10:
        case 9:
            cout << "Xuat sac" << endl;
            break;
        case 8:
            cout << "Gioi" << endl;
            break;
        case 7:
            cout << "Kha" << endl;
            break;
        case 6:
            cout << "Trung binh" << endl;
            break;
        case 5:
        case 4:
        case 3:
        case 2:
    }
}

```

```

        case 1:
        case 0:
            cout << "Yeu" << endl;
            break;
        default:
            cout << "Diem khong hop le" << endl;
            break;
    }
    return 0;
}

```

Kết quả chương trình:

Nhap diem so: 4
Yeu

4.12. Thực đơn

KHÁI NIỆM Thực đơn (Menus) là một cách tổ chức giao diện người dùng để cho phép người dùng lựa chọn các chức năng hoặc hành động cần thực hiện. Thực đơn thường bao gồm một danh sách các lựa chọn, và người dùng có thể chọn một lựa chọn bằng cách nhập vào một số hoặc ký tự tương ứng.

Việc sử dụng thực đơn giúp chương trình trở nên dễ sử dụng và dễ quản lý hơn, đặc biệt là khi chương trình có nhiều tính năng và chức năng khác nhau.

Dưới đây là 2 ví dụ về thực đơn, một ví dụ sử dụng if và else if và một ví dụ sử dụng switch case.

Ví dụ 1: Chương trình hiển thị các chức năng của hoạt động thư viện. Tiếp theo, cho phép người sử dụng lựa chọn hoạt động. Cuối cùng hiển thị trạng thái của hoạt động đó. Thực đơn sử dụng if else if

Chương trình 4.20. Menu hoạt động thư viện sử dụng if else if

```

#include <iostream>
using namespace std;
int main() {
    int lua_chon;

    cout << "Chương trình quan ly thu vien" << endl;
    cout << "1. Hien thi danh sach sach" << endl;
    cout << "2. Them sach moi" << endl;
    cout << "3. Muon sach" << endl;
    cout << "4. Tra sach" << endl;
    cout << "5. Thoat" << endl;
    cout << "Vui long chon mot tuy chon (1-5): ";
    cin >> lua_chon;

    if (lua_chon == 1) {
        cout << "Dang hien thi danh sach sach..." << endl;
    }
}

```

```

    } else if (lua_chon == 2) {
        cout << "Dang them sach moi..." << endl;
    } else if (lua_chon == 3) {
        cout << "Dang muon sach..." << endl;
    } else if (lua_chon == 4) {
        cout << "Dang tra sach..." << endl;
    } else if (lua_chon == 5) {
        cout << "Dang thoat chuong trinh..." << endl;
    } else {
        cout << "Lua chon khong hop le. Vui long chon lai." << endl;
    }
    return 0;
}

```

Kết quả chương trình:

Chương trình quản lý thư viện

1. Hiện thị danh sách sách
2. Thêm sách mới
3. Muôn sách
4. Trả sách
5. Thoát

Vui lòng chọn một tùy chọn (1-5): 2

Dang them sach moi...

Ví dụ 2: Viết lại chương trình trên và sử dụng switch case

Chương trình 4.21. Menu hoạt động thư viện sử dụng switch case

```

#include <iostream>
using namespace std;
int main() {
    int lua_chon ;
    cout << "Chương trình quản lý thư viện" << endl;
    cout << "1. Hiện thị danh sách sách" << endl;
    cout << "2. Thêm sách mới" << endl;
    cout << "3. Muôn sách" << endl;
    cout << "4. Trả sách" << endl;
    cout << "5. Thoát" << endl;
    cout << "Vui lòng chọn một tùy chọn (1-5): ";
    cin >> lua_chon ;

    switch (lua_chon ) {
        case 1:
            cout << "Dang hien thi danh sach sach..." << endl;
            break;
        case 2:
            cout << "Dang them sach moi..." << endl;
            break;
        case 3:
            cout << "Dang muon sach..." << endl;

```

```

        break;
    case 4:
        cout << "Dang tra sach..." << endl;
        break;
    case 5:
        cout << "Dang thoat chuong trinh..." << endl;
        break;
    default:
        cout << "Lua chon khong hop le. Vui long chon lai." <<
endl;
    }
    return 0;
}

```

Kết quả chương trình:

Chương trình quản lý thư viện

1. Hiện thị danh sách sách
2. Thêm sách mới
3. Muôn sách
4. Tra sách
5. Thoát

Vui lòng chọn một tùy chọn (1-5): 2

Đang thêm sách mới...

4.13. Câu hỏi và Bài tập

4.13.1. Câu hỏi

Cho biết các câu là ĐÚNG hay SAI?

1. Toán tử = và toán tử == thực hiện cùng một phép toán khi được sử dụng trong một biểu thức Boolean.
2. Một khối if/else lồng nhau với một khối if/else khác giúp kiểm tra nhiều điều kiện.
3. Một câu lệnh được thực thi có điều kiện nên được thụt vào một cấp độ từ câu lệnh if.
4. Trình biên dịch báo cáo lỗi nếu các dòng trong một khối không được thụt đầu dòng.
5. Việc giả định rằng tất cả các biến chưa được khởi tạo tự động bắt đầu với giá trị 0 là an toàn.
6. Nếu một câu lệnh if lồng bên trong phần if của một câu lệnh khác, khối bên trong câu lệnh if bên trong sẽ được thực thi khi cả hai điều kiện đều sai.
7. Khi một câu lệnh if lồng trong phần else của một câu lệnh khác, như trong trường hợp if/else if, thì câu lệnh if bên trong chỉ được thực thi khi biểu thức của câu lệnh if bên ngoài là đúng.
8. Phạm vi của một biến bị giới hạn trong khối mà nó được định nghĩa.
9. Bạn có thể sử dụng các toán tử quan hệ để so sánh các đối tượng chuỗi.

10. Biểu thức 'a' == 'A' || 0 == 0 là đúng.

11. $y < x$ tương đương với $x \geq y$

12. $x \geq y$ không tương đương với $(x > y \ \&\& \ x = y)$

Giả sử các biến $x = 5$, $y = 6$ và $z = 8$. Cho biết bằng cách khoanh tròn T hoặc F xem mỗi điều kiện sau là Đúng hay Sai:

13. $x == 5 \ \&\& \ y > 3$

14. $x == 5 \ \&\& \ (y \leq 10 \ \&\& \ z > 10)$

15. $2 != y \ \&\& \ z != 4$

16. $!(x < 5 \ \&\& \ z > 5)$

4.13.2. Bài tập

1. Tìm Max/Min (Bắt buộc)

Viết chương trình yêu cầu người dùng nhập vào hai số, sử dụng toán tử điều kiện (toán tử ? :) để in ra số lớn hơn và số nhỏ hơn trong 2 số.

2. Chuyển đổi số La Mã

Viết chương trình yêu cầu người dùng nhập vào một số tự nhiên trong khoảng từ 1 đến 10, sử dụng câu lệnh switch để in ra dạng số La Mã tương ứng của số đó.

Chú ý: kiểm tra đầu vào, không chấp nhận các số nhỏ hơn 1 hoặc lớn hơn 10.

3. Ngày phép thuật (Bắt buộc)

Ngày 10 tháng 6 năm 1960 được viết dưới dạng 6/10/60 có đặc điểm là tháng nhân với ngày thì bằng năm (chỉ tính 2 chữ số cuối của năm) thì được coi là ngày phép thuật

Viết chương trình yêu cầu người dùng nhập vào ngày, tháng, năm (chỉ nhập 2 chữ số cuối). In ra ngày đó có phải là ngày phép thuật không?

4. Diện tích hình chữ nhật

Viết chương trình yêu cầu người dùng nhập chiều dài và rộng của 2 hình chữ nhật. sau đó cho biết hình nào có diện tích lớn hơn, nếu bằng nhau thì thông báo bằng nhau

5. Chỉ số BMI

Chỉ số BMI được tính theo công thức lấy cân nặng (kg) chia cho bình phương của chiều cao (tính theo mét). Nếu BMI lớn hơn 25 là thừa cân, nhỏ hơn 18.5 là gầy, trong khoảng 18.5 đến 25 là bình thường.

Viết chương trình yêu cầu người dùng nhập chiều cao và cân nặng của họ. sau đó cho biết họ đang ở mức cân nặng nào trong 3 loại trên

6. Khối lượng và Trọng lượng

Viết chương trình nhập vào cân nặng của đồ vật và in ra trọng lượng của nó. Nếu trọng lượng đạt trên 100 thì thông báo vật quá nặng, nếu trọng lượng nhỏ hơn 10 thì thông báo vật quá nhẹ. Biết trọng lượng của 1 vật được tính bằng công thức lấy khối lượng (kg) nhân với 9.8.

7. Tính giờ (Bắt buộc)

Viết chương trình yêu cầu người dùng nhập vào số giây. Sau đó in ra khoảng thời gian đó là mấy ngày, giờ, phút, giây.

Chú ý:

- Nếu số ngày bằng không thì không cần in ra số ngày
- Nếu số giờ bằng không thì không cần in ra số giờ
- Nếu số phút bằng không thì không cần in ra số phút
- Nếu số giây bằng không thì không cần in ra số giây

8. Trộn màu

Ba màu cơ bản trong một chế độ màu nào đó là red, blue, và yellow. Khi trộn từng cặp 2 màu ta sẽ có:

- red + blue → purple (màu tím)
- red + yellow → orange (màu cam)
- blue + yellow → green (màu xanh lá cây)

Viết chương trình yêu cầu nhập tên 2 màu cơ bản, in ra màu kết quả khi pha trộn chúng. Nếu người dùng nhập không đúng màu cơ bản thì thông báo lỗi

9. Trò chơi đổi 1 Đô la

Viết chương trình yêu cầu người dùng nhập 4 số tương ứng số cent (1 đô la bằng 100 cent), nickel (bằng 5 cent), dime (bằng 10 cent), quarter (bằng 35 cent) để khi cộng lại thì được 1 đô la. Nếu cộng lại vừa đúng 1 đô la thì thông báo người chơi thắng, nếu không phải thì thông báo kết quả lớn hơn hay nhỏ hơn 1 đô la.

10. Số ngày của 1 tháng (Bắt buộc)

Viết chương trình yêu cầu người dùng nhập tháng và năm, cho biết tháng đó có bao nhiêu ngày. Sử dụng tiêu chí sau để xác định năm nhuận:

- Xác định xem năm có chia hết cho 100 hay không. Nếu đó là một năm nhuận nếu và chỉ nếu năm đó chia hết cho 400. Ví dụ, 2000 là năm nhuận nhưng 2100 không phải năm nhuận.
- Nếu năm không chia hết cho 100 thì đó là năm nhuận nếu và chỉ nếu năm đó chia hết 4. Ví dụ, năm 2008 là năm nhuận nhưng năm 2009 không phải năm nhuận.

Dưới đây là ví dụ chương trình trình chạy:

Nhập vào 1 tháng (1-12): 2 [Enter]

Nhập vào 1 năm: 2008 [Enter]

Số ngày: 29

11. Gia sư toán (Bắt buộc)

Viết một chương trình có thể sử dụng như một gia sư về toán cho các bạn nhỏ. Chương trình sẽ hiển thị hai số ngẫu nhiên để thực hiện phép cộng, ví dụ:

$$\begin{array}{r} 247 \\ +129 \\ \hline \end{array}$$

Sau đó chờ người dùng nhập tổng của 2 số trên. Nếu kết quả đúng thì thông báo **chúc mừng**. Nếu sai thì thông báo **kết quả không chính xác** và kết quả đúng là bao nhiêu.

12. Bán phần mềm

Một công ty phần mềm bán gói hàng sản phẩm giá \$99. Mức chiết khấu số lượng theo bảng sau.

Số lượng	Chiết khấu
10-19	20%
20-49	30%
50-99	40%
Trên 100	50%

Viết một chương trình yêu cầu số lượng đơn vị bán và tính tổng chi phí mua.

13. Câu lạc bộ sách

Một hiệu sách tính điểm thưởng cho người mua tùy theo số sách khách hàng đã mua như sau:

- 0 cuốn sách → 0 điểm
- 1 cuốn sách → 5 điểm
- 2 cuốn sách → 15 điểm
- 3 cuốn sách → 30 điểm
- từ 4 cuốn sách trở lên → 60 điểm

Viết chương trình nhập vào số sách của 1 khách hàng và in ra số điểm thưởng của họ là bao nhiêu.

14. Phí ngân hàng

Ngân hàng tính phí 10\$/1 tháng cộng với phí sử dụng séc như sau:

- 0.1\$ mỗi séc nếu số lượng không quá 20 séc
- 0.08\$ mỗi séc nếu số lượng séc trong khoảng 20 đến 39
- 0.06\$ mỗi séc nếu số lượng séc trong khoảng 40 đến 59
- 0.04\$ mỗi séc nếu số lượng séc trong từ 60 trở lên

Mặt khác sẽ tính thêm 15\$ nếu số dư trong tài khoản (trước khi trừ 2 loại phí ở trên) dưới 400\$

Viết chương trình nhập vào số dư tài khoản và số lượng séc sử dụng trong tháng sau đó in ra các khoản phí người dùng phải trả.

Chú ý: không chấp nhận người dùng nhập số âm.

15. Phí vận chuyển

Công ty vận tải hàng hóa nhanh thu phí sau:

Trọng lượng (Kilogram)	Mức phí trên mỗi dặm
Nhỏ hơn 2kg	\$1.10
Từ 2kg → 6kg	\$2.20
Từ 6kg → 10kg	\$3.70
Từ 10kg → 20kg	\$4.80

Viết một chương trình yêu cầu trọng lượng của gói và khoảng cách để vận chuyển, sau đó hiển thị các khoản phí.

Chú ý:

- Không chấp nhận giá trị 0 hoặc ít hơn cho trọng lượng của gói. Không chấp nhận trọng lượng hơn 20 kg (đây là trọng lượng tối đa của công ty).
- Không chấp nhận khoảng cách dưới 10 dặm hoặc hơn 3.000 dặm. Đây là khoảng cách tối thiểu và tối đa của công ty.

16. Chạy đua

Viết chương trình nhập vào tên và thời gian hoàn thành cuộc đua của 3 vận động viên. Sau đó in ra xếp hạng của 3 người theo thứ tự nhất nhì ba.

Chú ý: báo lỗi nếu nhập thời gian đua là số âm.

17. Cá nhân tốt nhất

Viết chương trình yêu cầu nhập tên vận động viên nhảy sào, ngày thi đấu, chiều cao nhảy (được tính bằng mét) của 3 vận động viên có thành tích tốt nhất. Sau đó chương trình sẽ hiển thị danh sách theo thứ tự thành tích đạt được.

Xác thực đầu vào: Chỉ chấp nhận các giá trị từ 2.0m đến 5.0m cho các chiều cao

18. Tính gram chất béo

Viết một chương trình yêu cầu nhập vào tổng lượng calo và gram chất béo trong một loại thực phẩm. Chương trình sẽ hiển thị phần trăm calo có trong chất béo. Nếu lượng calo có trong chất béo ít hơn 30% tổng lượng calo của thực phẩm thì hiển thị thông báo cho biết thực phẩm đó ít chất béo. Biết:

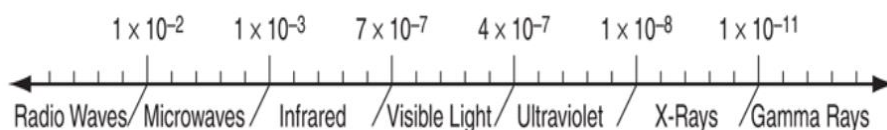
- 1 gram chất béo có 9 calo
- Lượng calo trong số gram chất béo = số gram chất béo*9
- Tỷ lệ calo có trong thực phẩm = Lượng calo trong số gram chất béo / Tổng lượng calo

Xác nhận đầu vào: Đảm bảo số lượng calo và gram chất béo không nhỏ hơn 0. Ngoài ra, số lượng calo từ chất béo không được lớn hơn tổng số calo (Số gram chất béo*9 phải nhỏ hơn Tổng lượng calo). Nếu điều đó xảy ra, hãy hiển thị thông báo lỗi cho biết rằng số gram calo hoặc chất béo đã được nhập không chính xác.

19. Phân loại sóng điện từ:

Viết chương trình nhập vào bước sóng của 1 sóng điện từ. Hãy chỉ ra nó là loại sóng gì biết:

- Sóng radio bước sóng $\geq 10^{-2}$
- Sóng của lò vi sóng từ 10^{-3} đến 10^{-2}
- Sóng hồng ngoại 7×10^{-7} đến 10^{-2}
- Sóng ánh sáng 4×10^{-7} đến 7×10^{-7}
- Sóng tử ngoại 1×10^{-8} đến 4×10^{-7}
- Tia X quang: 1×10^{-11} đến 1×10^{-8}
- Tia gamma: nhỏ hơn 1×10^{-11}



20. Tốc độ âm thanh

Bảng sau đây cho thấy tốc độ gần đúng của âm thanh trong không khí, nước và thép.

Môi trường	Tốc độ
Không khí	1.100 feet/giây
Nước	4.900 feet/giây
Thép	16.400 feet/giây

Viết chương trình hiển thị menu cho phép người dùng chọn: không khí, nước hoặc thép. Sau khi người dùng thực hiện lựa chọn, sẽ được yêu cầu nhập khoảng cách mà sóng âm thanh sẽ truyền trong môi trường đã chọn. Sau đó, chương trình sẽ hiển thị khoảng thời gian cần thiết. (Làm tròn câu trả lời đến bốn chữ số thập phân.)

Xác thực đầu vào: Kiểm tra xem người dùng đã chọn một trong các lựa chọn có sẵn từ menu chưa ? Không chấp nhận khoảng cách nhỏ hơn 0.

21. Tốc độ âm thanh trong khí

Khi âm thanh truyền qua chất khí, tốc độ của nó phụ thuộc chủ yếu vào mật độ của môi trường. Môi trường càng ít đặc thì tốc độ sẽ càng nhanh. Bảng sau đây cho thấy tốc độ gần đúng của âm thanh ở 0 độ C, được đo bằng mét trên giây, khi di chuyển qua carbon dioxide, không khí, helium và hydro.

Loại khí	Tốc độ (m/s)
Carbon Dioxide	258.0
Không khí	331.5
Helium	972.2
Hydro	1,270.0

Viết chương trình hiển thị menu cho phép người dùng chọn một trong bốn loại khí này. Sau khi lựa chọn được thực hiện, người dùng nên nhập số giây cần thiết để âm thanh di chuyển trong loại khí này từ nguồn của nó đến vị trí mà nó được phát hiện. Sau đó, chương trình sẽ báo cáo khoảng cách (tính bằng mét) nguồn phát âm thanh từ vị trí phát hiện.

Xác thực đầu vào: Kiểm tra xem người dùng đã chọn một trong các lựa chọn menu có đúng hay chưa. Không chấp nhận thời gian ít hơn 0 giây hoặc hơn 30 giây.

22. Điểm đông và sôi

Bảng sau liệt kê các điểm đông đặc và sôi của một số chất. Viết một chương trình yêu cầu người dùng nhập một nhiệt độ và sau đó hiển thị các chất sẽ đông đặc ở nhiệt độ đó và những chất sẽ sôi ở nhiệt độ đó. Ví dụ, nếu người dùng nhập -20, chương trình sẽ báo rằng nước sẽ đóng băng và oxy sẽ sôi ở nhiệt độ đó.

Chất	Nhiệt độ đông(°F)	Nhiệt độ sôi (°F)
Cồn Ethyl	-173	172
Thủy ngân	-38	676
Oxy	-362	-306
Nước	32	212

23. Tính hình học (Bắt buộc)

Viết chương trình hiển thị 1 thực đơn với các chức năng sau:

1. Tính diện tích hình tròn
2. Tính diện tích hình chữ nhật

3. Tính diện tích hình tam giác vuông

4. Thoát

Nhập lựa chọn [1-4] :

Yêu cầu chọn các chức năng từ 1 đến 4. Sau đó tùy lựa chọn của người dùng để thực hiện đúng chức năng. Biết :

- Diện tích hình tròn là πr^2
- Diện tích hình chữ nhật là dài x rộng.
- Diện tích tam giác vuông (biết độ dài 2 cạnh góc vuông a, b) là $0.5 \times ab$

24. Cuộc gọi đường dài

Một nhà cung cấp dịch vụ đường dài cho các cuộc gọi điện thoại:

Thời gian gọi	Tiền trên 1 phút (cent)
00:00-06 :59	0.05
07 :00-19 :00	0.45
19 :01-23 :59	0.20

Viết chương trình yêu cầu thời gian bắt đầu và số phút của cuộc gọi, đồng thời hiển thị cước phí. Chương trình sẽ yêu cầu thời gian được nhập dưới dạng số dấu chấm động ở dạng HH.MM. Ví dụ: 07:00 giờ sẽ được nhập là 07.00 và 16:28 giờ sẽ được nhập là 16.28.

Xác thực đầu vào: Chương trình không chấp nhận thời gian lớn hơn 23:59. Ngoài ra, không có số nào có hai chữ số cuối cùng lớn hơn 59 sẽ được chấp nhận. Gợi ý: Giả sử num là một biến dấu phẩy động, biểu thức sau sẽ cung cấp cho bạn phần phân số của nó: `num - static_cast <int> (num)`

25. Nhà cung cấp dịch vụ di động

Một nhà cung cấp dịch vụ điện thoại di động có ba gói đăng ký khác nhau cho khách hàng của mình:

- Gói A: Giá \$ 39,99 trên 1 tháng thì được cung cấp 450 phút. Số phút thêm là \$ 0,45 cho mỗi phút.
- Gói B: Giá \$ 59,99 trên 1 tháng thì được cung cấp 900 phút. Số phút thêm là \$ 0,40 cho mỗi phút.
- Gói C: Giá \$ 69,99 trên 1 tháng thì được cung cấp số phút không giới hạn.

Viết chương trình tính toán hóa đơn hàng tháng của khách hàng. Chương trình sẽ hỏi khách hàng đã mua gói nào và đã sử dụng bao nhiêu phút. Sau đó, nó sẽ hiển thị tổng số tiền đến hạn.

Xác thực đầu vào: Đảm bảo người dùng chỉ chọn gói A, B hoặc C

CHƯƠNG 5. VÒNG LẶP VÀ TỆP

Chủ đề	
5.1	Toán tử quan hệ
5.2	Câu lệnh if khuyết
5.3	Câu lệnh if/else
5.4	Câu lệnh if lồng nhau
5.5	Câu lệnh if/else if
5.6	Biến cờ (Flags)
5.7	Toán tử lôgic
5.8	Xác thực đầu vào
5.9	So sánh chuỗi và các kí tự

5.1. Các Toán Tử Tăng và Giảm

KHÁI NIỆM: ++ và -- là các toán tử tăng và giảm giá trị của toán hạng lên hoặc xuống 1 đơn vị.

Để tăng một giá trị có nghĩa là tăng nó lên một, và để giảm một giá trị có nghĩa là giảm nó đi một. Cả hai câu lệnh sau đều tăng giá trị của biến num:

```
num = num + 1;
```

hoặc `num += 1;`

Và num được giảm trong cả hai câu lệnh sau:

```
num = num - 1;
```

hoặc `num -= 1;`

C++ cung cấp một tập hợp các toán tử đơn giản chỉ dành riêng để tăng và giảm các biến. Toán tử tăng là ++, và toán tử giảm là --. Câu lệnh sau sử dụng toán tử ++ để tăng num:

```
num++;
```

Và câu lệnh sau đây giảm num:

```
num--;
```

Ví dụ trên đã giới thiệu về các toán tử tăng giá trị (increment) và giảm giá trị (decrement) được sử dụng theo chế độ hậu tố, tức là toán tử được đặt sau biến. Các toán tử này cũng có thể hoạt động theo chế độ tiền tố, nơi toán tử được đặt trước tên biến:

```
++num;
```

```
--num;
```

Trong cả hai chế độ hậu tố và tiền tố, những toán tử này tăng hoặc giảm 1 đơn vị cho biến của chúng. Ví dụ sau đây cho thấy cách hoạt động.

Chương trình 5.1. Minh họa toán tử ++, --

```
// Chương trình này minh họa các toán tử ++ và --.  
#include <iostream>
```

```

using namespace std;
int main()
{
    int num = 4; // num bat dau voi 4.

    // Hien thi gia tri trong num.
    cout << "Bien num la " << num << endl;
    cout << "Toi se tang num bay gio.\n\n";

    // Su dung ++ hau to de tang num.
    num++;
    cout << "Bay gio bien num la " << num << endl;
    cout << "Toi se tang num mot lan nua.\n\n";

    // Su dung ++ tien to de tang num.
    ++num;
    cout << "Bay gio bien num la " << num << endl;
    cout << "Toi se giam num bay gio.\n\n";

    // Su dung -- hau to de giam num.
    num--;
    cout << "Bay gio bien num la " << num << endl;
    cout << "Toi se giam num mot lan nua.\n\n";

    // Su dung -- tien to de giam num.
    --num;
    cout << "Bay gio bien num la " << num << endl;
    return 0;
}

```

Kết quả chương trình:

```

Bien num la 4
Toi se tang num bay gio.

Bay gio bien num la 5
Toi se tang num mot lan nua.

Bay gio bien num la 6
Toi se giam num bay gio.

Bay gio bien num la 5
Toi se giam num mot lan nua.

```

Sự khác biệt giữa các chế độ hậu tố và tiền tố là quan trọng khi các toán tử này được sử dụng trong các câu lệnh làm nhiều hơn là chỉ tăng hoặc giảm. Chẳng hạn, xem xét các câu lệnh sau:

```

num = 4;
cout << num++;

```

Câu lệnh `cout` này làm hai việc: (1) hiển thị giá trị của `num`, và (2) tăng `num`. Nhưng điều gì xảy ra trước? `cout` sẽ hiển thị một giá trị khác nếu `num` được tăng trước so với sau khi được tăng. Câu trả lời phụ thuộc vào chế độ của toán tử tăng.

Chế độ hậu tố khiến việc tăng xảy ra sau khi giá trị của biến được sử dụng trong biểu thức. Trong ví dụ, `cout` sẽ hiển thị 4, sau đó `num` được tăng lên 5. Tuy nhiên, chế độ tiền tố khiến việc tăng xảy ra trước. Trong các câu lệnh sau, `num` sẽ được tăng lên 5, sau đó `cout` sẽ hiển thị 5:

```
num = 4;
cout << ++num;
```

Sau đây là ví dụ minh họa thêm về toán tử tiền tố, hậu tố với `++`, `--`.

Chương trình 5.2. Minh họa toán tử tiền tố, hậu tố

```
// Chương trình này minh họa các chế độ tiền tố và hậu tố
// của các toán tử tăng và giảm.
#include <iostream>
using namespace std;

int main()
{
    int num = 4;

    cout << num << endl;    // Hiển thị 4
    cout << num++ << endl;  // Hiển thị 4, sau đó thêm 1 vào num
    cout << num << endl;    // Hiển thị 5
    cout << ++num << endl;  // Thêm 1 vào num, sau đó hiển thị 6
    cout << endl;          // Hiển thị một dòng trống

    cout << num << endl;    // Hiển thị 6
    cout << num-- << endl;  // Hiển thị 6, sau đó trừ 1 từ num
    cout << num << endl;    // Hiển thị 5
    cout << --num << endl;  // Trừ 1 từ num, sau đó hiển thị 4

    return 0;
}
```

Kết quả chương trình:

```
4
4
5
6

6
6
5
4
```

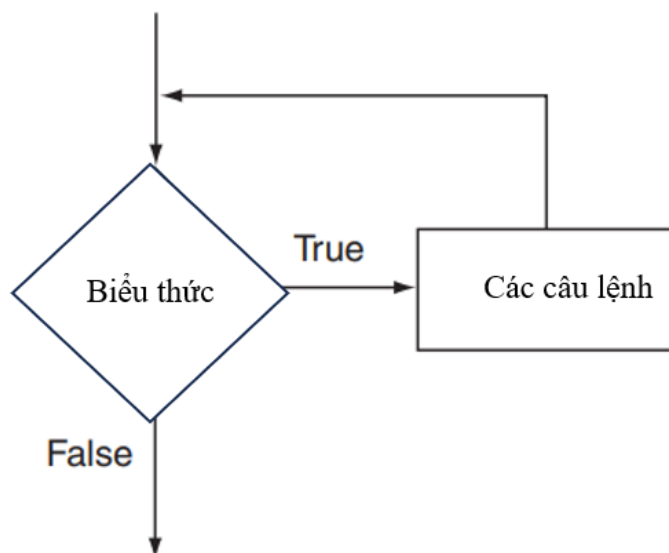
5.2. Giới thiệu về Vòng Lặp: Vòng Lặp while

KHÁI NIỆM: Một vòng lặp là một phần của chương trình lặp đi lặp lại.

Chương 4 đã giới thiệu về khái niệm các cấu trúc rẽ nhánh, điều hướng xử lý của chương trình. Một vòng lặp là một cấu trúc điều khiển khiến cho một câu lệnh hoặc một nhóm các câu lệnh được lặp lại. C++ có ba cấu trúc điều khiển vòng lặp: vòng lặp while, vòng lặp do-while và vòng lặp for. Sự khác biệt giữa các cấu trúc này là cách chúng điều khiển sự lặp lại.

Vòng Lặp while

Vòng lặp while có hai phần quan trọng: (1) một biểu thức được kiểm tra xem có giá trị đúng hay sai, và (2) một câu lệnh hoặc khối lệnh được lặp lại miễn là biểu thức đúng. Hình 5-1 cho thấy lô-gic của vòng lặp while.



Hình 5.1. Lô-gic của vòng lặp

Đây là cú pháp chung của vòng lặp while:

```
while (biểu thức)  
    câu lệnh;
```

Trong định dạng chung, biểu thức là bất kỳ biểu thức nào có thể được đánh giá là đúng hoặc sai, và câu lệnh là bất kỳ câu lệnh C++ hợp lệ nào. Dòng đầu tiên hiển thị trong định dạng này đôi khi được gọi là tiêu đề vòng lặp. Nó bao gồm từ khóa while theo sau là một biểu thức được đặt trong dấu ngoặc đơn.

Cách vòng lặp hoạt động như sau: biểu thức được kiểm tra, và nếu nó đúng, câu lệnh được thực thi. Sau đó, biểu thức được kiểm tra lại. Nếu nó đúng, câu lệnh được thực thi. Chu trình này lặp lại cho đến khi biểu thức sai.

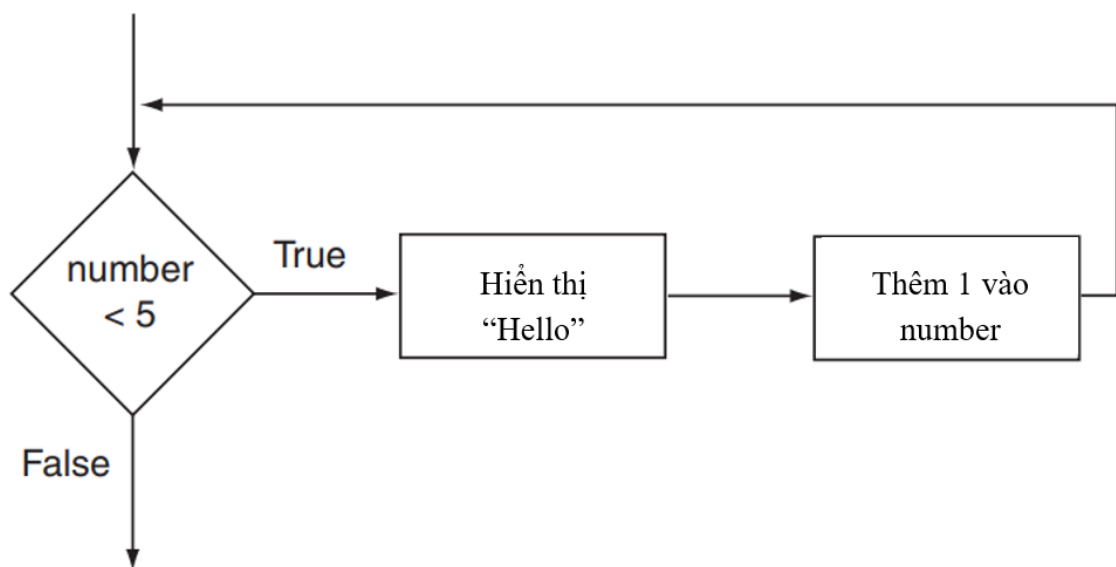
Câu lệnh được lặp lại được biết đến là phần thân của vòng lặp. Nó cũng được coi là một câu lệnh được thực thi có điều kiện, vì nó chỉ được thực thi dưới điều kiện biểu thức là đúng.

Chú ý không có dấu chấm phẩy sau biểu thức trong dấu ngoặc đơn. Giống như câu lệnh if, vòng lặp while không hoàn chỉnh nếu không có câu lệnh theo sau nó.

Nếu bạn muốn vòng lặp while lặp lại một khối các câu lệnh, định dạng của nó là:

```
while (biểu thức)
{
    câu lệnh;
    câu lệnh;
    // Đặt nhiều câu lệnh ở đây theo nhu cầu.
}
```

Vòng lặp while hoạt động giống như một câu lệnh if thực hiện đi thực hiện lại. Miễn là biểu thức bên trong dấu ngoặc đơn là đúng, câu lệnh hoặc khối câu lệnh được thực thi có điều kiện sẽ lặp lại. Chương trình sau sử dụng vòng lặp while để in “Hello” năm lần.



Hình 5.2. Sơ đồ thuật toán cho vòng lặp while

Chương trình 5.3. Sử dụng vòng lặp while để in 5 lần chữ “Hello”

```
// Đây là chương trình minh họa vòng lặp while.
#include <iostream>
using namespace std;
int main()
{
    int number = 0;
    while (number < 5)
    {
        cout << "Hello\n";
        number++;
    }
}
```

```

    }
    cout << "The thoi!\n";
    return 0;
}

```

Kết quả chương trình:

```

Hello
Hello
Hello
Hello
Hello
The thoi!

```

5.3. Sử Dụng Vòng Lặp while cho Kiểm Tra Dữ Liệu Đầu Vào

KHÁI NIỆM: Vòng lặp while có thể được sử dụng để tạo các thủ tục nhập liệu lặp lại cho đến khi dữ liệu nhập vào là chấp nhận được.

Câu nói nổi tiếng nhất của ngành công nghiệp máy tính là “rác vào, rác ra.” (“garbage in, garbage out.”). Tính toàn vẹn của đầu ra của một chương trình chỉ tốt bằng đầu vào của nó, vì vậy bạn nên cố gắng đảm bảo rằng rác không vào các chương trình của bạn. Kiểm tra dữ liệu đầu vào là quá trình kiểm tra dữ liệu do người dùng cung cấp cho chương trình và xác định liệu nó có hợp lệ hay không. Một chương trình tốt nên cung cấp hướng dẫn rõ ràng về loại đầu vào nào là chấp nhận được và không cho rằng người dùng đã tuân theo những hướng dẫn đó.

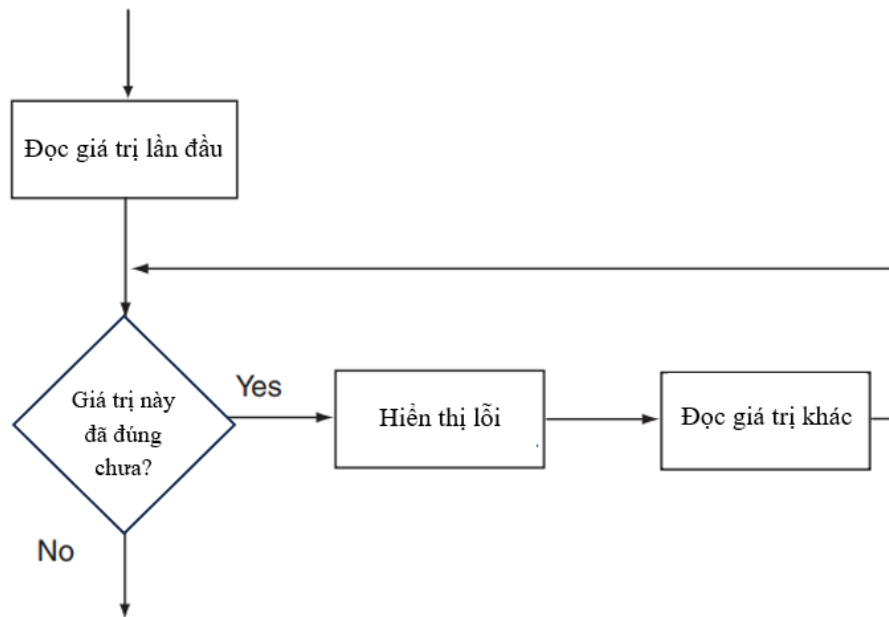
Vòng lặp while đặc biệt hữu ích để kiểm tra dữ liệu đầu vào. Nếu một giá trị không hợp lệ được nhập, một vòng lặp có thể yêu cầu người dùng nhập lại nhiều lần như cần thiết. Ví dụ, vòng lặp sau yêu cầu một số trong phạm vi từ 1 đến 100:

```

cout << "Nhập một số trong phạm vi 1-100: ";
cin >> so;
while (so < 1 || so > 100)
{
    cout << "LỖI: Nhập một giá trị trong phạm vi 1-100: ";
    cin >> so;
}

```

Đoạn chương trình trên đầu tiên cho phép người dùng nhập một số. Điều này diễn ra ngay trước vòng lặp. Nếu đầu vào hợp lệ, vòng lặp sẽ không thực thi. Nếu đầu vào không hợp lệ, tuy nhiên, vòng lặp sẽ hiển thị thông báo lỗi và yêu cầu người dùng nhập một số khác. Vòng lặp sẽ tiếp tục thực thi cho đến khi người dùng nhập một số hợp lệ. Lô-gic chung của việc thực hiện kiểm tra dữ liệu đầu vào được hiển thị trong hình dưới.



Hình 5.3. Sơ đồ thuật toán xác thực giá trị đầu vào

Việc đọc diễn ra ngay trước vòng lặp được gọi là đọc chuẩn bị. Nó cung cấp giá trị đầu tiên cho vòng lặp để kiểm tra. Các giá trị tiếp theo được lấy bởi vòng lặp.

5.4. Bộ Đếm

KHÁI NIỆM: Một bộ đếm là một biến được tăng hoặc giảm định kỳ mỗi khi vòng lặp lặp lại.

Đôi khi điều quan trọng đối với một chương trình là kiểm soát hoặc theo dõi số lần lặp lại của một vòng lặp. Ví dụ, Chương trình dưới đây hiển thị một bảng gồm các số từ 1 đến 10 và bình phương của chúng, vì vậy vòng lặp của nó phải lặp lại 10 lần.

Chương trình 5.4. In số và bình phương của số

```

// Chương trình này hiển thị một danh sách các số và bình phương của chúng.
#include <iostream>
using namespace std;
int main()
{
    const int MIN_NUMBER = 1, // Số bắt đầu để bình phương
            MAX_NUMBER = 10; // Giá trị tối đa để bình phương

    int num = MIN_NUMBER; // Bộ đếm

    cout << "Số        Số Bình Phương\n";
    cout << "-----\n";
    while (num <= MAX_NUMBER)
    {
        cout << num << "\t\t" << (num * num) << endl;
        num++; // Tăng bộ đếm.
    }
    return 0;
}

```

}	
Kết quả chương trình:	
So	So Bình Phương

1	1
2	4
3	9
4	16
5	25
6	36
7	49
8	64
9	81
10	100

Trong chương trình trên, biến num, bắt đầu từ 1, được tăng lên mỗi lần qua vòng lặp. Khi num đạt đến 11, vòng lặp dừng lại. num được sử dụng làm biến đếm, có nghĩa là nó được tăng định kỳ trong mỗi lần lặp lại của vòng lặp. Về bản chất, num giữ số lần lặp lại mà vòng lặp đã thực hiện.

5.5. Vòng lặp do-while

KHÁI NIỆM: Vòng lặp do-while là một vòng lặp kiểm tra sau, có nghĩa là biểu thức của nó được kiểm tra sau mỗi lần lặp.

Vòng lặp do-while trông giống như một vòng lặp while bị đảo ngược. Dưới đây là định dạng của vòng lặp do-while khi thân vòng lặp chỉ chứa một câu lệnh:

```
do
    câu lệnh;
while (biểu thức);
```

LƯU Ý: Vòng lặp do-while phải được kết thúc bằng dấu chấm phẩy (;)

Dưới đây là định dạng của vòng lặp do-while khi thân vòng lặp chứa nhiều câu lệnh:

```
do
{
    câu lệnh;
    câu lệnh;
    // Đặt nhiều câu lệnh ở đây theo nhu cầu.
} while (biểu thức);
```

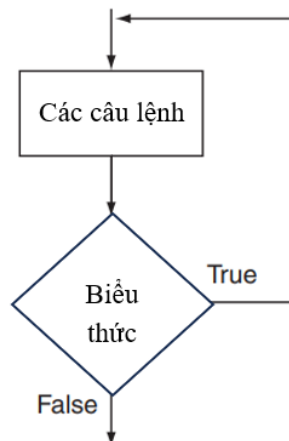
Vòng lặp do-while là một vòng lặp kiểm tra sau. Điều này có nghĩa là nó không kiểm tra biểu thức của mình cho đến khi nó đã hoàn thành một lần lặp. Do đó, vòng lặp do-while luôn thực hiện ít nhất một lần lặp, ngay cả khi biểu thức ban đầu là sai. Điều này khác với hành vi của vòng lặp while, như bạn nhớ là một vòng lặp kiểm tra trước. Ví dụ, trong vòng lặp while sau đây câu lệnh cout sẽ không thực thi một lần nào:

```
int x = 1;
while (x < 0)
    cout << x << endl;
```

Nhưng câu lệnh cout trong vòng lặp do-while sau đây sẽ thực thi một lần bởi vì vòng lặp do-while không đánh giá biểu thức $x < 0$ cho đến cuối lần lặp.

```
int x = 1;
do
    cout << x << endl;
while (x < 0);
```

Hình dưới đây minh họa lô-gic của vòng lặp do-while.



Hình 5.4. Lô-gic vòng lặp do-while

Bạn nên sử dụng vòng lặp do-while khi bạn muốn đảm bảo rằng vòng lặp thực thi ít nhất một lần. Ví dụ, Chương trình dưới đây tính trung bình một loạt ba điểm số thi cho một sinh viên. Sau khi hiển thị trung bình, nó hỏi người dùng liệu họ có muốn tính trung bình một bộ điểm số khác không. Chương trình lặp lại miễn là người dùng nhập Y cho có.

Chương trình 5.5. Nhập và tính điểm trung bình

```
/* Chương trình này tính trung bình 3 điểm số. Nó lặp lại nhiều lần theo ý
muốn của người dùng.*/
#include <iostream>
using namespace std;
int main()
{
    int score1, score2, score3; // Ba điểm số
    double average;             // Điểm trung bình
    char again;                 // Để giữ dấu vào Y hoặc N
    do
    {
        // Nhập ba điểm số.
        cout << "Nhập 3 điểm và tôi sẽ tính trung bình chúng: ";
        cin >> score1 >> score2 >> score3;
        // Tính toán và hiển thị điểm trung bình.
        average = (score1 + score2 + score3) / 3.0;
        cout << "Điểm trung bình là " << average << ".\n";
    }
```

```

        // Người dùng có muốn tính trung bình một bộ điểm số khác không?
        cout << "Ban co muon tinh trung binh "
                <<"mot bo diem so khac khong? (Y/N) ";

        cin >> again;
    } while (again == 'Y' || again == 'y');
    return 0;
}

```

Kết quả chương trình:

```

Nhap 3 diem va toi se tinh trung binh chung: 5 6 7
Diem trung binh la 6.
Ban co muon tinh trung binh mot bo diem so khac khong? (Y/N) y
Nhap 3 diem va toi se tinh trung binh chung: 8 7 9
Diem trung binh la 8.
Ban co muon tinh trung binh mot bo diem so khac khong? (Y/N) n

```

Khi chạy chương trình này, người lập trình không biết số lần vòng lặp sẽ lặp lại bao nhiêu lần. Điều này là do vòng lặp hỏi người dùng liệu họ có muốn lặp lại quá trình không. Loại vòng lặp này được gọi là vòng lặp do người dùng điều khiển, bởi vì nó cho phép người dùng quyết định số lần lặp lại.

Sử dụng Vòng Lặp do-while với Menu

Vòng lặp do-while là một lựa chọn tốt để lặp lại một menu. Hãy nhớ lại Chương trình 4-27, đã hiển thị một menu các gói hội viên sức khỏe. Chương trình dưới đây là một sửa đổi của chương trình đó, sử dụng vòng lặp do-while để lặp lại chương trình cho đến khi người dùng chọn mục 4 từ menu.

Chương trình 5.6. Sử dụng while để lặp lại menu

```

#include <iostream>
using namespace std;
int main() {
    int lua_chon ;
    do
    {
        cout << "Chương trình quản lý thư viện" << endl;
        cout << "1. Hiện thị danh sách sách" << endl;
        cout << "2. Thêm sách mới" << endl;
        cout << "3. Muốn sách" << endl;
        cout << "4. Trả sách" << endl;
        cout << "5. Thoát" << endl;
        cout << "Vui lòng chọn một tùy chọn (1-5): ";
        cin >> lua_chon ;

        switch (lua_chon ) {
            case 1:
                cout << "Đang hiện thị danh sách sách..." << endl;
            case 2:
                cout << "Đang thêm sách mới..." << endl;
            case 3:

```

```

        cout << "Dang muon sach..." << endl;
    case 4:
        cout << "Dang tra sach..." << endl;
    case 5:
        cout << "Dang thoat chuong trinh..." << endl;
    default:
        cout << "Lua chon khong hop le. Vui long chon lai." <<
endl;
    }

    }while (lua_chon!=5);
    cout<<"Tam biet!";
    return 0;
}

```

Kết quả chương trình:

```

Chương trình quan ly thu vien
1. Hien thi danh sach sach
2. Them sach moi
3. Muon sach
4. Tra sach
5. Thoat
Vui long chon mot tuy chon (1-5): 3
Dang muon sach...
Dang tra sach...
Dang thoat chuong trinh...
Lua chon khong hop le. Vui long chon lai.
Chương trình quan ly thu vien
1. Hien thi danh sach sach
2. Them sach moi
3. Muon sach
4. Tra sach
5. Thoat
Vui long chon mot tuy chon (1-5): 4
Dang tra sach...
Dang thoat chuong trinh...
Lua chon khong hop le. Vui long chon lai.
Chương trình quan ly thu vien
1. Hien thi danh sach sach
2. Them sach moi
3. Muon sach
4. Tra sach
5. Thoat
Vui long chon mot tuy chon (1-5): 5
Dang thoat chuong trinh...

```

Lua chon khong hop le. Vui long chon lai.
Tam biet!

5.6. Vòng lặp for

KHÁI NIỆM: Vòng lặp for là lý tưởng để thực hiện một số lần lặp xác định.

Khi bạn biết chính xác số lần lặp mà một vòng lặp phải thực hiện. Một vòng lặp lặp lại một số lần nhất định được gọi là vòng lặp kiểm soát đếm. Ví dụ, nếu một vòng lặp yêu cầu người dùng nhập vào số tiền bán hàng cho mỗi tháng trong năm, nó sẽ lặp lại 12 lần. Về bản chất, vòng lặp đếm đến 12 và yêu cầu người dùng nhập số tiền bán hàng mỗi khi nó đếm. Một vòng lặp kiểm soát đếm phải có ba yếu tố:

1. Nó phải khởi tạo một biến đếm với một giá trị bắt đầu.
2. Nó phải kiểm tra biến đếm bằng cách so sánh nó với một giá trị tối đa. Khi biến đếm đạt đến giá trị tối đa của nó, vòng lặp sẽ kết thúc.
3. Nó phải cập nhật biến đếm trong mỗi lần lặp. Thông thường, điều này được thực hiện bằng cách tăng giá trị của biến.

Vòng lặp kiểm soát đếm rất phổ biến đến nỗi C++ cung cấp một loại vòng lặp đặc biệt cho chúng. Đó là vòng lặp for. Vòng lặp for được thiết kế đặc biệt để khởi tạo, kiểm tra và cập nhật một biến đếm. Đây là định dạng của vòng lặp for khi nó được sử dụng để lặp lại một câu lệnh đơn:

```
for (khởi tạo; kiểm tra; cập nhật)  
    câu lệnh;
```

Định dạng của vòng lặp for khi nó được sử dụng để lặp lại một khối lệnh là:

```
for (khởi tạo; kiểm tra; cập nhật)  
{  
    câu lệnh;  
    câu lệnh;  
    // Đặt nhiều câu lệnh ở đây theo nhu cầu.  
}
```

Dòng đầu tiên của vòng lặp for là tiêu đề vòng lặp. Sau từ khóa for, có ba biểu thức trong dấu ngoặc đơn, được phân tách bởi dấu chấm phẩy. (Lưu ý rằng không có dấu chấm phẩy sau biểu thức thứ ba.) Biểu thức đầu tiên là biểu thức khởi tạo. Nó thường được sử dụng để khởi tạo một biến đếm với giá trị bắt đầu của nó. Đây là hành động đầu tiên được thực hiện bởi vòng lặp, và nó chỉ được thực hiện một lần. Biểu thức thứ hai là biểu thức kiểm tra. Đây là biểu thức điều khiển việc thực hiện của vòng lặp. Miễn là biểu thức này đúng, thân của vòng lặp for sẽ lặp lại. Vòng lặp for là một vòng lặp kiểm tra trước, vì vậy nó đánh giá biểu thức kiểm tra trước mỗi lần lặp. Biểu thức thứ ba là biểu thức cập nhật. Nó thực thi vào cuối mỗi lần lặp. Thông thường, đây là câu lệnh tăng biến đếm của vòng lặp.

Dưới đây là một ví dụ về một vòng lặp for đơn giản in ra "Hello" năm lần:

```
for (count = 0; count < 5; count++)  
    cout << "Hello" << endl;
```

Trong vòng lặp này, biểu thức khởi tạo là `count = 0`, biểu thức kiểm tra là `count < 5`, và biểu thức cập nhật là `count++`. Thân của vòng lặp có một câu lệnh, đó là câu lệnh `cout`. Hình dưới đây minh họa chuỗi các sự kiện diễn ra trong quá trình thực hiện của vòng lặp. Lưu ý Rằng các Bước 2 đến 4 được lặp lại miễn là biểu thức kiểm tra đúng.

Bước 1: Thực hiện biểu thức khởi tạo.

Bước 2: Đánh giá biểu thức kiểm tra. Nếu đúng, tiến tới Bước 3.

Nếu không, kết thúc vòng lặp.

Bước 4: Thực hiện biểu thức cập nhật, sau đó quay lại Bước 2.

for (count = 0; count < 5; count++)

cout << "Hello" << endl; ← **Bước 3:** Thực hiện thân của vòng lặp.

Chương trình sau in ra các số từ 1 đến 10 và bình phương của nó.

Chương trình 5.7. In các số từ 1 đến 10 và bình phương của các số

```
// Chương trình này hiển thị các số từ 1 đến 10 và bình phương của chúng.  
#include <iostream>  
using namespace std;  
int main()  
{  
    const int MIN_NUMBER = 1; // Giá trị bắt đầu  
    const int MAX_NUMBER = 10; // Giá trị kết thúc  
    int num;  
    cout << "Số\tSố bình phương\n";  
    cout << "-----\n";  
    for (num = MIN_NUMBER; num <= MAX_NUMBER; num++)  
        cout << num << "\t" << (num * num) << endl;  
    return 0;  
}
```

Kết quả chương trình:

Số	Số bình phương

1	1
2	4
3	9
4	16

5	25
6	36
7	49
8	64
9	81
10	100

Sử dụng vòng lặp for thay vì while hoặc do-while

Bạn nên sử dụng vòng lặp for thay vì vòng lặp while hoặc do-while trong bất kỳ tình huống nào rõ ràng yêu cầu một khởi tạo, sử dụng một điều kiện sai để dừng vòng lặp, và yêu cầu một cập nhật xảy ra vào cuối mỗi lần lặp lại. Chương trình trên là một ví dụ hoàn hảo. Nó yêu cầu biến num được khởi tạo là 1, nó dừng vòng lặp khi num lớn hơn 10, và nó tăng num vào cuối mỗi lần lặp lại.

Vòng lặp for là vòng lặp kiểm tra trước

Vì vòng lặp for kiểm tra biểu thức điều kiện của nó trước khi thực hiện một lần lặp, nó là một vòng lặp kiểm tra trước. Dưới đây là một ví dụ về vòng lặp for mà không lặp lần nào:

```
for (count = 11; count <= 10; count++)
    cout << "Hello" << endl;
```

Vì biến count được khởi tạo với một giá trị làm cho biểu thức kiểm tra sai ngay từ đầu, vòng lặp này kết thúc ngay khi nó bắt đầu.

Tránh sửa đổi biến Đếm trong thân của vòng lặp for

Hãy cẩn thận không đặt một câu lệnh sửa đổi biến đếm trong thân của vòng lặp for. Tất cả các sửa đổi của biến đếm nên diễn ra trong biểu thức cập nhật, được thực thi tự động vào cuối mỗi lần lặp. Nếu một câu lệnh trong thân của vòng lặp cũng sửa đổi biến đếm, vòng lặp có thể sẽ không kết thúc khi bạn mong đợi. Ví dụ, vòng lặp sau đây tăng x hai lần cho mỗi lần lặp:

```
for (x = 1; x <= 10; x++)
{
    cout << x << endl;
    x++; // Sai!
}
```

Các dạng khác của biểu thức cập nhật

Bạn không bị giới hạn chỉ sử dụng các câu lệnh tăng trong biểu thức cập nhật. Dưới đây là một vòng lặp hiển thị tất cả các số chẵn từ 2 đến 100 bằng cách cộng 2 vào biến đếm của nó:

```
for (num = 2; num <= 100; num += 2)
    cout << num << endl;
```

Và dưới đây là một vòng lặp đếm ngược từ 10 xuống 0:

```
for (num = 10; num >= 0; num--)
    cout << num << endl;
```

Tạo vòng lặp for do người dùng điều khiển

Đôi khi bạn muốn người dùng xác định giá trị tối đa của biến đếm trong một vòng lặp for, và do đó xác định số lần lặp lại của vòng lặp. Ví dụ, xem Chương trình dưới đây là một chương trình khác hiển thị danh sách các số và bình phương của chúng. Thay vì hiển thị các số từ 1 đến 10, chương trình này cho phép người dùng nhập giá trị tối thiểu và tối đa để hiển thị.

Chương trình 5.8. Dùng vòng for với khoảng giá trị

```
// Chương trình này chứng minh một vòng lặp for do người dùng điều khiển.
#include <iostream>
using namespace std;

int main()
{
    int minNumber, // Số bắt đầu để bình phương
        maxNumber; // Số kết thúc để bình phương

    // Nhận giá trị tối thiểu và tối đa để hiển thị.
    cout << "Tôi sẽ hiển thị một bảng các số và bình phương của chúng.\n";
    cout << "Nhập số bắt đầu: ";
    cin >> minNumber;
    cout << "Nhập số kết thúc: ";
    cin >> maxNumber;

    // Hiển thị bảng.
    cout << "Số \t\tSo Bình Phương\n";
    cout << "-----\n";

    for (int num = minNumber; num <= maxNumber; num++)
        cout << num << "\t\t" << (num * num) << endl;

    return 0;
}
```

Kết quả chương trình:

Tôi sẽ hiển thị một bảng các số và bình phương của chúng.

Nhập số bắt đầu: 3

Nhập số kết thúc: 9

Số	So Bình Phương
----	----------------

3	9
---	---

4	16
---	----

5	25
---	----

6	36
---	----

7	49
---	----

8	64
9	81

Trong vòng lặp này, biến num nhận các giá trị từ minNumber đến maxNumber, sau đó vòng lặp kết thúc.

Bỏ qua biểu thức trong ngoặc của vòng lặp for

Biểu thức khởi tạo có thể được bỏ qua bên trong ngoặc của vòng lặp for nếu nó đã được thực hiện trước đó hoặc không cần khởi tạo. Dưới đây là một ví dụ về vòng lặp for với khởi tạo được thực hiện trước vòng lặp:

```
int num = 1;
for ( ; num <= maxValue; num++)
    cout << num << "\t\t" << (num * num) << endl;
```

Cũng có thể bỏ qua biểu thức cập nhật nếu nó được thực hiện ở nơi khác trong vòng lặp hoặc nếu không cần thiết. Mặc dù loại mã này không được khuyến khích, vòng lặp for sau đây hoạt động giống như một vòng lặp while:

```
int num = 1;
for ( ; num <= maxValue; )
{
    cout << num << "\t\t" << (num * num) << endl;
    num++;
}
```

Thậm chí có thể bỏ qua tất cả ba biểu thức trong ngoặc của vòng lặp for. Tuy nhiên, hãy cảnh giác, bởi vì nếu bạn bỏ qua biểu thức kiểm tra, vòng lặp không có cách nào để kết thúc một cách tự nhiên. Dưới đây là một ví dụ:

```
for ( ; ; )
    cout << "Hello World\n";
```

Vì vòng lặp này không có cách nào để dừng lại, nó sẽ hiển thị "Hello World\n" mãi mãi (hoặc cho đến khi có điều gì đó làm gián đoạn chương trình).

5.7. Vòng lặp lồng nhau

KHÁI NIỆM: Vòng lặp nằm trong một vòng lặp khác được gọi là vòng lặp lồng nhau.

Vòng lặp lồng nhau là vòng lặp xuất hiện bên trong một vòng lặp khác. Đồng hồ là một ví dụ điển hình về một thứ hoạt động giống như một vòng lặp lồng nhau. Kim giây, kim phút và kim giờ đều quay quanh mặt đồng hồ. Mỗi lần kim giờ tăng thì kim phút tăng 60 lần. Mỗi lần kim phút tăng thì kim giây tăng 60 lần.

Đây là một đoạn chương trình có vòng lặp for mô phỏng một phần đồng hồ kỹ thuật số. Nó hiển thị số giây từ 0 đến 59.

```
cout << fixed << right;
cout.fill('0');
for (int seconds = 0; seconds < 60; seconds++)
    cout << setw(2) << seconds << endl;
```

LƯU Ý: Hàm thành viên điền của cout thay đổi ký tự điền, theo mặc định là khoảng trắng. Trong đoạn chương trình trên, hàm điền làm cho số 0 được in trước tất cả các số có một chữ số.

Chúng ta có thể thêm biến phút và lồng vòng lặp bên trên vào trong một vòng lặp khác có chu kỳ 60 phút.

```
cout << fixed << right;
cout.fill('0');
for (int minutes = 0; minutes < 60; minutes++)
{
    for (int seconds = 0; seconds < 60; seconds++)
    {
        cout << setw(2) << minutes << ":";
        cout << setw(2) << seconds << endl;
    }
}
```

Để hoàn thiện đồng hồ mô phỏng, có thể thêm một biến và vòng lặp khác để đếm số giờ.

```
cout << fixed << right;
cout.fill('0');
for (int hours = 0; hours < 24; hours++)
{
    for (int minutes = 0; minutes < 60; minutes++)
    {
        for (int seconds = 0; seconds < 60; seconds++)
        {
            cout << setw(2) << hours << ":";
            cout << setw(2) << minutes << ":";
            cout << setw(2) << seconds << endl;
        }
    }
}
```

Đầu ra của đoạn chương trình trên như sau:

00:00:00

00:00:01

00:00:02

...

23:59:59

Vòng lặp trong cùng sẽ lặp lại 60 lần cho mỗi lần lặp của vòng lặp giữa. Vòng lặp giữa sẽ lặp lại 60 lần cho mỗi lần lặp của vòng lặp ngoài cùng. Khi vòng lặp ngoài cùng lặp lại 24 lần, vòng lặp giữa sẽ lặp lại 1,440 lần và vòng lặp trong cùng sẽ lặp lại 86,400 lần! Ví dụ về đồng hồ mô phỏng đưa ra một số điểm về các vòng lặp lồng nhau:

- Một vòng lặp bên trong đi qua tất cả các lần lặp của nó cho mỗi lần lặp của một vòng lặp bên ngoài.
- Các vòng lặp bên trong hoàn thành các lần lặp của mình trước so với các vòng lặp bên ngoài.
- Để tính tổng số lần lặp của một vòng lặp lồng nhau, nhân số lần lặp của tất cả các vòng lặp.

Chương trình sau là một chương trình khác để tính điểm trung bình của các bài kiểm tra. Nó yêu cầu người dùng nhập số lượng học sinh và số lượng điểm kiểm tra cho mỗi học sinh. Một vòng lặp bên trong lồng, từ dòng 26 đến 33, yêu cầu nhập tất cả các điểm kiểm tra cho một học sinh, lặp lại một lần cho mỗi điểm kiểm tra. Vòng lặp bên ngoài từ dòng 23 đến 37 lặp lại một lần cho mỗi học sinh.

Chương trình 5.9. Tính trung bình các bài kiểm tra

```
// Chương trình này tính điểm trung bình các bài kiểm tra. Nó yêu cầu
// người dùng nhập số lượng học sinh và số lượng điểm kiểm tra cho mỗi học
// sinh.
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    int numStudents, // Số lượng học sinh
        numTests; // Số lượng bài kiểm tra cho mỗi học sinh
    double total, // Biến tích lũy cho tổng điểm
        average; // Điểm trung bình của bài kiểm tra

    // Thiết lập định dạng đầu ra số.
    cout << fixed << showpoint << setprecision(1);

    // Lấy số lượng học sinh.
    cout << "Chương trình này tính điểm trung bình các bài kiểm tra.\n";
    cout << "Bạn có bao nhiêu học sinh? ";
    cin >> numStudents;

    // Lấy số lượng điểm kiểm tra cho mỗi học sinh.
    cout << "Mỗi học sinh có bao nhiêu điểm kiểm tra? ";
    cin >> numTests;

    // Xác định điểm trung bình của mỗi học sinh.
    for (int student = 1; student <= numStudents; student++)
    {
        total = 0; // Khởi tạo biến tích lũy.
        for (int test = 1; test <= numTests; test++)
        {
            double score;
            cout << "Nhập điểm " << test << " cho ";
            cout << "học sinh " << student << ": ";
            cin >> score;
            total += score;
        }
    }
}
```

```

    }
    average = total / numTests;
    cout << "Diem trung binh cho hoc sinh " << student;
    cout << " la " << average << ".\n\n";
}
return 0;
}

```

Kết quả chương trình:

Chương trình này tính điểm trung bình các bài kiểm tra.

Bạn có bao nhiêu học sinh? 3

Mỗi học sinh có bao nhiêu điểm kiểm tra? 2

Nhập điểm 1 cho học sinh 1: 5

Nhập điểm 2 cho học sinh 1: 6

Điểm trung bình cho học sinh 1 là 5.5.

Nhập điểm 1 cho học sinh 2: 7

Nhập điểm 2 cho học sinh 2: 8

Điểm trung bình cho học sinh 2 là 7.5.

Nhập điểm 1 cho học sinh 3: 8

Nhập điểm 2 cho học sinh 3: 8

Điểm trung bình cho học sinh 3 là 8.0.

5.8. Sử dụng tệp để lưu trữ dữ liệu

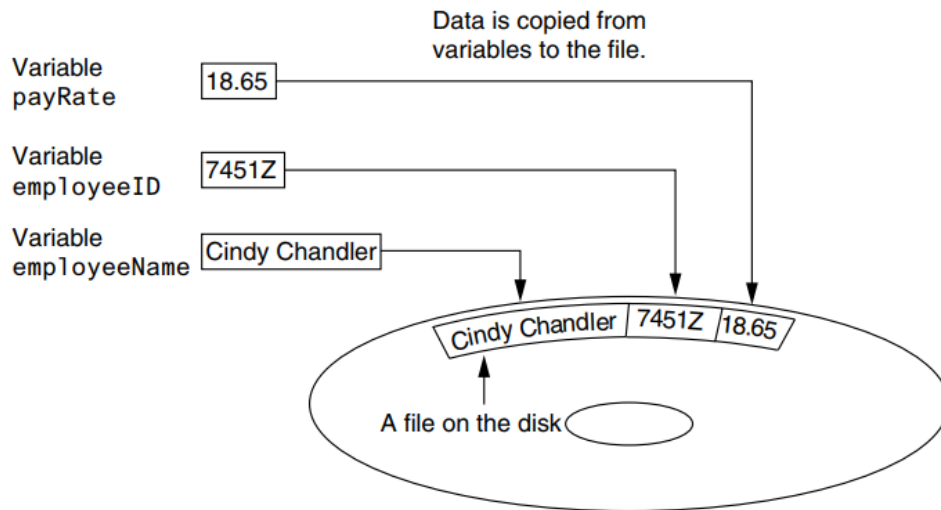
KHÁI NIỆM: Khi một chương trình cần lưu trữ dữ liệu để sử dụng sau này, nó sẽ ghi dữ liệu vào một tệp. Sau đó, dữ liệu có thể được đọc từ tệp vào một thời điểm sau.

Các chương trình mà bạn đã viết cho đến nay yêu cầu người dùng nhập lại dữ liệu mỗi khi chương trình được chạy, bởi vì dữ liệu được giữ trong các biến và thuộc tính điều khiển được lưu trữ trong RAM và biến mất ngay khi chương trình ngừng chạy. Nếu một chương trình cần giữ dữ liệu giữa các lần chạy, nó phải có một cách để lưu trữ nó. Dữ liệu được lưu trong một tệp, thường được lưu trữ trên đĩa của máy tính. Một khi dữ liệu được lưu trong tệp, nó sẽ ở lại đó sau khi chương trình ngừng chạy. Dữ liệu được lưu trong tệp sau đó có thể được truy xuất và sử dụng vào một thời điểm sau.

Hầu hết phần mềm thương mại mà bạn sử dụng hàng ngày lưu trữ dữ liệu trong các tệp. Ví dụ như: chương trình xử lý văn bản, hình ảnh, bảng tính, trò chơi, duyệt web,...

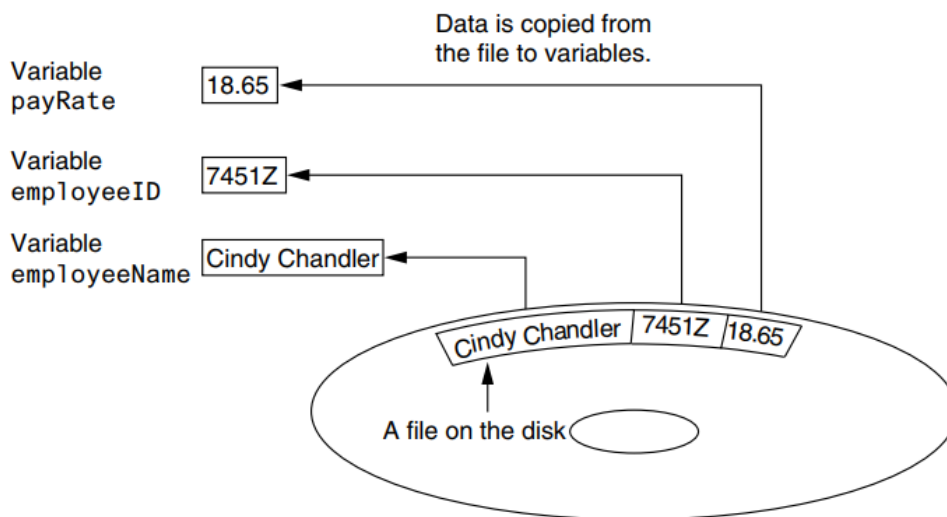
Chương trình được sử dụng trong các hoạt động kinh doanh hàng ngày phụ thuộc nhiều vào các tệp. Các chương trình tính lương giữ dữ liệu nhân viên trong các tệp, các chương trình kiểm kê giữ dữ liệu về sản phẩm của công ty trong các tệp, các hệ thống kế toán giữ dữ liệu về hoạt động tài chính của công ty trong các tệp, v.v.

Lập trình viên thường gọi quá trình lưu dữ liệu trong một tệp là ghi dữ liệu vào tệp. Khi một mẫu dữ liệu được ghi vào tệp, nó được sao chép từ một biến trong RAM vào tệp. Quá trình này được minh họa trong hình dưới. Một tệp đầu ra là một tệp mà dữ liệu được ghi vào. Nó được gọi là tệp đầu ra vì chương trình lưu đầu ra trong đó.



Hình 5.5. Ghi dữ liệu vào tệp

Quá trình truy xuất dữ liệu từ một tệp được gọi là đọc dữ liệu từ tệp. Khi một mẫu dữ liệu được đọc từ tệp, nó được sao chép từ tệp vào một biến trong RAM. Hình 5-13 minh họa quá trình này. Một tệp đầu vào là một tệp mà dữ liệu được đọc từ đó. Nó được gọi là tệp đầu vào vì chương trình nhận đầu vào từ tệp.



Hình 5.6. Đọc dữ liệu từ tệp

Phần này thảo luận về cách tạo các chương trình viết dữ liệu vào tệp và đọc dữ liệu từ tệp. Khi một tệp được sử dụng bởi một chương trình, ba bước phải được thực hiện.

1. Mở tệp: Mở một tệp tạo ra một kết nối giữa tệp và chương trình. Việc mở một tệp đầu ra thường tạo ra tệp trên đĩa và cho phép chương trình ghi dữ liệu vào đó. Mở một tệp đầu vào cho phép chương trình đọc dữ liệu từ tệp.

2. Xử lý tệp: Dữ liệu được viết vào tệp (nếu đó là một tệp đầu ra) hoặc được đọc từ tệp (nếu đó là một tệp đầu vào).

3. Đóng tệp: Sau khi chương trình hoàn thành việc sử dụng tệp, tệp phải được đóng lại. Đóng một tệp ngắt kết nối tệp khỏi chương trình.

Các loại tệp

Nói chung, có hai loại tệp: tệp văn bản và tệp nhị phân. Một tệp văn bản chứa dữ liệu đã được mã hóa dưới dạng văn bản, sử dụng một lược đồ như ASCII hoặc Unicode. Ngay cả khi tệp chứa số, những số đó được lưu trong tệp dưới dạng một chuỗi các ký tự. Do đó, tệp có thể được mở và xem trong trình soạn thảo văn bản như Notepad. Một tệp nhị phân chứa dữ liệu chưa được chuyển đổi thành văn bản. Do đó, bạn không thể xem nội dung của một tệp nhị phân bằng trình soạn thảo văn bản. Trong chương này, chúng ta chỉ làm việc với tệp văn bản.

Phương thức truy cập tệp

Có hai cách chung để truy cập dữ liệu được lưu trữ trong một tệp: truy cập tuần tự và truy cập trực tiếp. Khi bạn làm việc với một tệp truy cập tuần tự, bạn truy cập dữ liệu từ đầu tệp đến cuối tệp. Nếu bạn muốn đọc một mẫu dữ liệu được lưu trữ ở cuối tệp, phải đọc tất cả dữ liệu trước đó - không thể nhảy trực tiếp đến dữ liệu mong muốn. Điều này tương tự như cách mà các máy nghe băng cũ hoạt động. Nếu bạn muốn nghe bài hát cuối cùng trên một băng cassette, bạn phải tua nhanh qua tất cả các bài hát trước đó hoặc nghe chúng. Không có cách nào để nhảy trực tiếp đến một bài hát cụ thể.

Khi bạn làm việc với một tệp truy cập ngẫu nhiên (còn được gọi là tệp truy cập trực tiếp), bạn có thể nhảy trực tiếp đến bất kỳ mẫu dữ liệu nào trong tệp mà không cần đọc dữ liệu trước đó. Điều này tương tự như cách một máy nghe đĩa CD hoặc một máy nghe MP3 hoạt động. Bạn có thể nhảy trực tiếp đến bất kỳ bài hát nào mà bạn muốn nghe.

Chương này tập trung vào tệp truy cập tuần tự. Tệp truy cập tuần tự dễ dàng để làm việc, và bạn có thể sử dụng chúng để hiểu các thao tác cơ bản về tệp.

Tên tệp và đối tượng luồng tệp

Các tập tin trên đĩa được xác định bằng tên tập tin. Ví dụ, khi bạn tạo một tài liệu bằng một trình xử lý văn bản sau đó lưu tài liệu vào một tập tin, bạn phải chỉ định một tên tệp tin. Khi bạn sử dụng một tiện ích như Windows Explorer để xem nội dung trên đĩa máy tính, bạn sẽ thấy một danh sách các tên tập tin. Hình dưới cho thấy ba tập tin có tên cat.jpg, notes.txt và resume.doc có thể được đại diện trong Windows Explorer.



Hình 5.7. Một số loại tệp tin

Mỗi hệ điều hành có các quy tắc riêng cho việc đặt tên tệp tin. Nhiều hệ thống, bao gồm cả Windows, hỗ trợ việc sử dụng phần mở rộng tên tệp tin, đó là các chuỗi ký tự ngắn xuất hiện ở cuối tên tệp tin, được tiền bởi một dấu chấm (biết đến là một "dấu chấm"). Ví dụ, các tệp tin được mô tả trong Hình trên có các phần mở rộng là .jpg, .txt và .doc. Phần mở rộng thường chỉ ra loại dữ liệu được lưu trữ trong tệp tin. Ví dụ, phần mở rộng .jpg thường chỉ ra rằng tệp tin chứa một hình ảnh đồ họa được nén theo tiêu chuẩn hình ảnh JPEG. Phần mở rộng .txt thường chỉ ra rằng tệp tin chứa văn bản. Phần mở rộng .doc thường chỉ ra rằng tệp tin chứa một tài liệu Microsoft Word. Để một chương trình có thể làm việc với một tệp tin trên đĩa của máy tính, chương trình phải tạo ra một đối tượng luồng tệp tin trong bộ nhớ. Một đối tượng luồng tệp tin là một đối tượng được liên kết với một tệp tin cụ thể và cung cấp một cách cho chương trình làm việc với tệp tin đó. Nó được gọi là một đối tượng "luồng" vì một tệp tin có thể được coi là một dòng dữ liệu.

Các đối tượng luồng tệp tin hoạt động rất giống như các đối tượng cin và cout. Một luồng dữ liệu có thể được gửi đến cout, điều này gây ra các giá trị được hiển thị trên màn hình. Một luồng dữ liệu có thể được đọc từ bàn phím bởi cin và được lưu trữ trong các biến. Tương tự, các luồng dữ liệu có thể được gửi đến một đối tượng luồng tệp tin, mà sau đó viết dữ liệu vào một tệp tin. Khi dữ liệu được đọc từ một tệp tin, dữ liệu chảy từ đối tượng luồng tệp tin được liên kết với tệp tin vào các biến.

Thiết lập chương trình cho Đầu vào/Đầu ra Tệp tin

Giống như cin và cout yêu cầu file iostream được bao gồm trong chương trình, truy cập tệp tin C++ yêu cầu một tệp tiêu đề khác. Tệp <fstream> chứa tất cả các khai báo cần thiết cho các hoạt động tệp tin. Nó được bao gồm với câu lệnh sau:

```
#include <fstream>
```

Tệp tiêu đề <fstream> định nghĩa các loại dữ liệu ofstream, ifstream và fstream. Trước khi một chương trình C++ có thể làm việc với một tệp tin, nó phải xác định một đối tượng của một trong các loại dữ liệu này. Đối tượng sẽ được "liên kết" với một tệp tin thực tế trên đĩa của máy tính, và các hoạt động có thể được thực hiện trên tệp tin phụ thuộc vào loại dữ liệu trong ba loại này bạn chọn cho đối tượng luồng tệp tin. Bảng sau liệt kê và mô tả các loại dữ liệu luồng tệp tin.

Bảng 5.1. Bảng luồng tệp tin

Kiểu dữ liệu luồng tệp	Mô tả
ofstream	Luồng tệp tin đầu ra. Bạn tạo một đối tượng thuộc kiểu dữ liệu này khi bạn muốn tạo một tệp và ghi dữ liệu vào đó.
ifstream	Luồng tệp tin đầu vào. Bạn tạo một đối tượng thuộc kiểu dữ liệu này khi bạn muốn mở một tệp hiện có và đọc dữ liệu từ nó.
fstream	Tập hồ sơ. Các đối tượng thuộc kiểu dữ liệu này có thể được sử dụng để mở tệp để đọc, ghi hoặc cả hai.

LƯU Ý: Trong chương này, chúng ta chỉ thảo luận về các loại ofstream và ifstream. Loại fstream sẽ được bàn luận trong chương khác.

Tạo đối tượng tệp tin và mở một tệp tin

Trước khi dữ liệu có thể được viết vào hoặc đọc từ một tệp tin, những việc sau phải thực hiện:

- Một đối tượng luồng tệp tin phải được tạo ra.
- Tệp tin phải được mở và liên kết với đối tượng luồng tệp tin.

Đoạn mã sau đây minh họa một ví dụ về cách mở một tệp tin để đọc.

```
ifstream inputFile;  
inputFile.open("Customers.txt");
```

Câu lệnh đầu tiên định nghĩa một đối tượng ifstream với tên là inputFile. Câu lệnh thứ hai gọi hàm thành viên open của đối tượng, truyền chuỗi "Customers.txt" như một đối số. Trong câu lệnh này, hàm thành viên open mở tệp tin Customers.txt và liên kết nó với đối tượng inputFile. Sau khi mã này thực thi, bạn sẽ có thể sử dụng đối tượng inputFile để đọc dữ liệu từ tệp tin Customers.txt.

Đoạn mã sau đây minh họa một ví dụ về cách mở một tệp tin để đầu ra (ghi).

```
ofstream outputFile;  
outputFile.open("Employees.txt");
```

Câu lệnh đầu tiên định nghĩa một đối tượng ofstream với tên là outputFile. Câu lệnh thứ hai gọi hàm thành viên open của đối tượng, truyền chuỗi "Employees.txt" như một đối số. Trong câu lệnh này, hàm thành viên open tạo ra tệp tin Employees.txt và liên kết nó với đối tượng outputFile. Sau khi mã này thực thi, bạn sẽ có thể sử dụng đối tượng outputFile để ghi dữ liệu vào tệp tin Employees.txt. Quan trọng phải nhớ rằng khi bạn gọi hàm thành viên open của một đối tượng ofstream, tệp tin được chỉ định sẽ được tạo ra. Nếu tệp tin được chỉ định đã tồn tại, nó sẽ bị xóa và một tệp tin mới cùng tên sẽ được tạo ra.

Thường thì, khi mở một tệp tin, bạn sẽ cần chỉ định đường dẫn cũng như tên của nó. Ví dụ, trên hệ thống Windows, câu lệnh sau mở tệp tin C:\data\inventory.txt:

```
inputFile.open("C:\\data\\inventory.txt")
```

Trong câu lệnh này, tệp tin C:\\data\\inventory.txt được mở và liên kết với inputFile.

LƯU Ý: Chú ý việc sử dụng hai dấu gạch chéo ngược trong đường dẫn của tệp tin. Hai dấu gạch chéo ngược được sử dụng để biểu diễn một dấu gạch chéo ngược trong một chuỗi ký tự.

Cũng có thể định nghĩa một đối tượng luồng tệp tin và mở một tệp tin trong một câu lệnh. Dưới đây là một ví dụ:

```
ifstream inputFile("Customers.txt");
```

Câu lệnh này định nghĩa một đối tượng ifstream với tên là inputFile và mở tệp tin Customer.txt. Dưới đây là một ví dụ định nghĩa một đối tượng ofstream với tên là outputFile và mở tệp tin Employees.txt:

```
ofstream outputFile("Employees.txt");
```

Đóng một tệp tin

Ngược lại với việc mở một tệp tin là đóng nó. Mặc dù các tệp tin của chương trình sẽ tự động đóng khi chương trình tắt, nhưng chúng ta nên đóng các tệp sau khi sử dụng xong. Dưới đây là hai lý do mà một chương trình nên đóng các tệp tin:

1. Hầu hết các hệ điều hành tạm thời lưu trữ dữ liệu trong một bộ đệm tệp tin trước khi được ghi vào tệp tin. Một bộ đệm tệp tin là một phần nhỏ của bộ nhớ được sử dụng để lưu trữ dữ liệu liên quan đến tệp tin trước khi được ghi vào. Khi bộ đệm đầy, tất cả các dữ liệu được lưu trữ trong đó sẽ được ghi vào tệp tin. Kỹ thuật này cải thiện hiệu suất của hệ thống. Việc đóng một tệp tin làm cho bất kỳ dữ liệu còn lại trong bộ đệm được lưu vào tệp tin của nó.

2. Một số hệ điều hành giới hạn số lượng tệp tin có thể mở cùng một lúc. Khi một chương trình đóng các tệp tin không còn sử dụng nữa, nó sẽ không tiêu tốn thêm tài nguyên của hệ điều hành hơn cần thiết.

Gọi hàm thành viên close của đối tượng luồng tệp tin sẽ đóng một tệp tin. Dưới đây là một ví dụ:

```
inputFile.close();
```

Ghi dữ liệu vào tệp tin

Bạn đã biết cách sử dụng toán tử chèn luồng (<<) với đối tượng cout để ghi dữ liệu ra màn hình. Nó cũng có thể được sử dụng với các đối tượng ofstream để ghi dữ liệu vào một tệp tin. Giả sử outputFile là một đối tượng ofstream, câu lệnh sau minh họa việc sử dụng toán tử << để ghi một chuỗi ký tự vào tệp tin:

```
outputFile << "Tôi yêu lập trình C++\n";
```

Câu lệnh này ghi chuỗi ký tự "Tôi yêu lập trình C++\n" vào tệp tin được liên kết với outputFile. Như bạn có thể thấy, câu lệnh này giống như một câu lệnh cout, ngoại trừ tên của đối tượng ofstream thay thế cho cout. Đây là một câu lệnh ghi cả một chuỗi ký tự và nội dung của một biến vào tệp tin:

```
outputFile << "Giá: " << price << endl;
```

Câu lệnh trên ghi luồng dữ liệu vào outputFile chính xác như cout sẽ ghi nó ra màn hình: Nó ghi chuỗi "Giá: ", tiếp theo là giá trị của biến price, tiếp theo là một ký tự xuống dòng.

Chương trình dưới mình họa mở một tệp tin, ghi dữ liệu vào tệp tin và đóng tệp tin. Sau khi mã này được thực thi, chúng ta có thể mở tệp tin demofile.txt bằng một trình soạn thảo văn bản và xem nội dung của nó. Hình 5-15 cho thấy nội dung của tệp tin sẽ xuất hiện trong Notepad.

Chương trình 5.10. Ghi dữ liệu trên từng dòng vào tệp

```
// Chương trình này ghi dữ liệu vào một dòng duy nhất trong một tệp tin.
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
    ofstream outputFile;
    outputFile.open("demofile.txt");
    cout << "Đang ghi dữ liệu vào tệp tin.\n";

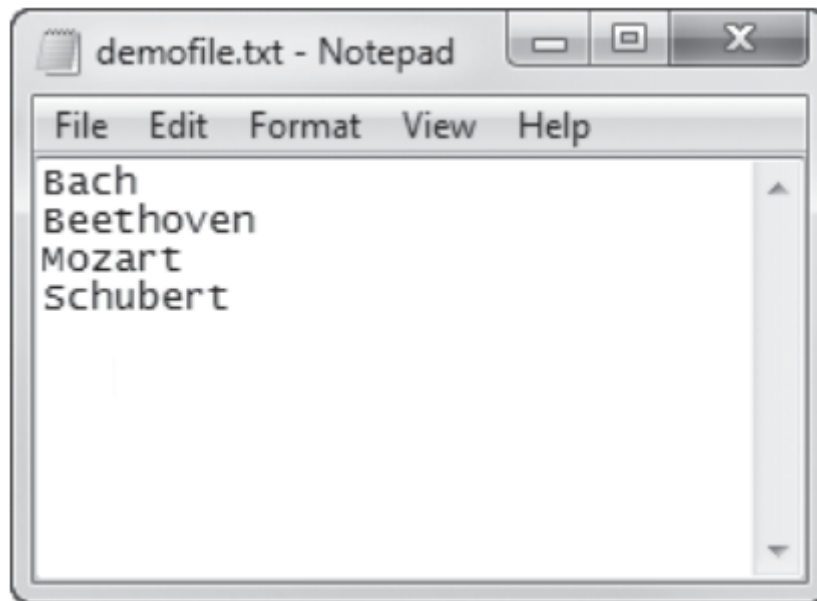
    // Ghi bốn tên vào tệp tin.
    outputFile << "Bach\n";
    outputFile << "Beethoven\n";
    outputFile << "Mozart\n";
    outputFile << "Schubert\n";

    // Đóng tệp tin
    outputFile.close();
    cout << "Hoàn tất.\n";
    return 0;
}
```

Kết quả chương trình:

Đang ghi dữ liệu vào tệp tin.
Hoàn tất.

Kết quả được ghi vào tệp demofile.txt như hình dưới



Hình 5.8. Kết quả được ghi trong tệp *demofile.txt* trên từng dòng

Lưu ý trong chương trình trên mỗi chuỗi được ghi vào tệp kết thúc bằng một chuỗi xuống dòng mới (\n). Dòng mới chỉ định phần cuối của một dòng văn bản. Vì một dòng mới được viết ở cuối mỗi chuỗi nên các chuỗi xuất hiện trên các dòng riêng biệt khi xem trong trình soạn thảo văn bản, như trong Hình 5-15.

Chương trình dưới đây cho thấy điều gì sẽ xảy ra nếu các tên được ghi vào tệp mà không có chuỗi xuống dòng mới (\n). Hình 5-16 hiển thị nội dung của tệp mà Chương trình dưới đây tạo ra. Như bạn có thể thấy, tất cả các tên đều xuất hiện trên cùng một dòng trong tệp.

Chương trình 5.11. Ghi dữ liệu trên 1 dòng vào tệp

```
// Chương trình này ghi dữ liệu vào một dòng duy nhất trong một tệp tin.
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
    ofstream outputFile;
    outputFile.open("demofile.txt");

    cout << "Đang ghi dữ liệu vào tệp tin.\n";

    // Ghi bốn tên vào tệp tin.
    outputFile << "Bach";
    outputFile << "Beethoven";
    outputFile << "Mozart";
    outputFile << "Schubert";

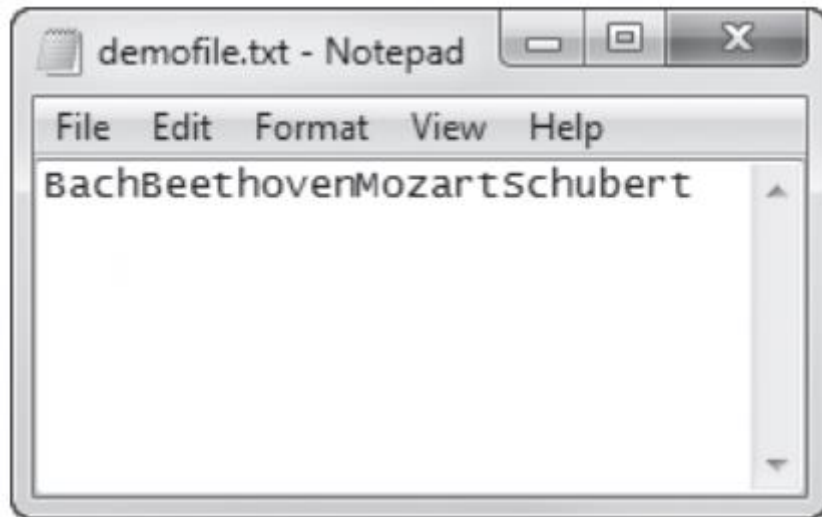
    // Đóng tệp tin
    outputFile.close();
    cout << "Hoan tat.\n";
}
```

```
    return 0;
}
```

Kết quả chương trình:

Dang ghi du lieu vao tep tin.
Hoan tat.

Kết quả được ghi vào tệp demofile.txt như hình dưới



Hình 5.9. Kết quả được ghi trong tệp demofile.txt trên 1 dòng

Chương trình dưới đây sẽ nhập 03 số từ bàn phím, sau đó ghi các giá trị của biến vào tệp Number.txt

Chương trình 5.12. Ghi dữ liệu người dùng nhập vào tệp

```
// Chương trình này ghi du lieu nhap tu nguoi dung vao mot tep.
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
    ofstream outputFile;
    int number1, number2, number3;

    // Mo mot tep de ghi.
    outputFile.open("Numbers.txt");

    // Lay ba so tu nguoi dung.
    cout << "Nhap mot so: ";
    cin >> number1;
    cout << "Nhap mot so khac: ";
    cin >> number2;
    cout << "Nhap mot lan nua. Nhap mot so: ";
    cin >> number3;

    // Ghi cac so vao tep.
```

```

outputFile << number1 << endl;
outputFile << number2 << endl;
outputFile << number3 << endl;
cout << "Cac so da duoc luu vao tep.\n";

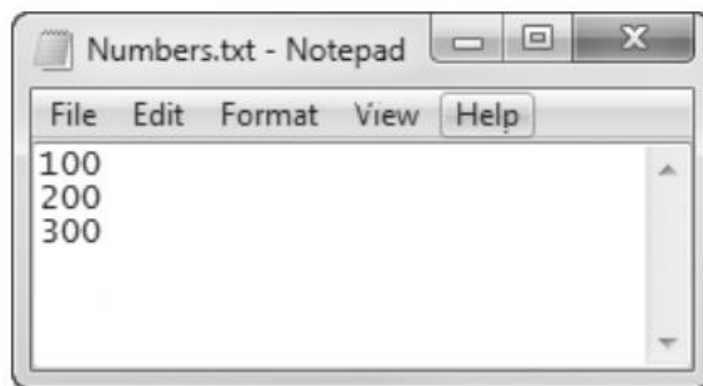
// Dong tep
outputFile.close();
cout << "Hoan tat.\n";
return 0;
}

```

Kết quả chương trình:

Nhập một số: 100
 Nhập một số khác: 200
 Nhập một lần nữa. Nhập một số: 300
 Các số đã được lưu vào tệp.
 Hoàn tất.

Kết quả được ghi vào tệp demofile.txt như hình dưới

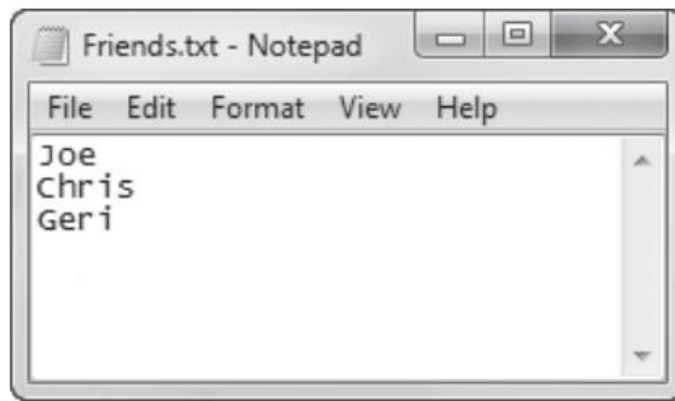


Hình 5.10. Kết quả được ghi vào tệp Numbers.txt

Đọc dữ liệu từ tệp

Toán tử >> không chỉ đọc dữ liệu đầu vào của người dùng từ đối tượng cin mà còn cả dữ liệu từ một tệp. Giả sử tệp đầu vào là một đối tượng luồng if, câu lệnh sau hiển thị toán tử >> đọc dữ liệu từ tệp vào tên biến name:

```
inputFile >> name;
```



Hình 5.11. Nội dung tệp Friends

Hãy xem một ví dụ. Giả sử tệp Friends.txt tồn tại và nó chứa các tên được hiển thị trong hình trên. Chương trình dưới đây mở tệp, đọc tên và hiển thị chúng trên màn hình, sau đó đóng tệp.

Chương trình 5.13. Đọc dữ liệu từ tệp ra các biến

```
// Chương trình này đọc dữ liệu từ một tệp.
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main()
{
    ifstream inputFile;
    string name;

    inputFile.open("Friends.txt");
    cout << "Đang đọc dữ liệu từ tệp.\n";

    inputFile >> name; // Đọc tên thứ nhất từ tệp
    cout << name << endl; // Hiển thị tên thứ nhất

    inputFile >> name; // Đọc tên thứ hai từ tệp
    cout << name << endl; // Hiển thị tên thứ hai

    inputFile >> name; // Đọc tên thứ ba từ tệp
    cout << name << endl; // Hiển thị tên thứ ba

    inputFile.close(); // Đóng tệp
    return 0;
}
```

Kết quả chương trình:

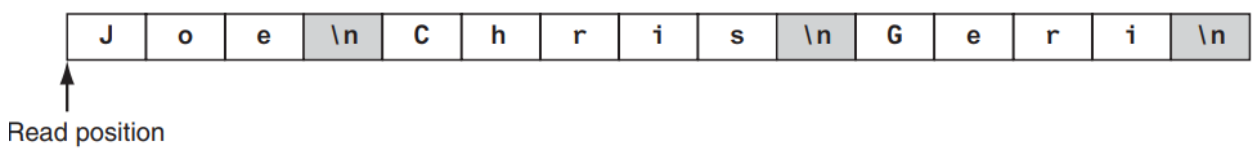
```
Đang đọc dữ liệu từ tệp
Joe
Chris
Geri
```

Vị trí đọc

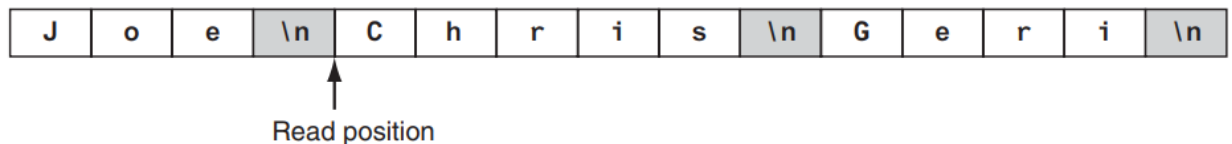
Khi một tệp đã được mở để nhập, đối tượng luồng tệp sẽ duy trì bên trong một giá trị đặc biệt được gọi là vị trí đọc. Vị trí đọc của tệp đánh dấu vị trí của byte tiếp theo sẽ được đọc từ tệp. Khi một tệp đầu vào được mở, vị trí đọc của nó ban đầu được đặt thành byte đầu tiên trong tệp. Vì vậy, thao tác đọc đầu tiên sẽ trích xuất dữ liệu bắt đầu từ byte đầu tiên.

Khi dữ liệu được đọc từ tệp, vị trí đọc sẽ di chuyển về phía trước, về phía cuối tệp.

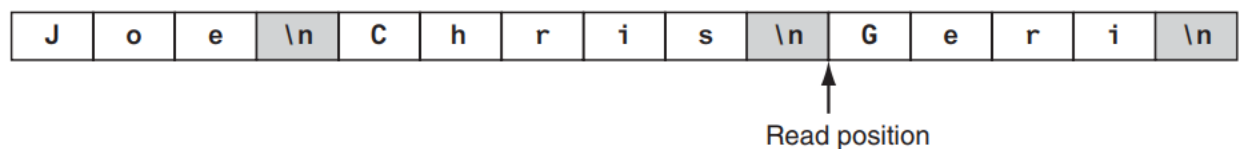
Hãy xem cách này hoạt động như thế nào với ví dụ được hiển thị trong Chương trình 5-19. Khi tệp `Friends.txt` được mở bằng câu lệnh ở dòng 12, vị trí đọc của tệp sẽ được định vị như trong Hình 5-19.



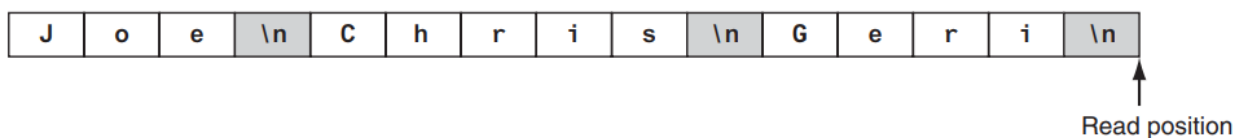
Hãy nhớ rằng khi toán tử `>>` trích xuất dữ liệu từ một tệp, nó sẽ đọc các phần dữ liệu được phân tách bằng ký tự khoảng trắng (dấu cách, tab hoặc dòng mới). Khi câu lệnh ở dòng 15 được thực thi, toán tử `>>` đọc dữ liệu từ vị trí đọc hiện tại của tệp, cho đến ký tự `\n`. Dữ liệu đọc từ tệp được gán cho đối tượng tên. Ký tự `\n` cũng được đọc từ tệp nhưng không được đưa vào như một phần của dữ liệu. Vì vậy, cái tên đối tượng sẽ giữ giá trị "Joe" sau khi câu lệnh này được thực thi. Vị trí đọc của tệp khi đó sẽ ở vị trí như trong Hình 5-20



Khi câu lệnh ở dòng 18 được thực thi, nó sẽ đọc mục tiếp theo từ tệp, đó là "Chris" và gán giá trị đó cho đối tượng tên. Sau khi câu lệnh này được thực thi, vị trí đọc của tệp sẽ được chuyển sang mục tiếp theo, như trong Hình 5-21.

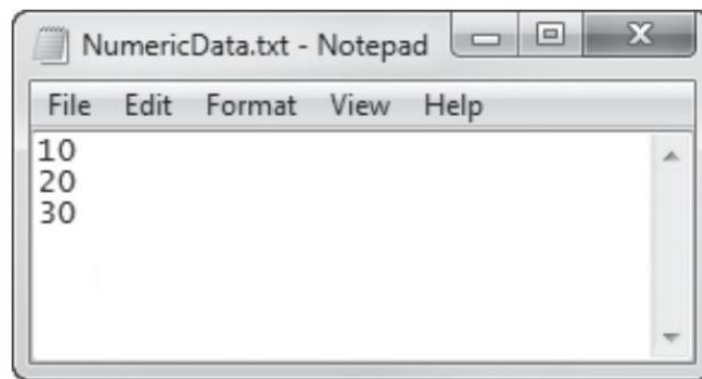


Khi câu lệnh ở dòng 21 được thực thi, nó sẽ đọc mục tiếp theo từ tệp, đó là "Geri" và gán giá trị đó cho đối tượng tên. Sau khi câu lệnh này được thực thi, vị trí đọc của tệp sẽ được chuyển đến cuối tệp, như trong Hình 5-22.



Đọc dữ liệu số từ tệp văn bản

Hãy nhớ rằng khi dữ liệu được lưu trữ trong một tệp văn bản, nó được mã hóa dưới dạng văn bản, sử dụng một hệ thống như ASCII hoặc Unicode. Ngay cả khi tệp chứa các con số, những con số đó cũng được lưu trong tệp dưới dạng một chuỗi các ký tự. Ví dụ, giả sử một tệp văn bản chứa dữ liệu số, như được hiển thị trong Hình 5-17. Các số mà bạn thấy được hiển thị trong hình được lưu trong tệp dưới dạng các chuỗi "10", "20" và "30". May mắn thay, bạn có thể sử dụng toán tử `>>` để đọc dữ liệu như vậy từ một tệp văn bản vào một biến số, và toán tử `>>` sẽ tự động chuyển đổi dữ liệu thành một kiểu dữ liệu số. Chương trình 5-20 cho thấy một ví dụ. Nó mở tệp được hiển thị trong Hình 5-23, đọc ba số từ tệp vào các biến `int`, và tính tổng của chúng.



Hình 5.12. Nội dung tệp *NumericData.txt*

Sau đây là một ví dụ minh họa thao tác đọc dữ liệu số ra các biến

Chương trình 5.14. Đọc các số từ tệp ra các biến

```
// Chương trình này đọc các số từ một tệp.
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
    ifstream inFile;
    int value1, value2, value3, sum;

    // Mở tệp.
    inFile.open("NumericData.txt");

    // Đọc ba số từ tệp.
    inFile >> value1;
    inFile >> value2;
    inFile >> value3;

    // Đóng tệp.
    inFile.close();

    // Tính tổng của các số.
```

```

sum = value1 + value2 + value3;

// Hien thi ba so.
cout << "Droi day la cac so:\n"
<< value1 << " " << value2 << " " << value3 << endl;

// Hien thi tong cua cac so.
cout << "Tong cua chung la: " << sum << endl;
return 0;
}

```

Kết quả chương trình:

```

Droi day la cac so:
10 20 30
Tong cua chung la 60

```

Sử dụng vòng lặp để xử lý tệp

Mặc dù một số chương trình sử dụng tệp để lưu trữ chỉ một lượng nhỏ dữ liệu, tệp thường được sử dụng để chứa các bộ sưu tập dữ liệu lớn. Khi một chương trình sử dụng tệp để ghi hoặc đọc một lượng lớn dữ liệu, thường có sự tham gia của một vòng lặp. Ví dụ, hãy nhìn vào mã trong chương trình dưới đây. Chương trình này lấy số liệu doanh số bán hàng cho một loạt các ngày từ người dùng và ghi các số liệu đó vào một tệp có tên là Sales.txt. Người dùng chỉ định số ngày dữ liệu doanh số bán hàng mà họ cần nhập. Trong lần chạy mẫu của chương trình, người dùng nhập số liệu doanh số cho 5 ngày. Hình 5-24 cho thấy nội dung của tệp Sales.txt chứa dữ liệu do người dùng nhập trong lần chạy mẫu.

Chương trình 5.15. Ghi doanh số vào tệp

```

// Chương trình này đọc dữ liệu từ một tệp.
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
    ofstream outputFile; // Đối tượng luồng tệp
    int numberOfDays; // Số ngày bán hàng
    double sales; // Số tiền bán hàng trong một ngày

    // Lấy số ngày.
    cout << "Bạn có bao nhiêu ngày bán hàng? ";
    cin >> numberOfDays;

    // Mở một tệp có tên là Sales.txt.
    outputFile.open("Sales.txt");

    // Lấy doanh số cho mỗi ngày và ghi
    // vào tệp.

```

```

for (int count = 1; count <= numberOfDays; count++)
{
    // Lay doanh so cho mot ngay.
    cout << "Nhap doanh so cho ngay " << count << ": ";
    cin >> sales;

    // Ghi doanh so vao tep.
    outputFile << sales << endl;
}

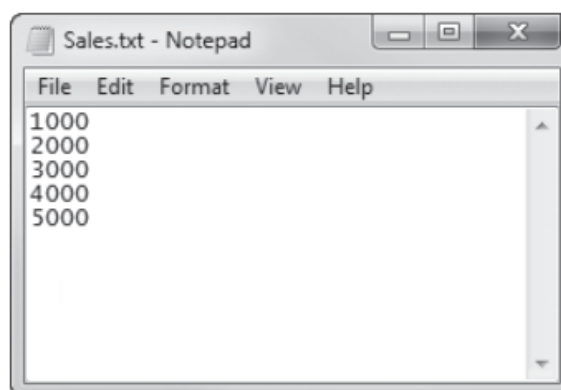
// Dong tep.
outputFile.close();
cout << "Du lieu da duoc ghi vao Sales.txt\n";
return 0;
}

```

Kết quả chương trình:

Ban co bao nhieu ngay ban hang? 5
 Nhap doanh so cho ngay 1: 1000
 Nhap doanh so cho ngay 2: 2000
 Nhap doanh so cho ngay 3: 3000
 Nhap doanh so cho ngay 4: 4000
 Nhap doanh so cho ngay 5: 5000
 Du lieu da duoc ghi vao Sales.txt

Kết quả được các dữ liệu được ghi vào tệp sales.txt như sau

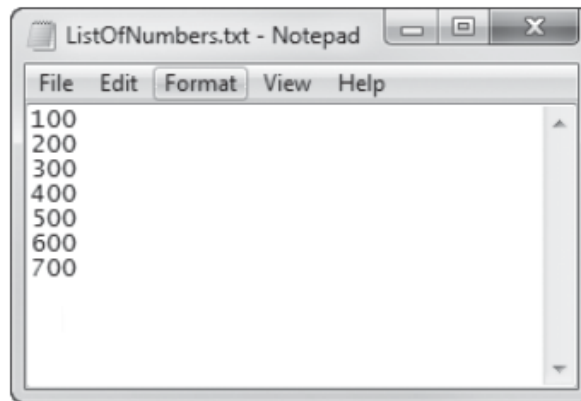


Hình 5.13. Nội dung tệp Sales.txt

Phát hiện cuối tệp tin

Phát hiện kết thúc của tệp tin là rất thường xuyên, một chương trình phải đọc nội dung của một tệp tin mà không biết số lượng mục được lưu trữ trong tệp tin đó. Ví dụ, giả sử bạn cần viết một chương trình hiển thị tất cả các mục trong một tệp tin, nhưng bạn không biết tệp tin chứa bao nhiêu mục. Bạn có thể mở tệp tin sau đó sử dụng một vòng lặp để đọc liên tục một mục từ tệp tin và hiển thị nó. Tuy nhiên, một lỗi sẽ xảy ra nếu chương trình cố gắng đọc vượt quá kết thúc của tệp tin. Chương trình cần một cách nào đó để biết khi đã đạt đến kết thúc của tệp tin để không cố gắng đọc vượt quá nó.

Như chúng ta đã biết, toán tử >> không chỉ đọc dữ liệu từ một tệp tin, mà còn trả về một giá trị đúng hoặc sai cho biết liệu dữ liệu có được đọc thành công hay không. Nếu toán tử trả về đúng, thì một giá trị đã được đọc thành công. Nếu toán tử trả về sai, điều đó có nghĩa là không có giá trị nào được đọc từ tệp tin. Hãy cùng xem một ví dụ. Một tệp tin có tên ListOfNumbers.txt, được hiển thị trong hình dưới đây, chứa một danh sách các số. Mà không biết tệp tin chứa bao nhiêu số, chương trình dưới đây sẽ mở tệp tin, đọc tất cả các giá trị có trong đó, và hiển thị chúng.



Hình 5.14. Nội dung tệp ListOfNumbers.txt

```
// Chương trình này đọc dữ liệu từ một tệp tin.
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
    ifstream inputFile;
    int number;
    // Mở tệp tin.
    inputFile.open("ListOfNumbers.txt");

    // Đọc các số từ tệp tin và
    // hiển thị chúng.
    while (inputFile >> number)
    {
        cout << number << endl;
    }

    // Đóng tệp tin.
    inputFile.close();
    return 0;
}
```

Kết quả chương trình:

100
200

300
400
500
600
700

Hãy xem kỹ hơn ở dòng sau:

```
while (inputFile >> number)
```

Lưu ý rằng câu lệnh trích xuất dữ liệu từ tệp tin được sử dụng làm biểu thức Boolean trong vòng lặp while. Nó hoạt động như sau:

- Biểu thức `inputFile >> number` thực thi.
- Nếu một mục được đọc thành công từ tệp tin, mục đó sẽ được lưu trong biến `number`, và biểu thức trả về `true` để chỉ ra rằng nó thành công. Trong trường hợp đó, câu lệnh trong vòng lặp sẽ thực thi và vòng lặp lặp lại.
- Nếu không còn mục nào để đọc từ tệp tin, biểu thức `inputFile >> number` sẽ trả về `false`, chỉ ra rằng nó không đọc được giá trị. Trong trường hợp đó, vòng lặp sẽ kết thúc.

Vì giá trị trả về từ toán tử `>>` điều khiển vòng lặp, nó sẽ đọc các mục từ tệp tin cho đến khi đạt đến kết thúc của tệp tin.

5.9. Câu lệnh Break/Continue

KHÁI NIỆM: Câu lệnh `break` làm cho vòng lặp kết thúc sớm. Câu lệnh `continue` làm cho vòng lặp dừng vòng lặp hiện tại và bắt đầu vòng lặp tiếp theo.

CẢNH BÁO! Hãy sử dụng câu lệnh `break` và `continue` một cách cẩn thận. Bởi vì chúng bỏ qua điều kiện bình thường điều khiển các vòng lặp, các câu lệnh này làm cho mã nguồn khó hiểu và khó gỡ lỗi. Vì lý do này, bạn nên cẩn thận khi sử dụng `break` và `continue`.

Vòng lặp `while` trong đoạn chương trình sau dường như được thực thi 10 lần, nhưng câu lệnh `break` khiến nó dừng sau lần lặp thứ năm.

```
int count = 0;
while (count++ < 10)
{
    cout << count << endl;
    if (count == 5)
        break;
}
```

Sử dụng break trong vòng lặp lồng nhau

Trong một vòng lặp lồng nhau, câu lệnh `break` chỉ làm gián đoạn vòng lặp mà nó được đặt vào. Đoạn chương trình sau đây hiển thị năm hàng dấu hoa thị trên màn hình. Vòng lặp

bên ngoài kiểm soát số lượng hàng và vòng lặp bên trong kiểm soát số lượng dấu hoa thị trong mỗi hàng. Vòng lặp bên trong được thiết kế để hiển thị 20 dấu hoa thị, nhưng câu lệnh break sẽ dừng nó ở lần lặp thứ 11.

```
for (int row = 0; row < 5; row++)
{
    for (int star = 0; star < 20; star++)
    {
        cout << '*';
        if (star == 10)
            break;
    }
    cout << endl;
}
```

Đầu ra của đoạn chương trình trên là:

```
*
*
*
*
*
```

Câu lệnh continue

Câu lệnh continue làm cho vòng lặp hiện tại kết thúc ngay lập tức. Khi gặp continue, tất cả các câu lệnh trong phần thân vòng lặp xuất hiện sau nó đều bị bỏ qua và vòng lặp chuẩn bị cho lần lặp tiếp theo.

Trong vòng lặp while, điều này có nghĩa là chương trình sẽ nhảy tới biểu thức kiểm tra ở đầu vòng lặp. Như thường lệ, nếu biểu thức vẫn đúng thì lần lặp tiếp theo sẽ bắt đầu. Trong vòng lặp do-while, chương trình nhảy tới biểu thức kiểm tra ở cuối vòng lặp để xác định xem lần lặp tiếp theo có bắt đầu hay không. Trong vòng lặp for, continue khiến biểu thức cập nhật được thực thi, sau đó biểu thức kiểm tra được đánh giá.

Đoạn chương trình sau đây minh họa cách sử dụng continue trong vòng lặp while :

```
int testVal = 0;
while (testVal++ < 10)
{
    if (testVal == 4)
        continue;
    cout << testVal << " ";
}
```

Vòng lặp này trông giống như hiển thị các số nguyên từ 1 đến 10. Tuy nhiên, khi testVal bằng 4, câu lệnh continue làm cho vòng lặp bỏ qua câu lệnh cout và bắt đầu lần lặp tiếp theo. Đầu ra của vòng lặp là

```
1 2 3 5 6 7 8 9 10
```

5.10. Câu hỏi và Bài tập

5.10.1. Câu hỏi

Cho biết các câu là ĐÚNG hay SAI?

1. Toán hạng của toán tử tăng và giảm có thể là bất kỳ biểu thức toán học hợp lệ nào.
2. Giá trị của biến `z` sau khi thực hiện các câu lệnh sau sẽ là 11:
$$x = y = 5;$$
$$z = x + ++y;$$
3. Giá trị của biến `z` sau khi thực hiện các câu lệnh sau sẽ là 10:
$$x = y = 5;$$
$$z = x++ + y;$$
4. Vòng lặp `while` là một vòng lặp kiểm tra trước.
5. Vòng lặp `do-while` là một vòng lặp kiểm tra sau.
6. Vòng lặp `for` là một vòng lặp kiểm tra trước.
7. Không cần thiết phải khởi tạo các biến đếm.
8. Bỏ qua biểu thức kiểm tra trong vòng lặp `for` dẫn đến một vòng lặp vô hạn.
9. Trong vòng lặp `for`, có thể có nhiều biểu thức kiểm tra và mỗi biểu thức kiểm tra được tách ra bởi dấu phẩy.
10. Biến có thể được định nghĩa bên trong thân của vòng lặp.
11. Biến có thể được định nghĩa trong biểu thức điều kiện của một vòng lặp `while`.
12. Trong một vòng lặp lồng nhau, vòng lặp bên ngoài thực thi nhanh hơn vòng lặp bên trong.
13. Trong một vòng lặp lồng nhau, vòng lặp bên trong thực hiện tất cả các lần lặp của nó cho mỗi lần lặp duy nhất của vòng lặp bên ngoài.
14. Trong một vòng lặp lồng nhau, nếu vòng lặp bên ngoài có m lần lặp và vòng lặp bên trong có n lần lặp, thì tổng số lần lặp sẽ là $m \times n$.
15. Câu lệnh `break` khiến một vòng lặp dừng lần lặp hiện tại và bắt đầu lần lặp tiếp theo.
16. Câu lệnh `continue` khiến một vòng lặp bị dừng tiếp tục.
17. Khi bạn gọi hàm thành viên `open` của một đối tượng `ofstream`, tệp được chỉ định sẽ bị xóa nếu nó đã tồn tại.

5.10.2. Bài tập

1. Tổng các số (Bắt buộc)

Viết chương trình yêu cầu người dùng nhập một giá trị nguyên dương. Chương trình nên sử dụng một vòng lặp để lấy tổng của tất cả các số nguyên từ 1 đến số được nhập. Ví dụ, nếu người dùng nhập 50, vòng lặp sẽ tìm tổng của 1, 2, 3, 4, ..., 50. Biến đầu vào: Không chấp nhận số âm.

2. Các ký tự mã ASCII

Viết chương trình sử dụng vòng lặp để hiển thị các ký tự cho các mã ASCII từ 0 đến 127. Hiển thị 16 ký tự trên mỗi dòng.

3. Mức độ của đại dương:

Giả sử mực nước biển hiện đang tăng khoảng 1,5 milimet mỗi năm, hãy viết một chương trình hiển thị bảng hiển thị số milimet mà đại dương sẽ tăng lên mỗi năm trong 25 năm tới.

4. Lượng calo bị đốt cháy (Bắt buộc)

Chạy trên một máy chạy bộ cụ thể, bạn đốt cháy 3,6 calo mỗi phút. Viết chương trình sử dụng vòng lặp để hiển thị số calo đã đốt cháy sau 5, 10, 15, 20, 25 và 30 phút.

5. Tăng phí thành viên

Một câu lạc bộ, hiện thu phí 2.500 đô la mỗi năm cho các thành viên và đã thông báo sẽ tăng phí thành viên lên 4% mỗi năm trong 6 năm tới. Viết chương trình sử dụng một vòng lặp để hiển thị mức phí dự kiến trong 6 năm tới.

6. Khoảng cách đi du lịch (Bắt buộc)

Quãng đường xe đi được có thể được tính như sau:

$$\text{distance} = \text{speed} * \text{time} \text{ (khoảng cách = tốc độ * thời gian)}$$

Ví dụ, nếu một chuyên tàu đi 40 dặm một giờ trong 3 giờ, quãng đường đi được là 120 dặm.

Viết một chương trình yêu cầu người dùng cho biết tốc độ của một chiếc xe (số dặm trên một giờ) và số giờ đã đi. Sau đó, chương trình sẽ sử dụng một vòng lặp để hiển thị quãng đường xe đã đi trong mỗi giờ của khoảng thời gian đó. Đây là một ví dụ về đầu ra:

Tốc độ của xe là bao nhiêu dặm trên một giờ? 40

Xe đã đi được bao nhiêu giờ? 3

Khoảng cách theo giờ mà xe đã đi

1 40

2 80

Xác thực đầu vào: Không chấp nhận số âm cho tốc độ và không chấp nhận bất kỳ giá trị nào nhỏ hơn 1 cho thời gian di chuyển

7. Đồng để thanh toán

Viết một chương trình tính toán số tiền một người sẽ kiếm được trong một khoảng thời gian nếu tiền lương của họ là một xu vào ngày đầu tiên và hai xu vào ngày thứ hai, và tiếp tục tăng gấp đôi mỗi ngày. Chương trình sẽ hỏi người dùng về số ngày. Hiện thị bảng hiển thị số tiền lương cho mỗi ngày, sau đó hiển thị tổng tiền lương vào cuối kỳ. Đầu ra phải được hiển thị bằng số đô la, không phải số xu.

Xác thực đầu vào:: Không chấp nhận số ngày nhỏ hơn 1 cho số ngày làm việc và 100 xu = 1 đô la.

8. Gia sư Toán (Bắt buộc)

Viết chương trình hiển thị ngẫu nhiên 2 số, sau đó hiển thị thực đơn cho người dùng lựa chọn phép toán: cộng, trừ, nhân, chia và chức năng thoát. Nếu người dùng chọn cộng thì chương trình thực hiện tính tổng 2 số ngẫu nhiên, tương tự với các phép toán. Sau khi người dùng hoàn thành bài toán, chương trình sẽ hiển thị lại thực đơn để người dùng có thể lựa chọn phép toán khác. Quá trình này được lặp lại cho đến khi người dùng chọn thoát khỏi chương trình.

Xác thực đầu vào: Nếu người dùng chọn một mục không có trong thực đơn, hiển thị thông báo lỗi và hiển thị lại thực đơn.

9. Người ở khách sạn

Viết chương trình tính tỷ lệ % sử dụng phòng trong khách sạn. Chương trình bắt đầu bằng cách hỏi người dùng xem khách sạn có bao nhiêu tầng. Sau đó, một vòng lặp sẽ lặp lại một lần cho mỗi tầng. Trong mỗi lần lặp, sẽ hỏi người dùng về số lượng phòng trong tầng và bao nhiêu phòng trong số đó được sử dụng. Sau tất cả các lần lặp lại, chương trình sẽ hiển thị khách sạn có bao nhiêu phòng, bao nhiêu phòng trong số đó đang được sử dụng, bao nhiêu phòng chưa sử dụng và tỷ lệ phòng có người. Tỷ lệ phần trăm có thể được tính bằng cách chia số phòng sử dụng cho tổng số phòng.

Theo truyền thống, hầu hết các khách sạn không có tầng mười ba. Vòng lặp trong chương trình này nên bỏ qua lần lặp thứ mười ba.

Xác thực đầu vào: Không chấp nhận giá trị số tầng nhỏ hơn 1. Không chấp nhận số lượng phòng ít hơn 10 trên một tầng.

10. Lượng mưa trung bình: (Bắt buộc)

Viết chương trình sử dụng các vòng lặp lồng nhau để thu thập dữ liệu và tính toán lượng mưa trung bình trong một khoảng thời gian hàng năm. Đầu tiên chương trình nên hỏi người

dùng số năm. Vòng lặp bên ngoài sẽ lặp lại một lần cho mỗi năm. Vòng lặp bên trong sẽ lặp lại 12 lần, mỗi tháng một lần. Mỗi lần lặp lại của vòng lặp bên trong sẽ hỏi người dùng về lượng mưa trong tháng đó. Sau tất cả các lần lặp, chương trình sẽ hiển thị số tháng, tổng lượng mưa và lượng mưa trung bình mỗi tháng trong toàn bộ thời kỳ

Xác thực đầu vào: Không chấp nhận số năm nhỏ hơn 1. Không chấp nhận số âm cho lượng mưa hàng tháng.

11. Quần thể

Viết chương trình dự đoán số lượng sinh vật trong một quần thể. Chương trình phải hỏi người dùng về số lượng sinh vật ban đầu, mức tăng trung bình hàng ngày của quần thể (theo phần trăm) và số ngày chúng sẽ nhân lên. Một vòng lặp sẽ hiển thị số lượng sinh vật cho mỗi ngày.

Xác thực đầu vào:

- Không chấp nhận số lượng ban đầu một số nhỏ hơn 2.
- Không chấp nhận mức tăng trung bình âm.
- Không chấp nhận số ngày chúng sẽ nhân lên nhỏ hơn 1.

12. Bảng độ C sang độ F:

Viết chương trình hiển thị bảng nhiệt độ C từ 0 - 20 và độ F tương đương. Biết công thức chuyển đổi từ độ C sang độ F như sau:

$$F = \frac{9}{5}C + 32$$

Trong đó: F là độ F; C là độ C

13. Lớn nhất và nhỏ nhất trong các số

Viết một chương trình với một vòng lặp cho phép người dùng nhập một dãy các số nguyên cho đến khi người dùng nhập vào số -99. Sau khi tất cả các số đã được nhập, chương trình sẽ hiển thị các số lớn nhất và nhỏ nhất đã nhập.

14. Xếp hàng học sinh

Một giáo viên đã yêu cầu tất cả học sinh của mình xếp hàng theo tên của họ. Ví dụ, trong một lớp học, Amy sẽ đứng đầu hàng, và Yolanda sẽ đứng cuối. Viết chương trình nhắc người dùng nhập số lượng học sinh trong lớp, sau đó lặp lại để đọc (nhập) tên từng học sinh. Khi tất cả các tên đã được đọc, nó sẽ báo cáo học sinh nào sẽ ở đầu hàng và học sinh nào sẽ ở cuối hàng. Giả sử rằng không có hai học sinh nào trùng tên.

Xác thực đầu vào: Không chấp nhận số lượng học sinh nhỏ hơn 1 hoặc lớn hơn 25.

15. Báo cáo tiền lương (Bắt buộc)

Viết chương trình hiển thị báo cáo lương hàng tuần. Một vòng lặp trong chương trình sẽ yêu cầu người dùng cung cấp mã số nhân viên, tổng lương, thuế tiểu bang, thuế liên bang và thuế thu nhập. Vòng lặp sẽ kết thúc khi nhập mã số nhân viên là 0. Sau khi dữ liệu được nhập, chương trình sẽ hiển thị tổng số tiền lương, thuế tiểu bang, thuế liên bang, thuế thu nhập và thanh toán ròng.

Xác thực đầu vào: Không chấp nhận số âm cho bất kỳ mục nào đã nhập. Không chấp nhận các giá trị khấu trừ của tiểu bang, liên bang hoặc thuế thu nhập lớn hơn tổng lương. Nếu tổng thuế tiểu bang + thuế liên bang + thuế thu nhập cho bất kỳ nhân viên nào lớn hơn tổng lương, hãy in thông báo lỗi và yêu cầu người dùng nhập lại dữ liệu cho nhân viên đó.

16. Số dư tài khoản tiết kiệm

Viết chương trình tính số dư của tài khoản tiết kiệm tại thời điểm cuối kỳ. Chương trình sẽ hỏi người dùng về lãi suất hàng năm, số dư ban đầu và số tháng đã trôi qua kể từ khi tài khoản được thành lập. Sau đó, một vòng lặp sẽ lặp lại mỗi tháng một lần, thực hiện như sau:

- A. Hỏi người dùng số tiền gửi vào tài khoản trong tháng. (Không chấp nhận số âm.) Số tiền này phải được cộng vào số dư.
- B. Hỏi người dùng số tiền rút từ tài khoản trong tháng. (Không chấp nhận số âm.) Số tiền này phải được trừ vào số dư.
- C. Tính tiền lãi hàng tháng. Lãi suất hàng tháng là lãi suất hàng năm chia cho 12. Nhân lãi suất hàng tháng với số dư, và cộng kết quả với số dư.

Sau lần lặp cuối cùng, chương trình sẽ hiển thị số dư cuối kỳ, tổng số tiền gửi, tổng số tiền rút và tổng tiền lãi thu được.

Chú ý: Nếu số dư âm được tính tại bất kỳ thời điểm nào, một thông báo sẽ hiển thị cho biết tài khoản đã bị đóng và vòng lặp sẽ kết thúc.

17. Biểu đồ thanh toán bán hàng (Bắt buộc)

Viết một chương trình yêu cầu người dùng nhập doanh số bán hàng của ngày hôm nay của 5 cửa hàng. Sau đó, chương trình sẽ hiển thị biểu đồ dạng thanh so sánh doanh số bán hàng của từng cửa hàng. Tạo mỗi thanh trong biểu đồ thể hiện doanh số bán hàng của một cửa hàng. Mỗi dấu hoa thị “*” tương đương với 100 đô la doanh thu.

Dưới đây là một ví dụ về đầu ra của chương trình:

```
Nhập vào doanh số bán hàng của cửa hàng thứ 1: 1000 Enter
Nhập vào doanh số bán hàng của cửa hàng thứ 2: 1200 Enter
Nhập vào doanh số bán hàng của cửa hàng thứ 3: 1800 Enter
Nhập vào doanh số bán hàng của cửa hàng thứ 4: 800 Enter
Nhập vào doanh số bán hàng của cửa hàng thứ 5: 1900 Enter
SALES BAR CHART
(Each * = $100)
```

Cửa hàng 1:
Cửa hàng 2:
Cửa hàng 3:
Cửa hàng 4:
Cửa hàng 5: *

18. Biểu đồ cột dân số (Bắt buộc)

Viết một chương trình tạo ra một biểu đồ thanh thể hiện sự gia tăng dân số của Prairieville, một thị trấn nhỏ ở giữa phía Tây của Hoa Kỳ, với khoảng thời gian 20 năm trong suốt 100 năm qua. Chương trình sẽ đọc các số liệu dân số (làm tròn đến 1.000 người gần nhất) cho các năm 1900, 1920, 1940, 1960, 1980 và 2000 từ một tệp. Mỗi năm, nó sẽ hiển thị năm và một thanh là các kí tự hoa thị '*', mỗi dấu hoa thị '*' tương ứng với 1.000 người. Dữ liệu có thể được tìm thấy trong tệp People.txt.

(Có thể lấy tệp People.txt trong tệp đính kèm trên trang elearning của bài tập)

Dưới đây là một ví dụ về cách biểu đồ có thể bắt đầu:

TĂNG TRƯỞNG DÂN SỐ PRAIRIEVILLE

(Mỗi kí tự '' đại diện cho 1.000 người)*

1900

1920

1940 *

19. Phân tích ngân sách:

Viết một chương trình yêu cầu người dùng nhập số tiền mà họ đã lập ngân sách cho một tháng. Sau đó, một vòng lặp sẽ hỏi người dùng nhập từng chi phí của họ trong tháng và tổng số tiền hiện có. Khi vòng lặp kết thúc, chương trình sẽ hiển thị số tiền mà người dùng vượt quá hoặc dư ngân sách.

20. Trò chơi đoán số ngẫu nhiên:

Viết chương trình tạo một số ngẫu nhiên và yêu cầu người dùng đoán số đó là gì. Nếu dự đoán của người dùng cao hơn số ngẫu nhiên, chương trình sẽ hiển thị "Rất cao, hãy thử lại!" Nếu dự đoán của người dùng thấp hơn số ngẫu nhiên, chương trình sẽ hiển thị "Rất thấp, hãy thử lại!" Chương trình nên sử dụng một vòng lặp lặp lại cho đến khi người dùng đoán đúng số ngẫu nhiên.

21. Cải tiến thêm ở bài 20:

Nâng cao chương trình mà bạn đã viết cho **bài 20** để nó lưu giữ số lần người dùng đoán. Khi người dùng đoán đúng số ngẫu nhiên, chương trình sẽ hiển thị số người đoán.

22. Màn hình vương

Viết chương trình yêu cầu người dùng nhập một số nguyên dương không lớn hơn 15. Sau đó, chương trình sẽ hiển thị một hình vuông trên màn hình bằng ký tự X'. Số người dùng nhập vào sẽ là độ dài của mỗi cạnh của hình vuông.

Ví dụ, nếu người dùng nhập 5, chương trình sẽ hiển thị như sau:

```
XXXXX
XXXXX
XXXXX
XXXXX
XXXXX
```

Nếu người dùng nhập 8, chương trình sẽ hiển thị như sau:

```
XXXXXXXXX
XXXXXXXXX
XXXXXXXXX
XXXXXXXXX
XXXXXXXXX
XXXXXXXXX
XXXXXXXXX
XXXXXXXXX
```

23. Màn hình mẫu

Viết chương trình hiển thị Hình 1 bằng 1 vòng lặp và Hình 2 bằng 1 vòng lặp

+

++

+++

++++

Hình 1

++++

+++

++

+

Hình 2

24. Xử lý số - Sử dụng tệp (Bắt buộc)

Các bạn có thể lấy file random.txt trong tệp đính kèm trên trang elearning của bài tập. Tệp này chứa một danh sách dài các số ngẫu nhiên. Sao chép tệp vào hệ thống của bạn, sau đó viết chương trình mở tệp, đọc tất cả các số từ tệp và tính toán những điều sau:

A. Số lượng các số trong tệp

- B. Tổng của tất cả các số trong tệp
- C. Giá trị trung bình của tất cả các số trong tệp

Chương trình sẽ hiển thị số lượng các số được tìm thấy trong tệp, tổng các số và giá trị trung bình của các số.

25. Sử dụng file – Xếp hàng học sinh

Sửa đổi chương trình Đội hình sinh viên được mô tả trong Bài 14 để chương trình lấy tên từ tệp. Tên nên được đọc cho đến khi không còn dữ liệu để đọc. Có thể sử dụng file lineUp.txt trong tệp đính kèm trên trang elearning của bài tập.

26. Tạo 1 trang web cá nhân:

Viết một chương trình hỏi người dùng tên của họ, sau đó yêu cầu người dùng nhập một câu mô tả về bản thân họ. Đây là một ví dụ về màn hình của chương trình:

Enter your name: Julie Taylor Enter

Describe yourself: I am a computer science major, a member of the Jazz club, and I hope to work as a mobile app developer after I graduate.

Khi người dùng đã nhập đầu vào được yêu cầu, chương trình sẽ tạo một tệp HTML, chứa đầu vào, cho một trang web đơn giản. Đây là một ví dụ về nội dung HTML, sử dụng đầu vào mẫu được hiển thị trước đó:

```
<html>
<head>
</head>
<body>
  <center>
    <h1>Julie Taylor</h1>
  </center>
  <hr />
  I am a computer science major, a member of the Jazz club,
  and I hope to work as a mobile app developer after I graduate.
  <hr />
</body>
</html>
```

27. Các bước trung bình đã thực hiện:

Máy theo dõi thể dục cá nhân là một thiết bị đeo được để theo dõi hoạt động thể chất của bạn, lượng calo đốt cháy, nhịp tim, kiểu ngủ, v.v. Một hoạt động thể chất phổ biến mà hầu hết các thiết bị này theo dõi là số bước bạn thực hiện mỗi ngày.

Các bạn có thể lấy tệp Steps.txt trong tệp đính kèm trên trang elearning của bài tập. Có 365 dòng trong tệp và mỗi dòng chứa số bước được thực hiện trong một ngày. (Dòng đầu tiên là số bước được thực hiện vào ngày 1 tháng 1, dòng thứ hai là số bước được thực hiện vào ngày 2 tháng 1, v.v.). Viết chương trình đọc tệp, sau đó hiển thị số bước trung bình được thực hiện cho mỗi tháng. (Dữ liệu từ một năm không phải là năm nhuận, vì vậy tháng Hai có 28 ngày.)

CHƯƠNG 6. HÀM

Chủ đề	
6.1	Mô đun chương trình
6.2	Định nghĩa và lời gọi hàm
6.3	Hàm nguyên mẫu
6.4	Truyền tham số cho hàm
6.5	Menu và hàm
6.6	Giá trị trả về từ hàm
6.7	Biến cục bộ và biến toàn cục
6.8	Đôi số mặc định
6.9	Biến tham chiếu là tham số

6.1. Mô đun chương trình

KHÁI NIỆM: Một chương trình có thể được chia thành các chức năng có thể quản lý được.

Một hàm là một tập hợp các câu lệnh thực hiện một nhiệm vụ cụ thể. Đến nay, bạn đã trải qua các hàm theo hai cách: (1) bạn đã tạo một hàm có tên là main trong mỗi chương trình bạn đã viết, và (2) bạn đã sử dụng các hàm thư viện như pow và strcmp. Trong chương này, bạn sẽ học cách tạo ra các hàm của riêng mình mà có thể được sử dụng như các hàm thư viện.

Hàm thường được sử dụng để chia nhỏ một vấn đề thành các phần nhỏ, dễ quản lý. Thay vì viết một hàm dài chứa tất cả các câu lệnh cần thiết để giải quyết một vấn đề, bạn có thể viết một số hàm nhỏ mỗi hàm giải quyết một phần cụ thể của vấn đề. Những hàm nhỏ này sau đó có thể được thực thi theo thứ tự mong muốn để giải quyết vấn đề.

Phương pháp này đôi khi được gọi là chia để trị vì một vấn đề lớn được chia thành một số vấn đề nhỏ hơn mà dễ dàng giải quyết. Hình 6-1 minh họa ý tưởng này bằng cách so sánh hai chương trình: một chương trình sử dụng một hàm dài, phức tạp chứa tất cả các câu lệnh cần thiết để giải quyết một vấn đề, và một chương trình khác chia vấn đề thành các vấn đề nhỏ hơn, mỗi vấn đề được xử lý bởi một hàm riêng biệt.

Một lý do khác để viết các hàm là chúng làm đơn giản hóa chương trình. Nếu một nhiệm vụ cụ thể được thực hiện ở nhiều nơi trong một chương trình, một hàm có thể được viết một lần để thực hiện nhiệm vụ đó, sau đó được thực thi bất cứ khi nào cần thiết. Lợi ích của việc sử dụng các hàm này được biết đến như là tái sử dụng mã vì bạn viết mã để thực hiện một nhiệm vụ một lần, sau đó sử dụng lại nó mỗi khi bạn cần thực hiện nhiệm vụ đó.

6.2. Định nghĩa và lời gọi hàm

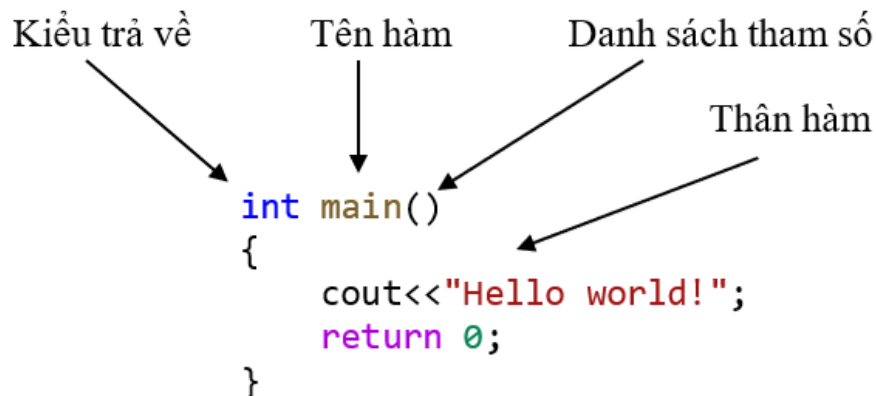
KHÁI NIỆM: Lời gọi hàm là một câu lệnh khiến một hàm được thực thi. Một định nghĩa hàm chứa các câu lệnh tạo nên hàm đó.

Khi tạo một hàm, bạn phải viết định nghĩa của nó. Tất cả các định nghĩa hàm đều có các phần sau:

Bảng 6.1. Bảng các thành phần của hàm

Kiểu trả về:	Một hàm có thể gửi một giá trị đến phần của chương trình đã thực thi nó. Loại trả về là kiểu dữ liệu của giá trị được gửi từ hàm đó.
Tên:	Bạn nên đặt tên mô tả cho mỗi hàm. Nói chung, các quy tắc giống nhau áp dụng cho tên biến cũng áp dụng cho tên hàm.
Danh sách tham số:	Chương trình có thể gửi dữ liệu vào một hàm. Danh sách tham số là một danh sách các biến giữ các giá trị được chuyển vào hàm.
Thân hàm:	Thân của một hàm là tập hợp các câu lệnh thực hiện hoạt động của hàm. Chúng được bao quanh bởi một cặp dấu ngoặc nhọn.

Hình ảnh sau đây mô tả các thành phần của hàm



Hình 6.1. Các thành phần của hàm

Dòng `int main()` được gọi là tiêu đề hàm

Hàm void

Bạn đã biết rằng một hàm có thể trả về một giá trị. Hàm `main` trong tất cả các chương trình bạn đã thấy trong cuốn sách này được khai báo để trả về một giá trị `int` cho hệ điều hành. Câu lệnh `return 0;` khiến cho giá trị 0 được trả về khi hàm `main` kết thúc thực thi.

Tuy nhiên, không phải tất cả các hàm đều cần phải trả về một giá trị. Một số hàm đơn giản chỉ thực hiện một hoặc nhiều câu lệnh, sau đó kết thúc. Chúng được gọi là hàm void. Hàm `displayMessage`, sau đây, là một ví dụ.

```
void displayMessage()
```

```

{
    cout << "Xin chào từ hàm displayMessage.\n";
}

```

Tên của hàm là displayMessage. Tên này cho biết chức năng của hàm: Hiển thị một thông điệp. Bạn nên luôn đặt tên cho các hàm phản ánh mục đích của chúng. Lưu ý kiểu trả về của hàm là void. Điều này có nghĩa là hàm không trả về giá trị cho phần của chương trình đã thực thi nó. Cũng lưu ý rằng hàm không có câu lệnh return. Nó chỉ đơn giản là hiển thị một thông điệp trên màn hình và kết thúc.

Gọi một hàm

Một hàm được thực thi khi nó được gọi. Hàm main được gọi tự động khi một chương trình bắt đầu, nhưng tất cả các hàm khác phải được thực thi bằng các câu lệnh gọi hàm.

Khi một hàm được gọi, chương trình nhánh tới hàm đó và thực thi các câu lệnh trong thân hàm của nó. Hãy xem xét chương trình dưới, trong đó chứa hai hàm: main và displayMessage.

Chương trình 6.1. Hàm hiển thị displayMessage

```

// Hàm displayMessage được gọi lặp đi lặp lại từ một vòng lặp.
#include <iostream>
using namespace std;

/**
// Định nghĩa của hàm displayMessage *
// Hàm này hiển thị một lời chào. *
/**

void displayMessage()
{
    cout << "Xin chào từ hàm displayMessage.\n";
}

/**
// Hàm main *
/**

int main()
{
    cout << "Xin chào từ hàm main.\n";
    for (int count = 0; count < 5; count++)
        displayMessage(); // Gọi hàm displayMessage
    cout << "Quay lại hàm main một lần nữa.\n";
    return 0;
}

```

Kết quả chương trình:

```

Xin chao tu ham main.
Xin chao tu ham displayMessage.

```

```
Xin chao tu ham displayMessage.  
Xin chao tu ham displayMessage.  
Xin chao tu ham displayMessage.  
Xin chao tu ham displayMessage.  
Quay lai ham main mot lan nua.
```

Có thể có nhiều hàm và lệnh gọi hàm trong một chương trình. Chương trình sau có ba hàm: main, first, second.

Chương trình 6.2. Minh họa thứ tự thực thi các hàm

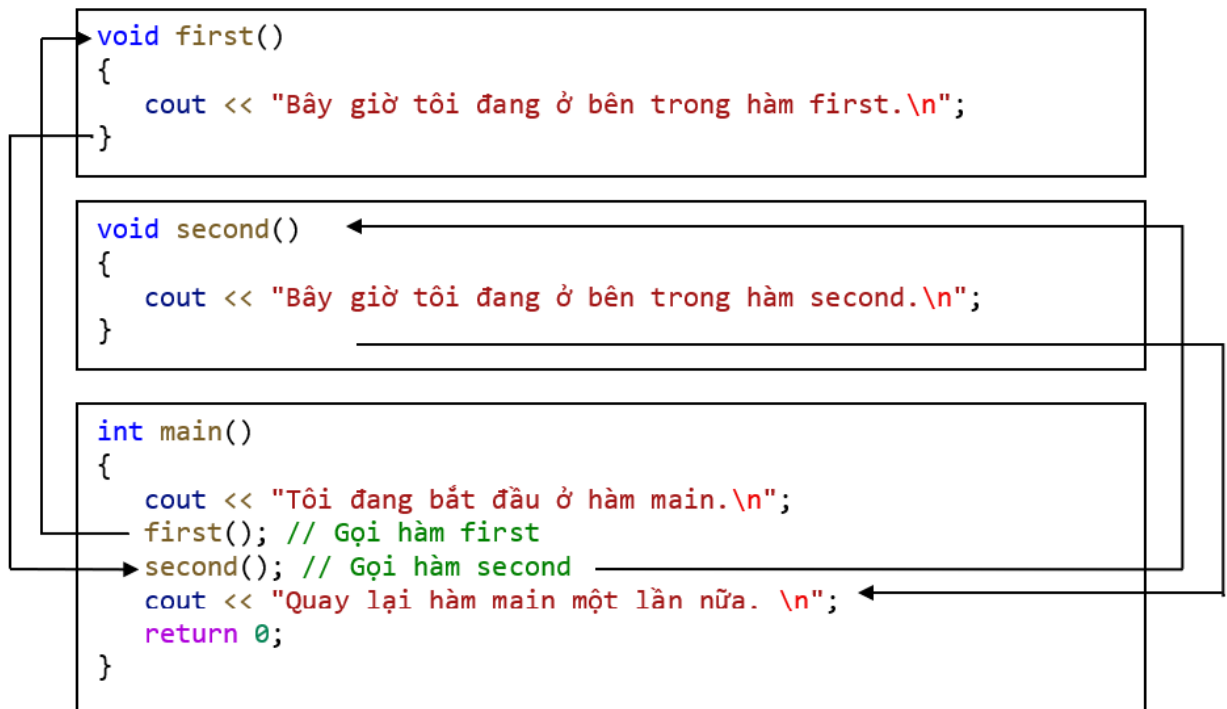
```
// Chương trình này có ba hàm: main, first và second.  
#include <iostream>  
using namespace std;  
/**  
// Định nghĩa của hàm first *  
// Hàm này hiển thị một thông điệp. *  
/**  
  
void first()  
{  
    cout << "Bây giờ tôi đang ở bên trong hàm first.\n";  
}  
  
/**  
// Định nghĩa của hàm second *  
// Hàm này hiển thị một thông điệp. *  
/**  
  
void second()  
{  
    cout << "Bây giờ tôi đang ở bên trong hàm second.\n";  
}  
  
/**  
// Hàm main *  
/**  
  
int main()  
{  
    cout << "Tôi đang bắt đầu ở hàm main.\n";  
    first(); // Gọi hàm first  
    second(); // Gọi hàm second  
    cout << "Quay lại hàm main một lần nữa. \n";  
    return 0;  
}
```

Kết quả chương trình:

Tôi đang bắt đầu ở hàm main

Bây giờ tôi đang ở bên trong hàm first
Bây giờ tôi đang ở bên trong hàm second
Quay lại hàm main một lần nữa

Hình 6-4 minh họa các đường dẫn của chương trình.



6.3. Hàm nguyên mẫu

KHÁI NIỆM: Một hàm nguyên mẫu (prototype) loại bỏ nhu cầu phải đặt một định nghĩa hàm trước tất cả các cuộc gọi đến hàm.

Trước khi trình biên dịch gặp một cuộc gọi đến một hàm cụ thể, nó phải đã biết loại trả về của hàm, số lượng tham số mà nó sử dụng, và kiểu của mỗi tham số. (Bạn sẽ học cách sử dụng tham số trong phần tiếp theo.)

Một cách để đảm bảo rằng trình biên dịch có thông tin này là đặt định nghĩa hàm trước tất cả các cuộc gọi đến hàm đó. Đây là phương pháp được áp dụng trong các Chương trình 6-1, 6-2, 6-3 và 6-4. Một phương pháp khác là khai báo hàm với một hàm prototype. Dưới đây là một hàm prototype cho hàm `displayMessage` trong Chương trình 6-1:

```
void displayMessage();
```

Hàm nguyên mẫu trông giống như tiêu đề hàm, trừ việc có một dấu chấm phẩy ở cuối.

Câu lệnh trên thông báo cho trình biên dịch rằng hàm `displayMessage` có một loại trả về `void` (nó không trả về giá trị) và không sử dụng tham số.

CHÚ Ý: Hàm nguyên mẫu cũng được biết đến là khai báo hàm.

CẢNH BÁO! Bạn phải đặt định nghĩa hàm hoặc hàm prototype trước tất cả các cuộc gọi đến hàm. Nếu không, chương trình sẽ không biên dịch được.

Hàm nguyên mẫu thường được đặt gần đầu chương trình để trình biên dịch sẽ gặp chúng trước bất kỳ cuộc gọi hàm nào. Chương trình 6-5 là một sửa đổi của Chương trình 6-3. Các định nghĩa của các hàm first và second đã được đặt sau main, và một hàm prototype đã được đặt sau câu lệnh using namespace std.

Chương trình 6.3. Minh họa hàm nguyên mẫu

```
// Chương trình này có ba hàm: main, first và second.
#include <iostream>
using namespace std;
// Hàm Prototypes
void first();
void second();

int main()
{
    cout << "Tôi bắt đầu ở hàm main.\n";
    first(); // Gọi hàm first
    second(); // Gọi hàm second
    cout << "Quay lại hàm main một lần nữa.\n";
    return 0;
}

/**
// Định nghĩa của hàm first. *
// Hàm này hiển thị một thông điệp. *
/**

void first()
{
    cout << "Bây giờ tôi đang ở bên trong hàm first.\n";
}

/**
// Định nghĩa của hàm second. *
// Hàm này hiển thị một thông điệp. *
/**

void second()
{
    cout << "Bây giờ tôi đang ở bên trong hàm second.\n";
}
```

Kết quả chương trình:

Tôi đang bắt đầu ở hàm main
Bây giờ tôi đang ở bên trong hàm first
Bây giờ tôi đang ở bên trong hàm second
Quay lại hàm main một lần nữa

6.4. Truyền tham số cho hàm

KHÁI NIỆM: Khi một hàm được gọi, chương trình có thể gửi các giá trị vào hàm.

Các giá trị gửi vào một hàm được gọi là đối số. Bạn đã quen thuộc với cách sử dụng đối số trong một cuộc gọi hàm. Trong câu lệnh sau, hàm `pow` đang được gọi và hai đối số, 2.0 và 4.0, được truyền cho nó:

```
result = pow(2.0, 4.0);
```

Bằng cách sử dụng tham số, bạn có thể thiết kế các hàm của riêng mình để chấp nhận dữ liệu theo cách này. Một tham số là một biến đặc biệt giữ một giá trị được chuyển vào một hàm. Đây là định nghĩa của một hàm sử dụng một tham số:

```
void displayValue(int num)
{
    cout << "Giá trị là " << num << endl;
}
```

Lưu ý định nghĩa biến số nguyên bên trong dấu ngoặc đơn (`int num`). Biến `num` là một tham số. Điều này cho phép hàm `displayValue` chấp nhận một giá trị số nguyên như một đối số. Chương trình 6-6 là một chương trình hoàn chỉnh sử dụng hàm này.

LƯU Ý: Trong chương trình này, các giá trị được truyền vào một hàm được gọi là đối số, và các biến nhận các giá trị đó được gọi là tham số. Có một số biến thể của các thuật ngữ này được sử dụng. Một số gọi các đối số là tham số thực và gọi các tham số là tham số hình thức. Người khác sử dụng các thuật ngữ đối số thực và đối số hình thức. Bất kể bộ thuật ngữ nào bạn sử dụng, điều quan trọng là phải nhất quán.

Chương trình 6.4. Hàm nguyên mẫu có tham số

```
// Chương trình này minh họa một hàm với một tham số.
#include <iostream>
using namespace std;

// Nguyên mẫu hàm
void displayValue(int);

int main()
{
    cout << "Tôi đang truyền 5 vào hàm displayValue.\n";
    displayValue(5); // Gọi hàm displayValue với đối số 5
    cout << "Bây giờ tôi quay lại trong hàm main.\n";
    return 0;
}
/**
// Định nghĩa của hàm displayValue. *
// Nó sử dụng một tham số nguyên mà giá trị được hiển thị. *
/**

void displayValue(int num)
```

```
{
    cout << "Giá trị là " << num << endl;
}
```

Kết quả chương trình:

Tôi đang truyền 5 vào hàm displayValue

Giá trị là 5

Bây giờ tôi quay lại trong hàm main

Đầu tiên, chú ý đến nguyên mẫu hàm cho hàm displayValue ở dòng 6:

```
void displayValue(int);
```

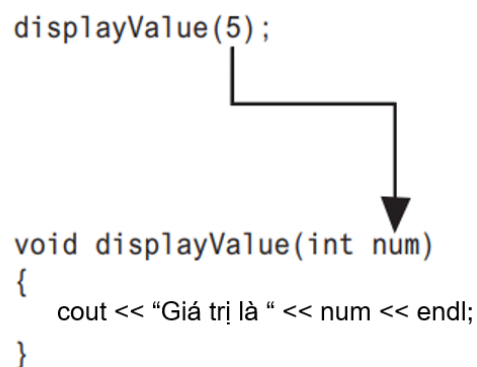
Không cần thiết phải liệt kê tên của biến tham số bên trong dấu ngoặc đơn. Chỉ cần kiểu dữ liệu của nó là cần thiết. Nguyên mẫu hàm được hiển thị ở trên có thể tùy ý được viết như

```
void displayValue(int num);
```

Tuy nhiên, trình biên dịch bỏ qua tên của biến tham số trong nguyên mẫu hàm.

Trong hàm main, hàm displayValue được gọi với đối số 5 bên trong dấu ngoặc đơn.

Số 5 được chuyển vào num, là tham số của displayValue. Điều này được minh họa trong Hình 6.2.



Hình 6.2. Truyền giá trị cho tham số

Bất kỳ đối số nào được liệt kê bên trong dấu ngoặc đơn của lệnh gọi hàm đều được sao chép vào biến tham số của hàm. Về bản chất, các biến tham số được khởi tạo bằng giá trị của các đối số tương ứng của chúng. Chương trình 6-7 hiển thị hàm displayValue được gọi nhiều lần với một đối số khác nhau được truyền vào mỗi lần.

Chương trình 6.5. Minh họa hàm có tham số

```
#include <iostream>
using namespace std;

// Nguyên mẫu hàm
```

```

void displayValue(int);

int main()
{
    cout << "Tôi đang truyền một số giá trị cho hàm displayValue.\n";
    displayValue(5); // Gọi hàm displayValue với đối số 5
    displayValue(10); // Gọi hàm displayValue với đối số 10
    displayValue(2); // Gọi hàm displayValue với đối số 2
    displayValue(16); // Gọi hàm displayValue với đối số 16
    cout << "Bây giờ tôi trở lại trong hàm main.\n";
    return 0;
}

/**
// Định nghĩa hàm displayValue.
// Nó sử dụng một tham số kiểu số nguyên và hiển thị giá trị của nó.
/**

void displayValue(int num)
{
    cout << "Giá trị là " << num << endl;
}

```

Kết quả chương trình:

Tôi đang chuyển một số giá trị tới displayValue.
 Giá trị là 5
 Giá trị là 10
 Giá trị là 2
 Giá trị là 16
 Bây giờ tôi quay lại main.

Thông thường, việc truyền nhiều đối số vào một hàm sẽ rất hữu ích. Chương trình 6-8 hiển thị định nghĩa của hàm có ba tham số.

Chương trình 6.6. Hàm có nhiều tham số

```

// Chương trình này minh họa một hàm có ba tham số.
#include <iostream>
using namespace std;

// Nguyên mẫu hàm
void showSum(int, int, int);

int main()
{
    int value1, value2, value3;

    // Nhận ba số nguyên.
    cout << "Nhập ba số nguyên và tôi sẽ hiển thị ";
    cout << "tổng của chúng: ";
    cin >> value1 >> value2 >> value3;

    // Gọi hàm showSum với ba đối số.

```

```

    showSum(value1, value2, value3);
    return 0;
}

/**
// Định nghĩa hàm showSum.
// No sử dụng ba tham số kiểu số nguyên. Tổng của chúng được hiển thị.
/**

void showSum(int num1, int num2, int num3)
{
    cout << (num1 + num2 + num3) << endl;
}

```

Kết quả chương trình:

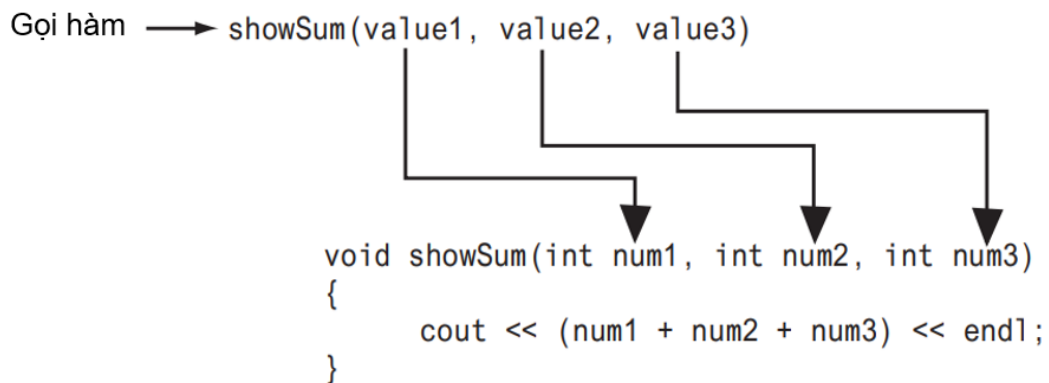
Nhập ba số nguyên và tôi sẽ hiển thị tổng của chúng: 6 7 8
21

Đầu ra chương trình với đầu vào ví dụ được hiển thị in đậm

Trong tiêu đề hàm của showSum, danh sách tham số chứa ba định nghĩa biến được phân tách bằng dấu phẩy:

```
void showSum(int num1, int num2, int num3)
```

Khi một hàm có nhiều tham số được gọi, các đối số sẽ được truyền cho các tham số theo thứ tự. Điều này được minh họa trong Hình 6.3.



Hình 6.3. Truyền nhiều giá trị cho các tham số

Lệnh gọi hàm sau sẽ truyền 5 vào tham số num1, 10 được chuyển vào num2 và 15 được chuyển vào num3:

```
showSum(5, 10, 15);
```

Tuy nhiên, lệnh gọi hàm sau sẽ khiến 15 được truyền vào tham số num1, 5 được truyền vào num2 và 10 được truyền vào num3:

```
showSum(15, 5, 10);
```

LƯU Ý:

- Nguyên mẫu hàm phải liệt kê kiểu dữ liệu của từng tham số.
- Giống như tất cả các biến, tham số có phạm vi. Phạm vi của một tham số được giới hạn trong phần thân của hàm sử dụng nó.

6.5. Minh họa: thực đơn với mỗi chức năng là 1 hàm

Trong chương trình sau, người dùng được yêu cầu nhập số lượng sản phẩm mỗi khi chọn một sản phẩm từ menu. Tổng tiền thanh toán được tính toán dựa trên số lượng sản phẩm mỗi khi mua hàng. Hóa đơn cuối cùng sẽ hiển thị tổng số tiền cần thanh toán sau khi người dùng kết thúc mua hàng.

Chương trình 6.7. Minh họa chức năng menu và hàm

```
#include <iostream>
#include <iomanip>
using namespace std;

// Function Prototypes
void displayMenu();
int buyItem(int itemCode);
void displayReceipt(int totalCost);

int main() {
    int totalCost = 0;
    char choice;

    do {
        // Hien thi menu
        displayMenu();

        // Nhap lua chon tu nguoi dung
        cout << "Chon mot san pham (1-3) hoac nhan '0' de ket thuc: ";
        cin >> choice;

        // Xu ly lua chon cua nguoi dung
        switch (choice) {
            case '0':
                cout << "Da ket thuc mua hang." << endl;
                break;
            case '1':
            case '2':
            case '3':
                totalCost += buyItem(choice);
                break;
            default:
                cout << "San pham khong hop le. Vui long chon lai." <<
endl;
        }
    } while (choice != '0');
```

```

        // Hien thi hoa don cuoi cung
        displayReceipt(totalCost);

        return 0;
    }

    // Hien thi menu san pham
    void displayMenu() {
        cout << "==== MENU BAN HANG =====> endl;
        cout << "1. San pham A - $10" << endl;
        cout << "2. San pham B - $20" << endl;
        cout << "3. San pham C - $30" << endl;
        cout << "0. Ket thuc mua hang" << endl;
        cout << "=====> endl;
    }

    // Ham mua san pham va tra ve tong tien cua san pham duoc mua
    int buyItem(int itemCode) {
        int price;
        switch (itemCode) {
            case 1:
                price = 10;
                break;
            case 2:
                price = 20;
                break;
            case 3:
                price = 30;
                break;
        }
        cout << "Ban da mua san pham " << itemCode << " voi gia $" << price << endl;
        return price;
    }

    // Hien thi hoa don cuoi cung
    void displayReceipt(int totalCost) {
        cout << "==== HOA DON CUOI CUNG =====> endl;
        cout << "Tong tien thanh toan: $" << totalCost << endl;
        cout << "=====> endl;
    }
}

```

Kết quả chương trình:

```
==== MENU BAN HANG =====
```

```
1. San pham A - $10
2. San pham B - $20
3. San pham C - $30
0. Ket thuc mua hang
```

```
=====
```

```
Chon mot san pham (1-3) hoac nhan '0' de ket thuc: 1
```

```
Ban da mua san pham 49 voi gia $17162608
```

```

===== MENU BAN HANG =====
1. San pham A - $10
2. San pham B - $20
3. San pham C - $30
0. Ket thuc mua hang
=====
Chon mot san pham (1-3) hoac nhan '0' de ket thuc: 2
Ban da mua san pham 50 voi gia $17162608
===== MENU BAN HANG =====
1. San pham A - $10
2. San pham B - $20
3. San pham C - $30
0. Ket thuc mua hang
=====
Chon mot san pham (1-3) hoac nhan '0' de ket thuc: 0
Da ket thuc mua hang.
===== HOA DON CUOI CUNG =====
Tong tien thanh toan: $34325216
=====

```

6.6. Giá trị trả về từ hàm

KHÁI NIỆM: Một hàm có thể gửi một giá trị trở lại phần chương trình gọi hàm đó.

Bạn đã thấy rằng dữ liệu có thể được truyền vào hàm bằng các biến tham số. Dữ liệu cũng có thể được trả về từ một hàm, quay trở lại câu lệnh đã gọi nó. Các hàm trả về một giá trị được gọi một cách thích hợp là các hàm trả về giá trị.

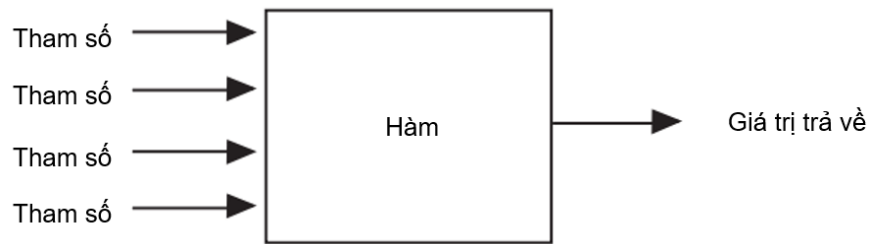
Hàm pow mà bạn đã thấy là một ví dụ về hàm trả về giá trị. Đây là một ví dụ:

```
double x;
```

```
x = pow(4.0, 2.0);
```

Dòng thứ hai trong đoạn mã này gọi hàm pow, truyền 4.0 và 2.0 làm đối số. Hàm tính giá trị của 4.0 lũy thừa 2.0 và trả về giá trị đó. Giá trị là 16,0 được gán cho biến x bởi toán tử =.

Mặc dù một số đối số có thể được truyền vào một hàm nhưng chỉ có một giá trị được trả về từ hàm đó. Hãy coi một hàm có nhiều kênh liên lạc để nhận dữ liệu (tham số), nhưng chỉ có một kênh để gửi dữ liệu (giá trị trả về). Điều này được minh họa trong Hình 6-4.



Hình 6.4. Hàm trả về giá trị

Định nghĩa hàm trả về giá trị

Khi viết một hàm trả về giá trị, bạn phải quyết định loại giá trị mà hàm sẽ trả về. Điều này là do bạn phải chỉ định kiểu dữ liệu của giá trị trả về trong tiêu đề hàm và trong nguyên mẫu hàm. Hãy nhớ lại rằng hàm void, không trả về giá trị, sử dụng từ khóa void làm kiểu trả về trong tiêu đề hàm.

Hàm trả về giá trị sẽ sử dụng int, double, bool hoặc bất kỳ loại dữ liệu hợp lệ nào khác trong tiêu đề của nó. Đây là một ví dụ về hàm trả về giá trị int:

```
int sum(int num1, int num2)
{
    int result;
    result = num1 + num2;
    return result;
}
```

Tên của hàm này là tổng. Lưu ý trong tiêu đề hàm kiểu trả về là int, như minh họa trong Hình 6.5.

Kiểu trả về
↓
int sum(int num1, int num2)

Hình 6.5. Định nghĩa hàm trả về giá trị

Đoạn mã này định nghĩa một hàm có tên sum chấp nhận hai đối số int. Các đối số được truyền vào các biến tham số num1 và num2. Bên trong hàm, một biến result được xác định. Các biến được xác định bên trong hàm được gọi là biến cục bộ. Sau khi định nghĩa biến, các biến tham số num1 và num2 được cộng vào và tổng của chúng được gán cho biến result. Câu lệnh cuối cùng trong hàm là

```
return result;
```

Câu lệnh này khiến hàm kết thúc và nó gửi giá trị của biến kết quả trở lại câu lệnh gọi hàm. Hàm trả về giá trị phải có câu lệnh return được viết theo định dạng chung sau:

```
return biểu thức;
```

Ở định dạng chung, biểu thức là giá trị được trả về. Nó có thể là bất kỳ biểu thức nào có giá trị, chẳng hạn như biến, biểu thức hằng hoặc biểu thức toán học. Giá trị của biểu thức được chuyển đổi thành kiểu dữ liệu mà hàm trả về và được gửi trở lại câu lệnh gọi hàm. Trong trường hợp này, hàm tổng trả về giá trị của biến result.

Tuy nhiên, chúng ta có thể loại bỏ biến result và trả về biểu thức $\text{num1} + \text{num2}$, như trong đoạn mã sau:

```
int sum(int num1, int num2)
{
    return num1 + num2;
}
```

Khi viết nguyên mẫu cho hàm trả về giá trị, hãy tuân theo các quy ước tương tự mà chúng ta đã đề cập trước đó. Đây là nguyên mẫu của hàm tính tổng:

```
int sum(int, int);
```

Gọi hàm trả về một giá trị

Chương trình 6.8 đưa ra một ví dụ về cách gọi hàm sum.

Chương trình 6.8. Hàm có giá trị trả về

```
// Chương trình này sử dụng một hàm trả về một giá trị.
#include <iostream>
using namespace std;

// Nguyên mẫu hàm
int tong(int, int);

int main()
{
    int gia_tri1 = 20, // Giá trị thứ nhất
    gia_tri2 = 40, // Giá trị thứ hai
    tong; // Để lưu tổng

    // Gọi hàm tong, truyền nội dung của
    // gia_tri1 và gia_tri2 làm đối số. Gán giá trị trả về
    // cho biến tong.
    tong = tong(gia_tri1, gia_tri2);

    // Hiển thị tổng của các giá trị.
    cout << "Tổng của " << gia_tri1 << " và "
    << gia_tri2 << " là " << tong << endl;
    return 0;
}

/**
// Định nghĩa hàm tong. Hàm này trả về *
// tổng của hai tham số của nó. *
/**

int tong(int num1, int num2)
```

```
{
    return num1 + num2;
}
```

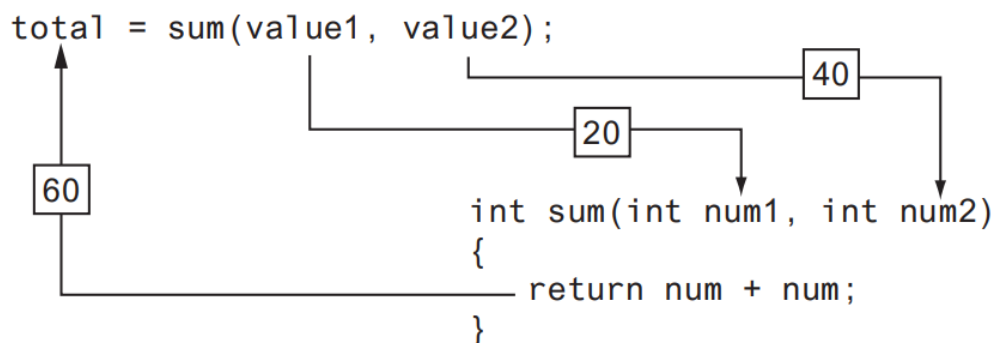
Kết quả chương trình:

Tổng của 20 và 40 là 60

Đây là câu lệnh ở dòng 17 gọi hàm tổng, truyền value1 và value2 làm đối số:

```
tong = tong(gia_tri1, gia_tri2);
```

Câu lệnh này gán giá trị được hàm sum trả về cho biến tổng. Trong trường hợp này, hàm sẽ trả về 60. Hình 6.6 cho thấy cách các đối số được truyền vào hàm và cách một giá trị được truyền trở lại từ hàm.



Hình 6.6. Truyền các đối số cho hàm và hàm trả về giá trị

Khi bạn gọi một hàm trả về giá trị, thường bạn muốn làm một điều có ý nghĩa với giá trị mà nó trả về. Chương trình 6-12 cho thấy giá trị trả về của một hàm được gán cho một biến. Điều này thường là cách mà các giá trị trả về được sử dụng, nhưng bạn cũng có thể làm nhiều điều khác với chúng. Ví dụ, đoạn mã sau cho thấy một biểu thức toán học sử dụng một cuộc gọi đến hàm tổng:

```
int x = 10, y = 15;
double average;
average = sum(x, y) / 2.0;
```

Trong câu lệnh cuối cùng, hàm tổng được gọi với x và y làm đối số của nó. Giá trị trả về của hàm là 25, được chia cho 2.0. Kết quả, 12.5, được gán cho biến average. Dưới đây là một ví dụ khác:

```
int x = 10, y = 15;
cout << "Tổng là " << sum(x, y) << endl;
```

Đoạn mã này gửi giá trị trả về của hàm sum đến cout để nó có thể được hiển thị trên màn hình. Thông báo "Tổng là 25" sẽ được hiển thị.

Một hàm trả về giá trị trả về một giá trị với một kiểu dữ liệu cụ thể. Bạn có thể sử dụng giá trị trả về của hàm ở bất kỳ chỗ nào như một giá trị thông thường của cùng kiểu dữ liệu đó.

Hãy xem một ví dụ khác. Chương trình 6-13, tính diện tích của một hình tròn, có hai hàm ngoài main. Một trong những hàm được đặt tên là square, và nó trả về bình phương của bất kỳ số nào được truyền vào như một đối số. Hàm square được gọi trong một câu lệnh toán học. Chương trình cũng có một hàm được đặt tên là getRadius, mà yêu cầu người dùng nhập bán kính của hình tròn. Giá trị được người dùng nhập vào được trả về từ hàm.

Chương trình 6.9. Hàm trả về giá trị

```
// Chương trình này minh họa hai hàm trả về giá trị.
// Hàm square được gọi trong một câu lệnh toán học.
#include <iostream>
#include <iomanip>
using namespace std;

// Nguyên mẫu hàm
double getRadius();
double square(double);

int main()
{
    const double PI = 3.14159; // Hằng số cho pi
    double banKinh; // Để lưu bán kính của hình tròn
    double dienTich; // Để lưu diện tích của hình tròn

    // Thiết lập định dạng đầu ra số
    cout << fixed << showpoint << setprecision(2);

    // Nhận bán kính của hình tròn.
    cout << "Chương trình này tính diện tích của ";
    cout << "một hình tròn.\n";
    banKinh = getRadius();

    // Tính diện tích của hình tròn.
    dienTich = PI * square(banKinh);

    // Hiển thị diện tích.
    cout << "Diện tích là " << dienTich << endl;
    return 0;
}

/**
 * Định nghĩa hàm getRadius. *
 * Hàm này yêu cầu người dùng nhập bán kính của *
 * hình tròn và sau đó trả lại số đó dưới dạng double.*
 */

double getRadius()
{
    double banKinh;
```

```

    cout << "Nhập bán kính của hình tròn: ";
    cin >> banKinh;
    return banKinh;
}

/**
// Định nghĩa hàm square. *
// Hàm này chấp nhận một đối số kiểu double và trả lại *
// bình phương của đối số đó dưới dạng double. *
//*/

double square(double so)
{
    return so * so;
}

```

Kết quả chương trình:

Chương trình này tính diện tích hình tròn.

Nhập bán kính hình tròn: 10 Enter

Diện tích là 314,16

Giả sử người dùng đã nhập 10 làm bán kính, và giá trị này được truyền vào hàm square như một đối số, hàm square sẽ trả về giá trị 100. Hình 6.7 minh họa cách giá trị 100 được truyền lại vào biểu thức toán học ở dòng lệnh sau:

```

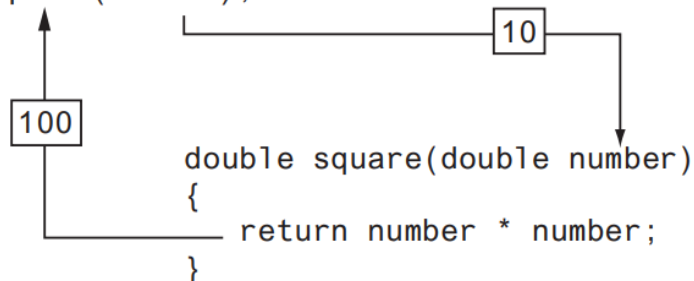
dienTich = PI * square(banKinh);

```

```

area = PI * square(radius);

```



Hình 6.7. Hàm là đối số của hàm khác

6.7. Biến cục bộ và toàn cục

KHÁI NIỆM

Biến toàn cục (Global Variable) là biến được khai báo bên ngoài tất cả các hàm và có phạm vi trên toàn bộ chương trình. Biến toàn cục có thể được truy cập và sử dụng bởi bất kỳ hàm nào trong chương trình.

Biến cục bộ (Local Variable) là biến được khai báo bên trong một hàm hoặc khối lệnh (block). Biến cục bộ chỉ có phạm vi trong hàm hoặc khối lệnh mà nó được khai báo, và không thể được truy cập từ bên ngoài phạm vi đó.

So sánh

Tiêu chí	Biến toàn cục	Biến cục bộ
Phạm vi truy cập	Toàn bộ chương trình	Cỉ trong hàm/Khối lệnh
Thời gian tồn tại	Tồn tại trong suốt thời gian chạy của chương trình	Chỉ tồn tại trong phạm vi hoạt động của hàm hoặc khối lệnh
Khởi tạo giá trị	Được khởi tạo tự động với giá trị mặc định (0 cho số nguyên, 0.0 cho số thực, false cho bool, '\0' cho ký tự)	Không được khởi tạo tự động và phải được khởi tạo giá trị trước khi sử dụng
Độ tin cậy	Kém tin cậy hơn biến cục bộ vì có thể bị sửa đổi bởi nhiều hàm khác nhau	Tin cậy hơn vì chỉ có thể được truy cập và sửa đổi trong phạm vi hạn hẹp

Ví dụ 1. Biến toàn cục

Chương trình 6.10. Minh họa biến toàn cục

```
//Biến toàn cục
#include <iostream>
using namespace std;

int globalVar = 10; // Biến toàn cục

void function1() {
    cout << "Trong function1, globalVar = " << globalVar << endl; // Truy cập biến toàn cục
}

void function2() {
    globalVar = 20; // Sửa đổi giá trị biến toàn cục
    cout << "Trong function2, globalVar = " << globalVar << endl;
}

int main() {
    cout << "Trong main, globalVar = " << globalVar << endl; // Truy cập biến toàn cục
    function1();
    function2();
    cout << "Trong main, globalVar = " << globalVar << endl; // Giá trị biến toàn cục đã bị thay đổi
}
```

```
    return 0;
}
```

Kết quả chương trình:

```
Trong main, globalVar = 10
Trong function1, globalVar = 10
Trong function2, globalVar = 20
Trong main, globalVar = 20
```

Ví dụ 2. Biến cục bộ

Chương trình 6.11. Minh họa biến cục bộ

```
// Biến cục bộ
#include <iostream>
using namespace std;

void function() {
    int localVar = 5; // Biến cục bộ
    cout << "Trong function, localVar = " << localVar << endl;
}

int main() {
    int localVar = 10; // Biến cục bộ khác
    cout << "Trong main, localVar = " << localVar << endl;
    function();
    // cout << localVar << endl; // Lỗi: Không thể truy cập biến cục bộ của
    // hàm function() từ main()

    return 0;
}
```

Kết quả chương trình:

```
Trong main, localVar = 10
Trong function, localVar = 5
```

Chú ý khi sử dụng

- *Sử dụng biến cục bộ nếu có thể:* Biến cục bộ được khuyến khích sử dụng nhiều hơn vì giúp đảm bảo tính rõ ràng và tin cậy của chương trình. Chúng chỉ có phạm vi hoạt động trong hàm hoặc khối lệnh, giảm thiểu khả năng xảy ra xung đột tên biến và giúp dễ dàng theo dõi và sửa đổi.
- *Hạn chế sử dụng biến toàn cục:* Biến toàn cục có thể dẫn đến các vấn đề về tính rõ ràng và tin cậy của chương trình. Chúng có thể bị sửa đổi bởi bất kỳ hàm nào trong chương trình, khiến cho việc theo dõi và sửa đổi trở nên khó khăn hơn.
- *Tránh sử dụng quá nhiều biến toàn cục:* Việc sử dụng quá nhiều biến toàn cục có thể làm tăng độ phức tạp của chương trình và khó khăn trong việc duy trì và mở rộng sau này.

- *Đặt tên biến rõ ràng*: Đặt tên biến rõ ràng và có ý nghĩa giúp dễ dàng hiểu và theo dõi chức năng của biến trong chương trình.
- *Khởi tạo biến cục bộ*: Luôn khởi tạo giá trị cho biến cục bộ trước khi sử dụng để tránh các lỗi không mong muốn.
- *Phạm vi biến cục bộ*: Chú ý đến phạm vi của biến cục bộ và tránh truy cập hoặc sử dụng biến cục bộ ngoài phạm vi của nó.

Bằng cách tuân thủ các nguyên tắc trên, bạn có thể sử dụng biến toàn cục và biến cục bộ một cách hiệu quả và giảm thiểu các lỗi trong chương trình C++.

6.8. Đối số mặc định

KHÁI NIỆM Đối số mặc định (Default Arguments) là một tính năng trong C++ cho phép bạn định nghĩa các giá trị mặc định cho các tham số của một hàm. Nếu không truyền giá trị cho tham số đó khi gọi hàm, giá trị mặc định sẽ được sử dụng.

Điều này giúp tăng tính linh hoạt cho hàm và làm cho việc gọi hàm trở nên đơn giản hơn trong một số trường hợp cụ thể.

Ví dụ, về đối số mặc định

Chương trình 6.12. Hàm với đối số mặc định

```
#include <iostream>
using namespace std;

// Hàm có đối số mặc định
void printMessage(string message = "Hello, World!") {
    cout << message << endl;
}

int multiply(int a, int b, int c = 1) {
    return a * b * c;
}

int main() {
    // Gọi hàm với đối số mặc định
    printMessage(); // Output: Hello, World!

    // Gọi hàm với đối số tùy chỉnh
    printMessage("Hello, C++!"); // Output: Hello, C++!

    // Gọi hàm multiply với đối số mặc định
    cout << multiply(2, 3) << endl; // Output: 6

    // Gọi hàm multiply với tất cả đối số
    cout << multiply(2, 3, 4) << endl; // Output: 24

    return 0;
}
```

Kết quả chương trình:

```
Hello, World!  
Hello, C++!  
6  
24
```

Giải thích:

- Hàm `printMessage` có một đối số `message` với giá trị mặc định là "Hello, World!". Khi gọi `printMessage()` mà không truyền đối số, nó sẽ in ra "Hello, World!".
- Hàm `multiply` có ba đối số, trong đó đối số `c` có giá trị mặc định là 1. Khi gọi `multiply(2, 3)`, nó sẽ nhân $2 * 3 * 1$ và trả về 6.

Chú ý

- *Đối số mặc định phải được đặt ở cuối danh sách tham số:* Nếu bạn có nhiều tham số có đối số mặc định, các tham số này phải được đặt ở cuối danh sách tham số.

```
// Đúng  
void myFunction(int a, int b = 2, int c = 3);  
// Sai  
void myFunction(int a = 1, int b, int c);
```

- *Đối số mặc định chỉ được định nghĩa một lần:* Bạn không thể định nghĩa đối số mặc định cho cùng một tham số trong nhiều khai báo hàm khác nhau.

```
// Đúng  
void myFunction(int a, int b = 2);  
// Sai  
void myFunction(int a, int b);  
void myFunction(int a, int b = 3); // Lỗi: Đối số mặc định đã được định nghĩa
```

- *Tránh sử dụng đối số mặc định cho các hàm có nhiều tham số:* Nếu một hàm có nhiều tham số, việc sử dụng đối số mặc định có thể làm cho việc gọi hàm trở nên khó hiểu và dễ gây nhầm lẫn.
- *Đối số mặc định được phân bổ khi biên dịch:* Giá trị mặc định của đối số được phân bổ trong quá trình biên dịch, do đó nó không thể là một biểu thức phức tạp hoặc một lời gọi hàm.
- *Kiểu dữ liệu của đối số mặc định phải tương thích:* Kiểu dữ liệu của giá trị mặc định phải tương thích với kiểu dữ liệu của tham số tương ứng trong hàm.

6.9. Sử dụng biến tham chiếu là tham số

KHÁI NIỆM Trong C++, khi bạn truyền một đối số vào một hàm, trình biên dịch sẽ tạo ra một bản sao của đối số đó trong bộ nhớ. Điều này có nghĩa là bất kỳ thay đổi nào đối

với giá trị của tham số trong hàm sẽ không ảnh hưởng đến giá trị của biến gốc bên ngoài hàm.

Tuy nhiên, nếu bạn muốn thay đổi giá trị của biến gốc từ bên trong hàm, bạn có thể sử dụng tham chiếu (reference) là tham số. Khi bạn truyền một tham chiếu vào hàm, bạn đang truyền địa chỉ bộ nhớ của biến gốc thay vì một bản sao của nó. Điều này cho phép bạn thay đổi giá trị của biến gốc từ bên trong hàm.

Ví dụ 1, phân biệt đối số là giá trị và đối số là tham chiếu

Chương trình 6.13. Hàm có đối số là tham chiếu

```
#include <iostream>
using namespace std;

// Hàm nhận đối số bằng giá trị
void incrementValue(int value) {
    value++;
    cout << "Giá trị trong hàm: " << value << endl;
}

// Hàm nhận đối số bằng tham chiếu
void incrementReference(int& ref) {
    ref++;
    cout << "Giá trị trong hàm: " << ref << endl;
}

int main() {
    int x = 5;
    cout << "Giá trị ban đầu của x: " << x << endl;

    incrementValue(x); // Truyền giá trị
    cout << "Giá trị của x sau khi gọi incrementValue: " << x << endl; // x
    vẫn là 5

    incrementReference(x); // Truyền tham chiếu
    cout << "Giá trị của x sau khi gọi incrementReference: " << x << endl;
    // x đã tăng lên 6

    return 0;
}
```

Kết quả chương trình:

```
Giá trị ban đầu của x: 5
Giá trị trong hàm: 6
Giá trị của x sau khi gọi incrementValue: 5
Giá trị trong hàm: 6
Giá trị của x sau khi gọi incrementReference: 6
```

Trong ví dụ trên, hàm incrementValue nhận đối số là giá trị, vì vậy, bất kỳ thay đổi nào đối với value bên trong hàm sẽ không ảnh hưởng đến giá trị của x bên ngoài hàm.

Ngược lại, hàm `incrementReference` nhận đối số là tham chiếu, vì vậy, bất kỳ thay đổi nào đối với `ref` bên trong hàm sẽ ảnh hưởng đến giá trị của `x` bên ngoài hàm.

Đây là một ví dụ khác về cách sử dụng tham chiếu làm tham số trong C++:

Ví dụ 2, hoán đổi giá trị của hai biến

Chương trình 6.14. Hàm hoán đổi giá trị

```
#include <iostream>
using namespace std;
void swap(int& a, int& b) {
    int temp = a;
    a = b;
    b = temp;
}

int main() {
    int x = 5, y = 10;
    cout << "Trước khi hoán đổi: x = " << x << ", y = " << y << endl;

    swap(x, y);

    cout << "Sau khi hoán đổi: x = " << x << ", y = " << y << endl;
    return 0;
}
```

Kết quả chương trình:

Trước khi hoán đổi: x = 5, y = 10
Sau khi hoán đổi: x = 10, y = 5

Trong ví dụ 2, hàm `swap` nhận hai tham chiếu `int&` đến các biến `a` và `b`. Bên trong hàm, nó hoán đổi giá trị của `a` và `b` bằng cách sử dụng một biến tạm thời `temp`. Nhờ sử dụng tham chiếu, hàm `swap` có thể thay đổi trực tiếp giá trị của `x` và `y` trong hàm `main`.

Chú ý: Khi sử dụng tham chiếu, bạn có quyền truy cập và thay đổi giá trị của biến gốc từ bên trong hàm. Do đó, cần phải cẩn thận để không vô tình thay đổi giá trị của biến gốc khi không cần thiết.

6.10. Một số vấn đề khác

Quá tải hàm:

KHÁI NIỆM: Quá tải hàm (Function Overloading) là khả năng cho phép định nghĩa nhiều hàm cùng tên nhưng khác nhau về số lượng hoặc kiểu dữ liệu của tham số. Trình biên dịch sẽ sử dụng kiểu và số lượng tham số để phân biệt và gọi đúng hàm tương ứng.

Ví dụ 1: Quá tải hàm cộng

Chương trình 6.15. Quá tải hàm

```
#include <iostream>
using namespace std;

// Hàm cộng hai số nguyên
int add(int a, int b) {
    return a + b;
}

// Hàm cộng hai số thực
double add(double a, double b) {
    return a + b;
}

int main() {
    int x = 5, y = 3;
    double m = 3.14, n = 2.72;

    cout << "Tong hai so nguyen: " << add(x, y) << endl; // Gọi hàm
    add(int, int)
    cout << "Tong hai so thuc: " << add(m, n) << endl; // Gọi hàm
    add(double, double)

    return 0;
}
```

Kết quả chương trình:

Tong hai so nguyen: 8
Tong hai so thuc: 5.86

Ví dụ 2: Quá tải hàm tính diện tích

Chương trình 6.16. Quá tải hàm tính diện tích

```
#include <iostream>
using namespace std;

// Hàm tính diện tích hình vuông
int area(int side) {
    return side * side;
}

// Hàm tính diện tích hình chữ nhật
int area(int length, int width) {
    return length * width;
}

int main() {
    int square_side = 5;
    int rect_length = 6, rect_width = 3;
```

```
cout << "Dien tich hinh vuong: " << area(square_side) << endl; // Gõi
hàm area(int)
cout << "Dien tich hinh chu nhat: " << area(rect_length, rect_width) <<
endl; // Gõi hàm area(int, int)

return 0;
}
```

Kết quả chương trình:

Dien tich hinh vuong: 25
Dien tich hinh chu nhat: 18

Chú ý

- Các hàm quá tải phải khác nhau về số lượng hoặc kiểu dữ liệu của tham số, không thể chỉ khác nhau về kiểu trả về.
- Nếu có nhiều hàm quá tải phù hợp, trình biên dịch sẽ gọi hàm có sự khớp tốt nhất (best match) dựa trên quy tắc chuyển đổi kiểu dữ liệu.
- Không thể quá tải hàm chỉ bằng cách thay đổi tên tham số, vì trình biên dịch chỉ nhìn vào kiểu dữ liệu của tham số, không phải tên.
- Quá tải hàm không liên quan đến kiểu trả về của hàm. Hai hàm có cùng tên, cùng số lượng và kiểu tham số, nhưng khác kiểu trả về sẽ bị coi là trùng lặp và gây ra lỗi.

Quá tải hàm giúp tăng tính linh hoạt và dễ đọc cho mã nguồn, đồng thời giảm sự cần thiết phải đặt tên khác nhau cho các hàm có chức năng tương tự nhưng hoạt động trên các kiểu dữ liệu khác nhau.

Hàm exit():

KHÁI NIỆM: Hàm exit() trong C++ là một hàm trong thư viện tiêu chuẩn <cstdlib> (hoặc <stdlib.h> trong C). Hàm này được sử dụng để kết thúc chương trình đang chạy một cách an toàn và trả về một giá trị thoát (exit status) cho hệ điều hành.

Cú pháp của hàm exit() là:

```
void exit(int exit_code);
```

Trong đó, exit_code là một giá trị nguyên dùng để chỉ ra trạng thái kết thúc của chương trình. Theo quy ước, giá trị 0 thường được sử dụng để chỉ ra rằng chương trình đã kết thúc một cách thành công, trong khi các giá trị khác được sử dụng để chỉ ra lỗi hoặc trạng thái kết thúc khác.

Ví dụ,

Chương trình 6.17. Minh họa sử dụng hàm exit

```
#include <iostream>
#include <cstdlib>
```

```
using namespace std;

int main() {
    int x, y;
    cout << "Nhap gia tri x: ";
    cin >> x;
    cout << "Nhap gia tri y: ";
    cin >> y;

    if (y == 0) {
        cout << "Khong the chia cho 0!" << endl;
        exit(1); // Kết thúc chương trình với mã lỗi 1
    }

    int result = x / y;
    cout << "Ket qua: " << result << endl;

    exit(0); // Kết thúc chương trình với mã thành công 0
}
```

Kết quả chương trình:

```
Nhap gia tri x: 4
Nhap gia tri y: 5
Ket qua: 0
```

Trong ví dụ trên, chương trình yêu cầu người dùng nhập hai giá trị x và y. Nếu y bằng 0, chương trình sẽ in ra thông báo lỗi và gọi `exit(1)` để kết thúc chương trình với mã lỗi 1. Nếu không có lỗi, chương trình sẽ tính và in ra kết quả của phép chia, sau đó gọi `exit(0)` để kết thúc chương trình với mã thành công 0.

Chú ý:

- Khi hàm `exit()` được gọi, các đối tượng tự động (automatic objects) trong phạm vi hiện tại sẽ không được hủy một cách thích hợp. Vì vậy, nên tránh sử dụng `exit()` trong các chương trình phức tạp có nhiều tài nguyên cần giải phóng.
- Nếu không có lệnh `exit()` nào được gọi trong chương trình, hệ điều hành sẽ tự động kết thúc chương trình khi kết thúc hàm `main()`.
- Trong một số trường hợp, như khi xử lý lỗi nghiêm trọng hoặc tình huống bất thường, việc sử dụng `exit()` là cần thiết để đảm bảo chương trình kết thúc một cách an toàn.
- Giá trị thoát (exit status) có thể được truy cập và sử dụng bởi các chương trình hoặc script khác sau khi chương trình kết thúc.

Tóm lại, hàm `exit()` là một cách để kết thúc chương trình một cách an toàn và trả về một giá trị thoát cho hệ điều hành. Tuy nhiên, nên sử dụng hàm này một cách thận trọng và chỉ trong các trường hợp cần thiết.

6.11. Câu hỏi và Bài tập

6.11.1. Câu hỏi

Cho biết các câu là ĐÚNG hay SAI?

1. Các hàm nên được đặt tên phản ánh mục đích của chúng.
2. Phần đầu của hàm được kết thúc bằng dấu chấm phẩy.
3. Một hàm có thể có nhiều hơn một câu lệnh return.
4. Nếu các hàm khác được định nghĩa trước main, chương trình vẫn bắt đầu thực thi tại hàm main.
5. Khi một hàm kết thúc, nó luôn chuyển đến main, bất kể nó được gọi từ đâu.
6. Các đối số được truyền cho các tham số của hàm theo thứ tự chúng xuất hiện trong lời gọi hàm.
7. Phạm vi của một tham số bị giới hạn trong hàm sử dụng nó.
8. Thay đổi tham số hàm luôn ảnh hưởng đến đối số gốc.
9. Trong một nguyên mẫu hàm, tên của các biến tham số có thể được bỏ qua.
10. Thời gian tồn tại của một biến cục bộ là trong suốt quá trình thực thi chương trình.
11. Một biến cục bộ không thể có cùng tên với một hằng toàn cục.
12. Các biến cục bộ tĩnh không bị hủy khi hàm trả về.
13. Tất cả các biến cục bộ tĩnh được khởi tạo mặc định là -1.
14. Khởi tạo các biến cục bộ tĩnh chỉ xảy ra một lần, bất kể hàm chứa chúng được gọi bao nhiêu lần.
15. Khi một hàm có đối số mặc định được gọi và một đối số bị bỏ qua, tất cả các đối số đến sau nó cũng phải bị bỏ qua.
16. Không thể có hàm với một số tham số có giá trị mặc định và một số không.
17. Hàm exit chỉ có thể được gọi từ main.

6.11.2. Bài tập

1. Giá trị mảng lớn nhất / nhỏ nhất

Viết chương trình cho phép người dùng nhập mười giá trị vào một mảng. Sau đó, chương trình sẽ hiển thị các giá trị lớn nhất và nhỏ nhất được lưu trữ trong mảng.

2. Thống kê lượng mưa (*Bắt buộc*)

Viết một chương trình cho phép người dùng nhập lượng mưa của mỗi tháng trong 12 tháng vào một mảng số **double**. Chương trình sẽ hiển thị tổng lượng mưa trong năm, lượng mưa trung bình hàng tháng và các tháng có lượng mưa cao nhất và thấp nhất.

Xác thực đầu vào: Không chấp nhận số âm cho số liệu lượng mưa hàng tháng.

3. Khoai tây chiên và Salsa (**Bắt buộc**)

Viết một chương trình cho phép nhà sản xuất khoai tây chiên và salsa theo dõi doanh số của năm loại salsa khác nhau: nhẹ, vừa, ngọt, nóng và vị chanh. Chương trình nên sử dụng cùng lúc hai mảng 5 phần tử: một mảng chuỗi chứa tên năm loại salsa và một mảng số nguyên chứa số lượng đã bán tháng qua cho mỗi loại salsa. Tên các loại salsa nên được lưu trữ bằng cách khởi tạo danh sách tại thời điểm tạo mảng tên. Chương trình yêu cầu người dùng nhập số lượng được bán cho từng loại. Khi dữ liệu bán hàng này đã được nhập, chương trình sẽ tạo ra một báo cáo hiển thị doanh số bán hàng cho từng loại salsa, tổng doanh số bán hàng và tên của các sản phẩm bán chạy nhất và bán chậm nhất.

Xác thực đầu vào: Không chấp nhận giá trị âm cho số lượng đã bán.

4. Lớn hơn n (**Bắt buộc**)

Trong chương trình, hãy viết một hàm chấp nhận ba đối số: một mảng, kích thước của mảng và một số n . Giả sử mảng chứa các số nguyên. Hàm cần hiển thị tất cả các số trong mảng lớn hơn số n .

5. Kinh doanh Khỉ

Một vườn thú địa phương muốn theo dõi xem mỗi con trong ba con khỉ của họ ăn bao nhiêu pound thức ăn mỗi ngày trong một tuần. Viết chương trình lưu trữ thông tin này trong một mảng hai chiều cỡ 3×5 , trong đó mỗi hàng đại diện cho một con khỉ và mỗi cột đại diện cho một ngày trong tuần. Trước tiên, chương trình cần để người dùng nhập dữ liệu cho mỗi con khỉ. Sau đó, nó sẽ hiển thị báo cáo gồm các thông tin sau:

- Lượng thức ăn trung bình mỗi ngày của cả gia đình khỉ.
- Lượng ăn ít nhất xét trên cả tuần và mọi con khỉ
- Lượng ăn nhiều nhất xét trên cả tuần và mọi con khỉ
- Xác thực đầu vào: Không chấp nhận số âm cho số pound thực phẩm đã ăn.

6. Mưa hay Nắng (**Bắt buộc**)

Một nhà khí tượng nghiệp dư cần thu thập thông tin thời tiết trong suốt ba tháng mùa hè năm vừa rồi; nên đã ghi chú một ngày là mưa ('R'), có mây ('C') hoặc nắng ('S'). Viết chương trình lưu trữ thông tin này dưới dạng mảng ký tự cỡ 3×30 , trong đó hàng để chỉ các tháng mùa hè (quy ước, 0 = tháng 6, 1 = tháng 7, 2 = tháng 8) và cột để chỉ các ngày trong tháng. Không cần ghi chép dữ liệu cho ngày 31 của bất kỳ tháng nào. Chương trình sẽ bắt đầu bằng cách đọc dữ liệu thời tiết từ một tệp. Sau đó, cho mỗi tháng và cho cả ba

tháng, chương trình sẽ hiển thị có bao nhiêu ngày mưa, ngày mây, và ngày nắng. Nó cũng sẽ báo cáo tháng nào trong số ba tháng có số ngày mưa lớn nhất. Dữ liệu cho chương trình cần lưu trong tệp RainOrShine.txt.

Gợi ý: Tệp RainOrShine.txt có cấu trúc như sau:

```
R C R A R S R R S R S C R C S S S S R S C A R C S S A C R S
C R R A S A S R C S R C R S S S R S C R R S C C S C S C R R
C R C C S C C R R R S A A C R C C C R C R S S S R S C A R A
```

7. Chương trình phân tích số

Viết chương trình yêu cầu người dùng nhập tên tệp. Giả sử tệp chứa một chuỗi số, mỗi số được viết trên một dòng. Chương trình sẽ đọc nội dung của tệp thành một mảng sau đó hiển thị các thông tin sau:

- Số thấp nhất trong mảng
- Số cao nhất trong mảng
- Tổng các số trong mảng
- Giá trị trung bình của các số trong mảng

Nếu bạn đã tải xuống mã nguồn của cuốn sách này, bạn sẽ tìm thấy một file tên number.txt trong thư mục Chương 07. Bạn có thể sử dụng file đó để kiểm tra chương trình.

8. Lo Shu Magic Square (Hình vuông kỳ ảo Lạc Thư)

Lo Shu Magic Square là một lưới có 3 hàng và 3 cột được thể hiện trong Hình 7-19. Lo Shu Magic Square có các thuộc tính sau:

4	9	2
3	5	7
8	1	6

Figure 7-19 Lo Shu Magic Square

Lưới chứa chính xác là các số từ 1 đến 9.

Tổng của mỗi hàng, mỗi cột và mỗi đường chéo đều cộng lại ra bằng nhau - được minh họa trong Hình 7-20

4	9	2	↗ 15
3	5	7	→ 15
8	1	6	→ 15
↓ 15	↓ 15	↓ 15	↘ 15

Figure 7-20 Sums of the rows, columns, and diagonals

Trong chương trình, bạn có thể mô phỏng hình vuông ma thuật trên bằng cách sử dụng một mảng hai chiều. Viết một hàm chấp nhận một mảng hai chiều làm đối số và xác định xem mảng đó có phải là một Lo Shu Magic Square hay không. Kiểm tra chức năng hàm này trong chương trình.

9. Tính lương (*Bắt buộc*)

Viết chương trình dùng các mảng sau:

empId: một mảng gồm bảy số nguyên dài, dùng để chứa số căn cước của các nhân viên. Mảng cần được khởi tạo với các số sau:

```
5658845  4520125  7895122  8777541
8451277  1302850  7580489
```

- **hours:** một mảng bảy số nguyên để chứa số giờ làm việc của mỗi nhân viên
- **payRate:** một mảng gồm bảy số double để lưu tỷ lệ lương theo giờ của mỗi nhân viên
- **wages:** một mảng bảy số double để giữ tổng tiền lương của mỗi nhân viên

Chương trình truy cập dữ liệu trong mỗi mảng một cách đồng bộ qua chỉ số mảng. Ví dụ: phần tử 0 của mảng **hours** sẽ lưu số giờ làm của nhân viên có căn cước lưu tại phần tử 0 của mảng **empId**. Tương ứng, tỷ lệ trả lương của nhân viên này được lưu trong phần tử 0 của mảng **payRate**.

Chương trình sẽ hiển thị số căn cước của từng nhân viên và yêu cầu người dùng nhập số giờ làm và mức lương (tỷ lệ lương theo giờ) của nhân viên đó. Sau đó, nó sẽ tính toán tổng tiền lương cho nhân viên đó (tỷ lệ lương theo giờ nhân với số giờ) và lưu trong mảng

wages. Sau khi dữ liệu được nhập cho tất cả nhân viên, chương trình sẽ hiển thị số căn cước từng nhân viên kèm tổng tiền lương của người đó.

Xác thực đầu vào: Không chấp nhận các giá trị âm cho số giờ, hoặc các số nhỏ hơn 15,00 đối với tỷ lệ lương theo giờ.

10. Kỳ thi lấy Bằng lái xe

Văn phòng Giấy phép Lái xe địa phương yêu cầu bạn viết một chương trình chấm điểm phần viết của một người thi lấy bằng lái xe. Đề thi có 20 câu hỏi trắc nghiệm, với đây là các đáp án:

1. A	6. B	11. A	16. C
2. D	7. A	12. C	17. C
3. B	8. B	13. D	18. A
4. B	9. C	14. B	19. D
5. C	10. D	15. D	20. B

Chương trình cần lưu trữ các đáp án trên trong một mảng. Chương trình sẽ yêu cầu nhập các câu trả lời của người thi cho 20 câu hỏi, và các câu trả lời này phải được lưu trong một mảng khác. Sau khi câu trả lời được nhập, chương trình sẽ hiển thị thông báo cho biết người thi đã vượt qua hay không trong kỳ thi (cần đúng ít nhất 15 trong số 20 câu để thi đỗ). Sau đó, chương trình cũng sẽ hiển thị tổng số câu trả lời đúng, tổng số câu trả lời sai và một danh sách số thứ tự các câu bị sai.

Xác thực đầu vào: Chỉ chấp nhận các chữ cái A, B, C hoặc D làm câu trả lời.

11. Phần mềm chấm thi (Bắt buộc)

Một người thầy đặt hàng bạn viết một chương trình chấm điểm thi cuối kỳ, với bài thi này gồm 20 câu hỏi trắc nghiệm và mỗi câu hỏi chỉ có bốn lựa chọn đáp án A, B, C hoặc D.

Tệp CorrectAnswers.txt sẽ chứa trả lời đáp án cho tất cả các câu, với mỗi đáp án được viết trên một dòng riêng biệt, dòng đầu tiên chứa đáp án cho câu đầu tiên, dòng thứ hai chứa đáp án cho câu thứ hai, v.v. (Tải xuống mã nguồn của cuốn sách từ Cổng thông tin Khoa học Máy tính tại www.pearsonhighering.com/gaddis. Bạn sẽ tìm thấy tệp trong thư mục Chương 07.) Viết chương trình đọc nội dung của tệp CorrectAnswers.txt vào một mảng *char*, sau đó đọc nội dung một tệp khác, chứa các câu trả lời của sinh viên, vào một mảng *char* thứ hai. (Bạn có thể sử dụng tệp StudentAnswers.txt cho mục đích kiểm tra. Tệp này cũng nằm trong thư mục mã nguồn Chương 07.) Chương trình sẽ xác định số câu hỏi mà sinh viên bị sai, sau đó hiển thị như sau:

- Danh sách các câu bị sai, và với mỗi câu này, hiển thị câu đáp án và câu trả lời sai của sinh viên.
- Tổng số câu trả lời sai

- Tỷ lệ phần trăm câu trả lời đúng, có thể được tính như sau:
- Số câu trả lời đúng / Tổng số câu hỏi
- Nếu tỷ lệ phần trăm trả lời đúng là 70 phần trăm trở lên, chương trình cần thông báo học sinh đã vượt qua kỳ thi. Nếu không, thông báo học sinh đã trượt kỳ thi.

Nội dung 2 tệp *CorrectAnswers.txt* & *StudentAnswers.txt* như sau:

Tệp CorrectAnswers.txt	Tệp StudentAnswers.txt
A	A
D	D
B	B
C	C
A	B
A	A
D	C
B	B
D	D
C	C
A	A
D	D
B	B
A	A
C	C
C	A
D	D
B	B
C	C
A	A

12. Sở điểm

Một giáo viên có năm học sinh và vừa làm bốn bài kiểm tra. Với một học sinh, dựa trên điểm trung bình từ bốn bài kiểm tra, giáo viên sẽ sử dụng thang điểm sau để chấm điểm dạng chữ cái:

Test Score	Letter Grade
90–100	A
80–89	B
70–79	C
60–69	D
0–59	F

Viết một chương trình sử dụng một mảng các đối tượng **string** để chứa tên năm học sinh, một mảng năm ký tự để chứa điểm chữ cái của năm học sinh và năm mảng chứa bốn số **double** ứng với tập hợp điểm kiểm tra của năm học sinh. Chương trình sẽ cho phép người dùng nhập tên của từng học sinh và lần lượt bốn điểm kiểm tra của họ. Sau đó, nó sẽ tính toán và hiển thị điểm kiểm tra trung bình và điểm chữ cái tương ứng của học sinh đó.

Xác nhận đầu vào: Không chấp nhận điểm kiểm tra nhỏ hơn 0 hoặc lớn hơn 100.

13. Sổ điểm nâng cấp

Sửa đổi ứng dụng sổ điểm trong bài trên thành tự động loại bỏ điểm thấp nhất khi đang tính điểm trung bình kiểm tra và điểm chữ cái (Vd sinh viên có 4 điểm 9.5; 7; 8.2; 3 thì chỉ tính điểm trung bình từ 9.5; 7; 8.2)

14. Ứng dụng xổ số

Viết chương trình mô phỏng một cuộc thi xổ số. Chương trình cần dùng một mảng gồm năm số nguyên được đặt tên là **lottery** và cần tự sinh ra một số ngẫu nhiên trong khoảng từ 0 đến 9 cho mỗi phần tử trong mảng này. Người dùng cũng phải nhập năm chữ số, số này sẽ được lưu trữ trong một mảng số nguyên khác có tên là **user**. Chương trình sẽ so sánh các phần tử tương ứng trong hai mảng và lưu số lượng số so khớp. Ví dụ: dưới đây là mảng **lottery** và mảng **user** với các các số mẫu cho trước. Ta thấy, ở đây có hai chữ số so khớp (phần tử 2 và 4).

– Mảng lottery:

Lottery array:

7	4	9	1	3
---	---	---	---	---

– Mảng user:

User array:

4	2	9	7	3
---	---	---	---	---

Chương trình cần hiển thị các số ngẫu nhiên đã được tạo ra và trong mảng lottery, cùng số chữ số so khớp. Nếu tất cả các chữ số đều khớp, hiển thị thông báo người đã thắng giải độc đắc.

15. Sửa đổi vector

Sửa đổi ví dụ về Ngân hàng Thương mại Quốc gia (National Commerce Bank) được trình bày trong Chương trình 7-23 để `pin2` và `pin3` là **vector** thay vì mảng. Ta cũng cần sửa đổi hàm `testPIN` để nhận đối số là một **vector** thay vì một mảng.

16. Nhà vô địch giải thể giới

Nếu bạn đã tải xuống mã nguồn của cuốn sách này, bạn sẽ tìm thấy các tệp sau trong thư mục của chương này:

`Teams.txt` - Tệp này chứa danh sách xếp theo thứ tự chữ cái của các đội bóng chày thuộc nhóm Những ông lớn (Major Leagues). Mỗi đội trong nhóm này đều đã vô địch Giải thể giới (World Series) ít nhất một lần.

`WorldSeriesWinners.txt` - Tệp này chứa danh sách xếp theo thứ tự thời gian của các đội chiến thắng Giải thể giới từ năm 1903 đến năm 2012. (Dòng đầu tiên trong tệp là tên của đội đã vô địch năm 1903 và dòng cuối cùng là tên của đội chiến thắng năm 2012. Lưu ý rằng World Series không tổ chức vào các năm 1904 và 1994.)

Viết chương trình hiển thị nội dung của tệp `Teams.txt` trên màn hình và nhắc người dùng nhập tên của một trong các nhóm. Sau đó, chương trình sẽ hiển thị số lần mà đội đó đã vô địch World Series trong khoảng thời gian từ 1903 đến 2012.

Gợi ý:

Đọc nội dung của tệp `WorldSeriesWinners.txt` thành một mảng hoặc **vector**. Sau khi người dùng nhập tên của một nhóm, chương trình sẽ duyệt qua mảng hoặc **vector** và đếm số lần nhóm này xuất hiện.

17. Tìm kiếm tên

Nếu bạn đã tải xuống mã nguồn của cuốn sách này, bạn sẽ tìm thấy các tệp sau trong thư mục của chương này:

- `GirlNames.txt` - Tệp này chứa danh sách 200 tên phổ biến nhất cho các bé gái sinh ra ở Mỹ từ năm 2000 đến năm 2009.
- `BoyNames.txt` - Tệp này chứa danh sách 200 tên phổ biến nhất cho các bé trai sinh ra ở Mỹ từ năm 2000 đến năm 2009.

Viết chương trình đọc nội dung của hai tệp trên vào hai mảng hoặc **vector** riêng biệt. Người dùng có thể nhập tên con trai, tên con gái hoặc cả hai và ứng dụng sẽ hiển thị thông báo cho biết liệu những tên đó có nằm trong số những tên phổ biến nhất hay không.

18. Trò chơi Tic-Tac-Toe

Viết một chương trình cho phép hai người chơi cùng chơi trò tic-tac-toe. Sử dụng mảng ký tự hai chiều với ba hàng và ba cột làm bảng trò chơi. Mỗi phần tử của mảng phải được khởi tạo bằng dấu hoa thị (*). Chương trình sẽ chạy vòng lặp sau:

- Hiển thị nội dung của mảng biểu diễn trò chơi
- Cho phép người chơi 1 chọn một vị trí trên bảng để điền X. Chương trình yêu cầu người dùng nhập số hàng và cột.
- Cho phép người chơi 2 chọn một vị trí trên bàn cờ để điền O. Chương trình sẽ yêu cầu người dùng nhập số hàng và cột.
- Xác định xem liệu một người chơi đã thắng hay trò chơi đã hoà. Nếu một người chơi đã thắng, chương trình sẽ tuyên bố người đó là người chiến thắng và kết thúc. Nếu tình huống hòa đã xảy ra, chương trình cũng hiển thị một thông báo tương ứng rồi kết thúc.

Người chơi 1 thắng khi có ba chữ X liên tiếp trên bảng trò chơi. Các chữ X này có thể xuất hiện trong một hàng, trong một cột hoặc theo đường chéo trên bảng. Người chơi 2 thắng khi có ba chữ O liên tiếp trên bảng trò chơi. Các chữ O này có thể xuất hiện trong một hàng, trong một cột hoặc theo đường chéo trên bảng. Tình huống hòa xảy ra khi tất cả các vị trí trên bàn cờ đã đầy đủ, nhưng không có ai chiến thắng.

19. Magic 8 Ball

Viết chương trình mô phỏng Magic 8 Ball, là một đồ chơi bói toán hiển thị một câu trả lời ngẫu nhiên cho một câu hỏi có/không. Trong các chương trình mẫu dành cho sinh viên cho cuốn sách này, bạn sẽ tìm thấy một tệp tên `8_ball_responses.txt`. Tệp chứa 12 câu trả lời, chẳng hạn như “Tôi không nghĩ vậy”, “Tất nhiên là có!”, “Tôi không chắc”, v.v. Chương trình sẽ đọc các phản hồi từ tệp thành một mảng hoặc **vector**. Nó sẽ nhắc người dùng đặt câu hỏi, sau đó hiển thị một trong các câu trả lời, được chọn ngẫu nhiên từ mảng hoặc **vector** này. Chương trình tiếp tục lặp lại việc hỏi đáp này cho đến khi người dùng muốn thoát.

Contents of 8_ball_responses.txt:

```
Yes, of course!
Without a doubt, yes.
You can count on it.
For sure!
Ask me later.
I'm not sure.
I can't tell you right now.
I'll tell you after my nap.
No way!
I don't think so.
Without a doubt, no.
The answer is clearly NO.
```

20. Giá gas dạo 1994

Trong các chương trình mẫu dành cho sinh viên cho cuốn sách này, bạn sẽ tìm thấy một tệp tên là `1994_Weekly_Gas_A Average.txt`. Tệp này chứa giá gas trung bình cho mỗi tuần

trong năm 1994 (Có 52 dòng trong tệp. Dòng 1 chứa giá gas trung bình vào tuần 1, dòng 2 chứa giá trung bình cho tuần 2, v.v.) Viết a chương trình đọc giá gas từ tệp vào một mảng hoặc một vector. Chương trình sẽ xử lý tiếp như sau:

- Hiển thị giá trung bình thấp nhất trong năm, cùng với số của tuần và tên của tháng xảy ra giá đó.
- Hiển thị giá trung bình cao nhất trong năm, cùng với số của tuần và tên của tháng xảy ra giá đó.
- Hiển thị giá gas trung bình cho mỗi tháng. (Để biết giá trung bình cho một tháng nhất định, hãy tính trung bình của giá trung bình các tuần của tháng đó.)

21. Thao tác trên mảng hai chiều (mảng 2D)

Viết chương trình sinh mảng hai chiều được khởi tạo với dữ liệu thử nghiệm. Sử dụng bất kỳ kiểu dữ liệu nào bạn muốn. Chương trình cần có các chức năng sau:

- getTotal - Hàm này nhận một mảng hai chiều làm đối số và trả về tổng các giá trị trong mảng.
- getAverage - Hàm này nhận một mảng hai chiều làm đối số và trả về trung bình của các giá trị trong mảng.
- getRowTotal - Hàm này nhận một mảng hai chiều làm đối số đầu tiên và một số nguyên làm đối số thứ hai. Đối số thứ hai ứng với chỉ số con của một hàng trong mảng. Hàm sẽ trả về tổng các giá trị trong hàng đó.
- getColumnTotal - Hàm này nhận một mảng hai chiều làm đối số đầu tiên và một số nguyên làm đối số thứ hai. Đối số thứ hai ứng với chỉ số con của một cột trong mảng. Hàm sẽ trả về tổng giá trị trong cột đó.
- getHighestInRow - Hàm này nhận một mảng hai chiều làm đối số đầu tiên và một số nguyên làm đối số thứ hai. Đối số thứ hai ứng với chỉ số con của một hàng trong mảng. Hàm sẽ trả về giá trị cao nhất trong hàng đó.
- getLowestInRow - Hàm này nhận một mảng hai chiều làm đối số đầu tiên và một số nguyên làm đối số thứ hai. Đối số thứ hai ứng với chỉ số con của một hàng trong mảng. Hàm sẽ trả về giá trị thấp nhất trong hàng đó.

Hãy demo từng chức năng trong chương trình này.

BÀI TẬP NHÓM

22. Chỗ ngồi trong nhà hát

Chương trình này nên được thiết kế và cài đặt bởi một nhóm sinh viên. Đây là một vài *Gợi ý*:

- Một sinh viên nên thiết kế hàm main, hàm này sẽ gọi các hàm khác trong chương trình. Các chức năng còn lại của sẽ được thiết kế bởi các thành viên khác trong nhóm.

- Các yêu cầu của chương trình cần được phân tích để mỗi sinh viên được giao cùng một khối lượng công việc.
- Các tham số và kiểu trả về của mỗi hàm nên được quyết định trước.
- Chương trình có thể được thực hiện như một chương trình nhiều tệp, hoặc tất cả các chức năng có thể được cắt và dán vào tệp chính.

Đây là nhiệm vụ:

Viết một chương trình có thể được dùng bởi một nhà hát nhỏ để bán vé cho các buổi biểu diễn. Khán phòng của nhà hát có 15 hàng ghế, mỗi hàng có 30 ghế. Chương trình sẽ hiển thị một màn hình hiển thị chỗ nào còn trống và chỗ nào đã mua vé. Ví dụ, màn hình sau đây hiển thị một biểu đồ mô tả từng chỗ ngồi trong rạp hát. Các ghế đã sử dụng được biểu thị bằng ký hiệu * và các ghế có sẵn được biểu thị bằng ký hiệu #.

	Seats																													
	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0
Row 1	*	*	*	#	#	#	*	*	*	#	#	#	*	*	*	#	#	#	*	*	*	#	#	#	*	*	*	#	#	#
Row 2	#	#	#	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	#	#	#	*	*	*	*	*	*
Row 3	*	*	#	#	#	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	#	#	#	*	*	*	*	*	*	*
Row 4	*	*	#	#	#	#	*	*	*	*	*	*	*	*	*	*	*	*	*	*	#	#	*	*	*	*	*	*	*	*
Row 5	*	*	*	*	*	*	#	#	#	#	*	*	*	*	*	*	*	*	*	*	#	#	#	#	*	*	*	*	*	*
Row 6	#	#	#	#	#	#	#	#	#	#	*	*	*	*	*	*	*	*	*	*	#	#	#	*	*	*	*	*	*	*
Row 7	#	#	#	#	#	#	*	*	*	*	*	*	*	*	*	*	*	*	*	*	#	#	#	#	#	*	*	*	*	*
Row 8	*	*	*	*	*	*	*	#	#	*	*	*	*	*	*	*	*	*	*	*	#	#	#	#	#	*	*	*	*	*
Row 9	#	#	#	#	#	#	*	*	*	*	*	*	*	*	*	*	*	*	*	*	#	#	#	#	#	*	*	*	*	*
Row 10	#	#	#	#	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	#	#	#	#	#	*	*	*	*	*
Row 11	#	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	#	#	#	#	#	*	*	*	*	*
Row 12	#	#	#	#	#	#	#	#	#	#	*	*	*	*	*	*	*	*	*	*	#	#	#	#	#	*	*	*	*	*
Row 13	#	#	#	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	#	#	#	#	#	*	*	*	*	*
Row 14	#	#	#	#	#	#	#	#	#	#	#	#	#	#	#	#	#	#	#	#	#	#	#	#	#	#	#	#	#	
Row 15	#	#	#	#	#	#	#	#	#	#	#	#	#	#	#	#	#	#	#	#	#	#	#	#	#	#	#	#	#	

Đây là danh sách các tác vụ mà chương trình cần thực hiện:

- Khi chương trình bắt đầu, nó sẽ yêu cầu người dùng nhập giá chỗ ngồi cho mỗi hàng. Giá có thể được lưu trữ trong một mảng riêng biệt. (Ngoài ra, giá có thể được đọc từ một tệp.)
- Sau khi giá được nhập, chương trình sẽ hiển thị sơ đồ chỗ ngồi tương tự như biểu đồ được hiển thị ở trên. Người dùng có thể nhập số hàng và số ghế cho vé đang bán. Sau khi mua vé hoặc nhóm vé, chương trình sẽ hiển thị tổng tiền vé vừa bán và cập nhật lại sơ đồ chỗ ngồi.
- Chương trình nên lưu trữ tổng số tiền bán vé trên cả rạp. Người dùng cần được cung cấp một tùy chọn để xem số tiền này.

- Chương trình cũng sẽ cung cấp cho người dùng một tùy chọn để xem danh sách các thông tin sau: bao nhiêu ghế đã được bán, bao nhiêu ghế còn trống trong từng hàng, và bao nhiêu ghế còn trống trong toàn bộ khán phòng.

Xác thực đầu vào: Khi vé đang được bán, không chấp nhận số hàng hoặc số ghế không tồn tại. Khi ai đó yêu cầu một chỗ ngồi cụ thể, chương trình cũng phải đảm bảo rằng chỗ ngồi đó đang trống trước khi nó được bán.

CHƯƠNG 7. MẢNG VÀ THUẬT TOÁN SẮP XẾP

Chủ đề	
7.1	Khái niệm mảng
7.2	Truy cập các phần tử
7.3	Phạm vi mảng
7.4	Mảng song song
7.5	Mảng là tham số của hàm
7.6	Mảng 2 chiều
7.7	Mảng nhiều chiều
7.8	Thuật toán tìm kiếm
7.9	Thuật toán sắp xếp

7.1. Khái niệm mảng

KHÁI NIỆM Trong lập trình, một mảng (array) là một cấu trúc dữ liệu cho phép lưu trữ một tập hợp các phần tử cùng kiểu dữ liệu. Mỗi phần tử trong mảng được đánh số thứ tự bắt đầu từ 0, gọi là chỉ số (index). Chỉ số này giúp ta có thể truy cập và thao tác với từng phần tử trong mảng.

Khai báo

kiểu_dữ_liệu tên_mảng[kích_thước];

Trong đó:

- *kiểu_dữ_liệu* là kiểu dữ liệu của các phần tử trong mảng (như int, float, char, double,...)
- *tên_mảng* là tên của biến mảng
- *kích_thước* là số lượng phần tử tối đa mà mảng có thể chứa

Ví dụ:

```
int arr[5]; // Khai báo mảng số nguyên có 5 phần tử
char name[20]; // Khai báo mảng ký tự có 20 phần tử
```

Bộ nhớ và mảng

Khi khai báo mảng, máy tính sẽ cấp phát một miền bộ nhớ liên tục để lưu trữ các phần tử của mảng. Mỗi phần tử trong mảng sẽ chiếm một vùng nhớ nhất định tùy thuộc vào kiểu dữ liệu của nó.

Các phần tử trong mảng được lưu trữ liên tục trong bộ nhớ, vì vậy ta có thể truy cập chúng thông qua chỉ số và địa chỉ của phần tử đầu tiên.

Giả sử chúng ta khai báo một mảng số nguyên như sau:

```
int arr[5] = {10, 20, 30, 40, 50};
```

Bộ nhớ sẽ được cấp phát và các phần tử trong mảng sẽ được lưu trữ như sau:

10	20	30	40	50
arr[0]	arr[1]	arr[2]	arr[3]	arr[4]

Trong đó:

- arr[0] lưu trữ giá trị 10 tại một vùng nhớ nào đó
- arr[1] lưu trữ giá trị 20 tại vùng nhớ kế tiếp
- arr[2] lưu trữ giá trị 30 tại vùng nhớ kế tiếp
- ...

Có thể truy cập và thay đổi giá trị của các phần tử trong mảng thông qua chỉ số tương ứng.

7.2. Truy cập các phần tử mảng

KHÁI NIỆM Trong C++, sử dụng chỉ số (index) để truy cập và thao tác với các phần tử trong mảng. Chỉ số của mảng bắt đầu từ 0.

Chú ý

- Khi truy cập phần tử mảng, chỉ số phải nằm trong khoảng từ 0 đến (kích thước mảng - 1). Nếu chỉ số vượt quá giới hạn này, chương trình sẽ gây ra lỗi.
- Mảng là kiểu dữ liệu tham chiếu, do đó khi gán một mảng cho một biến mảng khác, thực chất là gán địa chỉ của mảng đầu tiên cho biến mảng thứ hai.

Nhập và xuất mảng

Để nhập dữ liệu vào mảng, chúng ta có thể sử dụng vòng lặp và hàm cin để nhận giá trị từ người dùng cho từng phần tử của mảng.

Để xuất dữ liệu của mảng, chúng ta cũng có thể sử dụng vòng lặp và hàm cout để in ra từng phần tử của mảng.

Ví dụ: nhập và xuất mảng

Chương trình 7.1. Nhập và xuất mảng

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Nhap so luong phan tu cua mang: ";
    cin >> n;

    int arr[n];

    // Nhap du lieu vao mang
    for (int i = 0; i < n; i++) {
        cout << "Nhap phan tu thu " << i << ": ";
```

```

        cin >> arr[i];
    }

    // Xuất dữ liệu của mảng
    cout << "Mảng vừa nhập là: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }

    return 0;
}

```

Kết quả chương trình:

```

Nhập số lượng phần tử của mảng: 3
Nhập phần tử thứ 0: 1
Nhập phần tử thứ 1: 2
Nhập phần tử thứ 2: 3
Mảng vừa nhập là: 1 2 3

```

Khởi tạo mảng

Có thể khởi tạo mảng với các giá trị ban đầu khi khai báo mảng.

Ví dụ: khởi tạo mảng

```
int arr[] = {10, 20, 30, 40, 50}; // Khởi tạo mảng với các giá trị ban đầu
```

Nếu không chỉ định kích thước cho mảng, trình biên dịch sẽ tự động xác định kích thước mảng dựa trên số lượng phần tử được khởi tạo.

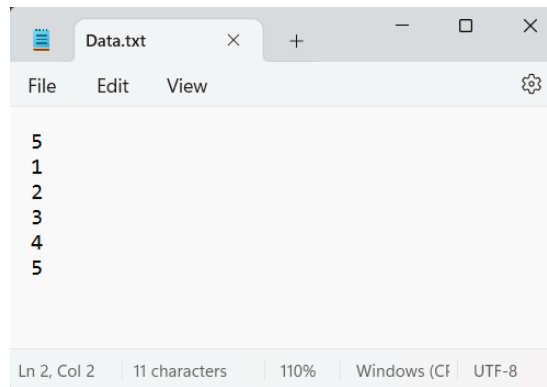
Khởi tạo mảng 1 phần

Trong trường hợp chỉ khởi tạo một số phần tử đầu tiên của mảng, phần còn lại sẽ được khởi tạo với giá trị mặc định tương ứng với kiểu dữ liệu của mảng.

Ví dụ: khởi tạo mảng 1 phần

```
int arr[5] = {10, 20}; // arr[0] = 10, arr[1] = 20, arr[2] = 0, arr[3] = 0, arr[4] = 0
```

Ví dụ: Giả sử có tệp Data.txt như hình dưới gồm: phần tử đầu tiên ghi số lượng phần tử, tiếp theo là các phần tử. Viết chương trình đọc dữ liệu từ tệp vào mảng



Chương trình 7.2. Đọc dữ liệu từ tệp vào mảng

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    int n;
    ifstream file("data.txt"); // Mở tệp đọc
    file >> n; // Đọc kích thước mảng từ tệp

    int arr[n];

    // Đọc dữ liệu từ tệp vào mảng
    for (int i = 0; i < n; i++) {
        file >> arr[i];
    }

    file.close(); // Đóng tệp

    // Xuất dữ liệu của mảng
    cout << "Mang doc tu tep: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }

    return 0;
}
```

Kết quả chương trình:

Mang doc tu tep: 1 2 3 4 5

Trong ví dụ trên, đọc kích thước của mảng từ tệp data.txt, sau đó đọc dữ liệu từ tệp vào mảng.

Ví dụ: ghi nội dung mảng vào tệp

Chương trình 7.3. Ghi mảng vào tệp

```
#include <iostream>
#include <fstream>
using namespace std;
```

```

int main() {
    int n;
    cout << "Nhap so luong phan tu cua mang: ";
    cin >> n;

    int arr[n];

    // Nhap du lieu vao mang
    for (int i = 0; i < n; i++) {
        cout << "Nhap phan tu thu " << i << ": ";
        cin >> arr[i];
    }

    ofstream file("Data1.txt"); // Mở tệp ghi
    file << n << endl; // Ghi kích thước mảng vào tệp

    // Ghi dữ liệu của mảng vào tệp
    for (int i = 0; i < n; i++) {
        file << arr[i] << " ";
    }

    file.close(); // Đóng tệp
    cout<<"Da ghi xong!";
    return 0;
}

```

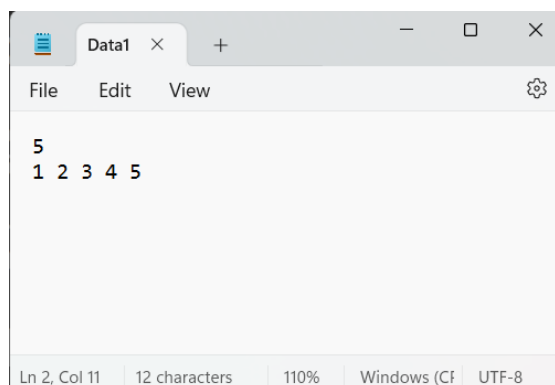
Kết quả chương trình:

```

Nhap so luong phan tu cua mang: 5
Nhap phan tu thu 0: 1
Nhap phan tu thu 1: 2
Nhap phan tu thu 2: 3
Nhap phan tu thu 3: 4
Nhap phan tu thu 4: 5
Da ghi xong!

```

Trong ví dụ trên, nhập dữ liệu vào mảng từ người dùng, sau đó ghi kích thước và nội dung của mảng vào tệp Data1.txt như hình dưới.



Hình 7.1. Nội dung tệp Data1.txt

7.3. Phạm vi mảng dựa trên vòng lặp

Mỗi lần vòng lặp dựa trên phạm vi lặp lại, nó sao chép một phần tử mảng vào biến phạm vi. Ví dụ, lần đầu tiên vòng lặp lặp lại, biến phạm vi sẽ chứa giá trị của phần tử 0, lần thứ hai vòng lặp lặp lại, biến phạm vi sẽ chứa giá trị của phần tử 1, và cứ tiếp tục như vậy.

Cú pháp:

```
for (dataType rangeVariable : array)
    statement;
```

Trong đó:

- *dataType* là kiểu dữ liệu của biến phạm vi. Nó phải cùng kiểu dữ liệu với các phần tử của mảng hoặc là kiểu dữ liệu mà các phần tử có thể tự động chuyển đổi sang.

- *rangeVariable* là tên của biến phạm vi. Biến này sẽ nhận giá trị của một phần tử mảng khác nhau trong mỗi lần lặp của vòng lặp. Trong lần lặp đầu tiên, nó nhận giá trị của phần tử đầu tiên; trong lần lặp thứ hai, nó nhận giá trị của phần tử thứ hai, ...

- *array* là tên của mảng muốn vòng lặp hoạt động trên đó. Vòng lặp sẽ lặp lại một lần cho mỗi phần tử trong mảng.

- *statement* là một câu lệnh thực thi trong mỗi lần lặp của vòng lặp. Nếu bạn cần thực thi nhiều câu lệnh trong vòng lặp, hãy đặt các câu lệnh trong một cặp dấu ngoặc nhọn.

Ví dụ 1:

```
int numbers[] = { 3, 6, 9 };
for (int val : numbers)
    cout << val << "\t";
```

sẽ in ra màn hình: 3 6 9

Ví dụ 2:

Chương trình 7.4. Vòng lặp for và phạm vi của mảng

```
// Chương trình này trình bày vòng lặp for dựa trên phạm vi.
#include <iostream>
#include <string>
using namespace std;

int main()
{
    string planets[] = { "Sao Thủy", "Sao Kim", "Trái Đất",
        "Sao Hỏa", "Sao Mộc", "Sao Thổ", "Sao Thiên Vương", "Sao Hải Vương", "Sao
        Diêm Vương (hành tinh lùn)" };

    cout << "Đây là các hành tinh:\n";

    // Hiển thị các giá trị trong mảng.
    for (string val: planets)
```

```

        cout << val << endl;

    return 0;
}

```

Kết quả chương trình:

Đây là các hành tinh:
 Sao Thủy
 Sao Kim
 Trái Đất
 Sao Hỏa
 Sao Mộc
 Sao Thổ
 Sao Thiên Vương
 Sao Hải Vương
 Sao Diêm Vương (hành tinh lùn)

7.4. Minh họa: sử dụng mảng song song

KHÁI NIỆM Mảng song song (Parallel Arrays) là một cấu trúc dữ liệu gồm hai hoặc nhiều mảng khác nhau, nhưng có mối liên hệ về ý nghĩa với nhau. Các phần tử tại cùng một chỉ số trong các mảng này được liên kết với nhau về mặt logic. Điều này có nghĩa là phần tử thứ i trong mảng thứ nhất tương ứng với phần tử thứ i trong mảng thứ hai, và cứ như vậy với các mảng khác.

Ví dụ 1:

Chương trình 7.5. Mảng song song

```

#include <iostream>
#include <string>
using namespace std;

int main() {
    const int SIZE = 3; // Kích thước của các mảng song song

    // Khai báo các mảng song song
    string tenSinhVien[SIZE] = {"Nguyen Van A", "Tran Thi B", "Le Hoang C"};
    int diem[SIZE] = {8, 7, 9};
    char danhGia[SIZE];

    // Xử lý dữ liệu trong các mảng song song
    for (int i = 0; i < SIZE; i++) {
        if (diem[i] >= 8) {
            danhGia[i] = 'A';
        } else if (diem[i] >= 6) {
            danhGia[i] = 'B';
        } else {
            danhGia[i] = 'C';
        }
    }
}

```

```

    }
}

// Xuất kết quả
cout << "Danh sach sinh vien va ket qua:" << endl;
for (int i = 0; i < SIZE; i++) {
    cout << tenSinhVien[i] << ": Diem = " << diem[i] << ", Danh gia = "
<< danhGia[i] << endl;
}

return 0;
}

```

Kết quả chương trình:

```

Danh sach sinh vien va ket qua:
Nguyen Van A: Diem = 8, Danh gia = A
Tran Thi B: Diem = 7, Danh gia = B
Le Hoang C: Diem = 9, Danh gia = A

```

Giải thích:

Trong ví dụ trên, chúng ta có ba mảng song song: **tenSinhVien**, **diem**, và **danhGia**. Mảng **tenSinhVien** lưu trữ tên của các sinh viên, mảng **diem** lưu trữ điểm số tương ứng của từng tên sinh viên, và mảng **danhGia** sẽ lưu trữ đánh giá tương ứng dựa trên điểm số.

Đầu tiên, chúng ta khai báo và khởi tạo các mảng song song với kích thước cố định là `SIZE = 3`. Tiếp theo, chúng ta sử dụng vòng lặp `for` để duyệt qua các phần tử của mảng **diem** và gán giá trị tương ứng cho mảng **danhGia** dựa trên điểm số. Cuối cùng, in ra kết quả bằng cách duyệt qua các mảng song song và xuất tên sinh viên, điểm số, và đánh giá tương ứng.

Chú ý

- Các mảng song song phải có cùng kích thước (số lượng phần tử) để đảm bảo tính nhất quán của dữ liệu.
- Cần phải duy trì mối liên hệ logic giữa các phần tử tương ứng trong các mảng song song khi thực hiện các thao tác như thêm, xóa, hoặc sửa đổi dữ liệu.
- Các mảng song song thường được sử dụng để lưu trữ dữ liệu có mối liên hệ với nhau, ví dụ như tên và tuổi của các sinh viên, mã sản phẩm và tên sản phẩm, v.v.

Mảng song song là một cấu trúc dữ liệu hữu ích khi cần lưu trữ và xử lý dữ liệu có mối liên hệ với nhau. Tuy nhiên, cần lưu ý một số vấn đề về tính nhất quán và quản lý dữ liệu khi sử dụng.

7.5. Mảng là tham số của hàm

KHÁI NIỆM Trong C++, có thể truyền một mảng làm tham số cho một hàm. Khi một mảng được truyền vào hàm, thực tế nó được truyền đi như một con trỏ chỉ đến phần tử đầu

tiên của mảng. Điều này có nghĩa rằng, bên trong hàm, chúng ta không thể sửa đổi kích thước của mảng, nhưng có thể sửa đổi giá trị của các phần tử trong mảng.

Ví dụ 1: Tính tổng các phần tử trong mảng

Chương trình 7.6. Hàm tính tổng các phần tử của mảng

```
#include <iostream>
using namespace std;

// Hàm tính tổng các phần tử trong mảng
int tinhTong(int arr[], int n) {
    int tong = 0;
    for (int i = 0; i < n; i++) {
        tong += arr[i];
    }
    return tong;
}

int main() {
    int mang[] = {1, 2, 3, 4, 5};
    int kíchThuoc = sizeof(mang) / sizeof(mang[0]); // Lấy kích thước của mảng

    int ketQua = tinhTong(mang, kíchThuoc);
    cout << "Tổng các phần tử trong mảng là: " << ketQua << endl;

    return 0;
}
```

Kết quả chương trình:

Tổng các phần tử trong mảng là: 15

Giải thích:

- Trong hàm **tinhTong**, chúng ta nhận vào hai tham số: arr là mảng số nguyên và n là kích thước của mảng.
- Bên trong hàm, chúng ta sử dụng vòng lặp for để duyệt qua tất cả các phần tử trong mảng và cộng chúng lại.
- Trong hàm main, chúng ta khởi tạo một mảng mang với 5 phần tử. Sau đó, chúng ta tính toán kích thước của mảng bằng cách sử dụng toán tử sizeof.
- Cuối cùng, chúng ta gọi hàm tinhTong với tham số là mảng mang và kích thước của mảng, sau đó in ra kết quả.

Ví dụ 2: Đếm số lần xuất hiện của một phần tử trong mảng

Chương trình 7.7. Đếm số lần xuất hiện của một phần tử trong mảng

```
#include <iostream>
using namespace std;
```

```

// Hàm đếm số lần xuất hiện của một phần tử trong mảng
int demSoLanXuatHien(int arr[], int n, int x) {
    int dem = 0;
    for (int i = 0; i < n; i++) {
        if (arr[i] == x) {
            dem++;
        }
    }
    return dem;
}

int main() {
    int mang[] = {1, 2, 3, 4, 2, 5, 2, 6, 2};
    int n = sizeof(mang) / sizeof(mang[0]); // Tính số lượng phần tử trong
mảng
    int x = 2; // Phần tử cần đếm số lần xuất hiện

    int soLanXuatHien = demSoLanXuatHien(mang, n, x);
    cout << "Phan tu " << x << " xuat hien " << soLanXuatHien << " lan
trong mang" << endl;

    return 0;
}

```

Kết quả chương trình:

Phan tu 2 xuat hien 4 lan trong mang

Giải thích:

- Hàm demSoLanXuatHien nhận ba đối số: arr là mảng các số nguyên, n là số lượng phần tử trong mảng, và x là phần tử cần đếm số lần xuất hiện.
- Bên trong hàm, ta sử dụng một biến dem để đếm số lần xuất hiện của phần tử x trong mảng.
- Ta duyệt qua từng phần tử của mảng bằng vòng lặp for. Nếu phần tử hiện tại bằng x, ta tăng biến dem lên 1.
- Sau khi kết thúc vòng lặp, hàm trả về giá trị của biến dem.
- Trong hàm main(), ta khai báo một mảng mang với các phần tử {1, 2, 3, 4, 2, 5, 2, 6, 2}.
- Sau đó, ta tính số lượng phần tử trong mảng bằng cách sizeof(mang) / sizeof(mang[0]) và gán vào biến n.
- Tiếp theo, ta khai báo biến x và gán giá trị là 2, đây là phần tử cần đếm số lần xuất hiện trong mảng.
- Ta gọi hàm demSoLanXuatHien với đối số là mảng mang, số lượng phần tử n, và phần tử cần đếm x. Kết quả được lưu vào biến soLanXuatHien.
- Cuối cùng, ta in ra thông báo hiển thị số lần xuất hiện của phần tử x trong mảng.

Trong ví dụ này, sử dụng mảng làm tham số cho hàm `demSoLanXuatHien`. Hàm này nhận ba đối số: mảng các số nguyên `arr`, số lượng phần tử trong mảng `n`, và phần tử cần đếm số lần xuất hiện `x`.

Bên trong hàm, ta duyệt qua từng phần tử của mảng và kiểm tra xem phần tử hiện tại có bằng `x` hay không. Nếu bằng, ta tăng biến đếm `dem` lên 1. Cuối cùng, hàm trả về giá trị của `dem`, đó là số lần xuất hiện của phần tử `x` trong mảng.

Trong hàm `main()`, ta khai báo mảng `mang`, tính số lượng phần tử, và gọi hàm `demSoLanXuatHien` với đối số là mảng, số lượng phần tử, và phần tử cần đếm. Kết quả được in ra màn hình.

Chú ý:

- Khi truyền một mảng làm tham số cho hàm, chúng ta thực sự đang truyền một con trỏ chỉ đến phần tử đầu tiên của mảng. Do đó, bên trong hàm, chúng ta không thể sửa đổi kích thước của mảng.
- Để biết kích thước của mảng, chúng ta cần truyền kích thước của mảng làm một tham số riêng biệt nữa cho hàm.
- Khi sử dụng mảng làm tham số, chúng ta nên cẩn thận với các phép toán có thể dẫn đến truy cập ngoài phạm vi mảng, vì điều này có thể gây ra lỗi và ảnh hưởng đến chương trình.

Mảng làm tham số cho hàm là một tính năng hữu ích trong C++, giúp chúng ta có thể xử lý dữ liệu mảng một cách linh hoạt và hiệu quả. Tuy nhiên, cần lưu ý một số vấn đề về kích thước mảng và truy cập phạm vi khi sử dụng tính năng này.

7.6. Mảng 2 chiều

KHÁI NIỆM Mảng 2 chiều là một dạng mảng đặc biệt, trong đó mỗi phần tử lại là một mảng khác. Mỗi phần tử trong mảng 2 chiều được xác định bởi hai chỉ số: chỉ số hàng và chỉ số cột.

Ví dụ 1: In ra các phần tử trong mảng 2 chiều

Chương trình 7.8. Mảng 2 chiều

```
#include <iostream>
using namespace std;

int main() {
    int arr[3][4] = { {1, 2, 3, 4},
                      {5, 6, 7, 8},
                      {9, 10, 11, 12} };

    int hang = sizeof(arr) / sizeof(arr[0]);
    int cot = sizeof(arr[0]) / sizeof(int);

    cout << "Mang 2 chieu: " << endl;
```

```

    for (int i = 0; i < hang; i++) {
        for (int j = 0; j < cot; j++) {
            cout << arr[i][j] << " ";
        }
        cout << endl;
    }

    return 0;
}

```

Kết quả chương trình:

Mang 2 chiều:

```

1 2 3 4
5 6 7 8
9 10 11 12

```

Giải thích:

- Trong ví dụ này, khai báo một mảng 2 chiều **arr** có 3 hàng và 4 cột. Các giá trị của mảng được khởi tạo ngay trong quá trình khai báo.
- Chúng ta tính số hàng của mảng bằng cách `sizeof(arr) / sizeof(arr[0])` và gán vào biến `hang`.
- Chúng ta tính số cột của mảng bằng cách `sizeof(arr[0]) / sizeof(int)` và gán vào biến `cot`.
- Sau đó, chúng ta sử dụng hai vòng lặp lồng nhau để duyệt qua từng phần tử của mảng 2 chiều.
 - Vòng lặp bên ngoài duyệt qua các hàng của mảng với chỉ số `i`.
 - Vòng lặp bên trong duyệt qua các cột của mảng với chỉ số `j`.
 - Ta in ra giá trị của phần tử tại vị trí `arr[i][j]`.
 - Sau khi kết thúc vòng lặp bên trong, ta xuống dòng để in phần tử trên hàng tiếp theo.

Ví dụ 2: Tính tổng các phần tử trên đường chéo chính của ma trận vuông

Chương trình 7.9. Tổng các phần tử trong đường chéo chính

```

#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Nhập kích thước ma trận vuông: ";
    cin >> n;

    int arr[n][n];
    cout << "Nhập các phần tử của ma trận: " << endl;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cin >> arr[i][j];
        }
    }
}

```

```

    }
}

int tong = 0;
for (int i = 0; i < n; i++) {
    tong += arr[i][i];
}

cout << "Tong cac phan tu tren duong cheo chinh: " << tong << endl;

return 0;
}

```

Kết quả chương trình:

```

Nhap kích thước ma trận vuông: 3
Nhap các phần tử của ma trận:
1 2 3
4 5 6
7 8 9
Tong các phần tử trên đường chéo chính: 15

```

Giải thích:

- Đầu tiên, chúng ta nhập kích thước của ma trận vuông n từ người dùng.
- Tiếp theo, chúng ta khai báo một mảng 2 chiều `arr` có kích thước $n \times n$.
- Chúng ta sử dụng hai vòng lặp lồng nhau để nhập các phần tử của ma trận từ người dùng.
- Sau khi nhập xong, chúng ta khởi tạo một biến `tong` để lưu tổng các phần tử trên đường chéo chính.
- Chúng ta sử dụng một vòng lặp để duyệt qua các phần tử trên đường chéo chính của ma trận (các phần tử có chỉ số hàng và cột bằng nhau).
- Tại mỗi vòng lặp, chúng ta cộng giá trị của phần tử `arr[i][i]` vào biến `tong`.
- Cuối cùng, chúng ta in ra màn hình giá trị của biến `tong`.

Chú ý:

- Khi khai báo mảng 2 chiều, chúng ta cần xác định số hàng và số cột của mảng.
- Để truy cập đến một phần tử trong mảng 2 chiều, chúng ta cần sử dụng hai chỉ số: chỉ số hàng và chỉ số cột.
- Chúng ta có thể sử dụng hai vòng lặp lồng nhau để duyệt qua tất cả các phần tử của mảng 2 chiều.
- Trong trường hợp cần nhập dữ liệu từ người dùng, chúng ta có thể sử dụng hai vòng lặp lồng nhau để nhập giá trị cho từng phần tử của mảng 2 chiều.
- Để tính toán trên mảng 2 chiều, chúng ta có thể sử dụng các vòng lặp và truy cập đến các phần tử theo chỉ số hàng và cột.

- Cần lưu ý đến việc kiểm tra giới hạn của mảng để tránh gây ra lỗi tràn bộ nhớ (buffer overflow).

7.7. Mảng nhiều chiều

KHÁI NIỆM Mảng nhiều chiều (hay mảng đa chiều) là một dạng mảng đặc biệt, trong đó mỗi phần tử lại là một mảng khác. Mỗi phần tử trong mảng nhiều chiều được xác định bởi nhiều chỉ số (tương ứng với số chiều của mảng). Mảng nhiều chiều có thể được xem như mở rộng của mảng 2 chiều.

Ví dụ 1: In ra các phần tử trong mảng 3 chiều

Chương trình 7.10. In các phần tử trong mảng 3 chiều

```
#include <iostream>
using namespace std;

int main() {
    int arr[2][3][4] = {
        {{1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12}},
        {{13, 14, 15, 16}, {17, 18, 19, 20}, {21, 22, 23, 24}}
    };

    int chieuDau = sizeof(arr) / sizeof(arr[0]);
    int chieuThu2 = sizeof(arr[0]) / sizeof(arr[0][0]);
    int chieuThu3 = sizeof(arr[0][0]) / sizeof(int);

    cout << "Mang 3 chieu: " << endl;
    for (int i = 0; i < chieuDau; i++) {
        for (int j = 0; j < chieuThu2; j++) {
            for (int k = 0; k < chieuThu3; k++) {
                cout << arr[i][j][k] << " ";
            }
            cout << endl;
        }
        cout << endl;
    }

    return 0;
}
```

Kết quả chương trình:

Mang 3 chieu:

1 2 3 4

5 6 7 8

9 10 11 12

13 14 15 16

17 18 19 20

21 22 23 24

Giải thích:

- Trong ví dụ này, khai báo một mảng 3 chiều arr có kích thước 2 x 3 x 4. Các giá trị của mảng được khởi tạo ngay trong quá trình khai báo.
- Tính số chiều đầu tiên của mảng bằng cách `sizeof(arr) / sizeof(arr[0])` và gán vào biến `chieuDau`.
- Tính số chiều thứ hai của mảng bằng cách `sizeof(arr[0]) / sizeof(arr[0][0])` và gán vào biến `chieuThu2`.
- Tính số chiều thứ ba của mảng bằng cách `sizeof(arr[0][0]) / sizeof(int)` và gán vào biến `chieuThu3`.
- Sau đó, sử dụng ba vòng lặp lồng nhau để duyệt qua từng phần tử của mảng 3 chiều.
 - Vòng lặp bên ngoài duyệt qua chiều đầu tiên của mảng với chỉ số `i`.
 - Vòng lặp thứ hai duyệt qua chiều thứ hai của mảng với chỉ số `j`.
 - Vòng lặp bên trong duyệt qua chiều thứ ba của mảng với chỉ số `k`.
 - Ta in ra giá trị của phần tử tại vị trí `arr[i][j][k]`.
- Sau khi kết thúc vòng lặp bên trong, ta xuống dòng để in phần tử trên hàng tiếp theo.
- Sau khi kết thúc vòng lặp thứ hai, ta xuống dòng để in phần tử trên chiều thứ hai tiếp theo.

Ví dụ 2: Tính tổng các phần tử trong mảng 4 chiều

Chương trình 7.11. Tính tổng các phần tử trong mảng 4 chiều

```
#include <iostream>
using namespace std;

int main() {
    int arr[2][2][2][2] = {
        {{1, 2}, {3, 4}}, {{5, 6}, {7, 8}},
        {{9, 10}, {11, 12}}, {{13, 14}, {15, 16}}
    };

    int chieuDau = sizeof(arr) / sizeof(arr[0]);
    int chieuThu2 = sizeof(arr[0]) / sizeof(arr[0][0]);
    int chieuThu3 = sizeof(arr[0][0]) / sizeof(arr[0][0][0]);
    int chieuThu4 = sizeof(arr[0][0][0]) / sizeof(int);

    int tong = 0;
    for (int i = 0; i < chieuDau; i++) {
        for (int j = 0; j < chieuThu2; j++) {
            for (int k = 0; k < chieuThu3; k++) {
                for (int l = 0; l < chieuThu4; l++) {
                    tong += arr[i][j][k][l];
                }
            }
        }
    }
}
```

```

    }

    cout << "Tong cac phan tu trong mang 4 chieu: " << tong << endl;

    return 0;
}

```

Kết quả chương trình:

Tong cac phan tu trong mang 4 chieu: 136

Giải thích:

- Trong ví dụ này, khai báo một mảng 4 chiều arr có kích thước 2 x 2 x 2 x 2. Các giá trị của mảng được khởi tạo ngay trong quá trình khai báo.
- Tính số chiều của mảng theo từng chiều bằng cách sử dụng sizeof và gán vào các biến tương ứng.
- Khởi tạo một biến tong để lưu tổng các phần tử trong mảng 4 chiều.
- Sau đó, sử dụng bốn vòng lặp lồng nhau để duyệt qua từng phần tử của mảng 4 chiều.
 - Vòng lặp bên ngoài duyệt qua chiều đầu tiên của mảng với chỉ số i.
 - Vòng lặp thứ hai duyệt qua chiều thứ hai của mảng với chỉ số j.
 - Vòng lặp thứ ba duyệt qua chiều thứ ba của mảng với chỉ số k.
 - Vòng lặp bên trong duyệt qua chiều thứ tư của mảng với chỉ số l.
 - Tại mỗi vòng lặp bên trong, chúng ta cộng giá trị của phần tử arr[i][j][k][l] vào biến tong.
- Cuối cùng, chúng ta in ra màn hình giá trị của biến tong.

Chú ý:

- Khi khai báo mảng nhiều chiều, chúng ta cần xác định số chiều và kích thước của mỗi chiều.
- Để truy cập đến một phần tử trong mảng nhiều chiều, cần sử dụng nhiều chỉ số tương ứng với số chiều của mảng.
- Có thể sử dụng nhiều vòng lặp lồng nhau để duyệt qua tất cả các phần tử của mảng nhiều chiều.

7.8. Một số thuật toán tìm kiếm và sắp xếp

7.8.1. Thuật toán tìm kiếm

KHÁI NIỆM Tìm kiếm là một trong những vấn đề cơ bản trong lập trình. Có nhiều thuật toán tìm kiếm khác nhau, trong đó tìm kiếm tuyến tính và tìm kiếm nhị phân là hai thuật toán đơn giản và phổ biến nhất.

Tìm kiếm tuyến tính

Tìm kiếm tuyến tính là quá trình duyệt qua tất cả các phần tử của một danh sách lần lượt cho đến khi tìm thấy phần tử cần tìm hoặc hết danh sách. Thuật toán này đơn giản nhưng hiệu suất không cao với danh sách lớn.

7.8.1.1 Thuật toán tìm kiếm tuyến tính

1. Bắt đầu từ đầu danh sách.
2. So sánh phần tử hiện tại với phần tử cần tìm.
 - Nếu bằng nhau, trả về vị trí của phần tử đó.
 - Nếu không, di chuyển sang phần tử tiếp theo.
3. Lặp lại bước 2 cho đến khi hết danh sách.
4. Nếu không tìm thấy, trả về -1 (hoặc một giá trị đặc biệt nào đó cho biết không tìm thấy).

Chương trình tìm kiếm tuyến tính

Chương trình 7.12. Tìm kiếm tuyến tính

```
#include <iostream>
using namespace std;

// Hàm tìm kiếm tuyến tính
int linearSearch(int arr[], int n, int x) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == x)
            return i; // Trả về chỉ số của phần tử tìm thấy
    }
    return -1; // Không tìm thấy, trả về -1
}

int main() {
    int arr[] = {5, 2, 8, 1, 9};
    int n = sizeof(arr) / sizeof(arr[0]);
    int x = 8; // Phần tử cần tìm

    int result = linearSearch(arr, n, x);
    if (result == -1)
        cout << "Không tìm thấy phần tử " << x << " trong mảng." << endl;
    else
        cout << "Tìm thấy phần tử " << x << " tại vị trí " << result << " trong mảng." << endl;

    return 0;
}
```

Kết quả chương trình:

Tim thay phan tu 8 tai vi tri 2 trong mang.

7.8.1.2 Tìm kiếm nhị phân

Tìm kiếm nhị phân chỉ áp dụng cho danh sách đã được sắp xếp. Thay vì duyệt lần lượt, thuật toán này sẽ chia đôi danh sách và loại bỏ một nửa danh sách không chứa phần tử cần tìm. Quá trình lặp lại cho đến khi tìm thấy phần tử hoặc danh sách trống. Thuật toán này có hiệu suất cao hơn tìm kiếm tuyến tính đối với danh sách lớn.

Thuật toán tìm kiếm nhị phân

1. Xác định giá trị left (chỉ số đầu danh sách) và right (chỉ số cuối danh sách).
2. Tính $mid = (left + right) / 2$ (chỉ số giữa danh sách).
3. So sánh phần tử tại vị trí mid với phần tử cần tìm.
 - Nếu bằng nhau, trả về mid.
 - Nếu nhỏ hơn, cập nhật $left = mid + 1$ (tìm ở nửa sau).
 - Nếu lớn hơn, cập nhật $right = mid - 1$ (tìm ở nửa trước).
4. Lặp lại bước 2 và 3 cho đến khi $left > right$.
5. Nếu không tìm thấy, trả về -1 (hoặc một giá trị đặc biệt nào đó cho biết không tìm thấy).

Chương trình tìm kiếm nhị phân

Chương trình 7.13. Tìm kiếm nhị phân

```
#include <iostream>
using namespace std;

// Hàm tìm kiếm nhị phân
int binarySearch(int arr[], int left, int right, int x) {
    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == x)
            return mid; // Trả về chỉ số của phần tử tìm thấy

        if (arr[mid] < x)
            left = mid + 1; // Tìm ở nửa sau
        else
            right = mid - 1; // Tìm ở nửa trước
    }

    return -1; // Không tìm thấy, trả về -1
}

int main() {
    int arr[] = {1, 3, 5, 7, 9};
    int n = sizeof(arr) / sizeof(arr[0]);
    int x = 5; // Phần tử cần tìm
```

```

int result = binarySearch(arr, 0, n - 1, x);
if (result == -1)
    cout << "Khong tim thay phan tu " << x << " trong mang." << endl;
else
    cout << "Tim thay phan tu " << x << " tai vi tri " << result << "
    trong mang." << endl;

return 0;
}

```

Kết quả chương trình:

Tim thay phan tu 5 tai vi tri 2 trong mang.

7.8.2. Thuật toán sắp xếp

KHÁI NIỆM Sắp xếp là một trong những vấn đề cơ bản và quan trọng trong lập trình. Nó liên quan đến việc sắp xếp các phần tử trong một danh sách theo một trật tự nhất định, như tăng dần hoặc giảm dần. Trong phần này, chúng ta sẽ tìm hiểu về ba thuật toán sắp xếp phổ biến: sắp xếp nổi bọt, sắp xếp lựa chọn và sắp xếp chèn.

7.8.2.1 Sắp xếp nổi bọt (Bubble Sort)

Sắp xếp nổi bọt là một thuật toán sắp xếp đơn giản, hoạt động bằng cách so sánh các cặp phần tử liên tiếp và hoán đổi chúng nếu chúng không theo thứ tự mong muốn. Quá trình này được lặp đi lặp lại cho đến khi danh sách được sắp xếp hoàn toàn.

Thuật toán sắp xếp nổi bọt

1. Đối với số lần lặp từ n đến 1, với n là số phần tử trong danh sách:

- So sánh các cặp phần tử liên tiếp từ đầu đến cuối danh sách.
- Nếu cặp phần tử hiện tại không theo thứ tự mong muốn (tăng dần hoặc giảm dần), hoán đổi chúng.

2. Lặp lại bước 1 cho đến khi danh sách được sắp xếp hoàn toàn.

Chương trình sắp xếp nổi bọt

Chương trình 7.14. Sắp xếp nổi bọt

```

#include <iostream>
using namespace std;

// Hàm sắp xếp nổi bọt
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                // Hoán đổi arr[j] và arr[j+1]
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

```

```

    }
}

int main() {
    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int n = sizeof(arr) / sizeof(arr[0]);

    cout << "Mang ban dau: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    bubbleSort(arr, n);

    cout << "Mang sau khi sap xep: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    return 0;
}

```

Kết quả chương trình:

Mang ban dau: 64 34 25 12 22 11 90

Mang sau khi sap xep: 11 12 22 25 34 64 90

7.8.2.2 Sắp xếp lựa chọn (Selection Sort)

Sắp xếp lựa chọn là một thuật toán sắp xếp đơn giản, hoạt động bằng cách tìm phần tử nhỏ nhất (hoặc lớn nhất) trong danh sách chưa được sắp xếp và đưa nó về đúng vị trí trong danh sách đã được sắp xếp. Quá trình này được lặp đi lặp lại cho đến khi toàn bộ danh sách được sắp xếp.

Thuật toán sắp xếp lựa chọn

1. Tìm phần tử nhỏ nhất (hoặc lớn nhất) trong danh sách chưa được sắp xếp.
2. Hoán đổi phần tử đó với phần tử đầu tiên trong danh sách chưa được sắp xếp.
3. Di chuyển ranh giới giữa danh sách đã sắp xếp và chưa sắp xếp một bước về phía trước (tăng vị trí bắt đầu của danh sách chưa sắp xếp).
4. Lặp lại bước 1 đến 3 cho đến khi toàn bộ danh sách được sắp xếp.

Chương trình sắp xếp lựa chọn

Chương trình 7.15. Sắp xếp lựa chọn

```
#include <iostream>
using namespace std;

// Hàm sắp xếp lựa chọn
void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        // Tìm phần tử nhỏ nhất trong danh sách chưa sắp xếp
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }

        // Hoán đổi phần tử nhỏ nhất với phần tử đầu tiên trong danh sách
        chưa sắp xếp
        int temp = arr[minIndex];
        arr[minIndex] = arr[i];
        arr[i] = temp;
    }
}

int main() {
    int arr[] = {64, 25, 12, 22, 11};
    int n = sizeof(arr) / sizeof(arr[0]);

    cout << "Mang ban dau: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    selectionSort(arr, n);

    cout << "Mang sau khi sap xep: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    return 0;
}
```

Kết quả chương trình:

Mang ban dau: 64 25 12 22 11

Mang sau khi sap xep: 11 12 22 25 64

7.8.2.3 Sắp xếp chèn (Insertion Sort)

Sắp xếp chèn là một thuật toán sắp xếp đơn giản và hiệu quả cho các danh sách nhỏ. Thuật toán này hoạt động bằng cách lấy một phần tử từ danh sách chưa được sắp xếp và chèn nó vào đúng vị trí trong danh sách đã được sắp xếp. Quá trình này được lặp đi lặp lại cho đến khi toàn bộ danh sách được sắp xếp.

Thuật toán sắp xếp chèn

1. Chia danh sách thành hai phần: danh sách đã sắp xếp (gồm một phần tử đầu tiên) và danh sách chưa sắp xếp (gồm các phần tử còn lại).
2. Lấy phần tử đầu tiên từ danh sách chưa sắp xếp.
3. Di chuyển các phần tử trong danh sách đã sắp xếp về phía sau để tạo chỗ trống cho phần tử mới.
4. Chèn phần tử mới vào đúng vị trí trong danh sách đã sắp xếp.
5. Lặp lại bước 2 đến 4 cho đến khi toàn bộ danh sách được sắp xếp.

Chương trình sắp xếp chèn

Chương trình 7.16. Sắp xếp chèn

```
#include <iostream>
using namespace std;

// Hàm sắp xếp chèn
void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;

        // Di chuyển các phần tử trong danh sách đã sắp xếp về phía sau
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }

        // Chèn phần tử mới vào đúng vị trí
        arr[j + 1] = key;
    }
}

int main() {
    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int n = sizeof(arr) / sizeof(arr[0]);

    cout << "Mang ban dau: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
```

```

insertionSort(arr, n);

cout << "Mang sau khi sap xep: ";
for (int i = 0; i < n; i++) {
    cout << arr[i] << " ";
}
cout << endl;

return 0;
}

```

Kết quả chương trình:

Mang ban dau: 64 34 25 12 22 11 90

Mang sau khi sap xep: 11 12 22 25 34 64 90

7.9. Câu hỏi và Bài tập

7.9.1. Câu hỏi

Cho biết các câu là ĐÚNG hay SAI?

1. Một mảng có thể chứa các phần tử với các kiểu dữ liệu khác nhau.
2. Để tính lượng bộ nhớ được sử dụng bởi một mảng, nhân số lượng phần tử với số byte mà mỗi phần tử sử dụng.
3. Các phần tử riêng lẻ của một mảng được truy cập và chỉ mục bằng các số duy nhất.
4. Chỉ số của một mảng có thể là một số dấu phẩy động.
5. Nếu giá trị của một chỉ số hoặc chỉ mục lớn hơn chỉ số hợp lệ lớn nhất của một mảng, trình biên dịch sẽ không báo cáo bất kỳ lỗi nào.
6. Các phần tử trong một mảng có thể được lưu trữ ở các vị trí không liên tục trong bộ nhớ chính.
7. Mảng có thể có chỉ số có cả giá trị dương và âm.
8. Bạn có thể viết chương trình sử dụng chỉ số không hợp lệ cho một mảng.
9. Mảng không thể được khởi tạo khi chúng được định nghĩa. Một vòng lặp hoặc phương tiện khác phải được sử dụng.
10. Các giá trị trong danh sách khởi tạo được lưu trữ trong mảng theo thứ tự chúng xuất hiện trong danh sách.
11. Một mảng hằng số luôn phải có danh sách khởi tạo.
12. Nếu một mảng được khởi tạo một phần, có thể khởi tạo vài phần tử cuối cùng của mảng mà không cần khởi tạo phần đầu.

13. Nếu bạn để một phần tử chưa khởi tạo, bạn không cần phải để tất cả những phần tử sau nó chưa khởi tạo.
14. Nếu bạn bỏ qua khai báo kích thước của một định nghĩa mảng, bạn không cần phải bao gồm danh sách khởi tạo.
15. Các phần tử chưa khởi tạo của một mảng chuỗi sẽ tự động được đặt giá trị "0".
16. Bạn không thể sử dụng toán tử gán để sao chép nội dung của một mảng sang mảng khác trong một câu lệnh duy nhất.
17. Khi tên mảng được sử dụng mà không có dấu ngoặc vuông và chỉ số, nó được coi là giá trị của phần tử đầu tiên trong mảng.
18. Để truyền một mảng cho một hàm, hãy truyền tên của mảng.
19. Khi định nghĩa một biến tham số để lưu đối số mảng một chiều, bạn không cần phải bao gồm khai báo kích thước.
20. Khi một mảng được truyền cho một hàm, hàm có quyền truy cập vào mảng gốc.
21. Một mảng hai chiều giống như nhiều mảng giống nhau được đặt cùng nhau.
22. Tốt nhất là nghĩ về mảng hai chiều như có hàng và cột.
23. Khai báo kích thước đầu tiên (trong khai báo của một mảng hai chiều) đại diện cho số lượng cột.
24. Khai báo kích thước thứ hai đại diện cho số lượng hàng.
25. Trong một mảng hai chiều được xác định là `myarray[10][20]`, phần tử cuối cùng được truy cập bằng cú pháp `myarray[9][19]`.
26. C++ cho phép bạn tạo mảng với ba hoặc nhiều chiều hơn.

7.9.2. Bài tập

1. Giá trị mảng lớn nhất / nhỏ nhất

Viết chương trình cho phép người dùng nhập mười giá trị vào một mảng. Sau đó, chương trình sẽ hiển thị các giá trị lớn nhất và nhỏ nhất được lưu trữ trong mảng.

2. Thống kê lượng mưa (*Bắt buộc*)

Viết một chương trình cho phép người dùng nhập lượng mưa của mỗi tháng trong 12 tháng vào một mảng số **double**. Chương trình sẽ hiển thị tổng lượng mưa trong năm, lượng mưa trung bình hàng tháng và các tháng có lượng mưa cao nhất và thấp nhất.

Xác thực đầu vào: Không chấp nhận số âm cho số liệu lượng mưa hàng tháng.

3. Khoai tây chiên và Salsa (*Bắt buộc*)

Viết một chương trình cho phép nhà sản xuất khoai tây chiên và salsa theo dõi doanh số của năm loại salsa khác nhau: nhẹ, vừa, ngọt, nóng và vị chanh. Chương trình nên sử dụng cùng lúc hai mảng 5 phần tử: một mảng chuỗi chứa tên năm loại salsa và một mảng số nguyên chứa số lọ đã bán tháng qua cho mỗi loại salsa. Tên các loại salsa nên được lưu trữ bằng cách khởi tạo danh sách tại thời điểm tạo mảng tên. Chương trình yêu cầu người dùng nhập số lượng lọ được bán cho từng loại. Khi dữ liệu bán hàng này đã được nhập, chương trình sẽ tạo ra một báo cáo hiển thị doanh số bán hàng cho từng loại salsa, tổng doanh số bán hàng và tên của các sản phẩm bán chạy nhất và bán chậm nhất.

Xác thực đầu vào: Không chấp nhận giá trị âm cho số lượng lọ đã bán.

4. Lớn hơn n (*Bắt buộc*)

Trong chương trình, hãy viết một hàm chấp nhận ba đối số: một mảng, kích thước của mảng và một số n . Giả sử mảng chứa các số nguyên. Hàm cần hiển thị tất cả các số trong mảng lớn hơn số n .

5. Kinh doanh Khỉ

Một vườn thú địa phương muốn theo dõi xem mỗi con trong ba con khỉ của họ ăn bao nhiêu pound thức ăn mỗi ngày trong một tuần. Viết chương trình lưu trữ thông tin này trong một mảng hai chiều cỡ 3×5 , trong đó mỗi hàng đại diện cho một con khỉ và mỗi cột đại diện cho một ngày trong tuần. Trước tiên, chương trình cần để người dùng nhập dữ liệu cho mỗi con khỉ. Sau đó, nó sẽ hiển thị báo cáo gồm các thông tin sau:

- Lượng thức ăn trung bình mỗi ngày của cả gia đình khỉ.
- Lượng ăn ít nhất xét trên cả tuần và mọi con khỉ
- Lượng ăn nhiều nhất xét trên cả tuần và mọi con khỉ
- Xác thực đầu vào: Không chấp nhận số âm cho số pound thực phẩm đã ăn.

6. Mưa hay Nắng (*Bắt buộc*)

Một nhà khí tượng nghiệp dư cần thu thập thông tin thời tiết trong suốt ba tháng mùa hè năm vừa rồi; nên đã ghi chú một ngày là mưa ('R'), có mây ('C') hoặc nắng ('S'). Viết chương trình lưu trữ thông tin này dưới dạng mảng ký tự cỡ 3×30 , trong đó hàng để chỉ các tháng mùa hè (quy ước, 0 = tháng 6, 1 = tháng 7, 2 = tháng 8) và cột để chỉ các ngày trong tháng. Không cần ghi chép dữ liệu cho ngày 31 của bất kỳ tháng nào. Chương trình sẽ bắt đầu bằng cách đọc dữ liệu thời tiết từ một tệp. Sau đó, cho mỗi tháng và cho cả ba tháng, chương trình sẽ hiển thị có bao nhiêu ngày mưa, ngày mây, và ngày nắng. Nó cũng sẽ báo cáo tháng nào trong số ba tháng có số ngày mưa lớn nhất. Dữ liệu cho chương trình cần lưu trong tệp RainOrShine.txt.

Gợi ý: Tệp RainOrShine.txt có cấu trúc như sau:

R C R A R S R R S R S C R C S S S S R S C A R C S S A C R S

C R R A S A S R C S R C R S S S R S C R R S C C S C S C R R
C R C C S C C R R R S A A C R C C C R C R S S S R S C A R A

7. Chương trình phân tích số

Viết chương trình yêu cầu người dùng nhập tên tệp. Giả sử tệp chứa một chuỗi số, mỗi số được viết trên một dòng. Chương trình sẽ đọc nội dung của tệp thành một mảng sau đó hiển thị các thông tin sau:

- Số thấp nhất trong mảng
- Số cao nhất trong mảng
- Tổng các số trong mảng
- Giá trị trung bình của các số trong mảng

Nếu bạn đã tải xuống mã nguồn của cuốn sách này, bạn sẽ tìm thấy một file tên number.txt trong thư mục Chương 07. Bạn có thể sử dụng file đó để kiểm tra chương trình.

8. Lo Shu Magic Square (Hình vuông kỳ ảo Lạc Thư)

Lo Shu Magic Square là một lưới có 3 hàng và 3 cột được thể hiện trong Hình 7-19. Lo Shu Magic Square có các thuộc tính sau:

4	9	2
3	5	7
8	1	6

Figure 7-19 Lo Shu Magic Square

Lưới chứa chính xác là các số từ 1 đến 9.

Tổng của mỗi hàng, mỗi cột và mỗi đường chéo đều cộng lại ra bằng nhau - được minh họa trong Hình 7-20

4	9	2	↗ 15
3	5	7	→ 15
8	1	6	→ 15
↓ 15	↓ 15	↓ 15	↘ 15

Figure 7-20 Sums of the rows, columns, and diagonals

Trong chương trình, bạn có thể mô phỏng hình vuông ma thuật trên bằng cách sử dụng một mảng hai chiều. Viết một hàm chấp nhận một mảng hai chiều làm đối số và xác định xem mảng đó có phải là một Lo Shu Magic Square hay không. Kiểm tra chức năng hàm này trong chương trình.

9. Tính lương (*Bắt buộc*)

Viết chương trình dùng các mảng sau:

empId: một mảng gồm bảy số nguyên dài, dùng để chứa số căn cước của các nhân viên. Mảng cần được khởi tạo với các số sau:

```
5658845  4520125  7895122  8777541
8451277  1302850  7580489
```

- **hours:** một mảng bảy số nguyên để chứa số giờ làm việc của mỗi nhân viên
- **payRate:** một mảng gồm bảy số double để lưu tỷ lệ lương theo giờ của mỗi nhân viên
- **wages:** một mảng bảy số double để giữ tổng tiền lương của mỗi nhân viên

Chương trình truy cập dữ liệu trong mỗi mảng một cách đồng bộ qua chỉ số mảng. Ví dụ: phần tử 0 của mảng **hours** sẽ lưu số giờ làm của nhân viên có căn cước lưu tại phần tử 0 của mảng **empId**. Tương ứng, tỷ lệ trả lương của nhân viên này được lưu trong phần tử 0 của mảng **payRate**.

Chương trình sẽ hiển thị số căn cước của từng nhân viên và yêu cầu người dùng nhập số giờ làm và mức lương (tỷ lệ lương theo giờ) của nhân viên đó. Sau đó, nó sẽ tính toán tổng tiền lương cho nhân viên đó (tỷ lệ lương theo giờ nhân với số giờ) và lưu trong mảng

wages. Sau khi dữ liệu được nhập cho tất cả nhân viên, chương trình sẽ hiển thị số căn cước từng nhân viên kèm tổng tiền lương của người đó.

Xác thực đầu vào: Không chấp nhận các giá trị âm cho số giờ, hoặc các số nhỏ hơn 15,00 đối với tỷ lệ lương theo giờ.

10. Kỳ thi lấy Bằng lái xe

Văn phòng Giấy phép Lái xe địa phương yêu cầu bạn viết một chương trình chấm điểm phần viết của một người thi lấy bằng lái xe. Đề thi có 20 câu hỏi trắc nghiệm, với đây là các đáp án:

1. A	6. B	11. A	16. C
2. D	7. A	12. C	17. C
3. B	8. B	13. D	18. A
4. B	9. C	14. B	19. D
5. C	10. D	15. D	20. B

Chương trình cần lưu trữ các đáp án trên trong một mảng. Chương trình sẽ yêu cầu nhập các câu trả lời của người thi cho 20 câu hỏi, và các câu trả lời này phải được lưu trong một mảng khác. Sau khi câu trả lời được nhập, chương trình sẽ hiển thị thông báo cho biết người thi đã vượt qua hay không trong kỳ thi (cần đúng ít nhất 15 trong số 20 câu để thi đỗ). Sau đó, chương trình cũng sẽ hiển thị tổng số câu trả lời đúng, tổng số câu trả lời sai và một danh sách số thứ tự các câu bị sai.

Xác thực đầu vào: Chỉ chấp nhận các chữ cái A, B, C hoặc D làm câu trả lời.

11. Phần mềm chấm thi (Bắt buộc)

Một người thầy đặt hàng bạn viết một chương trình chấm điểm thi cuối kỳ, với bài thi này gồm 20 câu hỏi trắc nghiệm và mỗi câu hỏi chỉ có bốn lựa chọn đáp án A, B, C hoặc D.

Tệp CorrectAnswers.txt sẽ chứa trả lời đáp án cho tất cả các câu, với mỗi đáp án được viết trên một dòng riêng biệt, dòng đầu tiên chứa đáp án cho câu đầu tiên, dòng thứ hai chứa đáp án cho câu thứ hai, v.v. (Tải xuống mã nguồn của cuốn sách từ Cổng thông tin Khoa học Máy tính tại www.pearsonhighering.com/gaddis. Bạn sẽ tìm thấy tệp trong thư mục Chương 07.) Viết chương trình đọc nội dung của tệp CorrectAnswers.txt vào một mảng **char**, sau đó đọc nội dung một tệp khác, chứa các câu trả lời của sinh viên, vào một mảng **char** thứ hai. (Bạn có thể sử dụng tệp StudentAnswers.txt cho mục đích kiểm tra. Tệp này cũng nằm trong thư mục mã nguồn Chương 07.) Chương trình sẽ xác định số câu hỏi mà sinh viên bị sai, sau đó hiển thị như sau:

- Danh sách các câu bị sai, và với mỗi câu này, hiển thị câu đáp án và câu trả lời sai của sinh viên.
- Tổng số câu trả lời sai

- Tỷ lệ phần trăm câu trả lời đúng, có thể được tính như sau:
- Số câu trả lời đúng / Tổng số câu hỏi
- Nếu tỷ lệ phần trăm trả lời đúng là 70 phần trăm trở lên, chương trình cần thông báo học sinh đã vượt qua kỳ thi. Nếu không, thông báo học sinh đã trượt kỳ thi.

Nội dung 2 tệp *CorrectAnswers.txt* & *StudentAnswers.txt* như sau:

Tệp CorrectAnswers.txt	Tệp StudentAnswers.txt
A	A
D	D
B	B
C	C
A	B
A	A
D	C
B	B
D	D
C	C
A	A
D	D
B	B
A	A
C	C
C	A
D	D
B	B
C	C
A	A

12. Sổ điểm

Một giáo viên có năm học sinh và vừa làm bốn bài kiểm tra. Với một học sinh, dựa trên điểm trung bình từ bốn bài kiểm tra, giáo viên sẽ sử dụng thang điểm sau để chấm điểm dạng chữ cái:

Test Score	Letter Grade
90–100	A
80–89	B
70–79	C
60–69	D
0–59	F

Viết một chương trình sử dụng một mảng các đối tượng **string** để chứa tên năm học sinh, một mảng năm ký tự để chứa điểm chữ cái của năm học sinh và năm mảng chứa bốn số **double** ứng với tập hợp điểm kiểm tra của năm học sinh. Chương trình sẽ cho phép người dùng nhập tên của từng học sinh và lần lượt bốn điểm kiểm tra của họ. Sau đó, nó sẽ tính toán và hiển thị điểm kiểm tra trung bình và điểm chữ cái tương ứng của học sinh đó.

Xác nhận đầu vào: Không chấp nhận điểm kiểm tra nhỏ hơn 0 hoặc lớn hơn 100.

13. Sổ điểm nâng cấp

Sửa đổi ứng dụng sổ điểm trong bài trên thành tự động loại bỏ điểm thấp nhất khi đang tính điểm trung bình kiểm tra và điểm chữ cái (Vd sinh viên có 4 điểm 9.5; 7; 8.2; 3 thì chỉ tính điểm trung bình từ 9.5; 7; 8.2)

14. Ứng dụng xổ số

Viết chương trình mô phỏng một cuộc thi xổ số. Chương trình cần dùng một mảng gồm năm số nguyên được đặt tên là **lottery** và cần tự sinh ra một số ngẫu nhiên trong khoảng từ 0 đến 9 cho mỗi phần tử trong mảng này. Người dùng cũng phải nhập năm chữ số, số này sẽ được lưu trữ trong một mảng số nguyên khác có tên là **user**. Chương trình sẽ so sánh các phần tử tương ứng trong hai mảng và lưu số lượng số so khớp. Ví dụ: dưới đây là mảng **lottery** và mảng **user** với các số mẫu cho trước. Ta thấy, ở đây có hai chữ số so khớp (phần tử 2 và 4).

- Mảng lottery:

Lottery array:

7	4	9	1	3
---	---	---	---	---

- Mảng user:

User array:

4	2	9	7	3
---	---	---	---	---

Chương trình cần hiển thị các số ngẫu nhiên đã được tạo ra và trong mảng lottery, cùng số chữ số so khớp. Nếu tất cả các chữ số đều khớp, hiển thị thông báo người đã thắng giải độc đắc.

15. Sửa đổi vector

Sửa đổi ví dụ về Ngân hàng Thương mại Quốc gia (National Commerce Bank) được trình bày trong Chương trình 7-23 để pin1, pin2 và pin3 là **vector** thay vì mảng. Ta cũng cần sửa đổi hàm testPIN để nhận đối số là một **vector** thay vì một mảng.

16. Nhà vô địch giải thể giới

Nếu bạn đã tải xuống mã nguồn của cuốn sách này, bạn sẽ tìm thấy các tệp sau trong thư mục của chương này:

Teams.txt - Tệp này chứa danh sách xếp theo thứ tự chữ cái của các đội bóng chày thuộc nhóm Những ông lớn (Major Leagues). Mỗi đội trong nhóm này đều đã vô địch Giải thể giới (World Series) ít nhất một lần.

WorldSeriesWinners.txt - Tệp này chứa danh sách xếp theo thứ tự thời gian của các đội chiến thắng Giải thể giới từ năm 1903 đến năm 2012. (Dòng đầu tiên trong tệp là tên của đội đã vô địch năm 1903 và dòng cuối cùng là tên của đội chiến thắng năm 2012. Lưu ý rằng World Series không tổ chức vào các năm 1904 và 1994.)

Viết chương trình hiển thị nội dung của tệp Teams.txt trên màn hình và nhắc người dùng nhập tên của một trong các nhóm. Sau đó, chương trình sẽ hiển thị số lần mà đội đó đã vô địch World Series trong khoảng thời gian từ 1903 đến 2012.

Gợi ý:

Đọc nội dung của tệp WorldSeriesWinners.txt thành một mảng hoặc **vector**. Sau khi người dùng nhập tên của một nhóm, chương trình sẽ duyệt qua mảng hoặc **vector** và đếm số lần nhóm này xuất hiện.

17. Tìm kiếm tên

Nếu bạn đã tải xuống mã nguồn của cuốn sách này, bạn sẽ tìm thấy các tệp sau trong thư mục của chương này:

- GirlNames.txt - Tệp này chứa danh sách 200 tên phổ biến nhất cho các bé gái sinh ra ở Mỹ từ năm 2000 đến năm 2009.
- BoyNames.txt - Tệp này chứa danh sách 200 tên phổ biến nhất cho các bé trai sinh ra ở Mỹ từ năm 2000 đến năm 2009.

Viết chương trình đọc nội dung của hai tệp trên vào hai mảng hoặc **vector** riêng biệt. Người dùng có thể nhập tên con trai, tên con gái hoặc cả hai và ứng dụng sẽ hiển thị thông báo cho biết liệu những tên đó có nằm trong số những tên phổ biến nhất hay không.

18. Trò chơi Tic-Tac-Toe

Viết một chương trình cho phép hai người chơi cùng chơi trò tic-tac-toe. Sử dụng mảng ký tự hai chiều với ba hàng và ba cột làm bảng trò chơi. Mỗi phần tử của mảng phải được khởi tạo bằng dấu hoa thị (*). Chương trình sẽ chạy vòng lặp sau:

- Hiển thị nội dung của mảng biểu diễn trò chơi
- Cho phép người chơi 1 chọn một vị trí trên bảng để điền X. Chương trình yêu cầu người dùng nhập số hàng và cột.
- Cho phép người chơi 2 chọn một vị trí trên bàn cờ để điền O. Chương trình sẽ yêu cầu người dùng nhập số hàng và cột.
- Xác định xem liệu một người chơi đã thắng hay trò chơi đã hoà. Nếu một người chơi đã thắng, chương trình sẽ tuyên bố người đó là người chiến thắng và kết thúc. Nếu tình huống hòa đã xảy ra, chương trình cũng hiển thị một thông báo tương ứng rồi kết thúc.

Người chơi 1 thắng khi có ba chữ X liên tiếp trên bảng trò chơi. Các chữ X này có thể xuất hiện trong một hàng, trong một cột hoặc theo đường chéo trên bảng. Người chơi 2 thắng khi có ba chữ O liên tiếp trên bảng trò chơi. Các chữ O này có thể xuất hiện trong một hàng, trong một cột hoặc theo đường chéo trên bảng. Tình huống hòa xảy ra khi tất cả các vị trí trên bàn cờ đã đầy đủ, nhưng không có ai chiến thắng.

19. Magic 8 Ball

Viết chương trình mô phỏng Magic 8 Ball, là một đồ chơi bói toán hiển thị một câu trả lời ngẫu nhiên cho một câu hỏi có/không. Trong các chương trình mẫu dành cho sinh viên cho cuốn sách này, bạn sẽ tìm thấy một tệp tên `8_ball_responses.txt`. Tệp chứa 12 câu trả lời, chẳng hạn như “Tôi không nghĩ vậy”, “Tất nhiên là có!”, “Tôi không chắc”, v.v. Chương trình sẽ đọc các phản hồi từ tệp thành một mảng hoặc **vector**. Nó sẽ nhắc người dùng đặt câu hỏi, sau đó hiển thị một trong các câu trả lời, được chọn ngẫu nhiên từ mảng hoặc **vector** này. Chương trình tiếp tục lặp lại việc hỏi đáp này cho đến khi người dùng muốn thoát.

Contents of 8_ball_responses.txt:

Yes, of course!
Without a doubt, yes.
You can count on it.
For sure!
Ask me later.
I'm not sure.
I can't tell you right now.
I'll tell you after my nap.
No way!
I don't think so.
Without a doubt, no.
The answer is clearly NO.

20. Giá gas dạo 1994

Trong các chương trình mẫu dành cho sinh viên cho cuốn sách này, bạn sẽ tìm thấy một tệp tên là 1994_Weekly_Gas_A Average.txt. Tệp này chứa giá gas trung bình cho mỗi tuần trong năm 1994 (Có 52 dòng trong tệp. Dòng 1 chứa giá gas trung bình vào tuần 1, dòng 2 chứa giá trung bình cho tuần 2, v.v.) Viết a chương trình đọc giá gas từ tệp vào một mảng hoặc một vector. Chương trình sẽ xử lý tiếp như sau:

- Hiển thị giá trung bình thấp nhất trong năm, cùng với số của tuần và tên của tháng xảy ra giá đó.
- Hiển thị giá trung bình cao nhất trong năm, cùng với số của tuần và tên của tháng xảy ra giá đó.
- Hiển thị giá gas trung bình cho mỗi tháng. (Để biết giá trung bình cho một tháng nhất định, hãy tính trung bình của giá trung bình các tuần của tháng đó.)

21. Thao tác trên mảng hai chiều (mảng 2D)

Viết chương trình sinh mảng hai chiều được khởi tạo với dữ liệu thử nghiệm. Sử dụng bất kỳ kiểu dữ liệu nào bạn muốn. Chương trình cần có các chức năng sau:

- getTotal - Hàm này nhận một mảng hai chiều làm đối số và trả về tổng các giá trị trong mảng.
- getAverage - Hàm này nhận một mảng hai chiều làm đối số và trả về trung bình của các giá trị trong mảng.
- getRowTotal - Hàm này nhận một mảng hai chiều làm đối số đầu tiên và một số nguyên làm đối số thứ hai. Đối số thứ hai ứng với chỉ số con của một hàng trong mảng. Hàm sẽ trả về tổng các giá trị trong hàng đó.
- getColumnTotal - Hàm này nhận một mảng hai chiều làm đối số đầu tiên và một số nguyên làm đối số thứ hai. Đối số thứ hai ứng với chỉ số con của một cột trong mảng. Hàm sẽ trả về tổng giá trị trong cột đó.

- `getHighestInRow` - Hàm này nhận một mảng hai chiều làm đối số đầu tiên và một số nguyên làm đối số thứ hai. Đối số thứ hai ứng với chỉ số con của một hàng trong mảng. Hàm sẽ trả về giá trị cao nhất trong hàng đó.
- `getLowestInRow` - Hàm này nhận một mảng hai chiều làm đối số đầu tiên và một số nguyên làm đối số thứ hai. Đối số thứ hai ứng với chỉ số con của một hàng trong mảng. Hàm sẽ trả về giá trị thấp nhất trong hàng đó.

Hãy demo từng chức năng trong chương trình này.

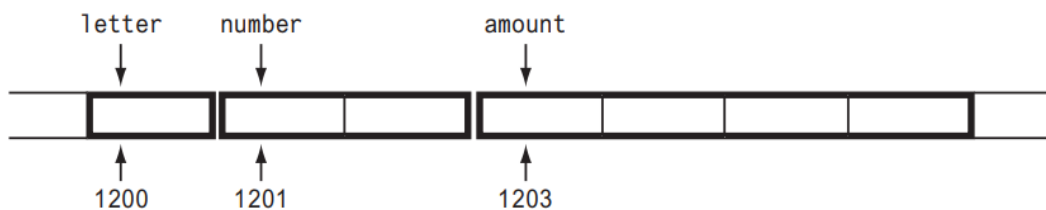
CHƯƠNG 8. CON TRỎ

Chủ đề	
8.1 Địa chỉ của biến	8.4 Con trỏ là tham số cho hàm
8.2 Biến con trỏ	8.5 Cấp phát động
8.3 Mảng và Con trỏ	8.6 Hàm trả về con trỏ

8.1. Địa chỉ của biến

KHÁI NIỆM:

- Trong lập trình, khi khai báo một biến, hệ điều hành sẽ cấp phát một vùng nhớ để lưu trữ giá trị của biến đó.
- Địa chỉ của biến (variable address) là vị trí của vùng nhớ đó trong bộ nhớ (RAM) của máy tính.
- Địa chỉ của biến được biểu diễn dưới dạng số hexadecimal (hệ cơ số 16).



Hình 8.1. Biến và địa chỉ của biến

Trong hình trên thì biến letter có địa chỉ là 1200, biến number có địa chỉ là 1201 và biến amount có địa chỉ là 1203.

Ví dụ:

Chương trình 8.1. Một số thao tác với địa chỉ

```
#include <iostream>
using namespace std;

int main() {
    int a = 10;      // Khai báo và khởi tạo biến a với giá trị 10
    int b = 20;      // Khai báo và khởi tạo biến b với giá trị 20

    // Sử dụng toán tử & để lấy địa chỉ của biến
    cout << "Địa chỉ của biến a: " << &a << endl;
    // In ra địa chỉ của biến a
    cout << "Địa chỉ của biến b: " << &b << endl;
    // In ra địa chỉ của biến b

    // Truy cập giá trị của biến thông qua địa chỉ
    cout << "Giá trị của biến a: " << *(&a) << endl;
```

```

// In ra giá trị của biến a
cout << "Gia tri cua bien b: " << *(&b) << endl;
// In ra giá trị của biến b

return 0;
}

```

Kết quả chương trình:

```

Dia chi cua bien a: 0x61febc
Dia chi cua bien b: 0x61feb8
Gia tri cua bien a: 10
Gia tri cua bien b: 20

```

Giải thích:

- Chương trình khai báo hai biến a và b kiểu int và khởi tạo giá trị cho chúng.
- Sử dụng toán tử & để lấy địa chỉ của biến a và b, sau đó in ra màn hình.
- Để truy cập giá trị của biến thông qua địa chỉ, chúng ta sử dụng toán tử * (toán tử con trỏ).
- Ví dụ, *(&a) sẽ trả về giá trị của biến a thông qua địa chỉ của nó.

Chú ý:

- Địa chỉ của biến có thể khác nhau trên mỗi hệ thống.
- Không nên thay đổi giá trị của biến thông qua địa chỉ trừ khi bạn thực sự hiểu rõ những gì bạn đang làm, vì việc này có thể gây ra lỗi và ảnh hưởng đến sự ổn định của chương trình.
- Trong lập trình C++, địa chỉ của biến thường được sử dụng với con trỏ (pointer) để truy cập và thao tác với giá trị của biến một cách gián tiếp.

8.2. Biến con trỏ

Khái niệm:

Các biến con trỏ, thường được gọi là con trỏ, được thiết kế để chứa các địa chỉ bộ nhớ. Với các biến con trỏ, bạn có thể thao tác gián tiếp dữ liệu được lưu trữ trong các biến khác.

Biến con trỏ, thường được gọi là con trỏ, là một biến đặc biệt chứa địa chỉ bộ nhớ. Giống như biến int được thiết kế để chứa số nguyên và biến double được thiết kế để chứa số dấu phẩy động, biến con trỏ được thiết kế để chứa địa chỉ bộ nhớ.

Địa chỉ bộ nhớ xác định các vị trí cụ thể trong bộ nhớ của máy tính. Một biến con trỏ chứa địa chỉ bộ nhớ, vì vậy nó có thể được sử dụng để lưu trữ vị trí của các phần dữ liệu khác. Điều này giải thích tại sao nó được gọi là con trỏ: Bởi vì nó "trỏ" đến một phần dữ liệu nào đó được lưu trữ trong bộ nhớ của máy tính. Biến con trỏ cũng cho phép bạn thao tác với dữ liệu mà chúng trỏ tới.

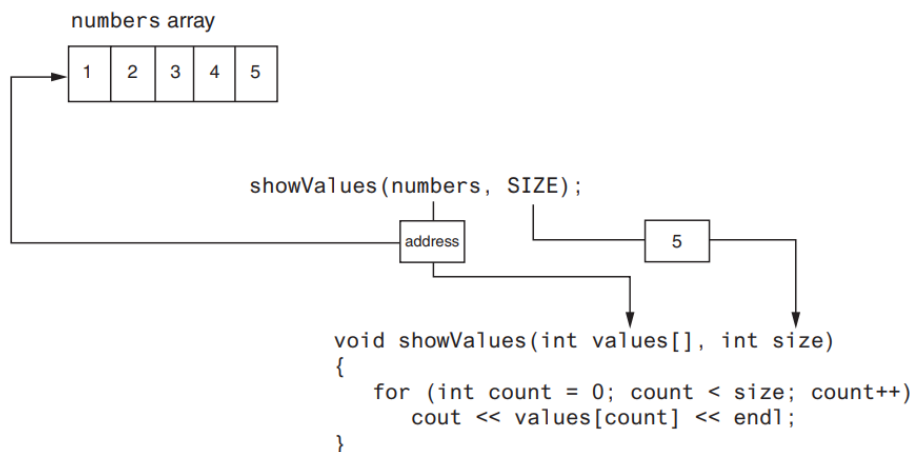
Trong thực tế, khi truyền một mảng vào hàm, chúng ta đang truyền địa chỉ bộ nhớ của phần tử đầu tiên trong mảng đó. Ví dụ: giả sử chúng ta có một mảng tên là `numbers` và chúng ta gọi hàm `showValues` như minh họa sau đây:

```
const int SIZE = 5;
int numbers[SIZE] = { 1, 2, 3, 4, 5 };
showValues(numbers, SIZE);
```

Trong đoạn mã này, truyền tên của mảng `numbers` và kích thước của mảng làm đối số cho hàm `showValues`. Dưới đây là định nghĩa của hàm `showValues`:

```
void showValues(int values[], int size)
{
    for (int count = 0; count < size; count++)
        cout << values[count] << endl;
}
```

Trong hình dưới đây, giá trị tham số nhận địa chỉ của mảng `numbers`. Nó hoạt động giống như một con trỏ vì nó “trỏ” tới mảng `numbers`.



Hình 8.2. Truyền một mảng cho hàm

Một biến tham chiếu đóng vai trò là tên gọi khác (bí danh) cho một biến khác. Nó được gọi là biến tham chiếu bởi vì nó tham chiếu (đề cập) đến một biến khác trong chương trình. Bất cứ thay đổi nào bạn thực hiện trên biến tham chiếu đều được áp dụng cho biến mà nó tham chiếu đến. Ví dụ: giả sử chúng ta có biến `JellyDonuts` và truyền biến đó vào hàm `getOrder`, như minh họa dưới đây:

```
int jellyDonuts;
getOrder(jellyDonuts);
//Dưới đây là định nghĩa hàm getOrder:
void getOrder(int &donuts)
{
    cout << "How many doughnuts do you want? ";
    cin >> donuts;
}
```

```
void getOrder(int &donuts)
{
    cout << "How many doughnuts do you want? ";
    cin >> donuts;
}
```

getOrder(jellyDonuts);

jellyDonuts variable

Bên trong hàm getOrder, tham số donuts tham chiếu đến biến jellyDonuts. Bất cứ điều gì được thực hiện trên tham số donuts thực chất là được thực hiện trên biến jellyDonuts. Khi người dùng nhập giá trị, câu lệnh cin sử dụng biến tham chiếu donuts để gián tiếp lưu trữ giá trị vào biến jellyDonuts.

Tạo và sử dụng biến con trỏ

kiểu dữ liệu *tên biến con trỏ;

- `kiểu_dữ_liệu`: là kiểu dữ liệu của biến mà con trỏ sẽ trỏ đến (ví dụ: `int`, `double`, `char`, v.v.).
- `*`: là toán tử con trỏ, được sử dụng để khai báo một biến là con trỏ.
- `tên biến con trỏ`: là tên của biến con trỏ.

Chương trình 8.2. Khai báo và sử dụng con trỏ

240

```

int a = 10;    // Khai báo và khởi tạo biến a
int *p;        // Khai báo biến con trỏ p

p = &a;        // Gán địa chỉ của biến a cho con trỏ p

cout << "Gia tri cua bien a: " << a << endl;    // In giá trị của
biến a
cout << "Dia chi cua bien a: " << &a << endl;    // In địa chỉ của
biến a
cout << "Gia tri duoc tro boi con tro p: " << *p << endl; // In giá trị
được trỏ bởi con trỏ p

*p = 20;        // Thay đổi giá trị của biến a thông qua con trỏ p

cout << "Gia tri moi cua bien a: " << a << endl;    // In giá trị mới
của biến a

return 0;
}

```

Kết quả chương trình:

```

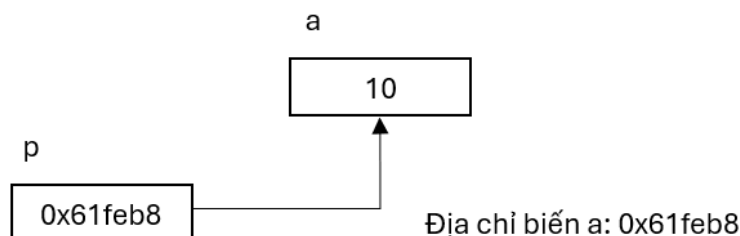
Gia tri cua bien a: 10
Dia chi cua bien a: 0x61feb8
Gia tri duoc tro boi con tro p: 10
Gia tri moi cua bien a: 20

```

Giải thích:

- Chương trình khai báo một biến a kiểu int và khởi tạo giá trị cho nó.
- Khai báo một biến con trỏ p kiểu int*.
- Gán địa chỉ của biến a cho con trỏ p bằng cách sử dụng toán tử &.
- In ra giá trị của biến a, địa chỉ của biến a và giá trị được trỏ bởi con trỏ p (sử dụng toán tử *).
- Thay đổi giá trị của biến a thông qua con trỏ p bằng cách gán giá trị mới cho *p.
- In ra giá trị mới của biến a.

Hình dưới đây mô tả sự liên quan của con trỏ *p và biến a



Hình 8.4. Mối quan hệ giữa con trỏ và biến

Ví dụ 2: Chương trình sau minh họa cách sử dụng con trỏ để tham chiếu và thao tác với các biến khác nhau trong bộ nhớ.

Chương trình 8.3. Con trỏ tham chiếu đến các biến

```
// Chương trình này minh họa việc sử dụng biến con trỏ để tham chiếu đến
// các biến khác nhau.
#include <iostream>
using namespace std;

int main()
{
    int x = 25, y = 50, z = 75; // Ba biến kiểu int
    int *ptr = nullptr; // Biến con trỏ

    // Hiển thị giá trị của x, y, và z.
    cout << "Đây là giá trị của x, y, và z:\n";
    cout << x << " " << y << " " << z << endl;

    // Sử dụng con trỏ để thao tác với x, y, và z.
    ptr = &x; // Lưu địa chỉ của x vào ptr.
    *ptr += 100; // Cộng 100 vào giá trị của x.
    ptr = &y; // Lưu địa chỉ của y vào ptr.
    *ptr += 100; // Cộng 100 vào giá trị của y.
    ptr = &z; // Lưu địa chỉ của z vào ptr.
    *ptr += 100; // Cộng 100 vào giá trị của z.

    // Hiển thị lại giá trị của x, y, và z.
    cout << "Một lần nữa, đây là giá trị của x, y, và z:\n";
    cout << x << " " << y << " " << z << endl;

    return 0;
}
```

Kết quả chương trình:

Đây là giá trị của x, y, và z:

25 50 75

Một lần nữa, đây là giá trị của x, y, và z:

125 150 175

Giải thích:

- Trong chương trình trên, ba biến x, y, và z được khởi tạo với các giá trị khác nhau. Sau đó, một biến con trỏ ptr được khởi tạo với giá trị nullptr. Giá trị của x, y, và z được in ra màn hình.
- Sau đó, con trỏ ptr được sử dụng để thao tác với các biến x, y, và z. Địa chỉ của x được gán cho ptr, và giá trị tại địa chỉ đó (tức là giá trị của x) được cộng thêm 100. Tương tự với y và z.
- Cuối cùng, giá trị mới của x, y, và z được in ra màn hình.

Chú ý:

- Con trỏ là một biến lưu trữ địa chỉ bộ nhớ của một biến khác.
- Khi khai báo con trỏ, cần chỉ định kiểu dữ liệu của biến mà con trỏ sẽ trỏ đến.

- Không nên trả con trỏ vào một vùng nhớ không hợp lệ hoặc truy cập giá trị của con trỏ NULL vì có thể gây ra lỗi.
- Sử dụng con trỏ một cách không đúng cách có thể dẫn đến lỗi bộ nhớ nghiêm trọng như tràn bộ đệm, truy cập bộ nhớ không hợp lệ, v.v.
- Con trỏ rất hữu ích trong việc quản lý và thao tác với bộ nhớ động, tuy nhiên cần cẩn thận khi sử dụng chúng để tránh gây ra lỗi.

8.3. Môi quan hệ giữa Mảng và Con trỏ

KHÁI NIỆM Tên của một mảng thực chất là một con trỏ hằng (constant pointer) trỏ đến phần tử đầu tiên của mảng. Điều này cho phép chúng ta sử dụng cú pháp của con trỏ để thao tác với các phần tử của mảng.

Ví dụ 1: Sử dụng con trỏ để duyệt qua mảng

Chương trình 8.4. Duyệt mảng bằng con trỏ

```
#include <iostream>
using namespace std;

int main() {
    int arr[] = {10, 20, 30, 40, 50}; // Khai báo và khởi tạo mảng
    int *ptr = arr;                    // Gán địa chỉ phần tử đầu tiên của
    mảng cho con trỏ

    cout << "Cac phan tu cua mang: ";
    for (int i = 0; i < 5; i++) {
        cout << *(ptr + i) << " "; // In giá trị của phần tử thông qua
    con trỏ
    }
    cout << endl;

    return 0;
}
```

Kết quả chương trình:

Cac phan tu cua mang: 10 20 30 40 50

Giải thích:

- Chương trình khai báo một mảng arr với 5 phần tử.
- Biến con trỏ ptr được khởi tạo với địa chỉ của phần tử đầu tiên trong mảng arr.
- Sử dụng vòng lặp for để duyệt qua các phần tử của mảng.
- Cú pháp `*(ptr + i)` được sử dụng để truy cập giá trị của phần tử thứ i trong mảng thông qua con trỏ.
- In ra giá trị của các phần tử trong mảng.

Ví dụ 2: Sử dụng con trỏ để tính tổng các phần tử trong mảng

Chương trình 8.5. Duyệt mảng bằng con trỏ

```
#include <iostream>
using namespace std;

int main() {
    int arr[] = {5, 10, 15, 20, 25}; // Khai báo và khởi tạo mảng
    int *ptr = arr; // Gán địa chỉ phần tử đầu tiên của mảng cho con trỏ
    int sum = 0; // Biến lưu trữ tổng các phần tử

    for (int i = 0; i < 5; i++) {
        sum += *(ptr + i); // Cộng giá trị của phần tử vào biến tổng
    }

    cout << "Tổng các phần tử của mảng là: " << sum << endl;

    return 0;
}
```

Kết quả chương trình:

Tổng các phần tử của mảng là: 75

Giải thích:

- Chương trình khai báo một mảng arr với 5 phần tử.
- Biến con trỏ ptr được khởi tạo với địa chỉ của phần tử đầu tiên trong mảng arr.
- Biến sum được khởi tạo với giá trị 0 để lưu trữ tổng các phần tử.
- Sử dụng vòng lặp for để duyệt qua các phần tử của mảng.
- Cú pháp *(ptr + i) được sử dụng để truy cập giá trị của phần tử thứ i trong mảng thông qua con trỏ.
- Giá trị của phần tử được cộng vào biến sum.
- In ra tổng các phần tử trong mảng.

Chú ý:

- Không nên thay đổi giá trị của tên mảng vì nó là một con trỏ hằng.
- Khi sử dụng con trỏ để truy cập các phần tử trong mảng, cần đảm bảo không vượt quá giới hạn của mảng (truy cập phần tử nằm ngoài mảng).
- Có thể sử dụng toán tử [] hoặc cú pháp con trỏ để truy cập các phần tử trong mảng.
- Khi truyền mảng cho hàm, thực chất là truyền địa chỉ của phần tử đầu tiên trong mảng.

8.4. Khởi tạo con trỏ

KHÁI NIỆM: Con trỏ có thể được khởi tạo bằng địa chỉ của một đối tượng hiện có.

Hãy nhớ rằng con trỏ được thiết kế để trỏ đến một đối tượng có kiểu dữ liệu cụ thể. Khi một con trỏ được khởi tạo bằng một địa chỉ, nó phải là địa chỉ của đối tượng mà con trỏ có thể trỏ tới.

Ví dụ: định nghĩa con trỏ pint lưu trữ địa chỉ của biến myValue:

```
int myValue;  
int *pint = &myValue;
```

Điều sau đây cũng hợp lệ vì ages là một mảng các số nguyên:

```
int ages[20];  
int *pint = ages;
```

Nhưng định nghĩa con trỏ pint sau đây là sai vì myFloat không phải là int:

```
float myFloat;  
int *pint = &myFloat; // Sai vì biến myFloat là kiểu float!
```

Con trỏ có thể được định nghĩa trong cùng một câu lệnh với các biến khác cùng loại. Câu lệnh sau định nghĩa một biến myValue là số nguyên, sau đó định nghĩa một con trỏ, pint, được khởi tạo bằng địa chỉ của biến myValue:

```
int myValue, *pint = &myValue;
```

Định nghĩa một mảng readings và gán mảng readings cho con trỏ marker :

```
double readings[50], *marker = readings;
```

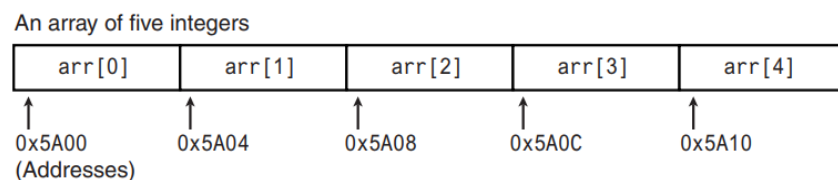
Các khởi tạo sau đây là sai vì con trỏ được gán cho địa chỉ một biến chưa tồn tại

```
int *pint = &myValue; // Sai!  
int myValue;
```

8.5. So sánh các con trỏ

Con trỏ trong C++ tuân theo quy tắc xếp thứ tự dựa trên vị trí của chúng trong bộ nhớ. Nếu một con trỏ nằm ở vị trí trước một con trỏ khác, thì con trỏ đó được coi là 'nhỏ hơn' so với con trỏ kia. Các toán tử quan hệ như <, >, <=, >= có thể được áp dụng để thực hiện việc so sánh giữa các giá trị con trỏ.

Trong một mảng, các phần tử được sắp xếp liên tiếp nhau trong bộ nhớ. Vì vậy, địa chỉ của phần tử tiếp theo trong mảng sẽ lớn hơn địa chỉ của phần tử trước đó. Sự sắp xếp này của các phần tử trong mảng theo thứ tự địa chỉ tăng dần được minh họa trong Hình 9-7.



Hình 8.5. Các phần tử của mảng

Do cấu trúc của mảng với các phần tử được sắp xếp liên tiếp và địa chỉ tăng dần từ phần tử này đến phần tử kế tiếp, nên các biểu thức so sánh địa chỉ trong các câu lệnh if dưới đây đều cho kết quả đúng:

```
if (&arr[1] > &arr[0])
```

```

if (arr < &arr[4])
if (arr == &arr[0])
if (&arr[2] != &arr[3])

```

Chú ý:

Việc so sánh hai con trỏ khác với việc so sánh giá trị của các đối tượng mà chúng trỏ tới. Ví dụ, câu lệnh if dưới đây thực hiện so sánh giữa các địa chỉ bộ nhớ được lưu trữ trong các biến con trỏ ptr1 và ptr2, chứ không phải so sánh giá trị của các đối tượng mà chúng trỏ tới:

```
if (ptr1 < ptr2)
```

Nhưng câu lệnh sau đây so sánh các giá trị mà ptr1 và ptr2 trỏ tới:

```
if (*ptr1 < *ptr2)
```

Chương trình sau, sử dụng con trỏ để hiển thị nội dung của mảng số nguyên

Chương trình 8.6. Sử dụng con trỏ để hiển thị mảng

```

// Chương trình này sử dụng một con trỏ để hiển thị nội dung của một mảng
số nguyên.
#include <iostream>
using namespace std;

int main()
{
    const int SIZE = 8;
    int numbers[SIZE] = {5, 10, 15, 20, 25, 30, 35, 40};
    int *ptr = numbers; // Cho ptr trỏ tới numbers

    // Hiển thị các số trong mảng.
    cout << "Các số là:\n";
    cout << *ptr << " "; // Hiển thị phần tử đầu tiên
    while (ptr < &numbers[SIZE - 1])
    {
        // Tiến ptr để trỏ tới phần tử tiếp theo.
        ptr++;
        // Hiển thị giá trị được trỏ bởi ptr.
        cout << *ptr << " ";
    }

    // Hiển thị các số theo thứ tự ngược lại.
    cout << "\nCác số theo thứ tự ngược lại là:\n";
    cout << *ptr << " "; // Hiển thị phần tử đầu tiên
    while (ptr > numbers)
    {
        // Di chuyển ngược về phần tử trước đó.
        ptr--;
        // Hiển thị giá trị được trỏ bởi ptr.
        cout << *ptr << " ";
    }
    return 0;
}

```

Kết quả chương trình:

Cac so la:

5 10 15 20 25 30 35 40

Cac so theo thu tu nguoc lai la:

40 35 30 25 20 15 10 5

8.6. Con trỏ là tham số cho hàm

Con trỏ có thể được sử dụng làm tham số truyền vào hàm. Điều này cho phép hàm có khả năng truy cập và thao tác trực tiếp lên đối tượng mà con trỏ đó đang trỏ tới, tương tự như cách hoạt động của tham chiếu trong việc truyền tham số.

Trong chương 6 đã được giới thiệu về khái niệm sử dụng biến tham chiếu làm tham số hàm. Một biến tham chiếu hoạt động như một bí danh cho biến gốc được sử dụng làm đối số. Điều này cho phép hàm truy cập vào biến đối số gốc, cho phép thay đổi nội dung của biến đó. Khi một biến được truyền vào tham số tham chiếu thì đối số được gọi là truyền tham chiếu.

Một cách khác để truyền đối số bằng tham chiếu là sử dụng biến con trỏ làm tham số. Nhưng sử dụng biến tham chiếu dễ sử dụng hơn nhiều so với con trỏ. Biến tham chiếu che giấu tất cả các "cơ chế" bỏ tham chiếu và gián tiếp. Tuy nhiên, bạn vẫn nên học cách sử dụng con trỏ làm đối số hàm, bởi vì một số tác vụ (đặc biệt khi đang xử lý chuỗi) sẽ hiệu quả nhất khi sử dụng con trỏ.

Dưới đây là định nghĩa của hàm sử dụng tham số con trỏ:

```
void doubleValue(int *val)
{
    *val *= 2;
}
```

Mục đích của hàm trên là nhân đôi giá trị của biến được trỏ tới bởi val bằng câu lệnh sau đây:

```
*val *= 2;
```

Khi val bị hủy tham chiếu thì toán tử *= sẽ thực hiện trên biến được trỏ tới bởi val. Câu lệnh này nhân giá trị của biến gốc, có địa chỉ được lưu trong val với 2. Khi hàm được gọi thì địa chỉ của biến cần được nhân đôi phải được dùng làm đối số thay vì chính biến đó. Đây là một ví dụ về cách gọi hàm doubleValue:

```
doubleValue(&number);
```

Câu lệnh này sử dụng toán tử địa chỉ (&) để chuyển địa chỉ của số vào tham số val. Sau khi hàm thực thi, nội dung của số sẽ được nhân với 2.

Ví dụ, hàm sử dụng con trỏ làm đối số

Chương trình 8.7. Hàm có con trỏ làm tham số

```
// Chương trình này sử dụng hai hàm nhận địa chỉ của các biến làm tham số.
#include <iostream>
using namespace std;

// Nguyên mẫu hàm
void getNumber(int *);
void doubleValue(int *);

int main()
{
    int number;

    // Gọi hàm getNumber và truyền địa chỉ của biến number.
    getNumber(&number);

    // Gọi hàm doubleValue và truyền địa chỉ của biến number.
    doubleValue(&number);

    // Hiển thị giá trị của biến number.
    cout << "Gia tri sau khi nhan doi la " << number << endl;
    return 0;
}

//
// Định nghĩa của hàm getNumber. Tham số input là một con trỏ. *
// Hàm này yêu cầu người dùng nhập một số. Giá trị nhập vào *
// được lưu trữ trong biến được trỏ bởi input. *
//

void getNumber(int *input)
{
    cout << "Nhap vao mot so nguyen: ";
    cin >> *input;
}

//
// Định nghĩa của hàm doubleValue. Tham số val là một con trỏ. *
// Hàm này nhân đôi giá trị của biến được trỏ bởi val. *
//

void doubleValue(int *val)
{
    *val *= 2;
}
```

Kết quả chương trình:

Nhap vao mot so nguyen: 4
Gia tri sau khi nhan doi la 8

Chương trình trên có hai hàm sử dụng con trỏ làm tham số:

```
void getNumber(int *);
```

```
void doubleValue(int *);
```

Hai hàm sử dụng kí hiệu `int*` để biểu thị tham số là một con trỏ nguyên (con trỏ đến kiểu dữ liệu `int`). Giống như tất cả các loại tham số khác, việc đặt tên cho biến tham số trong nguyên mẫu hàm là không bắt buộc. Tuy nhiên, dấu `*` là bắt buộc để chỉ ra rằng đó là một tham số con trỏ.

Con trỏ tới hằng số

KHÁI NIỆM Trong C++, có thể khai báo một con trỏ trỏ tới một hằng số (constant). Điều này có nghĩa là giá trị mà con trỏ đó trỏ tới sẽ không thể bị thay đổi thông qua con trỏ. Tuy nhiên, bạn vẫn có thể thay đổi giá trị mà con trỏ đó trỏ tới bằng cách sử dụng biến gốc.

Ví dụ, con trỏ đến tham số hằng số

Chương trình 8.8. Con trỏ hằng

```
// Chương trình này minh họa con trỏ đến tham số hằng số
#include <iostream>
using namespace std;

void displayValues(const int *, int);

int main()
{
    // Kích thước của mảng
    const int SIZE = 6;

    // Định nghĩa một mảng các số nguyên hằng số.
    const int array1[SIZE] = { 1, 2, 3, 4, 5, 6 };

    // Định nghĩa một mảng các số nguyên không hằng số.
    int array2[SIZE] = { 2, 4, 6, 8, 10, 12 };

    // Hiển thị nội dung của mảng hằng số.
    displayValues(array1, SIZE);

    // Hiển thị nội dung của mảng không hằng số.
    displayValues(array2, SIZE);
    return 0;
}

/**
// Hàm displayValues sử dụng một con trỏ đến *
// tham số để hiển thị nội dung của một mảng. *
**/

void displayValues(const int *numbers, int size)
{
    // Hiển thị tất cả các giá trị.
    for (int count = 0; count < size; count++)
    {
```

```

        cout << *(numbers + count) << " ";
    }
    cout << endl;
}

```

Kết quả chương trình:

```

1 2 3 4 5 6
2 4 6 8 10 12

```

Khi viết một hàm sử dụng tham số con trỏ và hàm này không nhằm mục đích thay đổi dữ liệu mà tham số con trỏ đó trỏ tới, thì luôn là một ý tưởng tốt để đặt tham số đó là một con trỏ trỏ tới hằng số (pointer to const). Không chỉ việc này sẽ bảo vệ khỏi việc vô tình viết mã trong hàm làm thay đổi đối số, mà hàm còn có thể chấp nhận được cả địa chỉ của các đối số hằng số lẫn không phải hằng số.

Chú ý:

- Khi khai báo một con trỏ trỏ tới hằng số, bạn có thể sử dụng từ khóa const trước hoặc sau dấu *, nhưng ý nghĩa sẽ khác nhau:
- `const int* ptr`: Con trỏ ptr trỏ tới một hằng số int (giá trị mà ptr trỏ tới không thể thay đổi).
- `int* const ptr`: ptr là một con trỏ hằng trỏ tới một int (giá trị mà ptr trỏ tới có thể thay đổi, nhưng chính ptr không thể thay đổi giá trị để trỏ tới một vị trí khác trong bộ nhớ).
- Khi khai báo một con trỏ trỏ tới hằng số và con trỏ hằng, bạn cũng có thể kết hợp cả hai từ khóa const: `const int* const ptr` (con trỏ hằng trỏ tới hằng số).
- Sử dụng con trỏ trỏ tới hằng số giúp đảm bảo an toàn cho chương trình, tránh thay đổi vô tình giá trị của các hằng số quan trọng.
- Khi sử dụng con trỏ trỏ tới hằng số, bạn có thể truy cập và sử dụng giá trị mà nó trỏ tới, nhưng không thể thay đổi giá trị đó thông qua con trỏ.

8.7. Cấp phát động

KHÁI NIỆM: Các biến có thể được tạo và hủy trong khi chương trình đang chạy.

Trước khi thực hiện một chương trình, bạn cần xác định số lượng biến cần thiết. Ví dụ, một chương trình tính diện tích hình chữ nhật sẽ yêu cầu ba biến: một cho chiều dài, một cho chiều rộng và một để lưu trữ diện tích. Nếu bạn viết chương trình tính lương cho 30 nhân viên, bạn có thể tạo một mảng gồm 30 phần tử để lưu trữ tiền lương của từng người.

Tuy nhiên, trong một số trường hợp, bạn không thể xác định trước số lượng biến cần thiết. Ví dụ, giả sử bạn muốn viết một chương trình tính điểm trung bình từ một số lượng bài kiểm tra không cố định. Rõ ràng, chương trình cần phải linh hoạt để xử lý bất kỳ số lượng bài kiểm tra nào. Vậy làm thế nào để lưu trữ điểm của từng bài kiểm tra nếu không

biết trước số lượng biến cần thiết? Giải pháp là cho phép chương trình tự động cấp phát bộ nhớ động để tạo các biến mới một cách linh hoạt. Điều này được gọi là cấp phát bộ nhớ động và chỉ có thể thực hiện được thông qua việc sử dụng con trỏ.

Cấp phát bộ nhớ động nghĩa là chương trình sẽ yêu cầu hệ thống cấp phát một vùng bộ nhớ chưa được sử dụng với kích thước đủ lớn để lưu trữ một biến thuộc kiểu dữ liệu cụ thể. Giả sử chương trình cần tạo một biến kiểu số nguyên, nó sẽ yêu cầu hệ thống cấp phát đủ byte để lưu trữ giá trị của biến đó. Khi nhận được yêu cầu, hệ thống sẽ tìm và cấp phát một vùng bộ nhớ chưa được sử dụng có kích thước phù hợp cho biến mới.

Tiếp theo, hệ thống sẽ cung cấp cho chương trình địa chỉ đầu tiên của vùng bộ nhớ vừa cấp phát. Chương trình chỉ có thể truy cập và sử dụng vùng bộ nhớ mới được cấp phát thông qua địa chỉ này, do đó cần sử dụng một con trỏ để thao tác với các byte bộ nhớ đó.

Trong ngôn ngữ lập trình C++ yêu cầu bộ nhớ được cấp phát động là thông qua toán tử new. Giả sử một chương trình có một con trỏ tới int được định nghĩa là

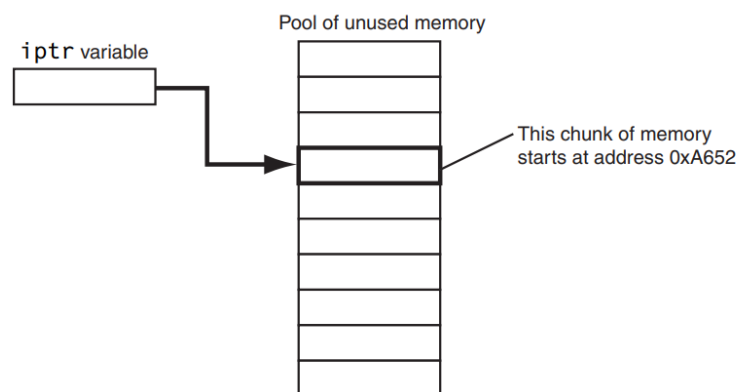
```
int *iptr = nullptr;
```

Đây là một ví dụ về cách sử dụng con trỏ này với toán tử new :

```
iptr = new int;
```

Câu lệnh này yêu cầu hệ điều hành cấp phát một vùng nhớ đủ để lưu trữ một giá trị kiểu nguyên (int). Kiểu dữ liệu của biến mới được tạo ra là kiểu của toán hạng trong toán tử new. Khi câu lệnh được thực hiện, con trỏ iptr sẽ chứa địa chỉ của vùng nhớ vừa được cấp phát. Điều này được minh họa trong hình dưới. Giá trị có thể được gán cho biến mới bằng cách sử dụng toán tử dẫn xuất (dereference) trên con trỏ như sau:

```
*iptr = 25;
```



Hình 8.6. Biến con trỏ iptr trỏ đến một đoạn bộ nhớ được cấp phát động

Bất kỳ thao tác nào khác có thể được thực hiện trên biến mới chỉ bằng cách sử dụng con trỏ đã được dẫn xuất. Đây là một số câu lệnh ví dụ:

```
cout << *iptr; // Hiển thị giá trị của biến mới.  
cin >> *iptr; // Cho phép người dùng nhập giá trị.  
total += *iptr; // Sử dụng biến mới trong một phép tính.
```

Mặc dù các câu lệnh trên minh họa cách sử dụng toán tử new, nhưng việc cấp phát động một biến đơn lẻ không có mục đích thiết thực. Một ứng dụng thực tế hơn của toán tử new là tạo động một mảng. Đây là ví dụ về cách cấp phát một mảng 100 phần tử kiểu nguyên:

```
iptr = new int[100];
```

Một khi mảng đã được tạo, con trỏ có thể được sử dụng với ký hiệu chỉ số để truy cập. Ví dụ, vòng lặp sau có thể được sử dụng để lưu giá trị 1 vào mỗi phần tử:

```
for (int count = 0; count < 100; count++)  
    iptr[count] = 1;
```

Nhưng nếu không đủ bộ nhớ đệm để đáp ứng yêu cầu thì sao? Nếu chương trình yêu cầu một khối bộ nhớ đủ lớn để chứa một mảng 100.000 phần tử kiểu số thực và lượng bộ nhớ đó không khả dụng? Khi bộ nhớ không thể được cấp phát động, C++ sẽ ném ra một ngoại lệ và kết thúc chương trình. Ném ngoại lệ có nghĩa là chương trình báo hiệu rằng đã xảy ra lỗi.

Khi chương trình hoàn tất sử dụng một khối bộ nhớ cấp phát động thì nên giải phóng khối đó để sử dụng trong tương lai. Toán tử delete được sử dụng để giải phóng bộ nhớ đã được cấp phát bằng new. Đây là ví dụ về cách sử dụng delete để giải phóng một biến đơn được trỏ đến bởi iptr:

```
delete iptr;
```

Nếu iptr trỏ đến một mảng được cấp phát động, ký hiệu [] phải được đặt giữa delete và iptr:

```
delete [] iptr;
```

Việc không giải phóng bộ nhớ được cấp phát động có thể khiến chương trình bị thất thoát bộ nhớ. Ví dụ: giả sử hàm GrabMemory sau đây có trong một chương trình:

```
void grabMemory()  
{  
    const int SIZE = 100;  
    // Cấp phát không gian cho 100 phần tử mảng số nguyên.  
    int *iptr = new int[SIZE];  
    // Hàm kết thúc mà không xóa bộ nhớ!  
}
```

Lưu ý hàm phân bổ động một mảng 100 phần tử kiểu int và gán địa chỉ của nó cho biến con trỏ iptr. Nhưng sau đó, hàm kết thúc mà không xóa bộ nhớ. Một khi hàm đã kết thúc, biến con trỏ iptr sẽ không còn tồn tại nữa. Kết quả là, chương trình sẽ không có con trỏ nào trỏ đến vùng nhớ đã được cấp phát động; do đó, vùng nhớ này sẽ không thể được truy cập hoặc xóa. Nó sẽ nằm đó không được sử dụng miễn là chương trình vẫn đang chạy.

Chú ý: Chỉ nên sử dụng các con trỏ trỏ đến vùng nhớ đã được cấp phát trước đó. Nếu bạn sử dụng một con trỏ trỏ đến vùng nhớ chưa được cấp phát động thì có thể gây ra các sự cố không mong muốn!

Chương trình sau minh họa việc sử dụng new và delete, yêu cầu nhập số liệu doanh thu cho bất kỳ số ngày nào. Các số liệu được lưu trữ trong một mảng được cấp phát động, sau đó được tính tổng và trung bình.

Chương trình 8.9. Cấp phát động

```
// Chương trình này tính tổng và trung bình số liệu doanh thu cho bất kỳ số
// ngày nào.
// Các số liệu được lưu trữ trong một mảng được cấp phát động.

#include <iostream>
#include <iomanip>
using namespace std;

int main()
{
    double *sales = nullptr, // Để cấp phát động một mảng
           total = 0.0, // Biến tích lũy
           average; // Để lưu trữ doanh thu trung bình
    int numDays, // Để lưu trữ số ngày bán hàng
        count; // Biến đếm

    // Nhập số ngày bán hàng
    cout << "Ban muon xu ly so lieu doanh thu cua bao nhieu ngay? ";
    cin >> numDays;

    // Cấp phát động một mảng đủ lớn để lưu trữ số liệu doanh thu cho số
    // ngày đó
    sales = new double[numDays];

    // Nhập số liệu doanh thu cho mỗi ngày
    cout << "Nhap so lieu doanh thu duoi day.\n";
    for (count = 0; count < numDays; count++)
    {
        cout << "Ngay " << (count + 1) << ": ";
        cin >> sales[count];
    }

    // Tính tổng doanh thu
    for (count = 0; count < numDays; count++)
    {
        total += sales[count];
    }

    // Tính doanh thu trung bình theo ngày
    average = total / numDays;

    // Hiển thị kết quả
    cout << fixed << showpoint << setprecision(2);
    cout << "\n\nTong Doanh Thu: $" << total << endl;
    cout << "Doanh Thu Trung Binh: $" << average << endl;

    // Giải phóng bộ nhớ được cấp phát động
    delete[] sales;
```

```

    sales = nullptr; // Đặt con trỏ sales về null

    return 0;
}

```

Kết quả chương trình:

Bạn muốn xử lý số liệu doanh thu của bao nhiêu ngày? 3

Nhập số liệu doanh thu dưới đây.

Ngày 1: 1000

Ngày 2: 2000

Ngày 3: 3000

Tổng Doanh Thu: \$6000.00

Doanh Thu Trung Bình: \$2000.00

Giải thích:

- Chương trình khai báo các biến cần thiết, bao gồm một con trỏ sales để lưu trữ mảng số liệu doanh thu, biến total để tích lũy tổng doanh thu, biến average để lưu trữ doanh thu trung bình, và các biến numDays và count để lưu trữ số ngày bán hàng và làm biến đếm.
- Chương trình yêu cầu người dùng nhập số ngày bán hàng bằng cách sử dụng cout và cin.
- Chương trình cấp phát động một mảng có kích thước numDays cho biến con trỏ sales bằng cách sử dụng toán tử new.
- Chương trình yêu cầu người dùng nhập số liệu doanh thu cho mỗi ngày bằng cách sử dụng vòng lặp for và cin.
- Chương trình tính tổng doanh thu bằng cách lặp qua mảng sales và tích lũy các giá trị vào biến total.
- Chương trình tính doanh thu trung bình bằng cách chia total cho numDays.
- Chương trình hiển thị kết quả tổng doanh thu và doanh thu trung bình bằng cách sử dụng cout và các thao tác định dạng fixed, showpoint, và setprecision(2).
- Cuối cùng, chương trình giải phóng bộ nhớ đã cấp phát động cho mảng sales bằng cách sử dụng toán tử delete[] và đặt sales về nullptr.

8.8. Hàm trả về con trỏ

Các hàm có thể trả về con trỏ, nhưng bạn phải đảm bảo rằng đối tượng mà con trỏ đó tham chiếu đến vẫn còn tồn tại.

Giống như bất kỳ kiểu dữ liệu nào khác, các hàm cũng có thể trả về con trỏ. Ví dụ sau đây có hàm trả về con trỏ kiểu int.

Chương trình 8.10. Hàm trả về con trỏ

```
#include<iostream>
using namespace std;
int *HamTraVeConTro()
{
    int *ptrInt;

    ptrInt = new int;

    cout<<"Nhập vào giá trị tại ptrInt trở về: ";
    cin>>*ptrInt;

    return ptrInt;
}
int main()
{
    int *ConTroInt;

    ConTroInt = HamTraVeConTro();
    cout<<"Giá trị vừa nhập: "<<*ConTroInt<<endl;

    return 0;
}
```

Kết quả chương trình:

Nhập vào giá trị tại ptrInt trở về: 99
Giá trị vừa nhập: 99

Chương trình sau cho thấy một ví dụ khác. Chương trình này sử dụng một hàm, getRandomNumbers, để nhận một con trỏ tới một mảng các số ngẫu nhiên. Hàm nhận một đối số nguyên là số lượng số ngẫu nhiên trong mảng. Hàm cấp phát động một mảng, sử dụng đồng hồ hệ thống để khởi tạo máy sinh số ngẫu nhiên, lấp đầy mảng với các giá trị ngẫu nhiên, sau đó trả về một con trỏ tới mảng.

Chương trình 8.11. Hàm trả về 1 mảng số ngẫu nhiên

```
// Chương trình này minh họa một hàm trả về con trỏ.

#include <iostream>
#include <cstdlib> // Cho rand và srand
#include <ctime> // Cho hàm time

using namespace std;

// Nguyên mẫu hàm
int *getRandomNumbers(int);

int main()
{
    int *numbers = nullptr; // Để trỏ tới các số

    // Nhận một mảng gồm 5 số ngẫu nhiên.
```

```

    numbers = getRandomNumbers(5);

    // Hiển thị các số.
    for (int count = 0; count < 5; count++)
        cout << "Số ngẫu nhiên thu " << count + 1 << ": " << numbers[count]
<< endl;

    // Giải phóng bộ nhớ.
    delete[] numbers;
    numbers = nullptr;

    return 0;
}

//
// Hàm getRandomNumbers trả về một con trỏ *
// tới một mảng các số nguyên ngẫu nhiên. Tham số *
// chỉ ra số lượng số được yêu cầu. *
//
int *getRandomNumbers(int num)
{
    int *arr = nullptr; // Mảng để lưu trữ các số

    // Trả về một con trỏ null nếu num là 0 hoặc âm.
    if (num <= 0)
        return nullptr;

    // Cấp phát động mảng.
    arr = new int[num];

    // Khởi tạo máy sinh số ngẫu nhiên bằng cách truyền
    // giá trị trả về của time(0) cho srand.
    srand(time(0));

    // Lấp đầy mảng với các số ngẫu nhiên.
    for (int count = 0; count < num; count++)
        arr[count] = rand();

    // Trả về một con trỏ tới mảng.
    return arr;
}

```

Kết quả chương trình:

```

Số ngẫu nhiên thu 1: 25861
Số ngẫu nhiên thu 2: 19236
Số ngẫu nhiên thu 3: 3810
Số ngẫu nhiên thu 4: 19464
Số ngẫu nhiên thu 5: 31592

```

Giải thích:

- Chương trình khai báo một con trỏ numbers để lưu trữ các số ngẫu nhiên.

- Chương trình gọi hàm getRandomNumbers với tham số là 5, nghĩa là sẽ nhận một mảng gồm 5 số ngẫu nhiên. Con trỏ numbers được gán địa chỉ của mảng này.
- Chương trình sử dụng một vòng lặp for để hiển thị các số ngẫu nhiên đã nhận được.
- Sau khi sử dụng xong, chương trình giải phóng bộ nhớ đã được cấp phát động cho mảng numbers bằng cách sử dụng toán tử delete[].
- Hàm getRandomNumbers nhận một tham số num chỉ định số lượng số ngẫu nhiên cần tạo ra.
- Nếu num là 0 hoặc âm, hàm sẽ trả về nullptr.
- Hàm cấp phát động một mảng có kích thước num bằng cách sử dụng toán tử new.
- Hàm khởi tạo máy sinh số ngẫu nhiên bằng cách truyền giá trị trả về của time(0) cho srand.
- Hàm sử dụng một vòng lặp for để lấp đầy mảng với các số ngẫu nhiên được tạo ra bởi rand().
- Cuối cùng, hàm trả về một con trỏ tới mảng đã được lấp đầy.

8.9. Câu hỏi và Bài tập

8.9.1. Câu hỏi

Cho biết các câu là ĐÚNG hay SAI?

1. Mỗi byte bộ nhớ được gán một địa chỉ duy nhất.
2. Toán tử & được sử dụng để lấy địa chỉ của một biến.
3. Một biến con trỏ có thể được tăng và giảm.
4. Ký hiệu & được gọi là toán tử gián tiếp.
5. Câu lệnh const double *dptr cho biết dptr là một con trỏ hằng số.
6. Khi toán tử gián tiếp được sử dụng với một biến con trỏ, bạn thực sự đang làm việc với giá trị mà con trỏ đang trỏ tới.
7. Tên của mảng không thể được giải quyết với toán tử gián tiếp.
8. Khi bạn cộng một giá trị vào con trỏ, bạn thực sự đang cộng số đó nhân với kích thước của kiểu dữ liệu được tham chiếu bởi con trỏ.
9. Con trỏ hằng số có nghĩa là con trỏ trỏ đến một biến hằng số.
10. Bạn không thể thay đổi địa chỉ mà tên mảng trỏ tới.
11. Bất kỳ phép toán nào, bao gồm nhân và chia, đều có thể được thực hiện trên một con trỏ.

12. Một khi con trỏ hằng số được khởi tạo với một địa chỉ, nó không thể trỏ đến bất cứ thứ gì khác.

13. Mặc dù một hàm có thể chấp nhận các biến con trỏ làm đối số, nhưng một hàm không bao giờ có thể trả về một con trỏ.

14. Toán tử new cấp phát bộ nhớ động.

15. Một biến con trỏ chưa được khởi tạo được gọi là con trỏ null.

16. Địa chỉ 0 thường được coi là không thể sử dụng.

17. Một con trỏ thông minh có thể tự động xóa bộ nhớ được cấp phát động khi nó không còn được sử dụng nữa, mà không cần phải gọi rõ ràng toán tử delete để làm điều đó.

8.9.2. Bài tập

1. Cấp phát mảng (*Bắt buộc*)

Viết một hàm cấp phát động một mảng số nguyên. Hàm nhận một đối số là số nguyên cho biết số lượng phần tử cần cấp phát. Hàm sẽ trả về một con trỏ đến mảng.

2. Điểm Kiểm tra # 1 (*Bắt buộc*)

Viết chương trình cấp phát động một mảng lớn đủ để chứa một số điểm bài kiểm tra do người dùng nhập. Một khi tất cả điểm số được nhập, có một hàm sắp xếp mảng theo thứ tự tăng dần. Một hàm khác là tính điểm trung bình. Các chương trình sẽ hiển thị danh sách điểm đã được sắp xếp và điểm trung bình, phía trên mỗi đầu điểm có tiêu đề thích hợp. Sử dụng con trỏ thay vì mảng bất cứ khi nào có thể.

Xác thực đầu vào: Điểm số không chấp nhận số âm.

3. Xóa điểm thấp nhất

Sửa Bài toán 2 ở trên sao cho loại bỏ điểm thấp nhất. Điểm này không được tính vào điểm trung bình cộng.

4. Điểm Kiểm tra # 2 (*Bắt buộc*)

Sửa đổi bài 2 để cho phép người dùng nhập vào các cặp tên - điểm. Với mỗi sinh viên làm bài kiểm tra, người dùng nhập tên của sinh viên và điểm kiểm tra dưới định dạng số nguyên. Sửa đổi chức năng sắp xếp để có một mảng chứa tên học sinh, và một mảng chứa điểm kiểm tra của học sinh. Khi danh sách được sắp xếp được hiển thị thì tên của sinh viên cũng được hiển thị tương ứng. Sử dụng con trỏ thay vì sử dụng mảng.

5. Viết lại con trỏ:

Hàm sau sử dụng các biến tham chiếu làm tham số. Viết lại hàm để nó sử dụng con trỏ thay vì các biến tham chiếu, sau đó thể hiện hàm trong một chương trình hoàn chỉnh.

```
int doSomething(int &x, int &y)
{
    int temp = x;
    x = y * 10;
    y = temp * 10;
    return x + y;
}
```

6. Hàm phân bố:

Trong thống kê, phân bố của một tập hợp các giá trị là giá trị xảy ra thường xuyên nhất hoặc với tần suất lớn nhất. Viết một hàm nhận các đối số như sau:

- Một mảng các số nguyên
- Một số nguyên cho biết số phần tử trong mảng

Hàm sẽ xác định phân bố của mảng. Nghĩa là, hàm sẽ xác định giá trị nào trong mảng xảy ra thường xuyên nhất. Phân bố là giá trị mà hàm sẽ trả về. Nếu mảng không có phân bố (không có giá trị nào xuất hiện nhiều hơn một lần), thì hàm sẽ trả về -1 . (Giả sử mảng sẽ luôn chứa các giá trị không âm.)

Sử dụng con trỏ thay vì mảng trong bài này.

7. Hàm Trung Vị (*Bắt buộc*)

Trong thống kê, khi một tập hợp các giá trị được sắp xếp theo thứ tự tăng dần hoặc giảm dần, giá trị trung vị của nó là giá trị giữa. Nếu tập hợp gồm chẵn giá trị thì trung vị là giá trị trung bình hoặc giá trị trung bình của hai giá trị giữa. Viết một hàm gồm các đối số như sau:

- Một mảng các số nguyên
- Một số nguyên cho biết số phần tử trong mảng

Hàm sẽ xác định trung vị của mảng. Giá trị này được trả về dưới dạng **double**. (Giả sử các giá trị trong mảng đã được sắp xếp.) Sử dụng con trỏ thay vì mảng trong bài này.

8. Mảng đảo ngược

Viết một hàm có đối số là 1 mảng các số nguyên và kích thước của mảng. Hàm phải tạo một bản sao của mảng, sao cho các giá trị phần tử phải được đảo ngược trong bản sao. Hàm sẽ trả về một con trỏ đến mảng mới. Thể hiện hàm để thành 1 chương trình hoàn chỉnh.

9. Mở rộng mảng

Viết một hàm có đối số là một mảng int và kích thước của mảng. Hàm sẽ tạo một mảng mới có kích thước gấp đôi kích thước của mảng đối số. Hàm nên sao chép nội dung của mảng đối số sang mảng mới và khởi tạo các phần tử chưa sử dụng của mảng thứ hai bằng 0. Hàm sẽ trả về một con trỏ đến mảng mới. Thể hiện hàm để thành 1 chương trình hoàn chỉnh.

10. Dịch chuyển các phần tử

Viết một hàm có đối số là một mảng int và kích thước của mảng. Hàm sẽ tạo một mảng mới lớn hơn một phần tử so với mảng đối số. Phần tử đầu tiên của mảng mới phải được đặt thành 0. Phần tử 0 của mảng đối số phải được sao chép sang phần tử 1 của mảng mới, phần tử 1 của mảng đối số phải được sao chép sang phần tử 2 của mảng mới, v.v.. Hàm sẽ trả về một con trỏ đến mảng mới.

11. Film Thống kê (*Bắt buộc*)

Viết một chương trình có thể được sử dụng để thu thập dữ liệu thống kê về số lượng phim mà sinh viên đại học xem trong một tháng. Chương trình sẽ thực hiện các bước sau:

1. Hỏi người dùng có bao nhiêu sinh viên được khảo sát. Sau đó, một mảng các số nguyên có nhiều phần tử này sẽ được cấp phát động.
2. Cho phép người dùng nhập số lượng phim mà mỗi học sinh đã xem vào mảng.
3. Tính toán và hiển thị giá trị trung bình, trung vị và phân bố của các giá trị đã nhập. (Sử dụng các hàm bạn đã viết trong bài 8 và 9 để tính giá trị trung bình và phân bố.)

Xác thực đầu vào: Không chấp nhận số âm cho đầu vào

CHƯƠNG 9. DỮ LIỆU CÓ CẤU TRÚC

Chủ đề	
9.1 Kiểu dữ liệu trừu tượng	9.5 Mảng cấu trúc
9.2 Dữ liệu có cấu trúc	9.6 Cấu trúc là tham số của hàm
9.3 Truy cập thành viên cấu trúc	9.7 Con trỏ trỏ đến cấu trúc
9.4 Khởi tạo cấu trúc	

9.1. Kiểu dữ liệu trừu tượng

KHÁI NIỆM: Các kiểu dữ liệu trừu tượng (ADTs) là các kiểu dữ liệu được tạo ra bởi nhà lập trình. ADTs có phạm vi (hoặc miền giá trị) dữ liệu riêng của chúng và các tập hợp hoạt động riêng có thể được thực hiện trên chúng.

Thuật ngữ kiểu dữ liệu trừu tượng, hoặc ADT, rất quan trọng trong khoa học máy tính và đặc biệt quan trọng trong lập trình hướng đối tượng. Chương này giới thiệu cho bạn cấu trúc, là một trong những cơ chế của C++ để tạo ra các kiểu dữ liệu trừu tượng.

Trừu tượng hóa

Một trừu tượng hóa là mô hình tổng quát của một cái gì đó. Đó là một định nghĩa chỉ bao gồm các đặc tính chung của một đối tượng. Ví dụ, thuật ngữ "chó" là một trừu tượng hóa. Nó xác định một loại động vật nói chung. Thuật ngữ này gói gọn bản chất của tất cả các loài chó mà không cần đặc thù hóa các đặc điểm chi tiết của bất kỳ loại chó cụ thể nào.

Trong cuộc sống thực tế, tuy nhiên, không có cái gọi là một "con chó" đơn thuần. Có những loại chó cụ thể, mỗi loại có tập hợp đặc điểm riêng. Có chó poodle, chó cocker spaniel, chó Đan Mạch khổng lồ, chó rottweiler, và nhiều giống khác. Có chó nhỏ và chó lớn. Có chó hiền lành và chó hung dữ. Chúng đến với đủ hình dạng, kích cỡ và tính khí khác nhau. Một con chó thực tế không phải là trừu tượng. Nó là cụ thể.

Kiểu dữ liệu

C++ có nhiều kiểu dữ liệu nguyên thủy, hoặc các kiểu dữ liệu được định nghĩa là một phần cơ bản của ngôn ngữ, như được thể hiện trong bảng sau.

Bảng 9.1. Các kiểu dữ liệu cơ bản

<code>bool</code>	<code>short int</code>	<code>long long int</code>
<code>char</code>	<code>unsigned short int</code>	<code>unsigned long long int</code>
<code>char_16_t</code>	<code>int</code>	
<code>char_32_t</code>	<code>unsigned int</code>	<code>float</code>
<code>wchar_t</code>	<code>long int</code>	<code>double</code>
<code>unsigned char</code>	<code>unsigned long int</code>	<code>long double</code>

Một kiểu dữ liệu xác định các giá trị mà một biến có thể chứa. Mỗi kiểu dữ liệu được liệt kê trong Bảng 11-1 có phạm vi giá trị riêng của nó, chẳng hạn như -32.768 đến +32.767 cho kiểu short, và tương tự. Các kiểu dữ liệu cũng xác định các giá trị mà một biến không thể chứa. Ví dụ, các biến kiểu nguyên không thể được sử dụng để chứa các số phân số.

Vì vậy, một kiểu dữ liệu xác định các giá trị mà một đối tượng có thể chứa, và các phép toán có thể được thực hiện trên đối tượng đó.

Kiểu dữ liệu trừu tượng:

Một kiểu dữ liệu trừu tượng (ADT) là một kiểu dữ liệu được tạo ra bởi lập trình viên và bao gồm một hoặc nhiều kiểu dữ liệu nguyên thủy. Lập trình viên sẽ quyết định các giá trị nào được chấp nhận cho kiểu dữ liệu đó, cũng như các phép toán có thể được thực hiện trên kiểu dữ liệu đó. Trong nhiều trường hợp, lập trình viên thiết kế các phép toán chuyên biệt của riêng mình.

Ví dụ, giả sử một chương trình được tạo ra để mô phỏng đồng hồ 12 giờ. Chương trình có thể chứa ba ADT: Giờ, Phút và Giây. Phạm vi giá trị cho kiểu dữ liệu Giờ sẽ là các số nguyên từ 1 đến 12. Phạm vi giá trị cho các kiểu dữ liệu Phút và Giây sẽ là từ 0 đến 59. Nếu một đối tượng Giờ được đặt là 12 và sau đó được tăng lên, nó sẽ nhận giá trị 1. Tương tự, nếu một đối tượng Phút hoặc đối tượng Giây được đặt là 59 rồi tăng lên, nó sẽ nhận giá trị 0.

Các kiểu dữ liệu trừu tượng thường kết hợp nhiều giá trị. Trong chương trình đồng hồ, các đối tượng Giờ, Phút và Giây có thể được kết hợp để tạo thành một đối tượng Đồng hồ duy nhất. Trong chương này, bạn sẽ học cách kết hợp các biến của các kiểu dữ liệu nguyên thủy để tạo ra các cấu trúc dữ liệu riêng của mình, hoặc ADT.

9.2. Cấu trúc

KHÁI NIỆM: C++ cho phép bạn nhóm nhiều biến lại với nhau thành một mục duy nhất được gọi là cấu trúc.

Trong các chương trước, chúng ta đã viết các chương trình lưu trữ dữ liệu trong các biến riêng lẻ. Nếu bạn cần nhóm các mục lại với nhau, C++ cho phép bạn tạo mảng. Tuy nhiên, hạn chế của mảng là tất cả các phần tử phải có cùng một kiểu dữ liệu. Đôi khi mối quan hệ tồn tại giữa các mục có các loại khác nhau. Ví dụ, một hệ thống lương có thể cần lưu giữ các biến được hiển thị trong bảng sau. Các biến này giữ dữ liệu cho một nhân viên duy nhất.

int empNumber;	Mã số nhân viên
string name;	Tên nhân viên
double hours;	Số giờ làm việc
double payRate;	Mức lương 1 giờ
double grossPay;	Tổng mức lương

Tất cả các biến được liệt kê trong bảng trên đều liên quan vì chúng có thể lưu giữ dữ liệu về cùng một nhân viên. Mặc dù các câu lệnh định nghĩa của chúng không làm rõ rằng chúng liên quan đến nhau, C++ cung cấp khả năng đóng gói chúng lại với nhau thành một cấu trúc.

Trước khi có thể sử dụng một cấu trúc, phải khai báo nó. Đây là định dạng tổng quát của một khai báo cấu trúc:

```
struct tên_cấu_trúc
{
    khai_báo_biến;
    // ... thêm khai báo
    // có thể tiếp theo...
};
```

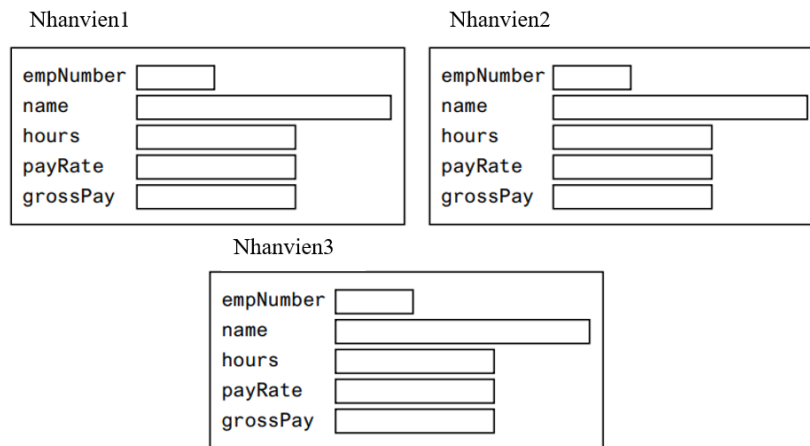
Như sẽ thấy sau này, nó được sử dụng giống như một tên kiểu dữ liệu. Các khai báo biến xuất hiện bên trong dấu ngoặc nhọn khai báo các thành viên của cấu trúc. Đây là một ví dụ về khai báo cấu trúc lưu trữ dữ liệu lương của nhân viên được mô tả ở phía trên.

```
struct TienLuong
{
    int empNumber; // Mã số nhân viên
    string name; // Tên nhân viên
    double hours; // Số giờ làm
    double payRate; // Tiền công 1 giờ
    double grossPay; // Tổng tiền lương
};
```

Với khai báo trên thì chương trình có kiểu dữ liệu kiểu TienLuong. Ví dụ sau khai báo 03 biến Nhanvien1, Nhanvien2, Nhanvien3 có kiểu dữ liệu TienLuong.

```
TienLuong Nhanvien1, Nhanvien2, Nhanvien3;
```

Khi đó, ta có cấu trúc 3 biến Nhanvien1, Nhanvien2, Nhanvien3 là giống nhau như hình dưới đây.



Hình 9.1. Các biến có cùng kiểu cấu trúc

9.3. Truy cập thành viên cấu trúc

C++ cung cấp toán tử dấu chấm (.) để truy cập từng thành viên của cấu trúc. Ví dụ để truy cập vào mã số nhân viên (empNumber) của biến Nhanvien1 và gán cho nó một giá trị 666, ta thực hiện như sau:

```
Nhanvien1.empNumber = 666;
```

Tương tự, cho các biến Nhanvien2, Nhanvien3 như sau:

```
Nhanvien2.empNumber = 888;
```

```
Nhanvien3.empNumber = 999;
```

Hoặc, có thể nhập dữ liệu cho mã số nhân viên, tên nhân viên của biến Nhanvien1 như sau:

```
cout<<"Nhập mã số nhân viên: ";
cin>>Nhanvien1.empNumber;
cin.ignore();
cout<<"Nhập họ tên:";
getline(cin,Nhanvien1.empNumber);
```

Dưới đây là một chương trình sử dụng kiểu dữ liệu có cấu trúc TienLuong (PayRoll)

Chương trình 9.1. Quản lý tính lương nhân viên

```
// Phan khai bao thu vien
#include <iostream> // Thu vien iostream de thuc hien nhap/xuat
#include <string>    // Thu vien string de xu ly chuoai
#include <iomanip>   // Thu vien iomanip de dinh dang dau ra
using namespace std;

// Khai bao cau truc PayRoll
struct PayRoll {
    int empNumber;    // So nhan vien
    string name;      // Ten nhan vien
    double hours;     // So gio lam viec
```

```

    double payRate;    // Luong theo gio
    double grossPay;   // Luong tho
};

int main() {
    PayRoll employee; // Khai bao bien employee co kieu PayRoll

    // Nhap so nhan vien
    cout << "Nhap ma so nhan vien: ";
    cin >> employee.empNumber;

    // Nhap ten nhan vien
    cout << "Nhap ten nhan vien: ";
    cin.ignore(); // Bo qua ky tu '\n' con sot lai trong bo dem
    getline(cin, employee.name);

    // Nhap so gio lam viec
    cout << "Nhap so gio lam viec cua nhan vien: ";
    cin >> employee.hours;

    // Nhap luong theo gio
    cout << "Luong theo gio cua nhan vien: ";
    cin >> employee.payRate;

    // Tinh luong tho
    employee.grossPay = employee.hours * employee.payRate;

    // Hien thi thong tin nhan vien
    cout << "Thong tin luong cua nhan vien:\n";
    cout << "Ten: " << employee.name << endl;
    cout << "So nhan vien: " << employee.empNumber << endl;
    cout << "So gio lam viec: " << employee.hours << endl;
    cout << "Luong theo gio: " << employee.payRate << endl;
    cout << fixed << showpoint << setprecision(2); // Dinh dang so thuc
    cout << "Luong tho: $" << employee.grossPay << endl;

    return 0;
}

```

Kết quả chương trình:

```

Nhap ma so nhan vien: 111
Nhap ten nhan vien: Phong
Nhap so gio lam viec cua nhan vien: 10
Luong theo gio cua nhan vien: 150
Thong tin luong cua nhan vien:
Ten: Phong
So nhan vien: 111
So gio lam viec: 10
Luong theo gio: 150
Luong tho: $1500.00

```

So sánh cấu trúc:

Không thể thực hiện các thao tác so sánh trực tiếp trên các biến cấu trúc. Ví dụ: giả sử `Nhanvien1` và `Nhanvien2` là các biến cấu trúc `TienLuong`. Câu lệnh sau sẽ gây ra lỗi:

```
if(Nhanvien1 > Nhanvien2) //Lỗi
```

Để so sánh hai cấu trúc, bạn phải so sánh từng thành phần riêng lẻ, như được hiển thị trong đoạn mã sau:

```
if(Nhanvien1.grossPay > Nhanvien2.grossPay)
```

9.4. Khởi tạo cấu trúc

KHÁI NIỆM: Các thành viên của một biến cấu trúc có thể được khởi tạo giá trị ban đầu khi biến cấu trúc đó được định nghĩa.

Một biến cấu trúc có thể được khởi tạo giá trị khi nó được định nghĩa, tương tự như việc khởi tạo một mảng. Giả sử có khai báo cấu trúc sau trong chương trình:

```
struct CityInfo
{
    string cityName;
    string state;
    long population;
    int distance;
};
```

Một biến có thể được định nghĩa với danh sách khởi tạo như sau:

```
CityInfo location = {"Asheville", "NC", 80000, 28};
```

Câu lệnh này định nghĩa biến `location`. Giá trị đầu tiên trong danh sách khởi tạo được gán cho thành viên đầu tiên được khai báo, giá trị thứ hai cho thành viên thứ hai, và cứ tiếp tục như vậy. Biến `location` được khởi tạo như sau:

- Chuỗi "Asheville" được gán cho `location.cityName`
- Chuỗi "NC" được gán cho `location.state`
- 80000 được gán cho `location.population`
- 28 được gán cho `location.distance`

Không nhất thiết phải cung cấp khởi tạo giá trị cho tất cả các thành viên của biến cấu trúc. Ví dụ, câu lệnh sau chỉ khởi tạo thành viên `cityName` của `location`:

```
CityInfo location = {"Tampa"};
```

Các thành viên `state`, `population` và `distance` không được khởi tạo. Câu lệnh sau chỉ khởi tạo các thành viên `cityName` và `state`, nhưng không khởi tạo `population` và `distance`:

```
CityInfo location = {"Atlanta", "GA"};
```

Nếu bạn bỏ qua khởi tạo một thành viên cấu trúc, bạn phải bỏ qua tất cả các thành viên đi sau nó. C++ không cung cấp cách để bỏ qua khởi tạo từng thành viên trong cấu trúc. Ví dụ, câu lệnh sau đây cố gắng bỏ qua khởi tạo thành viên population là không hợp lệ:

```
CityInfo location = {"Knoxville", "TN", , 90}; // Không hợp lệ!
```

9.5. Mảng cấu trúc

KHÁI NIỆM

- Mảng cấu trúc là một mảng chứa các phần tử có kiểu dữ liệu là cấu trúc (struct).
- Mỗi phần tử của mảng cấu trúc là một cấu trúc riêng biệt, gồm các trường dữ liệu khác nhau.
- Mảng cấu trúc cho phép lưu trữ và xử lý nhiều cấu trúc cùng loại một cách hiệu quả.

Ví dụ, Chương trình quản lý danh sách sinh viên

Chương trình 9.2. Quản lý tiền sinh viên

```
// Chương trình quản lý danh sách sinh viên
#include <iostream>
#include <string>
using namespace std;

// Khai báo cấu trúc SinhVien
struct SinhVien {
    int maSV;
    string hoTen;
    double diemTB;
};

int main() {
    const int MAX_SV = 100; // Số sinh viên tối đa
    SinhVien dsSinhVien[MAX_SV]; // Mảng cấu trúc chứa danh sách sinh viên
    int soSinhVien;

    // Nhập số lượng sinh viên
    cout << "Nhap so luong sinh vien: ";
    cin >> soSinhVien;
    cin.ignore(); // Bỏ qua ký tự '\n' còn sót lại trong bộ đệm

    // Nhập thông tin sinh viên
    for (int i = 0; i < soSinhVien; i++) {
        cout << "Nhap thong tin sinh vien thu " << i + 1 << ":\n";
        cout << "Ma so sinh vien: ";
        cin >> dsSinhVien[i].maSV;
        cin.ignore();
        cout << "Ho va ten: ";
        getline(cin, dsSinhVien[i].hoTen);
        cout << "Diem trung binh: ";
        cin >> dsSinhVien[i].diemTB;
    }
}
```

```

// Hiển thị danh sách sinh viên
cout << "\nDANH SACH SINH VIEN:\n";
for (int i = 0; i < soSinhVien; i++) {
    cout << "Sinh vien thu " << i + 1 << ":\n";
    cout << "Ma so sinh vien: " << dsSinhVien[i].maSV << "\n";
    cout << "Ho va ten: " << dsSinhVien[i].hoTen << "\n";
    cout << "Diem trung binh: " << dsSinhVien[i].diemTB << "\n\n";
}

return 0;
}

```

Kết quả chương trình:

```

Nhap so luong sinh vien: 3
Nhap thong tin sinh vien thu 1:
Ma so sinh vien: 111
Ho va ten: Dau Hai Phong
Diem trung binh: 8.5
Nhap thong tin sinh vien thu 2:
Ma so sinh vien: 222
Ho va ten: Tran Dang Cong
Diem trung binh: 9
Nhap thong tin sinh vien thu 3:
Ma so sinh vien: 333
Ho va ten: Pham Van Tiep
Diem trung binh: 8

```

DANH SACH SINH VIEN:

```

Sinh vien thu 1:
Ma so sinh vien: 111
Ho va ten: Dau Hai Phong
Diem trung binh: 8.5

```

```

Sinh vien thu 2:
Ma so sinh vien: 222
Ho va ten: Tran Dang Cong
Diem trung binh: 9

```

```

Sinh vien thu 3:
Ma so sinh vien: 333
Ho va ten: Pham Van Tiep
Diem trung binh: 8

```

Giải thích:

- Khai báo cấu trúc SinhVien với các thành viên maSV (kiểu int), hoTen (kiểu string), diemTB (kiểu double).
- Trong hàm main():
 - Khai báo hằng số MAX_SV để giới hạn số lượng sinh viên tối đa.
 - Khai báo mảng cấu trúc dsSinhVien có kiểu SinhVien với kích thước MAX_SV.
 - Khai báo biến soSinhVien để lưu số lượng sinh viên thực tế.
- Nhập số lượng sinh viên từ bàn phím.
- Sử dụng vòng lặp for để nhập thông tin của từng sinh viên:
 - Nhập maSV bằng cách đọc giá trị kiểu số nguyên vào dsSinhVien[i].maSV.
 - Nhập hoTen bằng cách sử dụng getline(cin, dsSinhVien[i].hoTen).
 - Nhập diemTB bằng cách đọc giá trị kiểu số thực vào dsSinhVien[i].diemTB.
- Sử dụng vòng lặp for để hiển thị thông tin của từng sinh viên ra màn hình.

Chú ý

- Khi khai báo mảng cấu trúc, cần xác định kích thước của mảng (số lượng phần tử tối đa).
- Truy cập đến các thành viên của phần tử trong mảng cấu trúc bằng cách sử dụng toán tử. (dấu chấm), ví dụ: dsSinhVien[i].maSV.
- Nếu cần nhập/xuất chuỗi ký tự (kiểu string), cần sử dụng getline() và cin.ignore() để xử lý đúng cách các ký tự \n trong bộ đệm.
- Khi làm việc với mảng cấu trúc, cần kiểm tra giới hạn chỉ số của mảng để tránh truy cập vùng nhớ không hợp lệ.

Mảng cấu trúc rất hữu ích trong việc lưu trữ và xử lý dữ liệu gồm nhiều thành phần khác nhau. Ví dụ trên cho thấy cách sử dụng mảng cấu trúc để quản lý danh sách sinh viên với các thông tin như mã số, họ tên và điểm trung bình.

9.6. Cấu trúc là tham số hàm

KHÁI NIỆM

Trong C++, cấu trúc (struct) có thể được sử dụng làm tham số cho một hàm. Điều này có nghĩa là một đối tượng cấu trúc có thể được truyền vào hàm như một đối số khi gọi hàm đó. Việc sử dụng cấu trúc làm tham số hàm giúp tăng tính linh hoạt và khả năng tái sử dụng của mã nguồn, đồng thời cũng làm cho mã dễ đọc và dễ bảo trì hơn.

Ví dụ, tính khoảng cách giữa 2 điểm

Chương trình 9.3. Tính khoảng cách giữa 2 điểm

```
#include <iostream>
#include<math.h>
using namespace std;

// Định nghĩa cấu trúc Diem
struct Diem {
    int x;
    int y;
};

// Hàm tính khoảng cách giữa hai điểm
double tinhKhoangCach(Diem p1, Diem p2) {
    int dx = p2.x - p1.x;
    int dy = p2.y - p1.y;
    return sqrt(dx * dx + dy * dy);
}

int main() {
    Diem diemA, diemB;

    // Nhập tọa độ cho điểm A
    cout << "Nhap toa do diem A (x y): ";
    cin >> diemA.x >> diemA.y;

    // Nhập tọa độ cho điểm B
    cout << "Nhap toa do diem B (x y): ";
    cin >> diemB.x >> diemB.y;

    // Gọi hàm tính khoảng cách và in kết quả
    double khoangCach = tinhKhoangCach(diemA, diemB);
    cout << "Khoang cach giữa diem A và diem B là: " << khoangCach << endl;

    return 0;
}
```

Kết quả chương trình:

```
Nhap toa do diem A (x y): 1 2
Nhap toa do diem B (x y): 3 4
Khoang cach giữa diem A và diem B là: 2.82843
```

Giải thích:

- Chúng ta định nghĩa một cấu trúc Diem gồm hai thành phần x và y để lưu trữ tọa độ của một điểm trên mặt phẳng.
- Hàm tinhKhoangCach nhận hai tham số p1 và p2 là các đối tượng cấu trúc Diem. Hàm tính khoảng cách Euclidean giữa hai điểm và trả về kết quả là một số thực.
- Trong hàm main, chúng ta khai báo hai đối tượng diemA và diemB của kiểu Diem.
- Người dùng nhập tọa độ cho diemA và diemB thông qua cin.

- Gọi hàm tínhKhoangCach với diemA và diemB làm đối số, và in ra kết quả khoảng cách giữa hai điểm.

Chú ý:

- Khi truyền một cấu trúc làm tham số hàm, giá trị của cấu trúc đó sẽ được sao chép. Nếu cấu trúc lớn, việc sao chép có thể gây ra hiệu suất chậm. Trong trường hợp này, bạn có thể truyền tham chiếu hoặc con trỏ đến cấu trúc để tránh sao chép không cần thiết.
- Nếu bạn muốn thay đổi giá trị của cấu trúc trong hàm, bạn phải truyền tham chiếu hoặc con trỏ đến cấu trúc đó.
- Cấu trúc cũng có thể được sử dụng làm kiểu trả về của hàm.

Ví dụ trên chỉ là một ví dụ đơn giản về việc sử dụng cấu trúc làm tham số hàm trong C++. Trong thực tế, cấu trúc có thể được sử dụng trong nhiều trường hợp phức tạp hơn và kết hợp với các khái niệm lập trình khác như lớp, đối tượng, con trỏ, v.v.

Hàm trả về kiểu dữ liệu cấu trúc:

KHÁI NIỆM

Trong C++, một hàm có thể trả về một cấu trúc (struct) làm kiểu trả về. Điều này cho phép chúng ta trả về nhiều giá trị từ một hàm dưới dạng các trường dữ liệu của cấu trúc. Việc sử dụng cấu trúc làm kiểu trả về giúp tăng tính linh hoạt và khả năng tái sử dụng của mã nguồn, đồng thời làm cho mã dễ đọc và dễ bảo trì hơn.

Ví dụ, chương trình dưới đây trả về Diem là kiểu dữ liệu có cấu trúc.

Chương trình 9.4. Hàm trả về kiểu cấu trúc

```
#include <iostream>
using namespace std;

// Định nghĩa cấu trúc Diem
struct Diem {
    int x;
    int y;
};

// Hàm tạo điểm với tọa độ nhập từ người dùng
Diem taoDoiTuong() {
    Diem p;
    cout << "Nhap toa do diem (x y): ";
    cin >> p.x >> p.y;
    return p;
}

int main() {
    Diem diemA = taoDoiTuong();
    Diem diemB = taoDoiTuong();
}
```

```

    cout << "Toa do diem A: (" << diemA.x << ", " << diemA.y << ")" <<
endl;
    cout << "Toa do diem B: (" << diemB.x << ", " << diemB.y << ")" <<
endl;

    return 0;
}

```

Kết quả chương trình:

```

Nhap toa do diem (x y): 1 2
Nhap toa do diem (x y): 3 4
Toa do diem A: (1, 2)
Toa do diem B: (3, 4)

```

Giải thích:

- Định nghĩa một cấu trúc Diem gồm hai thành phần x và y để lưu trữ tọa độ của một điểm trên mặt phẳng.
- Hàm taoDoiTuong không nhận tham số, nhưng trả về một đối tượng cấu trúc Diem. Trong hàm này, khai báo một đối tượng Diem tạm thời, yêu cầu người dùng nhập tọa độ, và trả về đối tượng đó.
- Trong hàm main, gọi hàm taoDoiTuong hai lần và gán kết quả trả về cho các đối tượng diemA và diemB.
- Sau đó, in ra tọa độ của diemA và diemB lên màn hình.

Chú ý:

- Khi hàm trả về một cấu trúc, giá trị của cấu trúc đó sẽ được sao chép. Nếu cấu trúc lớn, việc sao chép có thể gây ra hiệu suất chậm. Trong trường hợp này, bạn có thể trả về tham chiếu hoặc con trỏ đến cấu trúc để tránh sao chép không cần thiết.
- Nếu bạn muốn trả về một đối tượng cấu trúc tạm thời (không được khởi tạo trước), bạn có thể tạo và trả về đối tượng đó ngay trong hàm.
- Hàm trả về cấu trúc có thể được sử dụng để trả về nhiều giá trị từ một hàm, thay vì sử dụng tham chiếu hoặc con trỏ làm tham số đầu ra.
- Kiểu trả về của hàm không nhất thiết phải là một cấu trúc được định nghĩa trước, mà có thể là một cấu trúc được định nghĩa ngay trong hàm (cấu trúc vô danh).

9.7. Con trỏ trỏ đến cấu trúc

KHÁI NIỆM: Trong C++, có thể sử dụng con trỏ để trỏ đến một đối tượng cấu trúc (struct). Con trỏ là một biến lưu trữ địa chỉ bộ nhớ của một giá trị hoặc đối tượng khác. Khi con trỏ trỏ đến một cấu trúc, nó cho phép chúng ta truy cập và thao tác với các trường dữ liệu của cấu trúc đó thông qua con trỏ.

Ví dụ:

Chương trình 9.5. Con trỏ và kiểu dữ liệu có cấu trúc

```
#include <iostream>
using namespace std;

// Định nghĩa cấu trúc SinhVien
struct SinhVien {
    int maSV;
    string hoTen;
    float diemTB;
};

int main() {
    // Khai báo một đối tượng cấu trúc SinhVien
    SinhVien sv1 = {20201234, "Nguyen Van A", 8.5};

    // Khai báo một con trỏ trỏ đến cấu trúc SinhVien
    SinhVien* ptrSV = &sv1;

    // Truy cập và in thông tin sinh viên thông qua con trỏ
    cout << "Ma sinh vien: " << ptrSV->maSV << endl;
    cout << "Ho ten: " << ptrSV->hoTen << endl;
    cout << "Diem trung binh: " << ptrSV->diemTB << endl;

    // Thay đổi thông tin sinh viên thông qua con trỏ
    ptrSV->maSV = 20201235;
    ptrSV->hoTen = "Nguyen Van B";
    ptrSV->diemTB = 7.8;

    // In thông tin sinh viên sau khi thay đổi
    cout << "\nThong tin sinh vien sau khi thay doi:" << endl;
    cout << "Ma sinh vien: " << ptrSV->maSV << endl;
    cout << "Ho ten: " << ptrSV->hoTen << endl;
    cout << "Diem trung binh: " << ptrSV->diemTB << endl;

    return 0;
}
```

Kết quả chương trình:

Ma sinh vien: 20201234
Ho ten: Nguyen Van A
Diem trung binh: 8.5

Thong tin sinh vien sau khi thay doi:
Ma sinh vien: 20201235
Ho ten: Nguyen Van B
Diem trung binh: 7.8

Giải thích:

- Chúng ta định nghĩa một cấu trúc SinhVien gồm ba trường dữ liệu: maSV (kiểu int), hoTen (kiểu string), và diemTB (kiểu float).
- Trong hàm main, chúng ta khai báo một đối tượng sv1 của cấu trúc SinhVien và khởi tạo giá trị cho các trường dữ liệu.
- Chúng ta khai báo một con trỏ ptrSV của kiểu SinhVien* (con trỏ trỏ đến cấu trúc SinhVien) và gán địa chỉ của sv1 cho ptrSV bằng cách sử dụng toán tử &.
- Sử dụng toán tử -> (arrow operator) để truy cập và in thông tin sinh viên thông qua con trỏ ptrSV.
- Thay đổi thông tin sinh viên bằng cách gán giá trị mới cho các trường dữ liệu thông qua con trỏ ptrSV.
- In thông tin sinh viên sau khi thay đổi.

Chú ý:

- Khi sử dụng con trỏ, chúng ta phải cẩn thận với các vấn đề như con trỏ NULL, trỏ sai địa chỉ, và quản lý bộ nhớ đúng cách.
- Để truy cập các trường dữ liệu của cấu trúc thông qua con trỏ, chúng ta sử dụng toán tử -> (arrow operator) thay vì dấu chấm (.).
- Con trỏ không chỉ có thể trỏ đến các đối tượng cấu trúc mà còn có thể trỏ đến các biến, mảng, và các đối tượng khác trong C++.
- Việc sử dụng con trỏ đôi khi có thể làm cho mã nguồn trở nên phức tạp và khó hiểu, đặc biệt đối với những người mới bắt đầu học lập trình. Do đó, chúng ta nên sử dụng con trỏ một cách cẩn thận và chỉ khi thực sự cần thiết.

9.8. Câu hỏi và Bài tập

9.8.1. Câu hỏi

Cho biết các câu là ĐÚNG hay SAI?

1. Cần có dấu chấm phẩy sau dấu ngoặc nhọn đóng của một khai báo cấu trúc.
2. Các kiểu dữ liệu trừu tượng được định nghĩa là các phần cơ bản của một ngôn ngữ lập trình.
3. Nội dung của một biến cấu trúc có thể hiển thị bằng cách truyền biến cấu trúc cho đối tượng cout.
4. Các biến thành viên của cấu trúc có thể được khởi tạo trong phần khai báo của cấu trúc.
5. Trong danh sách khởi tạo biến cấu trúc, bạn không cần phải cung cấp giá trị khởi tạo cho tất cả các thành viên.
6. Có thể cấp phát động một mảng các cấu trúc.

7. Khai báo cấu trúc cấp phát không gian bộ nhớ cho tất cả các thành viên của nó.
8. Cấu trúc lồng nhau nghĩa là một biến cấu trúc bên trong một biến cấu trúc khác.
9. Một cấu trúc hoàn chỉnh không thể được truyền cho một hàm làm đối số.
10. Một hàm có thể trả về một con trỏ đến biến cấu trúc.
11. Khi một hàm trả về cấu trúc, luôn cần thiết cho hàm có một biến cấu trúc cục bộ để giữ các giá trị thành viên cần trả về.
12. Toán tử bấm có độ ưu tiên cao hơn toán tử dấu chấm.

9.8.2. Bài tập

1. Điểm trong mặt phẳng

- Định nghĩa một cấu trúc Diem gồm hai trường x và y kiểu số nguyên.
- Viết các hàm để nhập, in, tính khoảng cách giữa hai điểm, và kiểm tra hai điểm có bằng nhau hay không.

2. Sinh viên

- Định nghĩa một cấu trúc SinhVien gồm các trường thông tin như mã sinh viên, họ tên, ngày sinh, điểm trung bình.
- Viết chương trình cho phép nhập thông tin của một số sinh viên, in ra danh sách sinh viên và tìm sinh viên có điểm trung bình cao nhất.

3. Thời gian

- Định nghĩa một cấu trúc ThoiGian gồm các trường giờ, phút, giây.
- Viết các hàm để nhập, in, tính tổng hai thời gian, và kiểm tra xem một thời gian có hợp lệ hay không.

4. Phân số

- Định nghĩa một cấu trúc PhanSo gồm hai trường tử số và mẫu số.
- Viết các hàm để nhập, in, rút gọn phân số, cộng, trừ, nhân, chia hai phân số.

5. Hình chữ nhật

- Định nghĩa một cấu trúc HinhChuNhat gồm các trường chiều dài, chiều rộng.
- Viết các hàm để nhập, in, tính chu vi, diện tích, và kiểm tra xem hai hình chữ nhật có bằng nhau hay không.

6. Hình tròn

- Định nghĩa một cấu trúc HinhTron gồm trường bán kính.
- Viết các hàm để nhập, in, tính chu vi, diện tích hình tròn.

7. Ngày tháng năm

- Định nghĩa một cấu trúc NgayThangNam gồm các trường ngày, tháng, năm.
- Viết các hàm để nhập, in, tính số ngày của một tháng và năm, kiểm tra ngày hợp lệ.

8. Hộ dân cư

- Định nghĩa một cấu trúc NguoiDan gồm các trường họ tên, năm sinh, nghề nghiệp.
- Định nghĩa một cấu trúc HoDanCu gồm các trường số nhà, đường, quận/huyện và một mảng các NguoiDan.
- Viết chương trình cho phép nhập thông tin về các hộ dân cư và in ra danh sách các hộ dân cư.

9. Sản phẩm

- Định nghĩa một cấu trúc SanPham gồm các trường mã sản phẩm, tên sản phẩm, đơn giá, số lượng tồn kho.
- Viết chương trình cho phép nhập thông tin về các sản phẩm, tính tổng giá trị tồn kho, và in ra danh sách các sản phẩm có giá trị tồn kho lớn hơn một giá trị nhất định.

10. Nhân viên

- Định nghĩa một cấu trúc NhanVien gồm các trường mã nhân viên, họ tên, ngày sinh, chức vụ, lương cơ bản.
- Viết chương trình cho phép nhập thông tin về các nhân viên, tính lương thực nhận (lương cơ bản cộng với các khoản phụ cấp), và in ra danh sách các nhân viên có lương thực nhận cao nhất.

11. Hình tam giác

- Định nghĩa một cấu trúc Diem như bài 1.
- Định nghĩa một cấu trúc HinhTamGiac gồm ba điểm đại diện cho ba đỉnh của tam giác.
- Viết các hàm để nhập, in, tính chu vi, diện tích hình tam giác.

12. Quản lý sách

- Định nghĩa một cấu trúc Sach gồm các trường mã sách, tên sách, tác giả, năm xuất bản, số lượng.
- Viết chương trình cho phép nhập thông tin về các quyển sách, tìm kiếm sách theo tên hoặc tác giả, và in ra danh sách các sách có số lượng lớn hơn một giá trị nhất định.

13. Quản lý hóa đơn

- Định nghĩa một cấu trúc SanPham như bài 9.
- Định nghĩa một cấu trúc ChiTietHoaDon gồm các trường mã sản phẩm, số lượng, đơn giá.
- Định nghĩa một cấu trúc HoaDon gồm mã hóa đơn, ngày lập, danh sách các ChiTietHoaDon.
- Viết chương trình cho phép nhập thông tin về các hóa đơn, tính tổng tiền của mỗi hóa đơn, và in ra danh sách các hóa đơn có tổng tiền lớn hơn một giá trị nhất định.

14. Quản lý thư viện

- Định nghĩa một cấu trúc Sach như bài 12.
- Định nghĩa một cấu trúc DocGia gồm các trường mã độc giả, họ tên, ngày sinh, địa chỉ.
- Định nghĩa một cấu trúcMuonTra gồm mã sách, mã độc giả, ngày mượn, ngày trả.
- Viết chương trình cho phép nhập thông tin về sách, độc giả, và lịch sử mượn trả sách. In ra danh sách các sách đang được mượn và danh sách các độc giả có sách quá hạn.

15. Dữ liệu Phim

Viết một chương trình sử dụng một cấu trúc có tên MovieData để lưu trữ các thông tin sau về một bộ phim:

- Tựa đề
- Đạo diễn
- Năm phát hành
- Thời lượng (tính bằng phút)

Chương trình nên tạo hai biến MovieData, lưu trữ giá trị trong các thành viên của chúng, và truyền mỗi biến lần lượt cho một hàm để hiển thị thông tin về bộ phim theo cách định dạng rõ ràng.

16. Lợi nhuận Phim

Sửa đổi chương trình được viết cho Bài tập Lập trình 1 (Dữ liệu Phim) để bao gồm hai thành viên bổ sung lưu trữ chi phí sản xuất và doanh thu năm đầu tiên của bộ phim. Sửa đổi hàm hiển thị dữ liệu phim để hiển thị tựa đề, đạo diễn, năm phát hành, thời lượng và lợi nhuận hoặc thua lỗ trong năm đầu tiên.

17. Báo cáo về Động đất

Trung tâm Thông tin Động đất Quốc gia muốn ghi lại dữ liệu về ba trận động đất đã xảy ra ở đất nước trong hai năm qua. Bộ phận này cần lưu trữ các dữ liệu sau về mỗi trận động đất:

- Tên động đất
- Vị trí xảy ra động đất
- Năm xảy ra
- Độ lớn theo thang độ Richter
- Số người thiệt mạng

Viết một chương trình khai báo một cấu trúc với các trường để lưu trữ dữ liệu trên. Chương trình nên khai báo ba biến của cấu trúc trên để đại diện cho ba trận động đất. Người dùng sẽ được yêu cầu nhập chi tiết của mỗi trận động đất. Sau đó, chương trình sẽ hiển thị một phân tích về các trận động đất. Chi tiết cần được hiển thị trong phân tích là:

- Tổng số người thiệt mạng do ba trận động đất.
- Độ lớn trung bình của ba trận động đất.
- Tên và độ lớn của trận động đất có độ lớn lớn nhất.
- Tên của trận động đất gây ra số người thiệt mạng lớn nhất.

18. Thống kê Thời tiết

Viết một chương trình sử dụng một cấu trúc để lưu trữ các dữ liệu thời tiết sau đây cho một tháng nhất định:

- Tổng lượng mưa
- Nhiệt độ cao nhất
- Nhiệt độ thấp nhất
- Nhiệt độ trung bình

Chương trình nên có một mảng gồm 12 cấu trúc để lưu trữ dữ liệu thời tiết cho cả một năm. Khi chương trình chạy, nó sẽ yêu cầu người dùng nhập dữ liệu cho từng tháng. (Nhiệt độ trung bình nên được tính toán.) Sau khi đã nhập dữ liệu cho tất cả các tháng, chương trình sẽ tính toán và hiển thị lượng mưa trung bình hàng tháng, tổng lượng mưa cho cả năm, nhiệt độ cao nhất và thấp nhất trong năm (và tháng xảy ra), và trung bình của tất cả các nhiệt độ trung bình hàng tháng.

Xác nhận dữ liệu đầu vào: Chỉ chấp nhận nhiệt độ trong khoảng từ -100 đến +140 độ F.

19. Cấu hình Máy tính

Viết một chương trình sử dụng cấu trúc để lưu trữ các dữ liệu sau về một máy tính:

- Tốc độ đồng hồ của bộ vi xử lý
- Số lõi CPU
- Lượng RAM

- Dung lượng ổ cứng
- Điểm hiệu suất (cao hơn là nhanh hơn)
- Giá của máy tính

Chương trình nên có giao diện điều khiển bằng menu với các tùy chọn để thêm cấu hình máy tính mới, thay đổi giá của cấu hình hiện có và hiển thị đầy đủ dữ liệu của tất cả máy tính. Người dùng chọn cấu hình bằng cách nhập số của nó.

Xác nhận dữ liệu đầu vào: Không chấp nhận giá trị âm cho điểm hiệu suất hoặc giá. Khi nhập số của cấu hình, chỉ chấp nhận số hiện có.

20. Cơ quan Diễn giả

Viết một chương trình giữ theo dõi một cơ quan diễn giả. Chương trình nên sử dụng cấu trúc để lưu trữ các dữ liệu sau về một diễn giả:

- Tên
- Số điện thoại
- Chủ đề thuyết trình
- Phí yêu cầu

Chương trình nên sử dụng một mảng ít nhất 10 cấu trúc. Nó nên cho phép người dùng nhập dữ liệu vào mảng, thay đổi nội dung của bất kỳ phần tử nào và hiển thị tất cả dữ liệu được lưu trữ trong mảng. Chương trình nên có giao diện điều khiển bằng menu.

Xác nhận dữ liệu đầu vào: Khi dữ liệu cho một diễn giả mới được nhập, hãy đảm bảo người dùng nhập dữ liệu cho tất cả các trường. Không nên nhập số âm cho phí của diễn giả.

TÀI LIỆU THAM KHẢO

- [1]. Tony Gaddis, Starting Out with C++ from Control Structures through Objects, Ninth Edition, Global Edition, 2020.
- [2]. Paul Deitel, Harvey Deitel, C++ How to Program 10th Edition, 2023.
- [3]. Vũ Việt Vũ, Phùng Thị Thu Hiền, Giáo Trình Ngôn Ngữ C++, 2017, NXBKHK
- [4]. Trần Thông Quế, Bài tập lập trình với ngôn ngữ C++ từ cơ bản đến nâng cao, 2022, NXB Thông tin và Truyền thông.
- [5]. <https://www.w3schools.com/cpp/>
- [6]. <https://www.guru99.com/vi/cpp-programming-tutorial.html>